
XFaceMaker/UIMS

Version 4.0/Motif 2.1

Tutorial

Nova Software Labs

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



Nova Software Labs

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: sales@NovaSoftwareLabs.com or sales@nsl.fr.

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker is a registered trademark of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: XFMUIMSTutorial v. 7.1.1.

Document date: November 29, 2002

Acknowledgments

The following people have contributed to the development of XFaceMaker:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

Table of Contents

1. Introduction	1
2. Getting Started	5
2.1. Launching XFM	5
2.2. XFaceMaker's Main Window	7
2.2.1. The Message Pane	8
2.2.2. The Toolkit Pane	9
2.2.3. The Interface Pane	11
2.3. Our Main Window	13
2.4. Saving Your Work	15
3. Basic Editing Techniques	17
3.1. Manipulating the Interface	17
3.1.1. The Selection Process	17
3.1.2. Moving and Sizing	21
3.2. The Resource Window	22
3.2.1. Identifying the Widget	23
3.2.2. Widget Resources	25
3.2.3. Editing Resources	29
3.2.4. Making It Happen	29
4. Editing Resources	31
4.1. Strings and XmStrings	31
4.2. Ints and Other Numbers	34
4.3. Enumerations	35
4.4. Colors	36
5. The Menu Editor	39
5.1. A Basic Menu	42
5.2. The Font Selector	45
5.3. More Menus	47
6. Dialog Boxes	51

6.1. The DialogShell	51
6.2. DialogShell Resources	53
6.3. The Bulletin Board	54
6.4. Completing the Dialog Box	55
6.5. Some Fine Points	57
7. Forms	59
7.1. The Form As Dialog Box	60
7.1.1. Form Layout	60
7.1.2. Default Form Fonts	61
7.2. Using XFaceMaker to Attach	63
8. Callbacks and the FACE Programming Language	69
8.1. A Brief Introduction to FACE	70
8.2. The xfmCreateCallback Resource	71
8.3. Pushing Buttons	74
8.4. Try Mode	76
9. More Editing Techniques	79
9.1. Cutting and Pasting	79
9.2. More Attachment Issues	84
9.2.1. Building from Right to Left	84
9.2.2. Spacing Between Widgets	85
9.3. Offsets and Positions	88
9.3.1. Attaching Opposite Sides	89
9.3.2. Centering on a Position	90
9.3.3. Separators as Spacing	90
10. Creating the Application	93
10.1. Saving the Interface	93
10.2. Application Files	94
10.3. The Application Makefile	97
10.4. Makefile Variables	102
10.5. Makefile Rules	108
10.6. Making the Program	111

11. Generating C Code	113
11.1. Option Box Basics	113
11.2. C Code Options	115
11.3. Saving C Code	120
12. FACE Constructs	123
12.1. Structures	123
12.2. Declarations	124
12.3. Arrays	126
12.4. The Address Book	127
13. Advanced Editing Techniques	131
13.1. Reparenting.	131
13.2. Changing Class.	132
13.3. Dragging Resources	133
13.4. Adjusting the Main Window	134
13.5. Displaying the Address.	135
14. The File Selector	139
14.1. Creating a File Selector	139
14.2. More Menu Editing	140
14.3. Opening the File Selector	142
15. Application Functions	145
15.1. Function Declarations.	145
15.2. The OK Callback	146
15.3. Some Loose Ends	150
16. Application C Code	151
16.1. Saving Files	151
16.2. Attaching Functions	151
16.3. Filling in the Blanks	153
16.4. Making the Application	158
17. The Dual Process Architecture	161
17.1. Design Process Status.	161

17.2. Restarting the Design Process	163
18. Active Values	165
18.1. The Active Value Window	166
18.2. Creating Active Values	168
18.3. Calling Active Values	169
19. Groups	177
19.1. Creating a Group	177
19.2. Using a Group	180
19.3. Modifying a Group	181
20. Templates	185
20.1. Creating a Template	185
20.2. Using a Template	189
20.3. Modifying a Template Instance	190
20.4. Modifying a Template Definition	191
20.5. Template Resources	193
21. Creating a Widget Class	197
21.1. Build the class sub-tree	197
21.2. Add Internal Behavior	198
21.3. Define Class Resources	204
21.4. Save the Class	207
21.5. Generate the C Code	210
22. Creating a C++ Class	213
22.1. The“DoubleList” class	213
22.2. Defining the class members	215
22.2.1. Data members	215
22.2.2. Member functions	217
22.3. Defining the OK and Cancel scripts	220
22.3.1. The OK button	220
22.3.2. the Cancel button	220
22.4. Saving the class	220
22.5. Using the class	221

22.5.1. In detail	222
23. Motif - General Concepts	229
23.1. Introduction	229
23.2. Widgets and Gadgets	229
23.2.1. Resources	229
23.2.2. Classes	230
23.3. Motif Class Hierarchy	230
24. Motif - The Metaclasses	235
24.1. The Core Class	235
24.2. The Composite Class	238
24.3. The Constraint Class	238
24.4. The XmManager Class	239
24.5. The XmPrimitive Class	241
24.6. The XmGadget Class	244
25. Motif - The Shell Widgets	247
25.1. Introduction	247
25.2. The Shell Class	249
25.3. The OverrideShell Class	250
25.4. The XmMenuShell Class	251
25.5. The WMShell Class	251
25.6. The VendorShell Class	253
25.7. The TransientShell Class	256
25.8. The XmDialogShell Class	256
25.9. The TopLevelShell Class	256
25.10. The ApplicationShell Class	257
26. Motif - The Composite Widgets	259
26.1. Introduction	259
26.2. XmBulletinBoard	260
26.3. XmForm	263
26.4. XmRowColumn	265
26.5. XmScrolledWindow	270

26.6. XmMainWindow	273
26.7. XmDrawingArea	275
26.8. XmPanedWindow	276
26.9. XmFrame	277
26.10. XmScale	279
27. Motif - The Primitive Widgets	281
27.1. Introduction	281
27.2. XmLabel	281
27.3. XmSeparator	284
27.4. XmPushButton	285
27.5. XmToggleButton	286
27.6. XmDrawnButton	288
27.7. XmArrowButton	289
27.8. XmCascadeButton	289
27.9. XmList	290
27.10. XmText	293
27.11. XmTextField	296
27.12. XmScrollBar	296
. Index	299

CHAPTER 1: Introduction

Welcome to XFaceMaker, NSL's fully featured user interface management system. With XFaceMaker you design the graphical user interface to your application, both in terms of how it looks and how it behaves.

In the first part of this tutorial, we lead you step-by-step through the creation of an interface for a simple application, an address book.

In the beginning, we introduce you to the major features of XFaceMaker. Later, we explain more advanced techniques.

You may get further help by looking at the examples of interfaces built with XFaceMaker in the **demos** directory (if it has been installed).

In the second part of this manual, we cover the essentials of the Motif toolkit, as seen through XFaceMaker. You will find there Motif reference information on the widgets and their resources, as well as tips on how to implement some of the Motif concepts with XFaceMaker.

Terminology

The software industry is full of acronyms, jargon, and technical words. This book assumes that you are familiar with the vocabulary and concepts of C programming with some basic knowledge of the X Window System and OSF/Motif.

If you are unfamiliar with a word, you might get some help in the chapters that deal with Motif concepts in the back of this tutorial. Many of the Motif keywords have been put in the index to help you find the information you need.

If you are new to the world of interface building, here are some brief definitions to get you started:

- | | |
|--------------------|--|
| application | A software program. |
| user | The person using the software, also called an end-user . |
| interface | A means of communication. A user interface is the means of communication between the application and the user: how the application presents information to the user and how the user requests information from the application. |
| graphic | (or graphical) When applied to interfaces, this refers to an interface of objects (such as buttons or icons) on a display, as opposed to simply text on a terminal. |

window There are many ways in which this word is used in the software industry. For now, we point out the obvious one, a rectangular workspace on your screen where the application presents its interface to the user. An application can use more than one window at a time.

A **window manager** is the tool that lets you control the windows on your display. Most window managers will let you position windows, change their size, and open and close them. Others offer more advanced features. Note: the capitalized name “Windows” often refers to the product Microsoft Windows.

X Windows is a popular term for the **X Window System**, also known as just plain **X**, or **X11**, or **X11R5** (the number after the **R** can vary). The makers of **X** do not endorse the term **X Windows** to denote their windowing system, but they have not been able to stem the tide of popular usage in this respect.

iconify To temporarily close a window and replace it with a small icon on the display. You use the icon to re-open the window on the display, usually by double-clicking on it. In the OSF/Motif world, this is also called **minimize**.

widget Any graphical user interface object such as a label, a button, or an icon. A widget may also be a higher-level object, such as a form or row column that contains other widgets. Outside the X11 world, a widget is more commonly referred to simply as an object.

gadget A special kind of widget that uses less of the computer’s resources.

intrinsics The low-level support library for creating widgets in the X Window System, also referred to as **Xt Intrinsics**.

toolkit A support library in the X Window System that contains a collection of widgets that you can use to build an application’s interface.

Motif A toolkit provided by the Open Software Foundation.

X (also X11 and X11R5) The X Window System.

XFM XFaceMaker

FACE XFaceMaker’s programming language, used to program your interface’s behavior.

NSL Nova Software Labs

GUI graphical user interface.

UIMS user interface management system

WM window manager

OSF The Open Software Foundation, the creators of Motif

MIT The Massachusetts Institute of Technology, where the X Window System was created.

If most of these terms are new to you, you might want to read some documentation on graphical user interfaces, in particular the X Window System and OSF/Motif, before reading this tutorial, in order to familiarize yourself with the concepts involved. Here are some places where you could start:

Tim O'Reilly, Quercia. *Volume 3: X Window System User's Guide (OSF/Motif Edition)*, O'Reilly and Associates, Inc.

Open Software Foundation. *OSF/Motif User's Guide*, Prentice Hall, Englewood Cliffs, New Jersey.

Open Software Foundation. *OSF/Motif Programmer's Guide*, Prentice Hall, Englewood Cliffs, New Jersey.

Open Software Foundation. *OSF/Motif Style Guide*, Prentice Hall, Englewood Cliffs, New Jersey.

Open Software Foundation. *OSF/Motif Programmer's Reference*, Prentice Hall, Englewood Cliffs, New Jersey.

Typography

Here are some typographical conventions used in this document.

Italics are used in the normal way, to emphasize words, present titles, etc. They are also used to name parameters to functions when defining them and when describing them.

Boldface is used to name something particular to the development process. It could be the name of a key on the keyboard (such as **Control**), the name of a mouse button (such as **Select**), a function name (**FmInitialize**), or a global variable or constant (such as **NULL** or **True**).

Bold sans-serif indicates text found in an interface, including XFaceMaker's own interface. For example, we may talk about an **OK** button, or a **Name** text field.

Typewriter indicates text in an interface that would appear in typewriter font, such as in a text field or a list.

Bold typewriter indicates FACE code or some other code, such as C or C++ source, commands for make, a shell script, etc. It may also indicate text output, for instance during a compilation. Finally, it may indicate text input that you should enter, such as a command.

The Mouse

In the X environment, the mouse normally has three buttons. In the OSF/Motif world, these three buttons are called **Select**, **Drag**, and **Custom**. Sometimes the third button is called **Menu** instead of **Custom**. We refer to the mouse buttons using these names.

These three buttons correspond to the left, center, and right buttons on your mouse. This may differ, if, for example, you have a left-handed mouse. If your mouse has fewer than three buttons, there should be a way to create the action equivalent to pressing the missing second or third button. See your system's documentation or your system administrator.

The Keyboard

Not all keyboards and mice are the same. However, most people are familiar enough with keyboards to understand the various names for keys. Just to be sure, here is a list of important key names:

Shift Changes a lower case key to upper case, or a symbol in the lower part of a key (such as a digit on U.S. keyboards) to the one in the upper. Sometimes, this key is labeled with a simple upward-pointing arrow.

Control Similar to shift, but changes a key to a special value. Sometimes this key is labeled **Ctrl**, or even **Ctl**.

Alt Another modifier key, similar to **Shift** and **Control**, but resulting in a different character. On some keyboards, this is the same as the **Meta** key. On others, the **Meta** key and **Alt** key are different. On some Sun keyboards, the **Left** and **Right** keys are used as **Alt** keys.

These three keys are independent of each other, and normally do not represent a character unless sent with another key. For example, we may say **Control-A** to indicate the **A** key while the **Control** key is pressed, or **Alt-Shift-F1** to indicate the **F1** function key while the **Alt** and **Shift** modifiers are pressed. If there is more than one key for the same modifier, as there always is with **Shift**, often with **Alt**, and sometimes with **Control**, it does not matter whether you use the left or right modifier key.

The other important key is the **Enter** key. On some keyboards, this key is labeled **Return**. On others, there is only a left-pointing bent arrow, like this:  Some keyboards have two such keys, one by the alphabetic portion of the keyboard, and another by the numeric keypad. These two keys are not always equivalent.

CHAPTER 2: Getting Started

2.1 Launching XFM

First, we explain what happens when you launch XFaceMaker. Your system administrator should have already installed XFaceMaker on your system. Depending on how XFaceMaker was installed, you may need to set or change certain variables in your environment.

A few environment variables must be set before you can work efficiently with XFaceMaker. Once the NSL products are installed, you can execute a script (`nslnv.sh` or `nslnv.csh` depending on the Unix shell you are using) to set these variables. The script sets:

- `NSLHOME` to the name of the directory where the NSL products are installed.
- `XFM_LICENSE_NAME` to `xfm3`. If you do not have a full XFaceMaker license, you may need to change this to `xfm3/il` or `xfmr/idt` - otherwise, XFaceMaker will start up in Demo mode and will not let you save your work.
- `WX_LICENSE_NAME` to `wx2`. `Wx` is the NSL text editor that you can use from XFaceMaker to edit scripts.
- `DM_LICENSE_NAME` to `xdm2`. Useful if you are working with XFaceMaker/WF, the package that includes the XDraw library.

The script modifies:

- `PATH` to add `$NSLHOME/bin` to the existing setting.
- `LD_LIBRARY_PATH` (`LIBPATH` on IBM) to add `$NSLHOME/lib` to the existing setting.

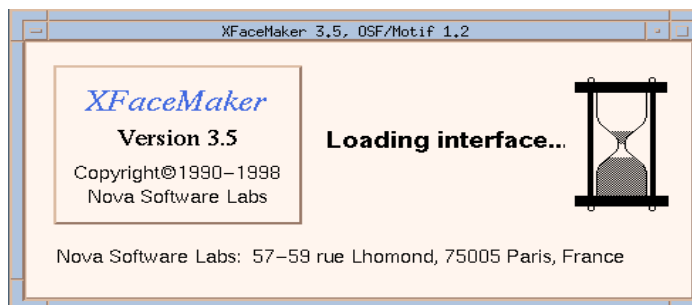
The script should also set:

- `NSL_LICENSE_HOST` to the name of the machine running the NSL license server, `NSLlicense`. This variable may not be set by your script, if your system administrator has not added it to the script. Make sure it is properly set. Otherwise, XFaceMaker may start in demo mode. In that case, it won't let you save your work.

To launch your XFaceMaker session, enter the following command:

xfm

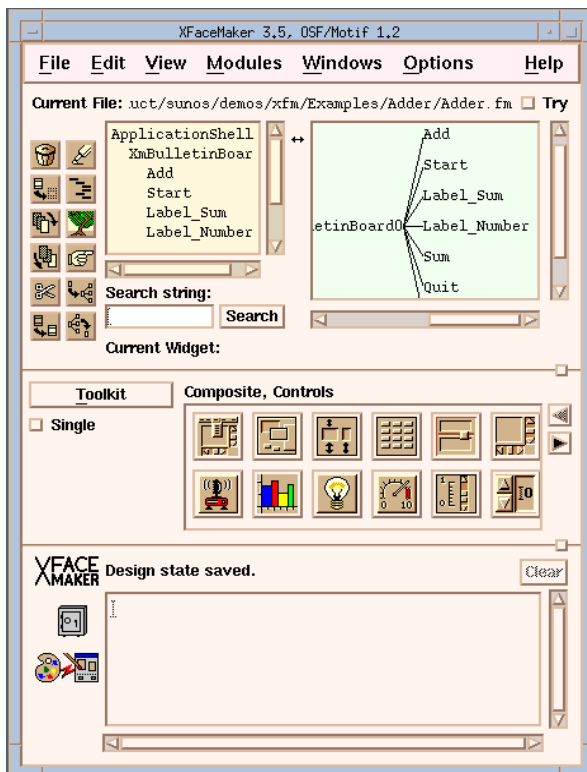
You will at first see XFaceMaker's "welcome" window. The sand in the hour glass shows XFaceMaker's progress as it loads its own interface and performs other initialization tasks. XFaceMaker's user interface was designed with XFaceMaker itself.



2.2 XFaceMaker's Main Window

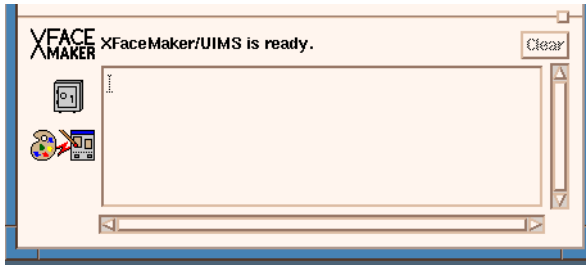
When the sand reaches the bottom, XFaceMaker's main window appears. It contains a menu bar and three "panes". Horizontal lines separate the panes, and you can drag the small square knob on the right end of the line to change the size of panes. From top to bottom, the three panes contain the interface hierarchy, the toolkit, and messages.

Before explaining the main window in more detail, we will point out the most important menu command. The last item in the **File** menu is the **Exit** command. You use this to leave XFaceMaker. If you have done any work on your interface, you will probably want to save your work in a file before you exit. Saving interface files is described in section 2.4.



2.2.1 The Message Pane

Let's start at the bottom with the message pane.



There is a one-line status message in bold type. When you launch XFaceMaker, you will see the message

XFaceMake/UIMS is ready.

when XFaceMaker is through with its initialization and ready for editing.

If, instead, XFaceMaker starts up in demo mode, ask your system administrator to check that NSLicense is running, with a valid license file. You will not be able to save your work if XFaceMaker is launched in demo mode.

Beneath the status line is a message area. XFaceMaker prints errors and warnings here as you construct your interface. The messages remain in the message area, and you can use the scroll bar to review messages that have scrolled out of the message area. You may also clear the message area by pressing the **Clear** button above it on the right.

To the left of the status line is the XFaceMaker logo, and beneath the logo are three status indicators. These indicators show the state of XFaceMaker in relation to its communications with other processes. We will explain these indicators in detail later.

2.2.2 The Toolkit Pane

Above the message pane is the toolkit pane. Press the **Toolkit** button to pop up the menu of toolkit widget categories.



Some of these categories represent Xt Intrinsics superclasses, such as **Shell**, **Composite**, or **Primitive**, while other categories represent logical groupings, such as **Menu** or **Dialog**.

Select a category, and you can see its widgets appear to the right in the toolkit box. When you place the cursor over an Icon, a name appears to the left of the toolkit, the name of the widget class that the icon represents. If there are too many widgets to fit in the box, you can use the arrow buttons to scroll to the left or right. The name of the currently selected category appears above the box.

When you select a widget category by clicking on its name, a small indicator appears to the left of the name. If you select a new button, the previously selected button will be deselected.

Beneath the **Toolkit** button is the **Single** toggle button, which is turned on by default. If you turn off this button by clicking on it, you can select more than one category, and the icons for the widgets in all the categories selected appear in the toolkit box.

Now, we will use the toolkit to start to build your interface. First, make sure the **Composite** button is selected in the toolkit menu. Then, find the **XmMainWindow** icon, which looks like this:



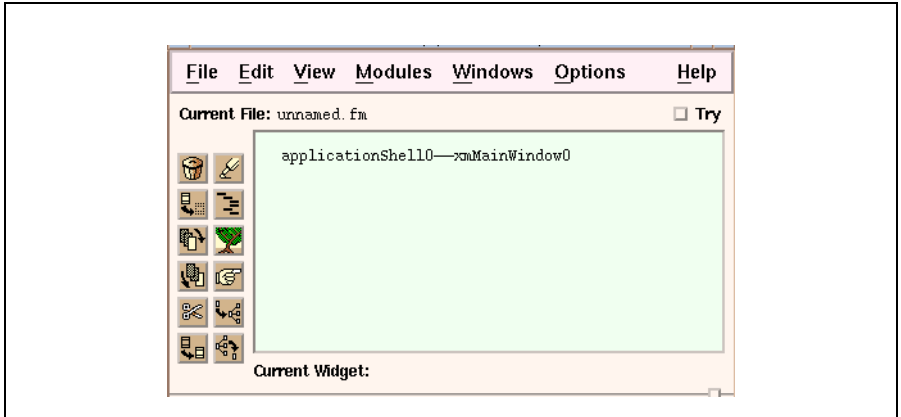
To create a main window, do the following:

1. Click the **Select** mouse button on the **XmMainWindow** icon. The cursor will change to a cross, like this: +
2. Choose where you want the top left corner of the main window. It cannot be over one of XFaceMaker's windows, but you can move it after it is created. Move the cursor to your chosen position and press the **Select** button down without lifting it.
3. With the **Select** button still down, drag the cursor to where you want the bottom right corner to be, then release the button. The cursor should change back to normal. Make the window about two-thirds the width and height of XFaceMaker's main window.

Once you have created this main window, you can change its size and position using the window manager, in the same way as with any other window on your display. XFaceMaker automatically notes the window's changed information.

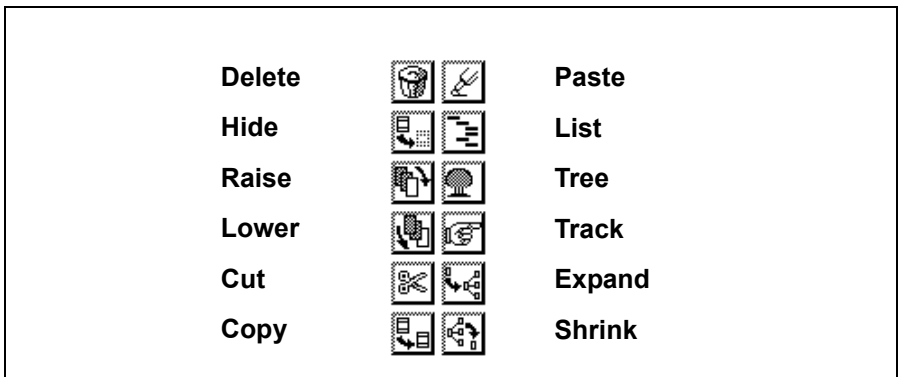
2.2.3 The Interface Pane

The topmost pane displays the widget hierarchy of the interface you are building. Now that you have created a main window, you can see it in that pane.

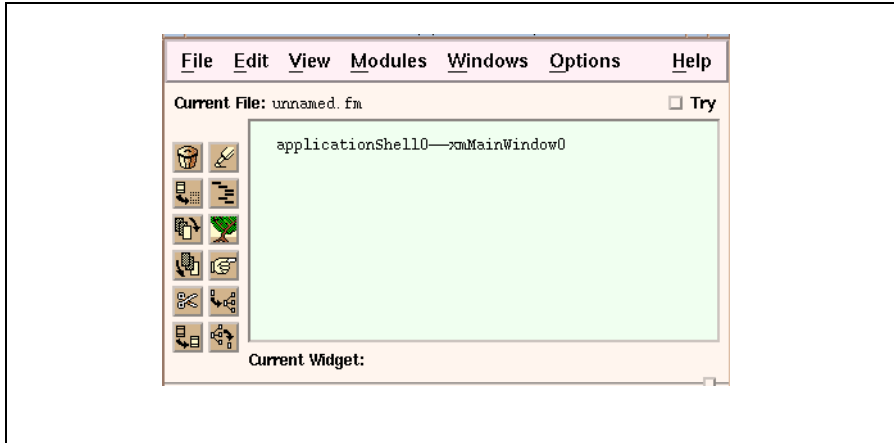


At the top of the pane is the file name for this interface. Up to now, we have not specified a file name, so XFaceMaker displays the default name, `unnamed.fm`. To the left below the file name appear some icon buttons for commonly used commands. All of these commands are also available in the menu bar.

The icons available are:



Next to the icons is the widget tree. In it, you see that we in fact created two widgets, **applicationShell0** and **xmMainWindow0**.



When XFaceMaker creates a widget directly on the display, as we asked it to do with our main window, XFaceMaker automatically puts it in a **Shell** widget which communicates with the window manager. Also, when XFaceMaker creates a widget, it automatically gives it a name based on the widget's class. We will show you how to change that name later.

Note the line connecting **applicationShell0** and **xmMainWindow0**. This shows the relationship between the two widgets, that **xmMainWindow0** is a child of **applicationShell0**, or that **applicationShell0** contains **xmMainWindow0**. The children of a widget appear to the right of the widget connected by a line.

Finally, beneath the widget tree is the name of the currently selected widget. The currently selected widget indicates which widget you are editing for specific properties. For now, no widget is selected.

At this point we are ready to fill the main window with our interface.

2.3 Our Main Window

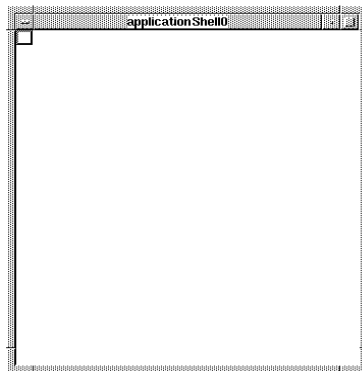
OSF provides the **XmMainWindow** widget class as a basis for interface main windows. The **XmMainWindow** class automatically handles certain interface designs typical of OSF/Motif applications. An **XmMainWindow** widget can contain up to three children: a menu bar, a workspace, and a command space. We will create only the first two. First we add the menu bar:

1. Open the **Toolkit** menu and select the **Menu** category.
2. Click on the **MenuBar** icon, which looks like this:
3. Move the crosshair cursor to the tree in XFaceMaker's main window, and click on the name **xmMainWindow0**. You now see a new child, **menuBar0**, created as a child of **xmMainWindow0**.



```
applicationShell0—xmMainWindow0—menuBar0
```

This illustrates how to create children directly in the widget tree. For now, the menu bar appears as a small square at the top left corner of the window, but it will change as we build our interface. On some platforms, the menu bar will span the width of the window.



Now we will show another way to create widgets, this time directly in the interface.

1. Select the **Composite** category in the **Toolkit** menu.
2. Click on the **XmBulletinBoard** icon, which looks like this: 
3. Move the crosshair cursor inside the main window we created earlier, and click anywhere except over the small square representing the menu bar.

On some platforms, the menu bar seems to have disappeared from the interface, but in fact, it's only hidden because it has no menus in it. You can verify in the widget tree that **xmMainWindow0** has two children, **menuBar0** and **xmBulletinBoard0**.

```
applicationShell0—xmMainWindow0└─menuBar0
                                   └─xmBulletinBoard0
```

The **XmMainWindow** widget class does not recognize the **BulletinBoard** as either a menu bar or a command box, so it assumes the **BulletinBoard** is a workspace and arranges for it to fill the main window. After we add some menus to the menu bar, it will reappear (if it had disappeared earlier).

2.4 Saving Your Work

It is good practice to save your work from time to time. Also, if you exit XFaceMaker before finishing this tutorial you will want to save the work you have done so far. This section describes how to save the interface you are creating. You can find more details in the *XFaceMaker User's Guide*, in the chapter called “Getting Started”.

While doing this tutorial, we suggest you save the current interface from time to time. We suggest you name your file **addrbook.fm**, which corresponds to the interface you will create in this tutorial. You can save your interface as follows:

1. Select the **Save As** item in XFaceMaker's **File** menu.
2. When the file selector pops up, enter the file name **addrbook.fm** into the text field under **XFaceMaker file to save**.
3. Click the **OK** push button.

After saving your file the first time, you may use the **Save** command in the **File** menu to save your interface into the same file. In this case, XFaceMaker will not pop up the file selector.

If you exit from XFaceMaker, you can resume editing your interface by selecting the **Open** item in the **File menu**, then selecting the file you saved in the file selector.

CHAPTER 3: Basic Editing Techniques

We will build a simple version of our main window in order to demonstrate basic editing techniques with XFaceMaker. In the previous chapter, we created the basis of a main window that could be used for any interface. Now we proceed to add specific elements for our application, an address book.

3.1 Manipulating the Interface

Our address book will have the currently selected address displayed at the top of the main window, and the list of addresses beneath it. We will use an **XmLabel** widget to display the currently selected address, and an **XmList** widget to display the list.

First, create the current address label as follows:

1. Select the **Primitive** category in the **Toolkit** menu.
2. Click on the **XmLabel** icon, which looks like this:
3. Move the crosshair cursor to the main window, and press the **Select** mouse button where you want the top left corner of the address label to be.
4. Without releasing the **Select** button, drag the mouse until the cursor reaches where you want the lower right corner of the address label to be. You should make the address label take up the top quarter or so of the main window.



Note that because there is a default value set for the `marginWidth` and `marginHeight` resources, it may not be possible to enlarge or drag the label to the edge of the window. To do so, change the resource value.

Next, create the list from which we will select addresses:

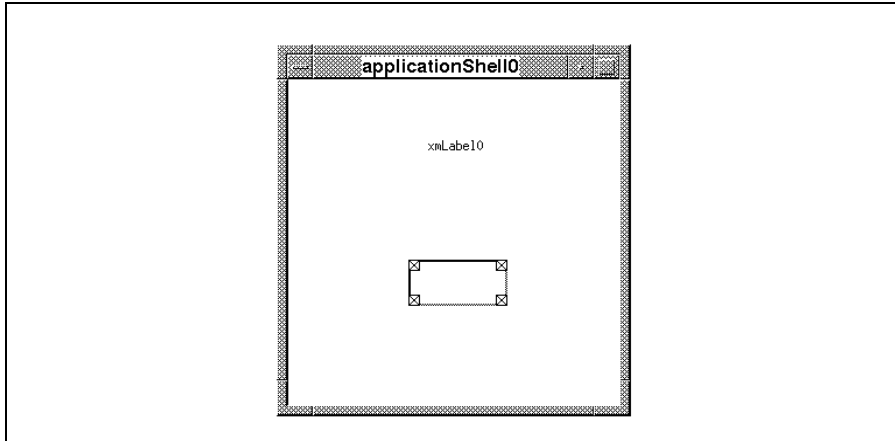
5. In the **Toolkit**, click on the **XmList** icon, which looks like this:
6. Move the crosshair cursor to the main window, and press the **Select** mouse button in the middle of the main window beneath the address label we just created.
7. Without releasing the **Select** button, drag the mouse to create the list widget as a small rectangle. We will enlarge it later.



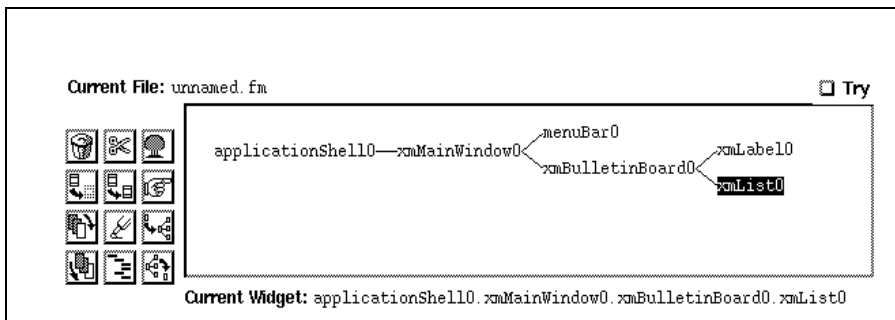
3.1.1 The Selection Process

We will use the list widget to examine how XFaceMaker lets you manipulate your widgets directly in your interface. To select a widget for manipulation, you click on it with the **Select** button of your mouse. Click on the list widget we just created. You should see several things happen:

- The widget now has “handles” to show it is the current selection. These appear as little boxes with x’s in them on the four corners of the widget.



- The name of the widget is highlighted in the widget tree. The name of the path of the widget inside the widget hierarchy appears in the **Current Widget** field beneath the widget tree.

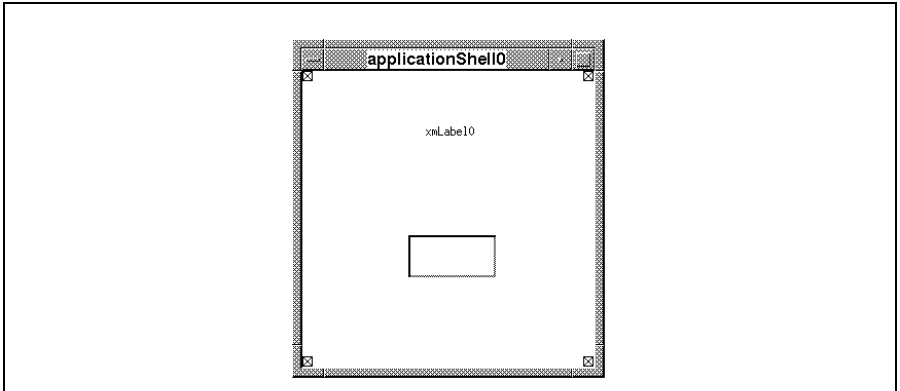


To select a widget that contains another widget, you use the **Custom** mouse button. If you click the **Custom** button on the interface when a widget is selected, you open a popup menu that has three submenus: one to operate on the current selection, one to operate on the current object, and one to operate on the object under the cursor, if the cursor is not on the current object. Using the button that selects the current object’s parent, in the popup menu, you can climb up the widget tree.

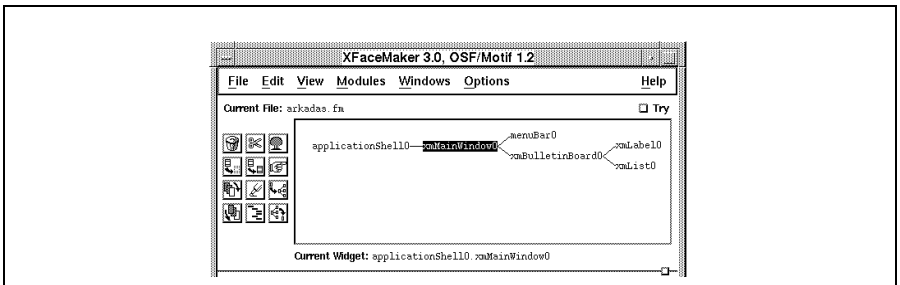
We can watch this in action with our interface:

1. Click the **Custom** button with the mouse somewhere the main window. Select

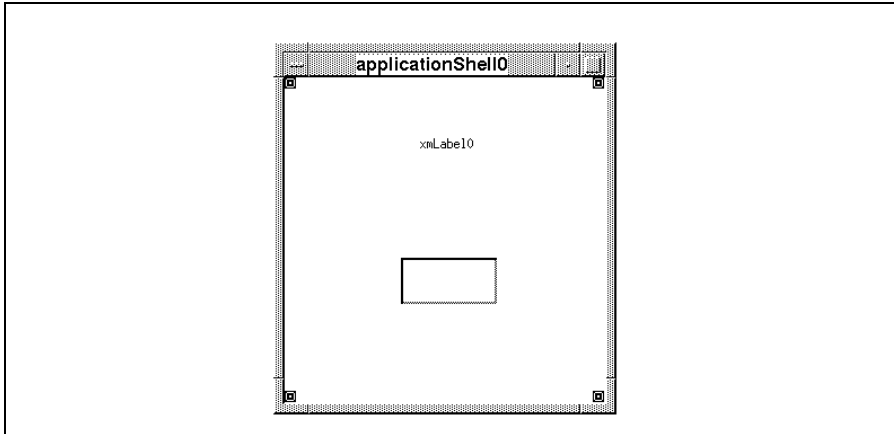
the parent of the list, in this case, the bulletin board. The handles have moved to the bulletin board, as shown here:



2. Click the **Custom** button again and select the parent of the bulletin board. The main window is now selected. The handles have not moved, since the bulletin board and the main window occupy the same space on your screen. However, the widget tree and **Current Widget** field reflect the new selection:



3. Click the **Custom** button again. You have selected the applicationShell, the parent of the main window. When you select a shell, XFaceMaker uses black handles with circles to indicate the shell:



Since a shell is always exactly the same size as its child, you can use the handles to see whether the shell or its child is selected.

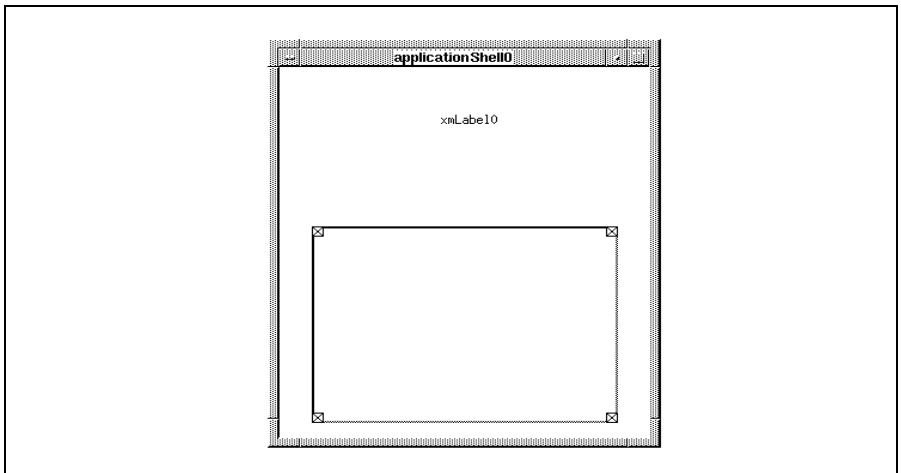
4. Once you have reached the application shell, you cannot change the selection with the Custom button. You can clear the selection, by pressing the **Control** key on your keyboard while you click the **Select** button. Ctrl Click Select toggles the current selection: if the selection is not empty, it clears it.

If you click the **Custom** button when nothing is selected, XFaceMaker lets you select the container widget where you clicked. If you click the **Select** button when nothing is selected, XFaceMaker selects the Primitive widget on which you clicked. You may also select widgets directly in the widget tree using the same techniques as in the interface itself. In the widget tree, you may select any widget using the **Select** button, even container widgets.

3.1.2 Moving and Sizing

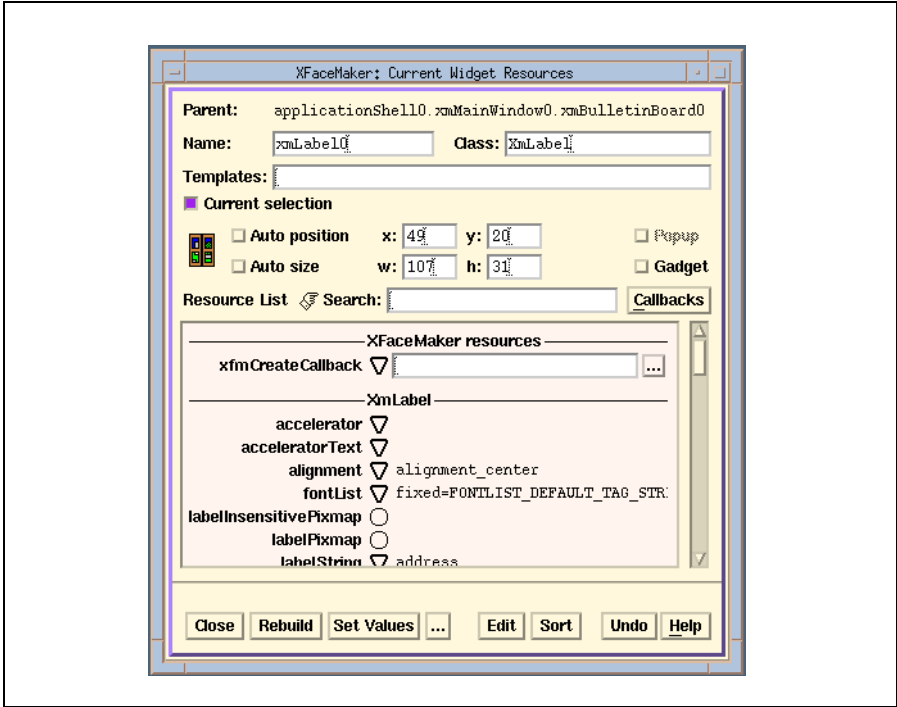
Once you have selected a widget, you can move it and change its size using the **Select** button. To move it, simply click anywhere within the widget except over the handles, then while holding the **Select** button down, drag the widget to its new position, and release the button. You can select and drag a widget with the same button press. Try this with the list widget to position its top left corner. You can also use the **Drag** button instead of the **Select** button to position a widget without selecting it first.

To change the size of a widget, press the **Select** button over one of its four handles, then drag the handle. You change the size by changing the position of this corner while the opposite corner remains fixed. Try this with the list widget so that it fills the bottom portion of the window. The default values of the **marginHeight** and **marginWidth** resources will prevent you from dragging the widget too close to the edges of the window. Your interface should now look something like the figure below. (On some platforms, the menu bar can be seen at the top of the window).



3.2 The Resource Window

For the time being, we will place a sample address in the address label. To do this, we need to set its **labelString** resource using an XFaceMaker resource window. To open a resource window, select the **New Resource Window** item in the **Windows** menu¹. After the window is open, select the label by clicking on it in the interface. The resource window should now look something like the figure below.



This window contains three sections: The top section identifies the widget, the middle section shows its resources, and the bottom section provides the various operations that the window can perform.

1. You can also use the Custom button: the mouse cursor being over the label, press the Custom button to open the popup menu and choose the item labeled Resources for xmLable0. This opens the resource window and selects the xmLabel0 widget.

3.2.1 Identifying the Widget

The top of the resource window tells you which widget the resource window is editing. The parent field contains the path of the widget's parent within the widget hierarchy. In our case,

```
applicationShell0.xmlMainWindow0.xmlBulletinBoard0
```

means that the parent of the label is **xmlBulletinBoard0**, whose parent is **xmlMainWindow0**, whose parent is **applicationShell0**.

The name of the widget is in the **Name** field. You can edit this field to change the widget's name. Try this out by changing the name of this address label to **address**. To see the change take effect, press the **Rebuild** or the **Set Values** button at the bottom of the box. After pressing **Rebuild**, you can see the changed name in the widget tree.

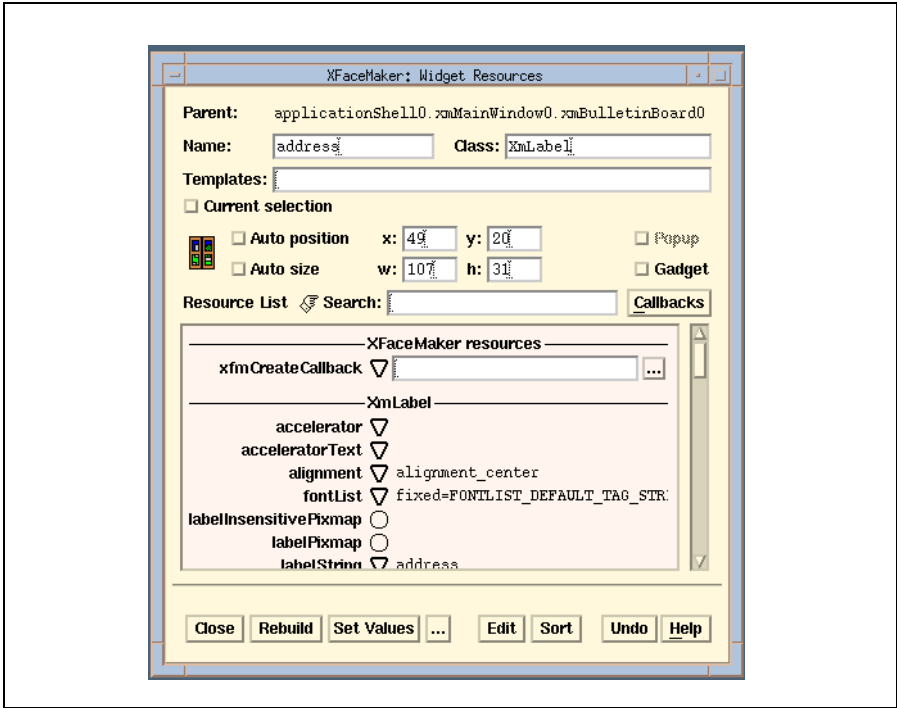
To the right of the widget name is the widget class. This field is also editable. You can change the class of a widget by entering the name of the new widget class in this text field. You may also click on an icon in the toolkit, then, after the cursor has changed to a crosshair, click on this text field to enter the name of the corresponding widget class in this text field. In either case, you need to press **Rebuild** or **Set Values** for the change to take effect. For now, leave the class as is.

The **Templates** field contains the list of templates used to create this widget. We explain templates in a later chapter.

The last piece of identification for the widget is the **Current selection** toggle button. When this button is selected, it indicates that the window will show the resources for the currently selected widget, the one with the x-ed handles  in the interface. If you select another widget, the resource window will be updated with the contents of the new widget.

If you deselect the current selection button, then whatever widget you are editing in that resource window remains in that window, even if you select another widget. Because you can have more than one resource window open at a time, the appearance of the window changes when you deselect the current selection button.

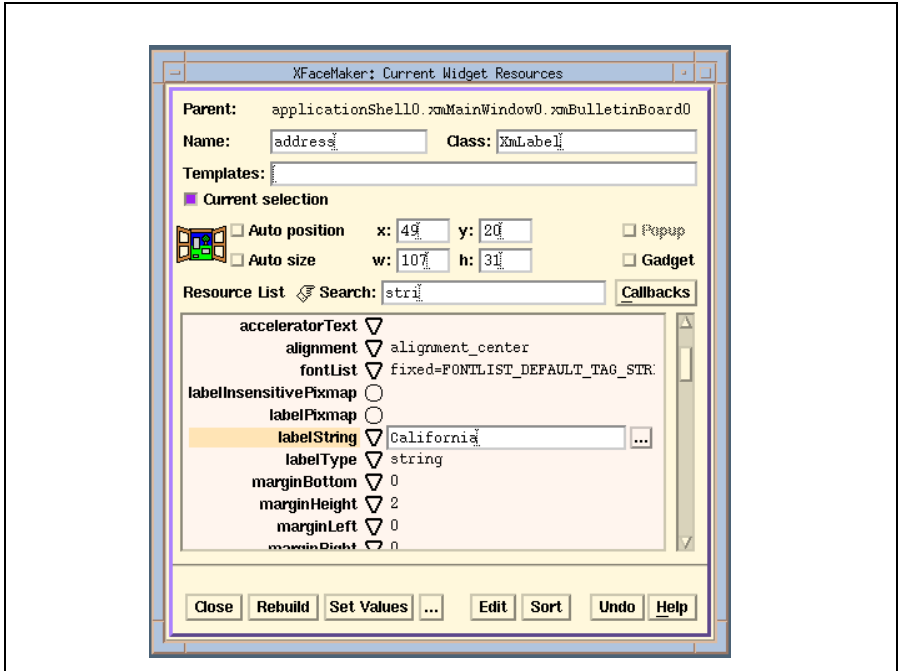
The title of the window changes (as well as its name when iconified), and the 3-d shadow inside its border disappears. Click on the toggle button to see how the window looks when it is not the current selection. Click on it again to make it the current selection.



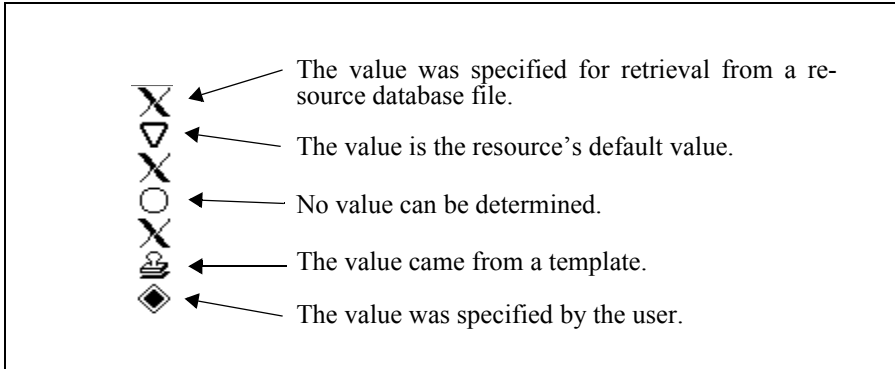
While you may have more than one resource window open at a time, only one of them may be the current selection. If you make a resource window the current selection, and there is another window that is already the current selection, XFaceMaker automatically deselects the latter window's current selection button.

3.2.2 Widget Resources

The center of the **Widget Resource** window shows the resources of the widget identified at the top of the window.



The list contains each resource on one line, with the name of the resource on the left, and its value on the right. If the value contains more than one line or is very long, only the beginning of the first line is shown. Between the resource name and value is a small icon showing where the value shown came from. Here are the possibilities:



The current resource has a text field next to it containing the value. You may edit the value in the text field to give the resource a specific value. In this case, the icon for that resource will be the one shown here on the right . You can use the arrow keys to access the parts of the text field that are not immediately visible, including any lines after the first line.

Next to the text field is a push button with an ellipsis in it. The ellipsis is the three dots, like this: ... When you press the button, you get a dialog box specially designed for editing the current resource value. This box will replace the entire resource window below the widget identification section. We will show you some of these boxes later.

There are several ways to choose the current resource.

- The most obvious way is to simply click on the line containing the resource. If you do not see the resource you want in the visible portion of the list, you can use the scroll bar to scroll the list. Try this with the resource list for the label widget. You can drag the rectangular knob (usually called the “slider”) in the center darkened portion of scrollbar to scroll the window up and down. You can also click above or below the slider to scroll one page up or down. Or, you can scroll up or down one line by clicking on one of the two arrow buttons.

- If you know the name of the resource you want, you may type one or more characters of its name in the **Search** field above the scrolled window, marked by the icon shown here , then press **Enter**. The current resource changes to the first resource after the current one that contains that letter or group of letters. If the search reaches the end of the list, it resumes at the beginning of the list.

For example, enter the letters **stri** in the **Search** text field. Now, each time you press **Enter** in the text field, the current resource moves between **label-String** and **stringDirection**.

- You may also type a single letter directly into the list to look for a resource beginning with that letter. You must be sure the keyboard focus is not in the text field. For users that use *explicit* keyboard focus (also called “click to type”), look for a black line around the icon next to the current resource, instead of around the text field. If you use *pointer* keyboard focus, make sure the cursor is inside the list, but not the text field.²
- Finally, there is a special type of resource called *callbacks* which are especially useful. You can edit one of these resources directly using the **Callbacks** button, which presents a menu of callback resources in the widget. We will explain callbacks in more detail when we talk about FACE code.

By default, XFaceMaker divides the resource list into sections according to the superclasses that the widget’s class inherited from. This is the basis of the Xt Intrinsics library’s object-oriented system. The superclasses show the different parts that make up the widget. You do not need to know the Xt Intrinsics library at this level of detail to use XFaceMaker, but expert users sometimes find it more informative to see the class structure of the widget. We will show you how to turn off this feature in the upcoming section on sorting options.

Above the resource list are the values for the resource geometry: its *x*-position, *y*-position, width and height. XFaceMaker separates these values since they are so essential to interface layout. They also appear in their normal place within the widget list.

There are also two toggle buttons to the left of the geometry values. The **Auto position** indicates that the widget will take its default position instead of the one you define. This is normally turned off, but you may want to turn it on if the widget’s par-

2. If you are not sure what kind of focus policy you have, the presence of the black line around the icon, text field, push button, or some other object in the window indicates explicit keyboard focus. The lack of any such highlighting indicates pointer keyboard focus.

ent is the kind that will control its children's position; for example, a shell, a main window, or a form. If you turn on this toggle, the shadow around the **x** and **y** values disappears to indicate that these values are no longer editable. Their icons in the resource list change from  to .

Similarly, the **Auto size** toggle indicates whether or not you want to define the widget's width and height and affects the **w** and **h** text fields in the same way that the **Auto position** toggle affects the **x** and **y** text fields.

To the right of the text fields are two other toggle buttons. These present fairly technical aspects of the widget that you may not be familiar with. If this is the case, you may simply ignore them and rely on XFaceMaker's default values.

The toggle button to the right of the **y** text field, **Popup**, is not really a resource, but it indicates how XFaceMaker will make the widget appear. If set, XFaceMaker uses the function **XtPopup** to make the widget appear, otherwise **XtManageChild**. This toggle is only sensitive if the widget is a shell widget. We recommend that you always leave this toggle set for shell widgets, otherwise you may witness bizarre behavior from shells, such as their size always changing to match the main window's size.

Beneath the **Popup** toggle is the **Gadget** toggle. If sensitive, it means that there is a gadget version of this widget that you may use instead of the widget version. Gadgets use less computing power than widgets, but they also have fewer capabilities. To use the gadget version of the widget, set the toggle. By default, it is unset. See the Motif section of this manual, or your OSF/Motif documentation for details about which widgets can be gadgets and what the differences are.

3.2.3 Editing Resources

When editing resources in the resource list, you can press the ellipsis button shown here  to edit that particular resource value using a dialog box specially designed for that resource's type. We will look at some of the basic dialog boxes for editing resource values. Other dialog boxes will come up as we build our interface.

3.2.4 Making It Happen

At the bottom of the window are the buttons that provide the basic actions of the resource window, as follows:

	Close this resource window.
	Apply these changes by rebuilding the object.
	Apply these changes by setting the values. The changes may not be immediately visible.
	Pop up the Apply Options box.
	Switch to resource text editor.
	Set sorting options.
	Undo last change.
	Get help on the resource window.

As you move the cursor over one of these buttons, and any other push button in the resource window, a brief description of the button's function appears just beneath the separator line.

The button that you use most frequently should be the **Rebuild** or the **Set Values** button. These buttons take all the changes you have made in this resource window and apply them to the widget. All changes you make, including the name, widget class, auto position and size toggles, gadget toggle, resource values, and so forth do not take effect until you press **Rebuild** or **Set Values**. The sole exception is the **Current selection** toggle, which takes effect immediately. Note that this toggle does not change the widget in any way. It simply changes XFaceMaker's behavior.

By default, when you edit resource values and click **Rebuild** or **Set Values**, XFaceMaker applies: **all resources** (all the resource modifications) to **self** (the widget being edited in that window).

In this tutorial, we will apply resource changes using the **Rebuild** option so that all modifications will be immediately visible.

Since XFaceMaker waits for you to press **Rebuild** or **Set Values** before applying changes, the values you see in the resource window may not match the widget you see in your interface. For example, you may have changed the value of the background resource to red, but the actual color on the interface will remain as before (blue, for example).

For this reason, XFaceMaker has an icon that indicates if you have changed anything in the resource window. The icon looks like a real window in a house. Behind the window is a representation of an interface.



If the icon is “closed”, then the resource window is up to date.



If the icon is “open”, then the resource window has been changed in some way, and you must press **Rebuild** or **Set Values** to make these changes take effect.

The **Apply** options specify how to apply resource values in special ways. You can pop up this box by clicking on the  ellipsis button at the bottom of the Resources window. We will explain how to use this dialog box later.

CHAPTER 4: Editing Resources

4.1 Strings and XmStrings

The most basic type of resource value is a string. You may enter any resource value as a string, the same as you would in a resource data base file.¹

XFaceMaker converts the string to the appropriate resource type when it creates the widget.

If a resource type has a special dialog box for editing its value, the result of that dialog box will ultimately be a string. You see this string as the value of that resource in the resource list. Since XFaceMaker represents all resource values as strings, it will use the **String** dialog box if it can find no other dialog box for a resource's type.

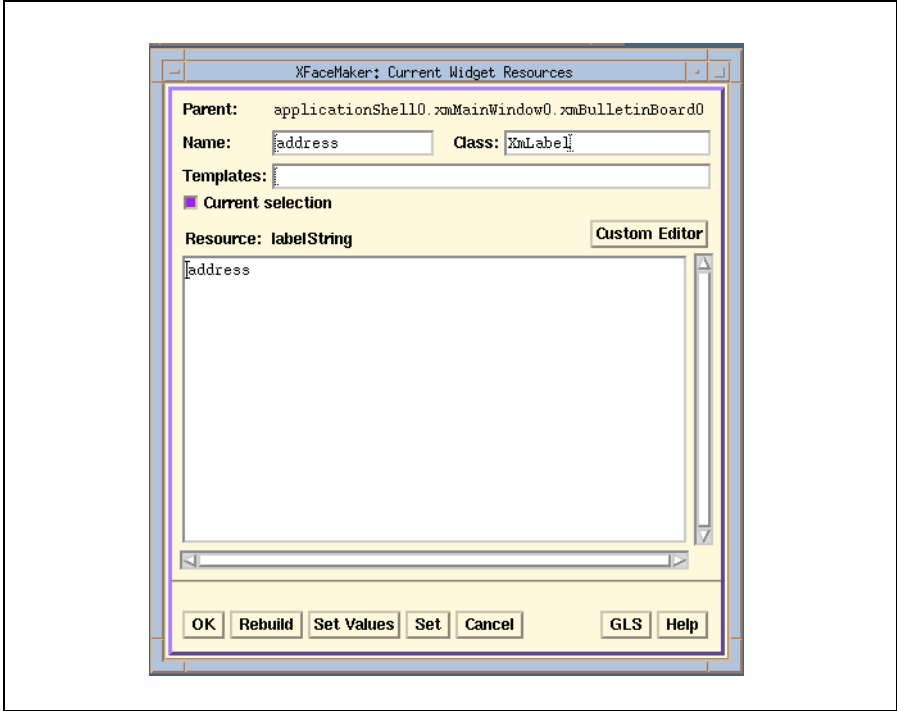
In addition to the string, the OSF/Motif toolkit adds the new type **XmString**, for complex strings that contain more information about the string than just the text, such as its character set. XFaceMaker uses the same dialog box to enter strings and **XmStrings**, since the vast majority of OSF/Motif applications do not use the additional properties that differentiate **XmStrings** from strings.

We use the string dialog box to illustrate the basic features of all resource dialog boxes. To bring up the string dialog box, do the following:

1. Open a resource window if there is not already one open. Make sure the **Current selection** toggle is set.
2. Select the label widget that we created to show the current address.
3. Select the **labelString** resource in the resource list.
4. Press the ... button to the right of the text field.

Note that everything below the **Current selection** toggle has changed to an editor for the resource value. Beneath the toggle is the name of the resource you are editing, then the main dialog box for editing the value. In this case, there is a text widget that contains the name of the label, **address**, since that is the name you entered in the **Name** field. By default, OSF/Motif label widgets have their name as the label string.

1. For those of you who have programmed in C, this can be a bit misleading at first. For example, if the label type is a pixmap, you must enter **pixmap**, not **XmPIXMAP**.



At the bottom are buttons for dealing with the new resource value. Like the resource list, this dialog box will tell you what a button does when you place the cursor over one. All resource dialog boxes have these buttons:

OK Confirm the new resource value, update the widget, and return to the list. Press this button if you are happy with the resource value, and you want to return to the resource list.

Rebuild Confirm the new resource value, update the widget by rebuilding it, but remain in this dialog box. Press this button if you want to see the effect the new resource value has on the widget, but you want to remain in the dialog box in case you want to change the value.

Set Values Confirm the new resource value, update the widget by setting a new value for it, but remain in this dialog box. Press this button if you want to remain in the dialog box in case you want to change the value.

-  **Set** Confirm the new resource value without updating the widget, and return to the list. Press this button if you want to use this new value, but you do not want to update the widget yet for some reason. For example, you may need to set another resource, or you may not want to take the time now to recreate the widget or widgets involved.
-  **Cancel** Return to the list without changing this resource value. Press this button if you have changed your mind about editing this resource value.
-  **Help** Get help on this resource editor.

The string dialog box has these two additional buttons:

-  **GLS** Insert the internationalization marker at the beginning of the string. See the section on Internationalization for details.
-  **Custom Editor** Use your own editor to edit this resource value. XFaceMaker determines which editor to use according to your FMEDITOR environment variable. By default, it uses **vi**.

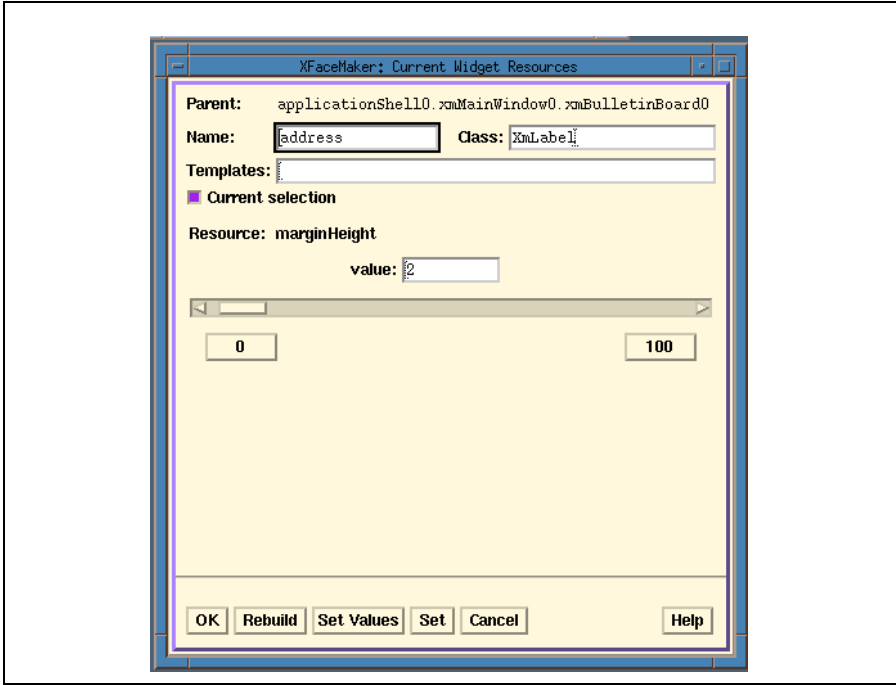
The editing box is an OSF/Motif **XmText** widget. You should see your OSF/Motif guide for complete documentation of this widget. Briefly, you type in the text that you want for the value, using the arrow keys and the mouse to move the cursor and the **Delete** key to delete characters.

You can also cut and paste text with the mouse, but the OSF/Motif model is slightly different from the standard **X11** model. When you paste text using the middle mouse button, it inserts the text at the text position where you click the mouse button, not where the text cursor is.

If you press the **Custom Editor** to start up your own editor, XFaceMaker saves the value in a file for your editor. When you leave the editor, XFaceMaker retrieves the modified value from the file and replaces the text. This may require a certain amount of time, especially if the editor needs to be run inside an **xterm** session. You need to decide whether the convenience of using your own editor is worth the expense of starting up a completely separate editing session for the changes you want to make. We explain the custom editor in detail in a later section.

4.2 Ints and Other Numbers

To look at how integers and other numbers are represented as dialog boxes, we shall change the margin around our label that displays the current address. If the string dialog box is still open, click on the **Cancel** button to return to resource list. Select the label for editing, and then select the **marginHeight** resource in the list. Press the ... button to bring up the integer dialog box.

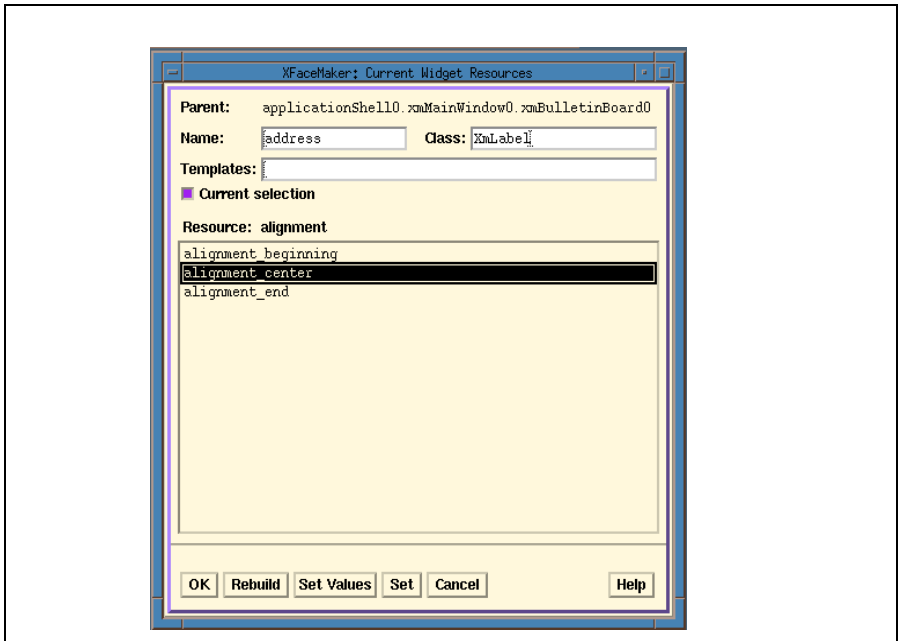


The text field at the top of the box indicates the current value. You may type a new value directly into this field. However, you may also use the scroll bar below the text field to adjust the value shown. Note that there are also buttons underneath the scrollbar. These buttons show the lower and upper bound of the scrollbar. You may press one of these buttons to change the bound corresponding to that button. Using the **Select** mouse button to press the button decreases the limit. Using the **Custom** mouse button increases it.

Use the scrollbars and/or the text field to set the margin height of the label to five, then press the **OK** button to apply the resource and return to the resource list. Select the **marginWidth** resource, and set its value to five in the same manner that you did for the margin height.

4.3 Enumerations

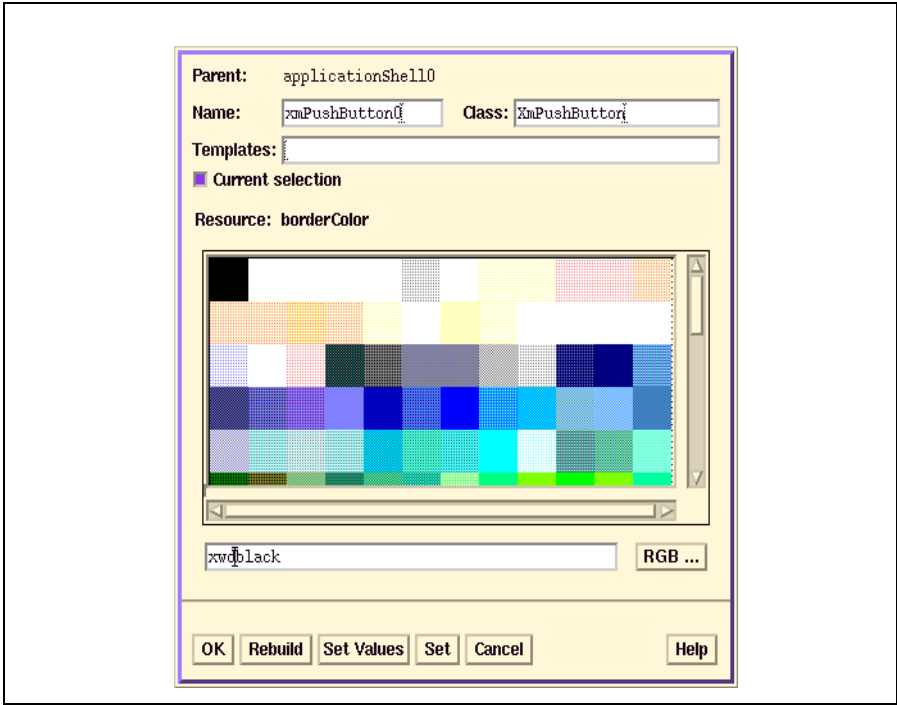
XFaceMaker presents the choices for an enumeration as a list. You select the item in the list that corresponds to the value you want to set for the resource. For example, you can choose the alignment of the address label by selecting the **alignment** resource, then pressing the ... button. You will see three choices, `alignment_beginning`, `alignment_center`, and `alignment_end`, as in the figure below.



The highlighted line shows the current setting, `alignment_center`. A label's alignment indicates how the label aligns each line of text with respect to the entire label. Thus, the three types of alignment correspond to flush left, centered, and flush right, respectively. Choose the alignment you want for your label, then press the **OK** button.

4.4 Colors

Some resources require a color. For example, select the background resource, and edit it by pressing the ... button. On monochrome screens, you see only two toggle buttons, so you must choose between black and white. However, if you are on a color screen, you get a palette of possible colors, as shown below.

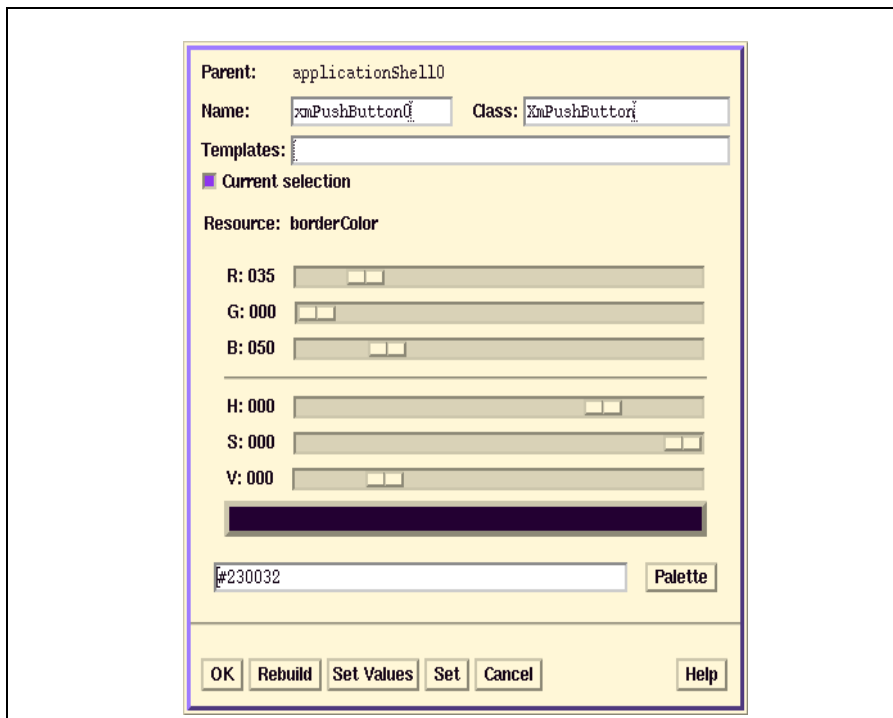


You may select a color by clicking the **Select** button on the square that contains the color you want. Use the scroll bar to see more colors. You may also enter the name of the color directly into the text field, or the exact specification of a color as *#rgb*, where *r*, *g*, and *b* are hexadecimal values for the red, green, and blue values of the color. Each hexadecimal value must have the exact same number of digits, with leading zeros where required.

If you press the **RGB** button, the dialog box changes to the RGB color viewer. The top half contains three scrollbars to set a color by its red, green, and blue color components. The bottom half of the screen contains another set of three scrollbars. You

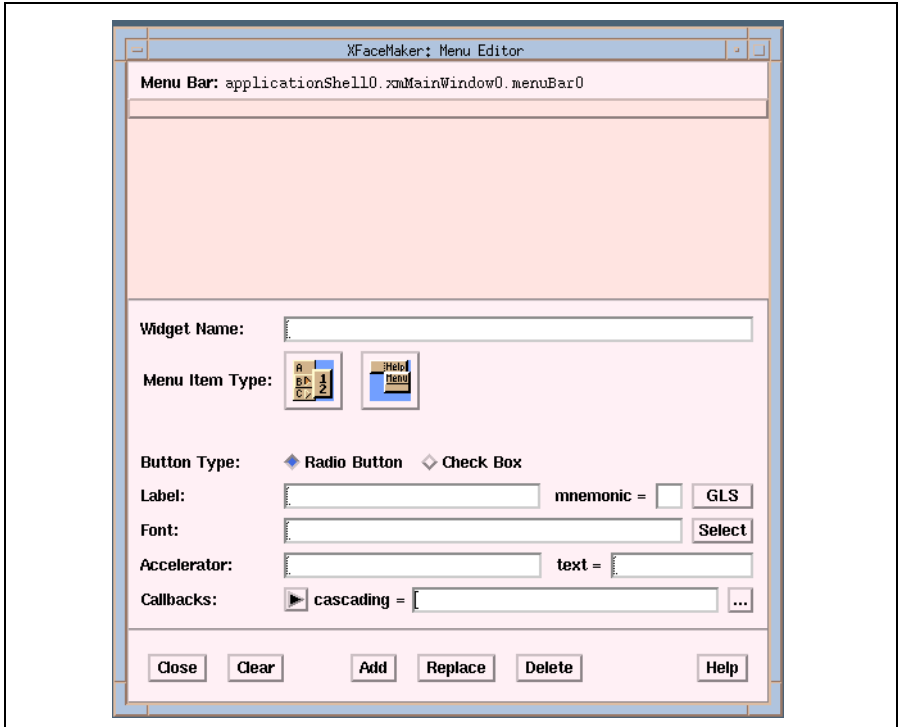
use these to set the color by its hue, saturation, and value components. You may use either set of scrollbars in any combination to choose your color, which XFaceMaker displays at the bottom of the dialog box, along with its text value. If you want to return to the color palette, press the **Palette** button.

Once you have chosen your color in either the color palette or **RGB** dialog box, press the **OK** button.



CHAPTER 5: The Menu Editor

Now we will show you how to use the menu editor to add some basic menu capabilities to our interface. Open the menu editor by selecting the **Menu Editor** item in the **Windows** menu. Then select the menu bar by moving the cursor over the name of the menu bar in the widget tree, and clicking with the **Select** button.



The name of the menu bar within the widget hierarchy appears at the top of the menu editor, `applicationShell0.xmMainWindow0.menuBar0`. A long thin rectangle representing the menu bar extends across the top of the screen.

The bottom portion of the menu editor contains a dialog box used to build menu elements. It starts with the widget name, followed by the possible menu item types. Normally, you will not have to enter a widget name, since the menu editor automatically assigns one based on the label value if you do not enter one yourself. Also, the portion of the dialog box below the menu item type will vary according to the type of menu item selected.

The examples below show the different fields that appear according to the item selected. We will explain how to display these fields in the next section. For example, for a label (used as a title), this area will contain only the Label and Font fields.

Widget Name:

Menu Item Type:

Label

OK

Label

Label:

a title

GLS

Font:

Select

Close

Clear

Add

Replace

Delete

Help

A pulldown menu will add the mnemonic and Callback fields.

Widget Name:

Menu Item Type:

Label

OK

Pulldown Menu

Label:

mnemonic =

GLS

Font:

Select

Callbacks:

cascading =

...

Close

Clear

Add

Replace

Delete

Help

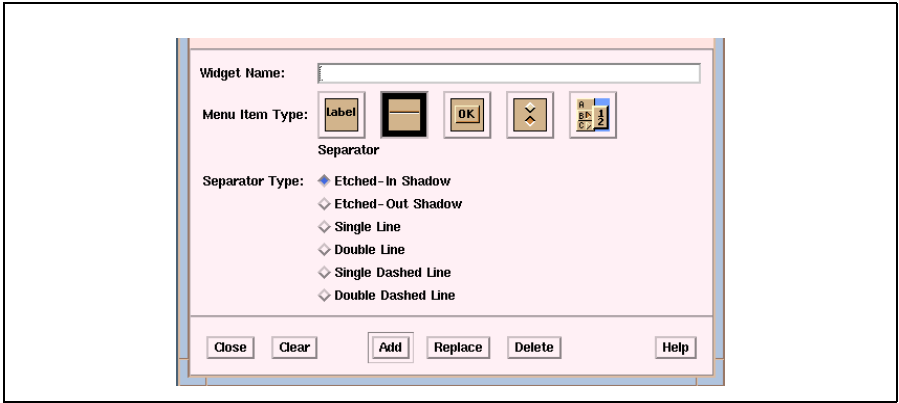
For a push button, you will also see the Accelerator and Accelerator text fields.

The screenshot shows a configuration window for a 'Push Button' widget. At the top, there is a 'Widget Name' text field. Below it, the 'Menu Item Type' section displays five icons: 'Label', a standard button, an 'OK' button, a dropdown arrow, and a radio button. The 'Push Button' title is centered. The 'Label' field is followed by a 'mnemonic' checkbox and a 'GLS' button. The 'Font' field has a 'Select' button. The 'Accelerator' field is followed by a 'text' field. The 'Callbacks' section features a play icon, the text 'activate =', a text field, and an ellipsis button. At the bottom, there are buttons for 'Close', 'Clear', 'Add', 'Replace', 'Delete', and 'Help'.

A toggle button will also include the toggle button type.

The screenshot shows a configuration window for a 'Toggle Button' widget. At the top, there is a 'Widget Name' text field. Below it, the 'Menu Item Type' section displays five icons: 'Label', a standard button, an 'OK' button, a dropdown arrow, and a radio button. The 'Toggle Button' title is centered. The 'Button Type' section has two options: 'Radio Button' (selected with a diamond) and 'Check Box' (unselected with a diamond). The 'Label' field is followed by a 'mnemonic' checkbox and a 'GLS' button. The 'Font' field has a 'Select' button. The 'Accelerator' field is followed by a 'text' field. The 'Callbacks' section features a play icon, the text 'value changed =', a text field, and an ellipsis button. At the bottom, there are buttons for 'Close', 'Clear', 'Add', 'Replace', 'Delete', and 'Help'.

A separator will contain none of these fields, but rather a choice of type of separator. This is the only case where you may want to provide a widget name, since there is no **Label** field from which to create a widget name automatically.



5.1 A Basic Menu

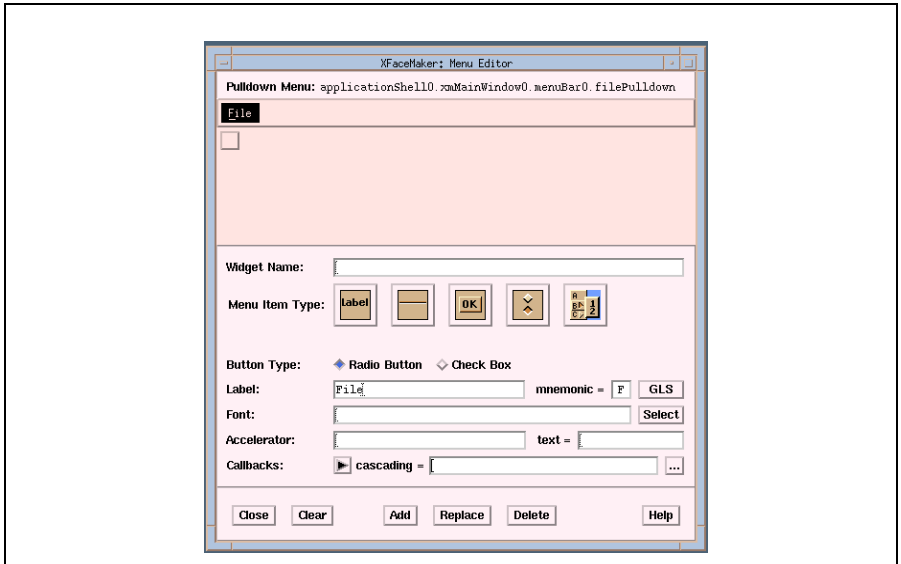
Let's return to our example and create a **File** menu for the menu bar:



1. Click on the **Pulldown menu** type icon. The dialog box changes to present only those attributes useful for pulldown menus.
2. Enter the word **File** in the **Label** text field. This is the label for the cascade button used to present the pulldown menu.
3. Type the letter **F** in the mnemonic text field.
4. Click on the **Add** button.

You may use the menu editor to select menu items to edit. Since most menu items are not normally visible in the interface you build, the menu editor shows a representation of the menus leading up to a particular menu item. Notice that once we added the File menu, it appeared in the menu bar representation at the top of the menu editor. You can then select the menu item within the menu bar by clicking on it.

Double-click on the **File** item to open up the pulldown menu associated with that item. In our case the menu is empty and appears as a small square. Note that there are now more choices in the Menu Item Type field, since a menu bar is restricted to only a pulldown menu or a help menu, while a pulldown menu is not.

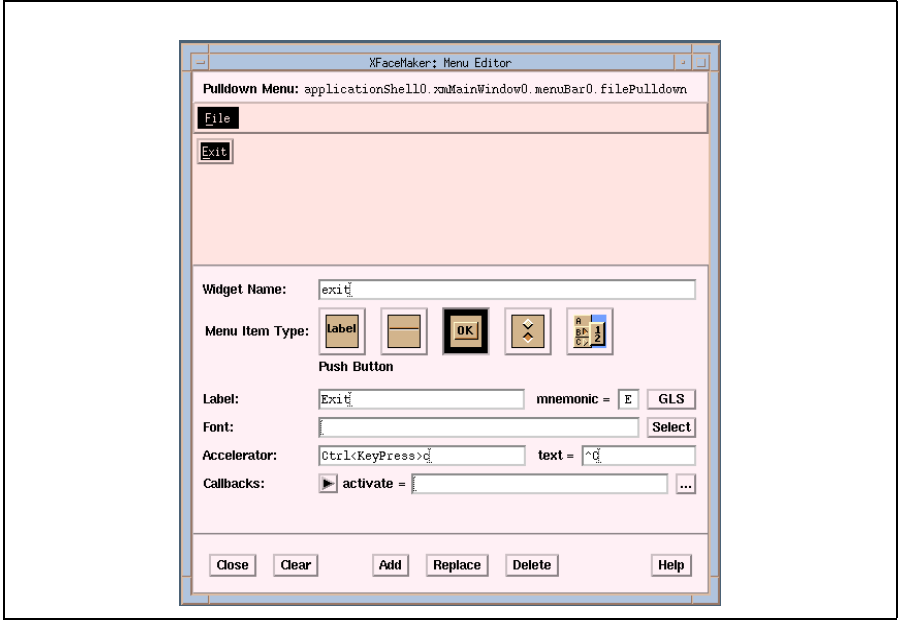


We can add an **Exit** button to the **File** menu as follows.

1. Click on the **Push Button menu** type icon, shown on the right.
2. Enter **Exit** as the label.
3. Enter **E** as the mnemonic.
4. Move the cursor over the **Accelerator** text field and enter a **Control-C**. The menu editor places the translation table event description of the keystroke you entered, and sets the default accelerator text. You may modify the accelerator text if you want, but in most cases it will not be necessary.
5. Click the **Add** button to add the item.



The **Exit** button appears in a smaller menu beneath the menu bar representation at the top of menu editor. The menu representation always shows the succession of menu selections requested to reach a particular menu selection. We can go back to the **File** item within the menu bar by clicking on it in the representation at the top of the menu editor, but don't do this just yet.

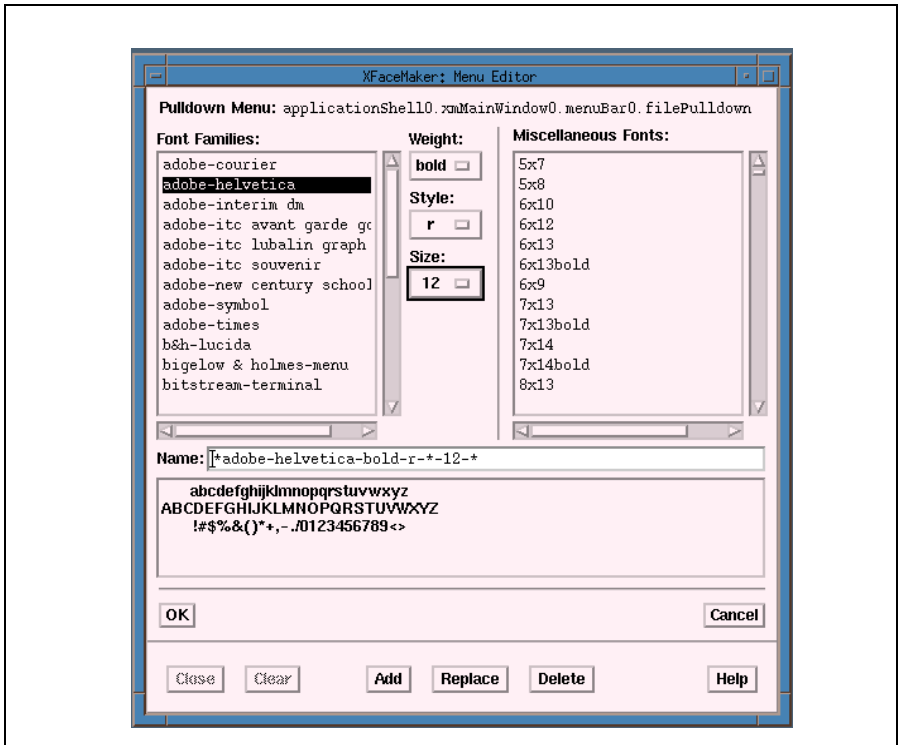


In addition to the **Add** button, there are **Replace** and **Delete** buttons. You use these with existing menu items. After selecting a menu item, you can edit it in the menu editor dialog box, then click the **Replace** button to replace it with the modified values. Or you can click on the **Delete** button to delete the selected item from the menu. After deleting an item, its values will remain in the menu editor so that if you had made a mistake, you may immediately click on the **Add** button to restore the menu item.

Up to now, we have not selected any fonts nor entered any callbacks. First, we will describe selecting fonts. You can select a font by entering its specification in the standard X11 style in the text field, or by pressing the **Select** button next to the **Font** text field to get a dialog box. We shall now select a font using the latter method.

5.2 The Font Selector

The font selector used by the menu editor is the same as the one used by the resource editor. Thus, you may apply everything you learn here to editing font resources in the resource window. The fonts available to you depend on your system configuration. Most X11 systems will have the fonts we select here. If your system does not have the font we describe, or you prefer another font, you may select a different font using the method described here. We open the font selector by clicking on the **Select** button next to the **Font** text field.



The font selector replaces the menu editor, except for the name of the menu which remains at the top of the screen, and the button row at the bottom. The **Clear** and **Close** buttons are insensitive because we should return to the menu editor before closing this window.

The top portion of the font selector contains two sets of font selections, separated by a vertical line. You use the set on the left of the line to select a font by family. You select the name of the font family from the list. Then you can select the font's weight, style, and size. These option menus present possibilities for the selected font family as they would appear in an X11 font specification. Typically, weight is one of **medium**, **bold**, or **demibold**. Style is usually a letter indicating one of the following styles:

r	roman
o	oblique (slanted)
i	italic
m	monochrome

The size is the size of the font in pixels. All option menus provide the option **any** to indicate that any value may be selected for that characteristic.

On our machine, we choose the font family **helvetica**. By clicking on the **Weight** menu, we see that there are **bold** and **medium** weights available. We select **bold**. The **Style** menu indicates that there are **r** (roman) and **o** (oblique) styles available. We choose roman. Finally, we select a pixel size of **12**. Each time we choose a font characteristic, the font selector updates the X11 font specification in the **Name** text field, and displays the font selected by that specification in the box below.

To the right of the line at the top is a list of miscellaneous fonts. These fonts are not available in weights, styles, and sizes, so selecting a font in the list specifies the font exactly. The font selector places the name in the **Name** text field and displays the font in the box below.

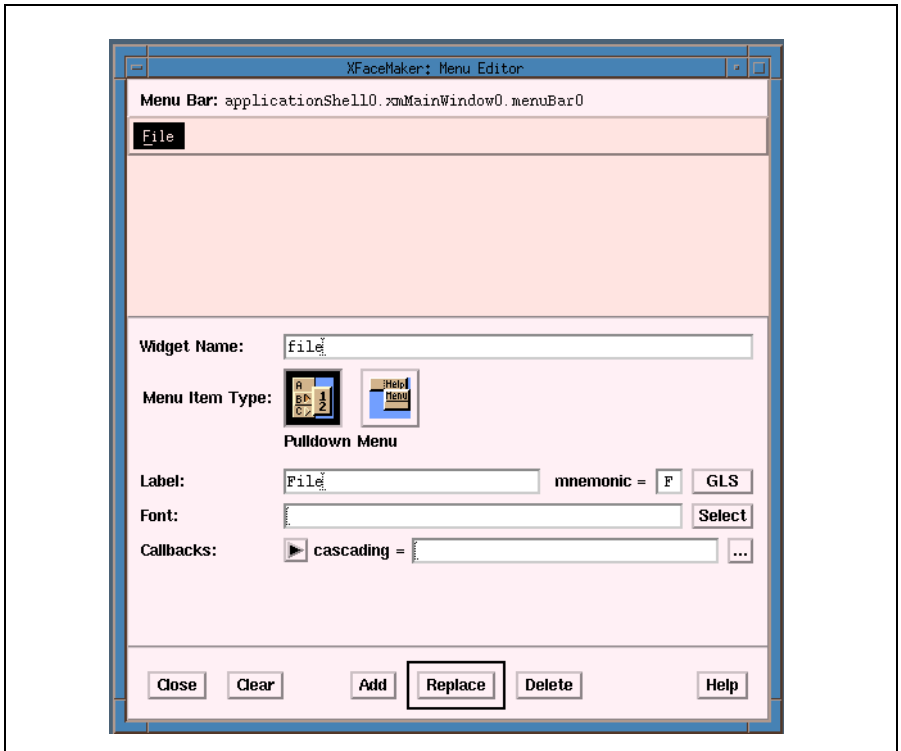
Once we are satisfied with the font we have selected, we can confirm our selection with the **OK** button, or we can ignore our selection by pressing the **Cancel** button. In either case, we return to the Menu Editor.

1. Click on **OK**. This confirms the font selection and returns us to the Menu Editor.
2. Click on the **Replace** button to replace the button we created earlier with a new one using this font.

5.3 More Menus

Let's add some more menus to our menu bar. To do this, we must return to the menu bar, instead of editing the **File** pulldown menu. We can do this either by selecting the menu bar, or by selecting one of the buttons within the menu bar. Note that in the widget tree, the menu bar has two children, one named `file` and one named `filePulldown`. The first child is the cascade button and the second child is the pulldown menu opened by that cascade button.

Select the `file` button in the widget tree by clicking on it. You should be editing the menu bar with the details for the **File** menu shown in the dialog box, as shown in the figure below.



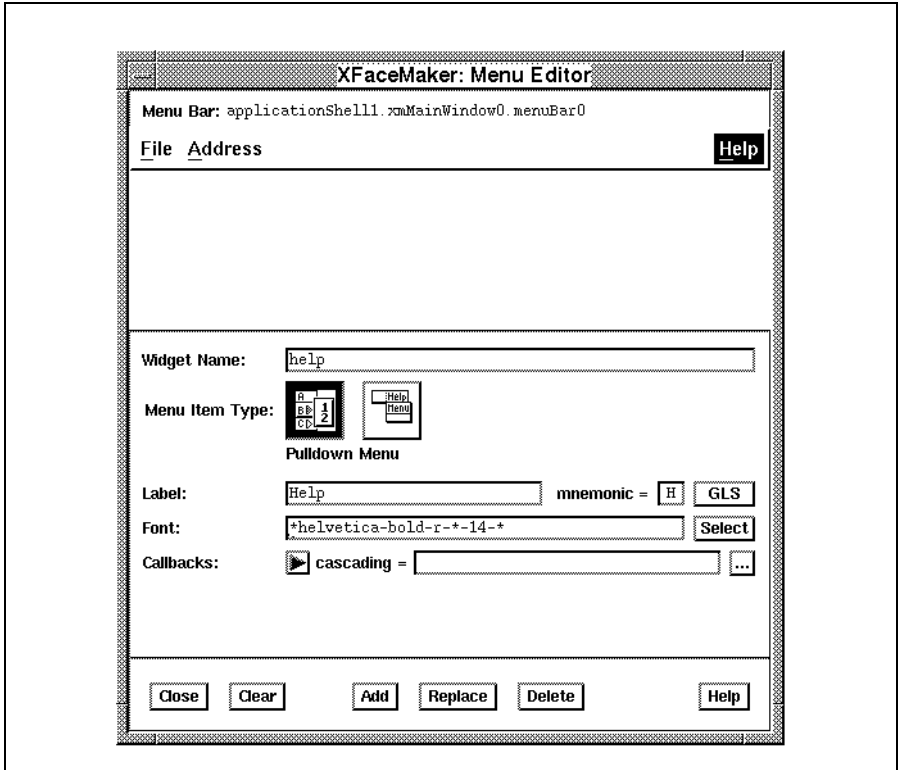
First of all, let's choose a font for the menu bar, since we didn't do that earlier. Click on the **Select** button next to the **Font** field to bring up the font dialog box, then select a font. In our example, we have selected the Helvetica font family, bold weight, roman (r), 14 pixels. Then, press the **Replace** button to replace the cascade button with the new font.

Now that we have done that, we are ready to show you how the menu editor lets you easily create consistent menus. Notice that all the information for the **File** menu button that we just created remains in the dialog portion of the menu editor, especially the selected font. This allows us to create a new button by simply changing the appropriate fields, then pressing **Add** for each button.

In particular, we need change only the **Label** and the **mnemonic** fields. If we change the **Label** field without changing the **Name**, XFaceMaker automatically creates a name based on the label, as it did for our previous menu items. It leaves the **Font** field alone, thus using the same font as in the previous item. Let's try this:

1. Change the **Label** field from **File** to **Address**.
2. Change the **mnemonic** field from **F** to **A**.
3. Press the **Add** button. You should now have an **Address** menu button next to the **File** menu button in your menu bar.
4. Change the **Label** field from **Address** to **Help**.
5. Change the **mnemonic** field from **A** to **H**.
6. Select the **Help Menu** icon instead of the **Pulldown Menu** icon, shown here on the right.
7. Press the **Add** button. You should now have a **Help** menu button to the right of the menu bar.





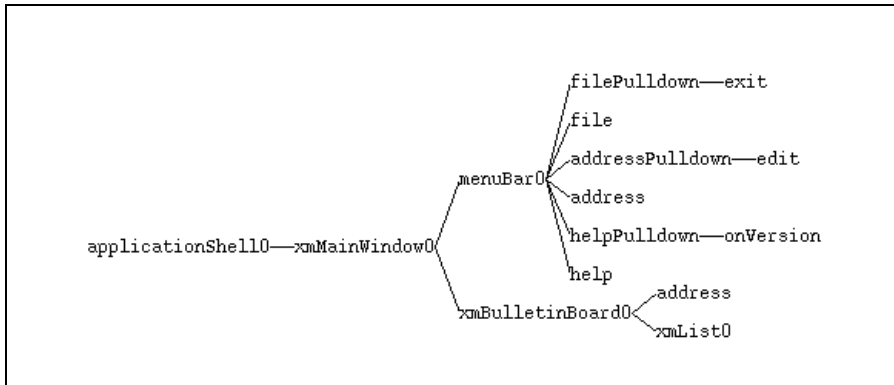
Note that because the menu editor left the **Font** field unchanged, the font selected for the **File** button was copied for the **Address** and **Help** buttons.

We can use the same trick to make menu items in our menus match the one we already created:

1. Select the button named `exit` in the widget tree. The dialog portion of the menu editor will contain the description of this button.
2. Select the widget named `helpPullDown` in the widget tree. This opens up the pulldown menu for the **Help** button without selecting any specific widget within the menu (and in fact, there are none to select). The previous values of the dialog portion remain in the window.
3. Change the **Label** field to **On Version...** and change the **mnemonic** field to **V**.

4. Click on the **Push Button** menu item type icon.
5. Delete the text in the **text =** field next to the **Accelerator** field. This deletes the accelerator.
6. Press the **Add** button. You have now added the **On Version** item to the **Help** menu. Even though the menu is a Help menu, it appears on the left side of the Menu Editor window, just like any other menu in the menu bar. The highlighted Help button indicates which menu you are editing. Its font is the same as the font of the **Exit** button in the **File** menu.
7. Select the widget named `addressPulldown` in the widget tree.
8. Click on the **Push Button** menu item type icon.
9. Change the **Label** field to **Edit . . .** and change the **mnemonic** field to **E**.
10. Press the **Add** button. You have now added the **Edit** button to the **Address** menu.

The widget tree now looks like this.



We will add behavior for these two new buttons in the upcoming chapters. We no longer need the Menu Editor. Click on its **Close** button to close it.

CHAPTER 6: Dialog Boxes

As an example of a dialog box, we will create the **Help On Version** window that we will display when the user chooses the **On Version** button in the **Help** menu. With this window, we illustrate the use of two important OSF/Motif components, the `dialogShell` and the bulletin board.

6.1 The `DialogShell`

After the `applicationShell`, the `dialogShell` is the second most important type of shell you will use in designing your interface. You use `dialogShells` for all windows that are subordinate to your main application shell but require a separate window. There are many reasons why you may want to place some of your interface in a separate window. For example:

- The window's dialog is temporary in nature, so a separate window allows you to make it appear and disappear without disrupting the rest of the interface.
- The window presents optional features that not every user of the application will want to use.
- The window contains a logically separate set of functions that the user would expect in a different window.
- The information in the window does not fit into the `applicationShell`, for either aesthetic or economic reasons. Applications with windows that are very large can become unwieldy and prevent the user from interacting with other applications on the same display.

The `dialogShell` provides a convenient vehicle for any or all of the above types of windows. The window manager considers a `dialogShell` to be subordinate to the `applicationShell`. So if you iconify an application's main window, the window manager also closes any open `dialogShells` for that main window. However, it will not create any icons for them. It re-opens the `dialogShells` when you re-open the application's main window.

NOTE: The Xt Intrinsics library includes a `transientShell` widget that provides the same kind of functionality as the `dialogShell`. In fact, the `dialog-Shell` is a subclass of the `transientShell`. However, the `dialogShell` belongs to the OSF/Motif toolkit, not the Xt Intrinsics library, and thus has specific capabilities for interacting with the OSF/Motif window manager and the various OSF/Motif widgets that you may want to use as child of the `dialog-Shell`. Therefore, we recommend that you always use OSF/Motif `dialog-Shells` for your application's secondary windows, not `transientShells`.

The only time when you would not want to use a `dialogShell`, is if the window is so logically separate and independent from the main window, that you would not want it to iconify at the same time as the main window. In that case, you would use a top level shell, which we describe later.

Let's create a window using a `dialogShell`:

1. Make sure the **Shell** toggle button is set in the toolkit menu in the toolkit pane of XFaceMaker's main window.
2. Click on the **XmDialogShell** icon, shown here on the right. The cursor  should change to a crosshair, like this: 
3. Move the crosshair to a spot on your root window where you want to place the top left corner of the `dialogShell`. It should *not* be over the main window of your interface.
4. Press the **Select** mouse button and while holding it down drag the cursor to the lower right corner of the window, then release the button.
5. Click on the **Composite** toggle button in the toolkit menu.
6. Click on the **XmBulletinBoard** icon, which looks like this: 
7. Move the crosshair cursor inside the `dialogShell` window you just created, and click the **Select** button.

The bulletin board will manage the elements of the version window. The **XmBulletinBoard** widget interacts with the **XmDialogShell** widget and the window manager, so that when you create the bulletin board widget within the `dialogShell`, the window manager centers the new window over the `applicationShell`. We will talk more about this later.

8. Click on the **Primitive** toggle button in the toolkit menu.
9. Click on the **XmLabel** icon, which looks like this: 
10. Move the crosshair cursor inside the `dialogShell` window, just inside the top left corner.
11. Press the **Select** mouse button and while holding it down, drag the cursor to make a label centered in the top two thirds of the window. This label will contain the version information about our application.
12. Click on the **XmPushButton** icon, which looks like this: 

13. Move the crosshair cursor inside the dialogShell window, at the bottom portion of the window, outside the area where you created the label in step 11.
14. Click the **Select** mouse button to create the push button. This button will close the window. Note that the OSF/Motif window manager also provides a means for closing the window, as do most window managers. However some window managers do not provide this feature, so it is usually a good idea to place a specific, visible means of closing the window in every application window.

6.2 DialogShell Resources

There are a few basic resources that you should be aware of when using dialogShells. Surprisingly enough, not all of them are in the **XmDialogShell** widget! Some of them are in the **XmBulletinBoard** widget. This is due to the interaction between the two widgets. First, we'll look at the resources of the dialogShell itself.

1. Select the dialogShell, **XmDialogShell0**, in the widget tree.
2. If there is not one open already, open a resource window and make sure it has the **Current selection** toggle set.
3. Select the resource named **title**. This is the title presented to the window manager. The OSF/Motif window manager, like most window managers, displays this title in a bar above the window itself. It defaults to the name of the widget.
4. Change the title to **Address Book Version**, or whatever you want the title of this window to be, then click **Rebuild**.
5. Select the resource named **deleteResponse**.
6. Press the ... button. This resource's type is an enumeration. This enumeration indicates what response the dialogShell makes when it receives a **WM_DELETE_WINDOW** message from the window manager. For a dialogShell, its default value is **unmap**, which means the window is closed, but may be re-opened later. The other two options are **destroy**, which destroys the shell and all its children, and **do_nothing**, which does nothing.
7. Press the **Cancel** button to leave the resource set to its default value.

6.3 The Bulletin Board

Now, we'll look at a few important resources of the bulletin board widget. Widgets that are subclasses of **XmBulletinBoard** inherit these resources as well, so they communicate with **XmDialogShell** in the same manner as with **XmBulletinBoard**. These widgets include the **XmForm**, **XmCommand**, **XmMessageBox**, **XmSelectionBox**, and **XmFileSelectionBox** widgets.

1. Select the bulletin board widget that is the child of the dialogShell.
2. Open a resource window if there is not one already open, and make sure it has the **Current selection** toggle set.
3. Select the **defaultPosition** resource in the resource list, and press the ... button.

As you can see, this is a Boolean resource. If **True**, the dialogShell's window centers itself over the applicationShell. If **False**, it uses the positions given to it as resource values. By default, this resource is **True**.

4. Press the **Cancel** button to return to the resource list.
5. Select the **autoUnmanage** resource in the resource list, and press the ... button.

This is a Boolean resource. If **True**, the window containing the dialog-Shell will unmap when any button in the bulletin board is pressed. This is its default value. We will take advantage of this in this window, since the only action we want when the **Close** button is pressed is to close the window. We do not need to describe any other behavior for the button.

6. Press the **Cancel** button to return to the resource list.
7. Select the **dialogStyle** resource in the resource list, and press the ... button. This enumeration describes the modality of the window. A window's modality describes its interaction with the rest of the interface. A modal dialog box blocks input with the rest of the interface until the dialog box is closed. A modeless dialog box lets you interact with the rest of the interface while the dialog box is open.

The default value, **dialog_modeless**, indicates a modeless dialog box. The bulletin board forces the value to be **dialog_work_area** if the bulletin board is not the child of a dialogShell. The other values indicate different types of modal dialog boxes.

8. Press the **Cancel** button to return to the resource list.

Note: there is also a **dialogTitle** resource. This resource serves the same purpose as the title resource in the dialogShell. However, the **title** resource has the type **String**,

while the **dialogTitle** resource has the type **XmString**. You may use either to specify the title of the window, although using the title resource of the shell widget is the more traditional way, and it sets it in the more obvious place. However, providing a value for the **dialogTitle** resource in the bulletin board will override any value specified in the title resource of the shell.

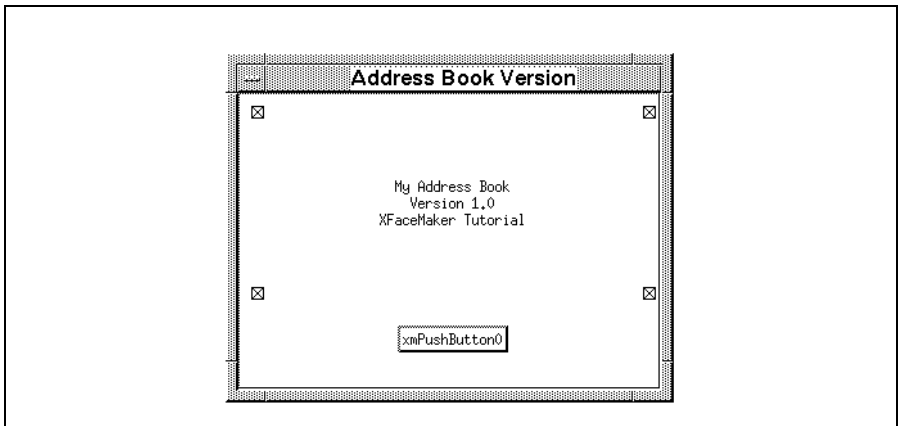
6.4 Completing the Dialog Box

Let's finish our dialog box by setting some other necessary resources.

1. Select the label widget in the dialog box by clicking on its name in the widget tree.
2. Select the **labelString** resource in the resource list, and press the ... button.
3. Enter the version description of your application. For our example:

My Address Book
Version 1.0
XFaceMaker Tutorial

4. Press the **OK** button to apply the change and return to the resource list. Notice that the label widget automatically centers each line.



5. If you want, choose another font for the label by selecting the **fontList** resource, and pressing the ... button to use the font selector.
6. Select the push button by clicking on it in the interface.
7. Select the **labelString** resource in the resource list.
8. Without pressing the ... button, change the text field so that it contains the word **Close**.
9. Press the **Rebuild** button. The button remains the same size as before, which was for a longer name. In this case, we will want the button to decide its own size based on the label string.
10. Set the **Auto size** toggle button. The highlighting around the **w** and **h** text fields should disappear, and the icons for the width and height resources should change from  to . Scroll the list to verify this.
11. If you want, choose a font for the button.
12. You may also want to change the margins for the button. Typically, you might change the **marginHeight** resource to add more space above and below the button label, and you might change the **marginWidth** resource to add more space to the left and right of the label.
13. Drag the push button so it is once again centered in the window beneath the version information.



Now we have finished the dialog box. Normally, this box is not visible, so we should hide it initially. To do this, we click on the **Hide** icon in the interface pane to the left of the widget tree.

The cursor will change to a pointed cross (+), and will drag with it a copy of the **Hide** icon. Move the cursor over the bulletin board in the widget tree and click the **Select** mouse button. The window is no longer visible, but it still exists in the widget tree. Thus the window is unmapped, or hidden.

6.5 Some Fine Points

We now present a few more interesting points about how dialogShells work and how XFaceMaker deals with them. Remember, when we were creating the dialogShell, that we instructed you not to place the top left corner of the shell in the main window? There was a reason for that. Even though all shell widgets are placed directly on the desktop, they can be children of any widget in your interface. If you had placed the top left corner of the shell widget inside a widget in your interface, or even in the widget tree, XFaceMaker would make the shell the child of that widget. Otherwise, XFaceMaker makes the shell widget the child of the topmost widget in the application, the application shell.

For the most part, it does not matter very much who the parent of dialog-Shell is. However, it does have two important effects. First, it determines when the dialogShell is iconified. If a top level shell is an ancestor of the dialogShell before the applicationShell, then the dialogShell is iconified with the top level shell, not the application window. Remember when we mentioned that top level shells are not iconified with the application shell? They are iconified separately and have their own icons. Like the applicationShell, they unmap any dialogShell among their descendants with them.

The other way that a dialogShell's parent matters has to do with the default positioning of the shell. The dialogShell's default position is centered over its parent. In our example, the parent is the applicationShell, so the dialogShell centers itself over the main window.

Finally, you can change the widget for which a dialogShell acts (instead of its parent) by setting its **transientFor** resource. However, the widget must be realized. Like many things in the X Window System, if you set the **transientFor** resource to a widget that has not been realized, your application will probably generate a fatal error and dump core. If you are not familiar with the subtleties of widget realization, you can use the rule of thumb that the widget must have already been visible at least once for it to be realized. As with all Xt Intrinsics and OSF/Motif matters, you should consult your documentation for the complete story.

CHAPTER 7: Forms

You will design one more dialog box before delving into the behavioral aspects of interface building. This dialog box is the one displayed when the user selects the **Edit** item in the **Address** menu. It allows the user to edit the list of addresses.

This address dialog box will use a commonplace and fundamental widget of the OSF/Motif toolkit, the **XmForm** widget. The form widget is a container similar to the bulletin board widget. In fact it is a subclass of the **XmBulletinBoard** widget. However, while the bulletin board takes almost no responsibility for where widgets are placed within it, the form widget allows you to specify the relationship between widgets in the form, and between the widgets and the form itself. You may specify constraints¹ for the widgets in the form, which limit or specify their position and size.

The most commonly used constraints added by a form to its children are attachments. They describe how to attach the top, bottom, left, and right sides of the widget. You may attach the side of the widget to another widget, the form, or a position in the form. Each attachment has up to four parameters:

- The type of attachment
- The widget to attach to (if any)
- The position within the form to attach to (if applicable)
- An offset from the attachment point

Rather than going into the details of attachments at this point, we will show you how to make the most commonly used kind of attachments in XFaceMaker. After that, we will explain the different attachment types in detail.

1. *Constraint* is also the word for a special class of resources in Xt Intrinsics parlance. **Constraint** resources are resources added to a widget by its parent, rather than being an intrinsic (pun intended) part of the widget. Usually, these resources constrain the widget in some way, hence the name, but this is not necessarily the case.

7.1 The Form As Dialog Box

First, create the window in which you will use the **XmForm** widget and attachments. We create the dialog box using the **XmDialogShell** widget, with the form as its child. Since the **XmForm** widget is a subclass of the **XmBulletinBoard** widget, it interacts with the **XmDialogShell** in the same way.

1. Select the **Shell** button in the **Toolkit** menu in the main window.
2. Click on the **XmDialogShell** icon, which looks like this:
3. Choose the top left corner of the dialogShell somewhere on your desktop, but not over your interface and not over the widget tree in the main window.
4. Drag a rectangle by moving the cursor to the top left corner of the dialogShell, pressing the **Select** mouse button, and while holding it down, dragging the mouse to the lower right corner of the dialogShell, then releasing the button.



5. Select the **Composite** button in the **Toolkit** menu.
6. Click on the **XmForm** icon, which looks like the one here:
7. Move the cursor inside the dialogShell you just created, and click the **Select** mouse button.
8. Open a resource window if one is not already open, make sure its **Current selection** toggle is set, and select the dialogShell in the widget tree.
9. Select the **title** resource.
10. Set the text field to **Address Editor**.
11. Press the **Rebuild** button in the resource window.
12. Select the form widget.
13. Select the **autoUnmanage** resource.
14. Change its value to **false**. You do not want the form to unmanage when the user presses the **Add** or **Help** buttons (which will be added later), only when he presses the **Close** button, in which case you will unmanage the window yourself.



7.1.1 Form Layout

We will now consider some basic layout issues for the form. The form lets you choose the default spacing of objects, both horizontally and vertically, as well as the size of the margin around the edges. In this dialog box, you will use a ten pixel margin around the edges of the form, five pixels of space between objects horizontally, and 2 pixels of space vertically. Here is how to set the appropriate resources:

1. Select the **marginHeight** resource.

2. Enter **10** in the text field.
3. If needed, adjust the scrollbar so that both the **marginHeight** and **marginWidth** resources are visible.
4. Move the cursor over value 10 for the **marginHeight** resource.
5. Press the **Drag** mouse button and hold it down. The cursor should change to the resource value cursor.
6. With the **Drag** button still down, drag the cursor to the value of the **marginWidth** resource and release the button. The value of the **marginWidth** resource changes to **10**.

This shows one way to drag resource values while editing. While it would not have been too difficult to select the **marginWidth** resource and enter **10** as its value, this ability is useful for more complicated resource values that are more difficult to enter.

7. Select the **horizontalSpacing** resource.
8. Enter **5** in the text field.
9. Select the **verticalSpacing** resource.
10. Enter **2** in the text field.
11. Press the **Rebuild** button.

You have now specified our default layout parameters for the form. Although you have specified spacing and margins for the form, most widgets also have their own internal margins, usually around two pixels. So the actual space along the margins and between pixels will in general be larger than the numbers you specified here.

7.1.2 Default Form Fonts

There is one more thing we would like you to specify for the form widget: the default fonts to use for the elements in the form.

1. Select the **buttonFontList** resource.
2. Press the ... button next to the text field. The font selector appears in the resource window.
3. Choose the default font to use for buttons in this dialog box. In this tutorial, we use the font family **Helvetica**, the weight **bold**, the style **r** (for Roman), and a pixel size of **12**.
4. Press the **Set** button to set this resource without applying it yet.

5. Scroll the resource list so that the **buttonFontList** and **labelFontList** resources are both visible. If you need to, you can make the resource window bigger.
6. Move the cursor over the value of the **buttonFontList** resource, press the **Drag** mouse button, and while holding it down, drag the cursor over the value of the **labelFontList**.

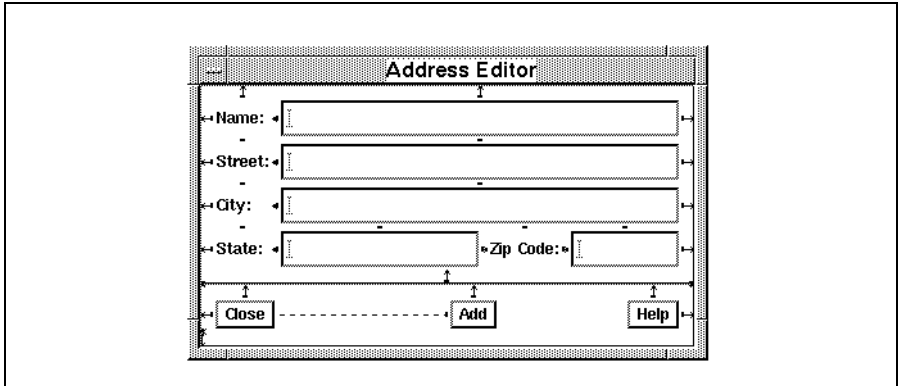
This copies the **buttonFontList** value to the **labelFontList**. If you prefer to use a different font, you may instead select it with the font selector by pressing the ... button.

7. Select the **textFontList**, and press the ... button to open the font selector.
8. Select the font for text fields. These examples use the **Courier** family, **bold** weight, Roman (**r**) style, and pixel size **12**.
9. Press the **OK** button. XFaceMaker applies all of the changes up to this point to the form.

These resources specify the default fonts for children of the form. By specifying them here, you will not have to specify the font for each individual widget in this form.

7.2 Using XFaceMaker to Attach

We will now show you how to build your dialog box, attaching each object in the dialog as you go along. Your finished dialog box will look something like the figure below.



We will explain in detail at first, and become more terse as you gain experience in building with forms.

1. Select the **Primitive** button in the **Toolkit** menu in the main window.
2. Click the **Select** mouse button on the **XmTextField** icon: 
3. Move the crosshair cursor inside the form you created earlier, and press the **Select** mouse button again.
4. Select the text field you just created.
5. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
6. Change the **Name** field in the resource window to **name**.

This text field will contain the name of the person in this address book entry. Generally, it is a good idea to give widgets a name based on their function in the interface. That way, if you need to refer to them later, especially in FACE code, it will be more obvious what is going on.

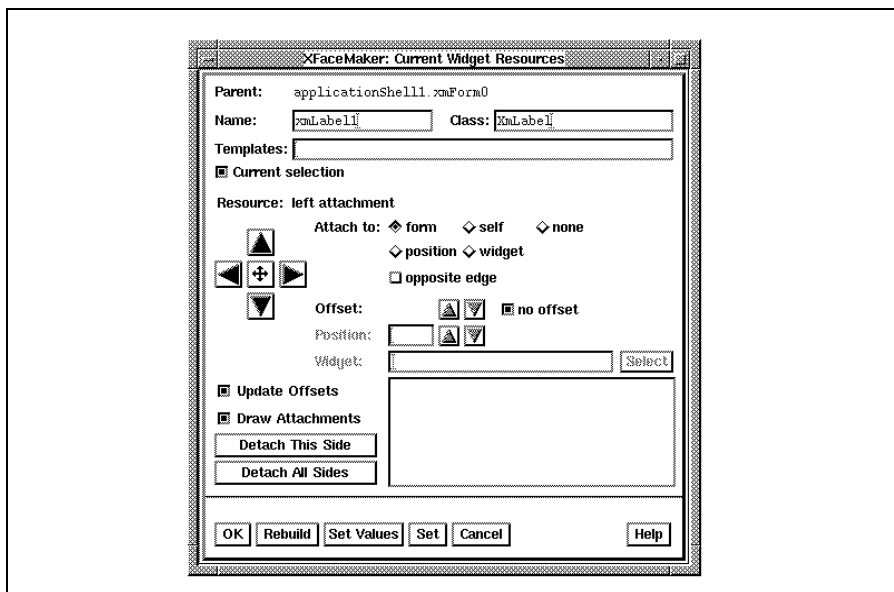
7. Press the **Rebuild** button in the resource window.
8. Click the **Select** mouse button on the **XmLabel** icon: 
9. Move the crosshair cursor inside the form and press the **Select** mouse button again, creating a label.

10. Select the label you just created.
11. Change the name of the widget to **nameLabel1**. In this dialog box, we will stick to this convention where we append **Label1** to the name of a field to name the field's label.
12. Select the **labelString** resource and change the text field to **Name**:
13. Set the **Auto position** and **Auto size** toggle buttons.
14. Press the **Rebuild** button in the resource window.

Note that the label moves to the top left corner of the form. This is because you set the **Auto position** toggle. You have not specified any attachments yet, so the position defaults to the top left corner. Also, because you set the **Auto size** toggle, the size of the label is exactly large enough to contain the text.

Now you are ready to attach these widgets. First you will attach the **nameLabel1** widget to the form, even though it already defaults to this position.

15. The widget **nameLabel1** should still be selected. Select the **Attachments** item in the **Edit** menu of the XFaceMaker main window. The resource window should now show the attachment dialog box.

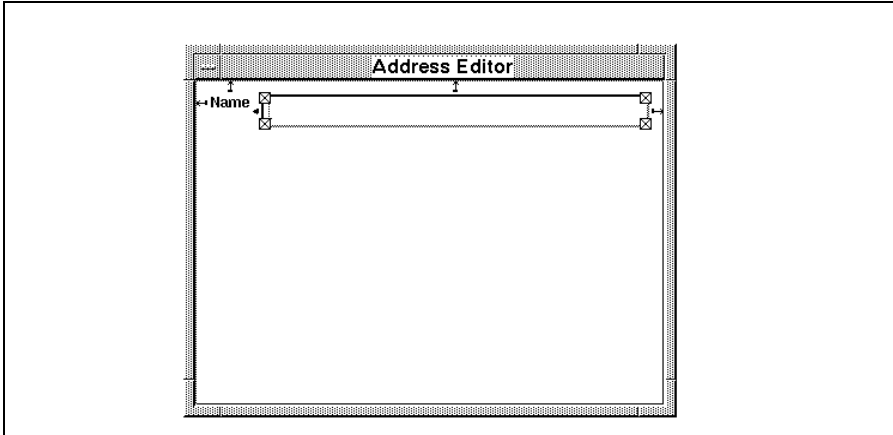


16. Click on the left arrow button in the group of four arrow buttons that look like this: 
17. The cursor should change to a question mark cursor . Move the cursor over form, but not the text or label widget, then click the **Select** mouse button.
18. Click on the up arrow button in attachment dialog box.
19. Move the  cursor again to the form, but not on the text or label widget and click the **Select** button.

You will see two small arrows from the **Name** label to the edge of the form. These indicate that the form is attached. If you do not want to see these arrows, you can turn them off by clearing the **Draw Attachments** toggle in either the attachment dialog box or the **View** menu in the main window.

Now that we have shown how to attach to a form, we will show how to attach to a widget:

20. Select the text field widget. The attachment box should remain open in the resource window.
21. Click the left arrow in the attachment box.
22. Move the  cursor over the label widget, either the actual widget in the interface or its name in the widget tree, then click the **Select** mouse button.
23. Click the up arrow in the attachment box.
24. Move the  cursor over the form (but not one of its children) and click the **Select** button.
25. Click the right arrow in the attachment box.
26. Just for variety, move the  cursor over the name of the form widget in the widget tree and click the **Select** button.
27. Press **Cancel** to return to the resource list.
28. Set the **Auto position** and **Auto size** toggle buttons. The form now controls the size and position of the text field.
29. Press the **Rebuild** button in the resource window.



When you attached the left side of the widget to the name label, you might have noticed that XFaceMaker changed the order of widgets in the widget tree. This is because a widget must already exist in order to be able to attach to it, and XFaceMaker creates widgets in the order they appear in the widget tree. When you attached the name widget to the name label, XFaceMaker moved the label before the text field in order to be able attach it.

If you scroll up and down the resource list, you will find the attachment resources. The attachment resources all begin with the side of the attachment, one of **bottom**, **left**, **right**, or **top**, followed by the parameter for the attachment, either **Attachment** (the type), **Offset**, **Position**, or **Widget**. For example, since the top of the widget is attached to the form, the **topAttachment** resource is set to **attach_form**.

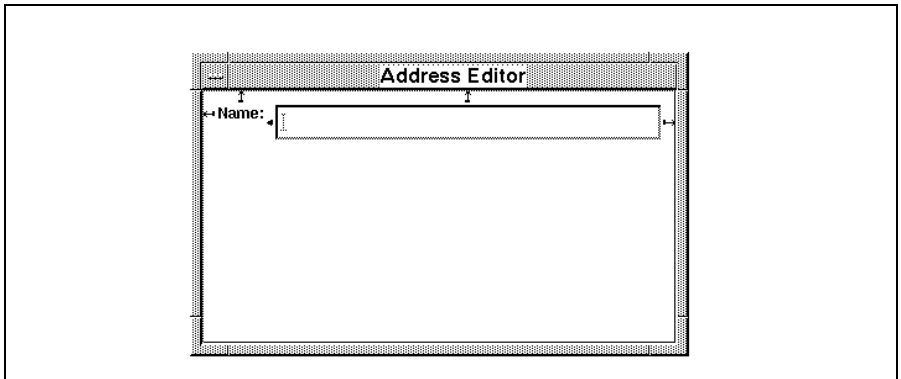
The left of the widget is attached to another widget, and thus the **leftAttachment** is **attach_widget**. Furthermore, the **leftWidget** is `parent.nameLabel`. This is your first introduction to the FACE programming language. In FACE, you can refer to other widgets in your interface by their position in the widget hierarchy. The widget `parent` refers to the parent of the current widget (where the reference is made), and `parent.nameLabel` means the child named `nameLabel` of the parent widget. We will explain more about FACE as you build your interface.

Next, look at the offset resources for the attachments. The **topOffset** is **10**, which corresponds to the form's **marginHeight** resource, and the **rightOffset** is also **10**, which corresponds to the form's **marginWidth** resource. The **leftOffset** is **5**, which corresponds to the form's **horizontalSpacing** resource.

Notice how the text field spans the width of the dialog box. This is because you attached the text field on both the left and right sides. Now, if you change the size of the dialog box, the size of the text field follows suit. Try changing the size of the dialog box to see the text field change size.

CHAPTER 8: Callbacks and the FACE Programming Language

In the address editor dialog box you are building, you may have noticed that the name label and text field do not line up very well, as in the figure below. This is because the two widgets do not have the same height, even though the fonts have the same size of twelve pixels.



The text widget has highlighting, shadowing, and wider margins than the label. You can compensate for this by changing the offset of the label to adjust for the difference in height. However, you do not always know the font being used by the widgets, since ultimately, you will want the user to be able to specify the font using the resource data base. The safer and easier way to make the two widgets more or less line up is to make their height the same. While not always the best method (since base-lines may not be aligned), this method generally works for fonts of the same or nearly the same size.

How do you decide what height to make the widgets? Remember that we set the **Auto size** toggles for both widgets, so they took whatever height they needed for the font being used. If we choose an arbitrary size, it may be too small for the user's chosen font, or it may make the upper and lower margins embarrassingly large. A good solution would be to find the two widgets' "natural" heights, and choose the larger of the two. However, you can only do this after the two widgets have been created. This brings us to a further introduction to FACE code, and our first **Callback** resource.

8.1 A Brief Introduction to FACE

FACE is a “fourth generation” programming language, that is, a language designed to perform a particular task, as opposed to a general purpose language such as C or C++. The task of the FACE programming language is to program user interface behavior, specifically for interfaces using the OSF/Motif toolkit.

Like all toolkits based on the Xt Intrinsics, the OSF/Motif toolkit depends heavily on the C programming language and libraries written in it. In this vein, much of FACE’s syntax comes from the C language syntax. In general, if you stick to simple C statement syntax when programming in FACE, you will be fine. Some things to watch out for:

- The following operators are not available:
`% ~ ^ | & ++ -- ? : ,`
- Operators that combine with assignment (e.g., `+=`, `-=`) are also not available.
- Assignments are not expressions. You cannot say `a = b = c;` in FACE.
- The logical operators `&&` and `||` do *not* short-circuit. Both operands are always evaluated.
- The `do` and `switch` statements do not exist in FACE.
- Function, structure, and array syntax is different.
- There are no unions in FACE.
- You cannot specify pointer types in FACE.

The *XFaceMaker Reference Manual* specifies FACE syntax in detail. We will point out its basic uses and various idiosyncracies as we go along. The FACE language has four keywords associated with widgets:

`self` is the current widget when the FACE script is executed.

`parent` is the parent of the current widget.

`root` is the top of the widget tree.

`oplevel` is the immediate descendant of `root` that contains the current widget, but only if it is a shell widget. Otherwise, it is the same as `root`.

The `.` operator refers to the widget hierarchy, *not* to a structure field as in C. Normally, the name after the `.` indicates the name of a child of the widget before it. Thus `a.b` would be the child named `b` of `a`, if `a` referred to a widget. The only time the `.`

operator does not refer to a child is when you specify more than one `parent` in order to ascend more than one level in the tree. Here are examples of widget references, where `w` is a widget.

`self.name` A child of the current widget named `name`
`parent.parent.foo` A child named `foo` of the current widget's grandparent.
Or you could say a sibling of the current widget's parent named `foo`.
`w.foo.bar` Child named `bar` of a child named `foo` of the widget `w`.
`w.parent.baz` A sibling of `w` named `baz`.
`parent.booo` A sibling of the current widget named `booo`.

You can only specify one or more `parent`'s at the beginning of a widget specification, or immediately after a widget variable. You cannot specify it in the middle of a specification already in progress. For example, `self.parent.foo` and `foo.bar.parent.baz` are not allowed.

You may specify an asterisk (*) between names to omit one or more widgets, in much the same way you would in a resource data base specification. However, you cannot specify * before `parent`. If a widget has more than one child with the same name, or a specification using * matches more than one possible widget, XFaceMaker does not guarantee which of the possible widgets will match the specification, so you should only use specifications which match exactly one widget. Normally, if every child of a given parent has a different name, there is no problem. You could even give every widget in your interface a different name, to avoid ambiguity even when using the * operator.

Unlike the resource data base syntax, FACE uses a different operator to specify the resource of a widget. The operator `:` introduces a resource name, and you follow it with the name of the resource as a string. This means you specify the name as you would for a resource data base, not the way you would in a C program. Typically this involves removing the prefix (such as **XtN** or **XmN**) from the name you would use in C. For example, `self:labelString` refers to the **labelString** resource of the current widget.

One more thing before we proceed. You call functions in the same way as in C. In fact, you may call most of the familiar C functions from FACE, including all those provided by the OSF/Motif toolkit, and most of the useful ones in the Xt Intrinsics library.

8.2 The xfmCreateCallback Resource

The **xfmCreateCallback** resource is not a real resource, but rather a convenience XFaceMaker provides for letting you execute FACE code after a widget is created, but before it is managed (i.e., before it is made visible). This allows you to make minor adjustments to resources that must be made at run time, or to initialize code needed to interact with application functions or other parts of the interface.

We will show you how to use the **xfmCreateCallback** resource to make the name label and text field the same height in your interface. You can do this in the **xfmCreateCallback** of the text field widget. Since you know that the text field will be created after the label, both widgets will exist when XFaceMaker calls this callback. Remember, you can look at the widget tree to determine the order of creation. Note that this order can be changed if new attachments are defined, as we will explain in a later section.

1. Select the name text field widget.
2. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
3. Select the **xfmCreateCallback** resource.
4. Press the ... button.

Now you should enter the FACE code to assign the height of the two widgets to the height of the tallest widget. Here is one possible solution:

```
if (self:height < parent.nameLabel:height)
    self:height = parent.nameLabel:height;
else
    parent.nameLabel:height = self:height;
```

However, XFaceMaker provides a convenience function for this kind of operation since it is very common, and writing the explicit code gets more complicated if more than two widgets are involved. The function is called **FmSetTallest**, and it takes a list of widgets as arguments, terminated by **NULL**. You should use this function in the **xfmCreateCallback**.

5. Enter this statement in the text widget for the **xfmCreateCallback**:

```
FmSetTallest (parent.nameLabel, self, NULL);
```

6. Press the **OK** button.

You should return to the Resource list, and the name label and text field should be better aligned now.

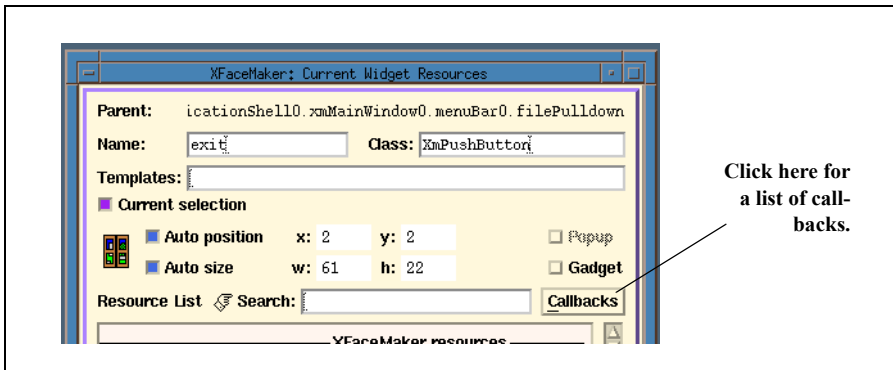
One remark about FACE function calls. FACE will only let you specify up to twelve arguments. Thus, you do not specify the **NULL** to **FmSetTallest** if you specify exactly twelve widgets. If you need to set a group of widgets larger than twelve, you need to make more than one call to **FmSetTallest**. (Proof that thirteen to twenty-three widgets require three function calls is left as an exercise for the user).

8.3 Pushing Buttons

While we are on the topic of FACE programming, we will take a small detour back to the version box that we created earlier. Push button widgets have an **activateCallback** resource, a real resource specified in the description of the widget. The widget calls this callback whenever it should take the action indicated by the button, normally this means when the user presses it¹.

With your newly acquired knowledge of the FACE programming language, you can add callbacks to the menu items you created in your main window.

1. In the tree, select the **exit** button in the **filePulldown** widget.
2. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
3. Press the **Callbacks** button, revealing the menu of callbacks. The Callbacks button is above the resource list, to the right.



4. Select the **activate** button.
5. Enter the following FACE code into the text box: **exit (0);**
6. Press the **Rebuild** button.
7. In the widget tree, select the **edit** button in the **addressPulldown** widget. Notice that the top of the resource window changes, but the **activateCallback** editing

-
1. We use this legalese in order to differentiate this callback from the **armCallback** that the widget calls when the user presses the button but has not yet released it. This is when the widget changes the button's color. Also, the widget calls its **activateCallback** when the user activates the button from the keyboard, such as through a mnemonic, accelerator, or default action.

box remains in place.

8. Press the **Select** pushbutton at the bottom of the resource window. If necessary, scroll the widget tree so that the Form corresponding to the address editor is visible.
9. Move the  cursor to the widget tree, on top of the form corresponding to the address editor and press the **Select** button on the mouse. The relative name of the widget should appear in the text box:

```
parent.parent.parent.parent.xmDialogShell1.xmForm0
```

This shows how you can use the **Select** button to insert the name of another widget in your interface without having to figure out its exact relation to the current widget.

10. Modify the line so that the widget specification just inserted is the argument to the function **FmShowWidget**. Now the line should look like this:

```
FmShowWidget
    (parent.parent.parent.parent.xmDialogShell1.xmForm0);
```

11. Press the **Rebuild** button.
12. In the widget tree, select the **onVersion** button in the **helpPulldown** widget.
13. Insert the name of the bulletin board containing the version label, using the same method you just used for the address dialog box.
14. Modify this widget specification so that it, too, is an argument to the function **FmShowWidget**. It should look like this:

```
FmShowWidget(parent.parent.parent.parent.
    xmDialogShell0.xmBulletinBoard1;
```

15. Press the **OK** button.

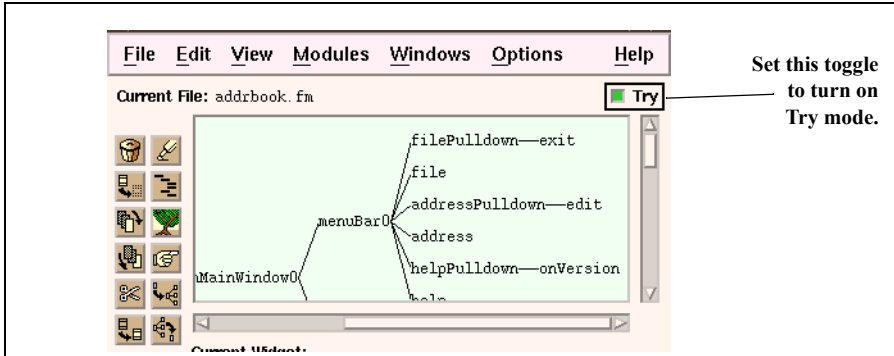
16. Hide the address dialog box by clicking on the Hide widget in the main window. Then move the cursor over the Form in the widget tree and press the **Select** mouse button.



You have now written your activate callbacks for your menu items. You can even test them out, in spite of the fact that you have not even finished your address book yet.

8.4 Try Mode

As we like to remind people, XFaceMaker lets you program and test the behavior of your interface, not simply lay out its appearance. Our brief journey into callbacks and FACE programming showed us how we program interface behavior. Now we will show you how to test our FACE code. Above the widget tree to the right, is the **Try** toggle button.



When you set this toggle button, XFaceMaker enters “try mode”, where you can try out your interface. The major differences between try mode and its counterpart, edit mode, are as follows. In try mode:

- There are no selection handles or attachment arrows drawn on the interface.
- Any mouse or keyboard activity in the interface takes the actual action programmed for the interface, instead of performing selection, the normal edit mode behavior.

All other aspects of interface building with XFaceMaker remain intact. You can still edit widget resources, save files, and perform other editing activities, just as if you were in edit mode.

You can use try mode to test the activate callbacks we have programmed so far:

1. Set the **Try** toggle button in the main window.
2. Select the **On Version** button in the **Help** menu in your interface. Your version dialog box should appear.

3. Press the **Close** button in the version box. The window closes because of the default behavior of the `dialogShell`. We did not have to program any behavior into the **Close** button.
4. Select the **Edit** button in the **Address** menu in your interface. Your partially completed address editor should appear.
5. Close the window using the window manager. If your window manager does not provide a means for you to close the window, you may either leave it open or hide it using the **Hide** icon in the main window.
6. Select the **Exit** button in the **File** menu in your interface. (Really, trust us!) You should receive the following message in the message box in XFaceMaker's main window.

Your application would now exit with status 0.

7. Clear the **Try** toggle button to return to edit mode.

Now we will continue building the address editor.

CHAPTER 9: More Editing Techniques

After that brief excursion into the world of the FACE programming language, you can now proceed to finish the address editor dialog box. We will use this opportunity to introduce you to more editing techniques.

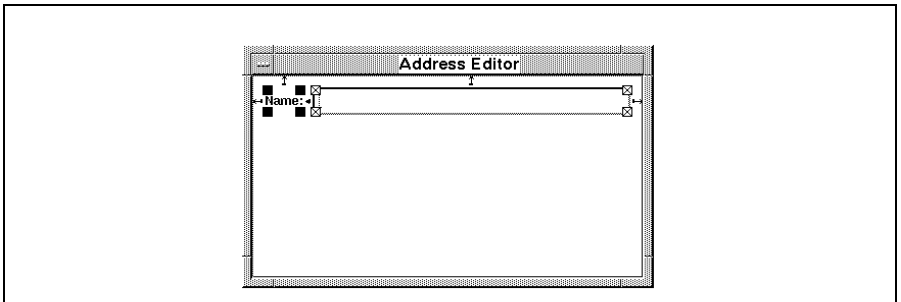
9.1 Cutting and Pasting

Most applications that involve creating and editing data use the “cut and paste” model for manipulating data. XFaceMaker is no exception. However, unlike a word processor where you might select a word, sentence, or paragraph for editing, in XFaceMaker you select widgets. We start by making the address editor visible, if it was not already.

1. In the widget tree, select the form that corresponds to the address window.
2. Select the **Selection** item in the **Show** sub-menu of the **View** menu. You can do this by clicking on the **View** menu button, then clicking on its **Show** menu button, then clicking on the **Selection** item. Or you can press the **Select** mouse button on the **View** menu button, drag the cursor to the **Show** menu item so that the sub-menu appears, then drag the cursor to the **Selection** item.

The dialogShell shows what has been built so far — the **Name** field. You can use the name field to create the next field, the **Street** field:

3. Select the nameLabel widget by clicking on the **Name** label in the interface.
4. With the **Control** key held down, click on the **Name** text field in the interface. You should see both widgets selected.



The label widget should have dark handles while the text field has the familiar selection handles. The ☒ handles indicate the current selection, while the ■ handles indicate other selected widgets. This method of selecting more than one widget

works with the widget tree also, and you can mix and match where you select your widgets depending on what is most convenient. The last widget you select will always be the current selection. Remember, if you do not hold the **Control** key down, XFaceMaker deselects all previous selections before selecting the new current selection.

The notion of *current selection* is important to XFaceMaker, since many operations only work with the current selection. For example, the menu editor always operates on the current selection, and it will change the current selection as you select widgets in it. For this reason, XFaceMaker displays the name of the current selection in the **Current Widget** field in the main window and highlights the resource window with the **Current Selection** toggle set.

When you learn about other dialog boxes, you will see other uses of the current selection. In spite of all this, the editing operations **Cut**, **Copy**, **Delete**, and **Duplicate** always work on *all* the selected objects, not just the current selection.

To use the **Name** field that you have selected as a basis for the **Street** field, you should copy it.

5. Move the cursor over the **Name** text field (the current selection), and press the **Select** mouse button and hold it down.
6. With the **Select** button still pressed, move the cursor over the **Copy** icon  in the main window. While dragging, you will see an outline of the two selected widgets follow the cursor to the **Copy** icon.
7. Release the **Select** button with the cursor over the **Copy** icon.
8. Click on the **Paste** icon. 
9. The cursor should re-appear, as well as the outline of the two widgets that you copied.
10. Move the cursor to the dialog box, but not over one of the **Name** field widgets, then click the **Select** mouse button. The copy you have created is not immediately visible because it is positioned directly over the first field.

You can also copy and paste using the **Edit** menu in the main window. This will also copy all selected widgets. However, there are two ways to copy that act on an individual widget only.

- First, you can click on the **Copy** icon in the main window, then click on a widget in your interface.
- Second, you can drag a widget using the **Drag** mouse button to the **Copy** icon in the main window.

In both of these cases, XFaceMaker copies only the single widget. Also, in both of these cases, you do not have to select the widget you want to copy. Only when you drag the current selection with the **Select** button do all selected widgets get copied. (If you try to drag a selection that is not the current selection using the **Select** button, the widget becomes the current selection, and if you do not hold down the **Control** button, the other selections get deselected.)

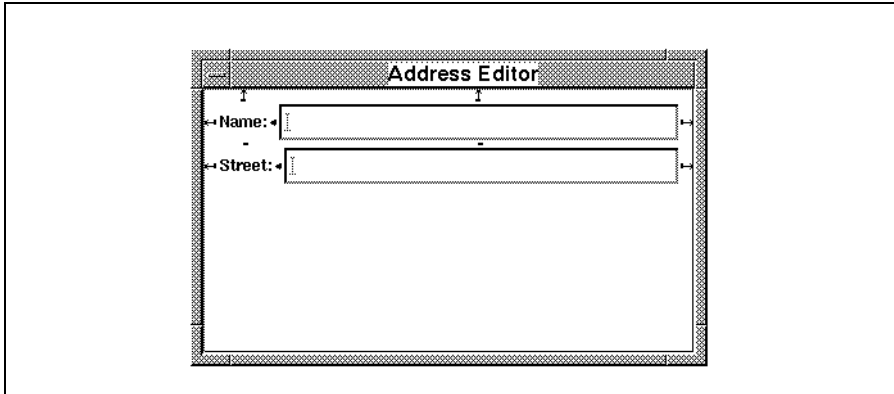
You will have undoubtedly noticed that your interface is rather muddled now. For one thing, the widget tree has two new children, **nameLabel_2** and **name_2**. For another, the selection handles have disappeared from the text field, even though it remains selected in the widget tree. However, there is also a second **Name** label closer to the top of the form.

This clue might help you figure out what really happened. When XFaceMaker copied the widgets, it copied them completely, including their resource values, which means including their attachments. So the new widgets ended up right on top of the old ones! You will need to change some resources so that they correctly reflect the Street field.

11. Select the **nameLabel_2** widget in the widget tree.
12. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
13. Change the Name field in the resource window to **streetLabel**.
14. Change the **labelString** resource to be **Street :**.
15. Press the **Rebuild** button. You will see the label change its name. Now we can change the attachments.
16. Select the **topAttachment** resource, then press the  button next to the text field. The attachment dialog box appears in the resource editor.
17. Press the up arrow. The cursor should change to .
18. Click on the **nameLabel** widget in the widget tree.
19. Press the **Cancel** button to return to the resource list.
20. Select the **name_2** widget in the widget tree.
21. Change the Name field in the resource window to **street**.
22. Select the **xfmCreateCallback** resource, then press the  button next to the text field. The callback editor appears. Note that XFaceMaker even copies the callback script in the duplicated widget.

23. Change the `nameLabel` widget name to `streetLabel` in the script, then press the **OK** button.
24. Change the **topAttachment** resource to attach the widget to the name widget, in the same way as you did in steps 16-19.

Your Street field should now be visible, as shown in the figure below.

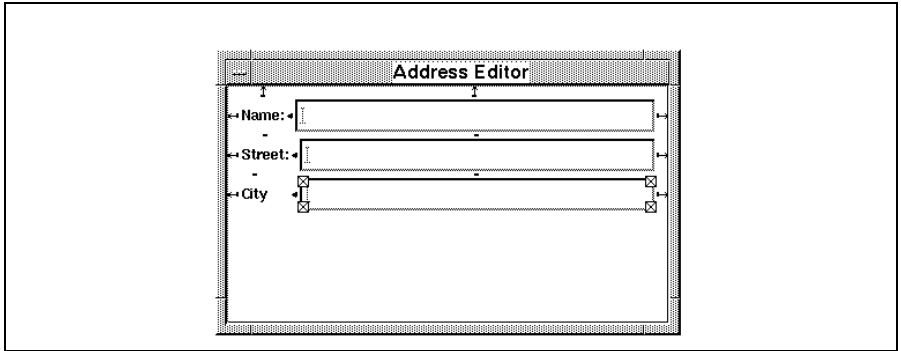


You should be able to add the City field using a similar technique. We will illustrate another editing command here.

25. Select the Street label widget in the interface.
26. With the **Control** key held down, select the text field for the Street field.
27. With the cursor over the dialog box, type the character **Control-D**.

The character **Control-D** is the accelerator for the **Duplicate** item in the **Edit** menu. It duplicates the currently selected widgets. Normally, XFaceMaker offsets the new widgets slightly from their originals, but since in this case the widgets are attached in a form, they appear directly on top, as they did when you copied and pasted earlier. If you want to copy and paste widgets within the same parent, the **Duplicate** command provides an easier means to perform the same function. You still need to copy and paste, however, to duplicate widgets in a different parent widget.

You can now use the method you used for the Street label to set the widget names, label string, and attachments for the City label. Remember to change the **xfmCreateCallback** resource in the new text field widget to refer to the new label. The result should look like the figure below.



9.2 More Attachment Issues

The State field presents a slightly different problem from the other fields: it has the zip code field to its right. We would like to leave the Zip Code field fixed, while the State field changes its size with the dialog box. This means that only the State text field will be attached on both the left and right sides. The other fields on this line will be attached on only one side; in this case, the right side instead of the left.

Start by creating a State field in the same way you created the Street and City fields. Once you have created the State field, remove its **rightAttachment** so that we can attach it to the zip code field later. To do so:

1. With the `state` text field widget selected, open a Resource window if one is not already open, set the **Current selection** toggle, and then open the attachments editor.
2. Click on the fleur icon in the middle or the attachment arrows, then click on the right arrow. This changes the resource being edited to **rightAttachment**.
3. Click on the **Detach This Side** button. This removes the right attachment from the selected widget. On most platforms, the state widget will resize to a smaller width. If it doesn't on your platform, resize the widget to leave space for the zip code fields to its right.
4. Click on **Cancel** to return to the resource list.

9.2.1 Building from Right to Left

We can now add the Zip Code field. We build it from right to left since they remain fixed to the right side of the form. After we finish, we can attach the state text field's right side to the zip code label.

1. Click on the text field icon in the main window, then move the cross-hair cursor (+) to the dialog box and click the **Select** button with the cursor over the dialog box, but not one of the label or text field widgets. 
2. Select the text field you just created.
3. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
4. Change the **Name** field in the resource window to `zip`.
5. Set the **Auto position** and **Auto size** toggle buttons.
6. Select the **columns** resource in the resource list.
7. Change the value in the text field to `12`. This indicates how many columns the text widget will have when it chooses its own size. We choose 12 to have plenty

- of space for modern nine-digit US zip codes.
8. Press the **Rebuild** button.
 9. Select the **Attachments** item in the **Edit** menu. The attachment box appears in the resource window.
 10. Attach the top of the widget to the widget named **city**.
 11. Attach the right side of the widget to the form by clicking in the window but not over any label or text field widgets.
 12. Click on **Cancel** to return to the resource list.
 13. Click on the **XmLabel** icon in the main window. Move the crosshair cursor (+) to the dialog box, but not over another widget, and press the **Select** button. 
 14. Select the label widget you just created.
 15. Change the **Name** field in the resource window to **zipLabel1**.
 16. Set the **Auto position** and **Auto size** toggle buttons.
 17. Select the **labelString** resource.
 18. Change the value in the text field to **Zip Code**:
 19. Press the **Rebuild** button.
 20. Select the **rightAttachment** resource.
 21. Press the  button next to the text field to open the attachment box in the resource window.
 22. Attach the right side of the widget to the widget named **zip**.
 23. Attach the top of the widget to the widget named **city**.
 24. Click on the **Rebuild** button.

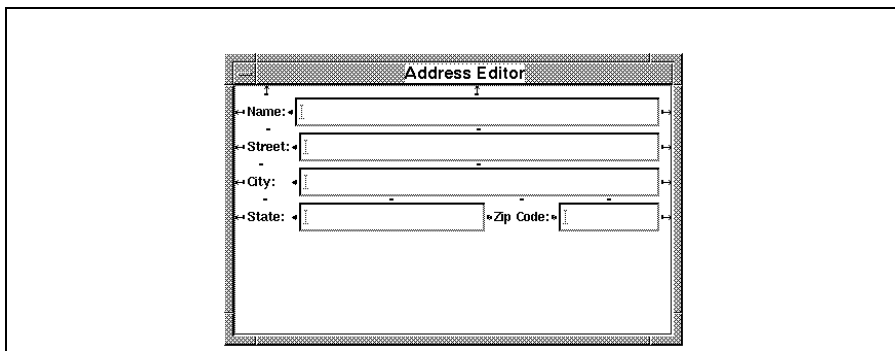
9.2.2 Spacing Between Widgets

As we have seen in other widgets, when we attach a widget on both its left and right sides, it spans the dialog box. With the state text field, we will produce the same effect, but between the state field's label and the Zip Code field. (Note that you may have to make your window wider to make the Zip Code field fit next to the State field).

To do this, we will change the order of the widgets in the widget tree. The state widget should be created after the pair of zip code widgets so that it can be attached to the zipLabel widget, the latter remaining attached to the Zip widget.

1. Select the state widget. The attachment box should remain in the resource window.
2. In the widget tree, press the Select button on the State widget and drag it over the ZipLabel widget. The **state** widget moves after the **zipLabel** widget.
3. Attach the right side of the widget to the **ZipLabel** widget.
4. Press the **Cancel** button to return to the resource list.
5. Select the **xfmCreateCallback** resource.
6. Press the  button next to the text field to open the callback editor.
7. Move the text cursor for the text widget so it is just before the first comma in the call to **FmSetTallest**.
8. Insert another comma (by pressing the , key) then press the Select pushbutton in the resource window. The cursor changes to **?**.
9. Move the **?** cursor over the zip widget and press the **Select** mouse button.
10. Move the text cursor again to just before the second comma.
11. Insert another comma, then press the **Select** button again.
12. Move the ? cursor over the **zipLabel** widget and press the **Select** mouse button.
13. Press the **OK** button.

Your dialog box should look something like the figure below.



The dialog box would be prettier if the left edges of the text fields lined up. Since the text fields are attached to the labels, we can align them by making the labels all have the same width:

1. Select the form containing the text field: click the **Custom** mouse button over the Address Editor dialog box to open the popup menu; select the current widget's parent (or the cursor object, depending where the cursor was when you clicked the **Custom** button).
2. Edit the **xfmCreateCallback** resource, either by selecting the **xfmCreate** item in the **Callbacks** menu, or by selecting the resource in the resource list, then pressing the ... button.
3. Enter the following script:

```
FmSetWidest(self.nameLabel,self.streetLabel,self.cityLabel,  
            self.stateLabel,NULL);
```

4. Press the **OK** button. Now the text fields should be aligned.

9.3 Offsets and Positions

All that remains is to add the separator line and buttons. We use these items to illustrate how to use attachment offsets and positions.



1. Click on the **XmSeparator** icon in the toolkit area of the main window.
2. Move the cursor to the dialog box and click it in the dialog box, but not over any of the label or text widgets we already created, then click the **Select** mouse button.
3. XFaceMaker creates a very small separator, since it defaults to 2x2 pixels. However, you can easily select it in the widget tree. (Do so.)
4. Set the **Auto position** and **Auto size** toggles.
5. Attach the top of the separator to the **state** widget.
6. Clear the **no offset** button in the attachment box. The text field next to it now has shadowing, and the value 2 appears in it. Do you remember that we set the vertical spacing in this form to 2? That is where the value comes from. However, we like more space around our separators. Change this text field to **10**, then press the **Rebuild** button.
7. Attach the left side of the separator to the form.
8. Set the offset to zero, then press the **Rebuild** button. We want the separator to go all the way to the edge of the form, in spite of the margin width value we gave.
9. Attach the right side of the separator to the form.
10. Set the offset to zero, then press the **OK** button.
11. Click on the **XmPushButton** icon in the toolkit of the main window.
12. Move the cursor into the dialog box beneath the separator, then click the **Select** mouse button.
13. Select the push button you just created.
14. Change the **Name** field in the resource window to **ok**.
15. Set the **Auto position** and **Auto size** toggles.
16. Select the **labelString** resource and change the value of the resource to **OK**.
17. Press the **Rebuild** button.
18. Select the **topAttachment** resource and press the  button next to the text field.
19. Attach the top of the widget to the separator. It will probably be easier to select

the separator in the widget tree.

20. Clear the **no offset** toggle button.
21. Change the offset value to **10**. Again, we would like the space around the separator to be more than the default 2 pixels.
22. Press the **Rebuild** button in the attachment box.
23. Attach the left of the widget to the form, setting the offset to 10.
24. Press **OK**.

9.3.1 Attaching Opposite Sides

By attaching opposite another widget, you attach to the side of a widget opposite of the one you normally would. For example, normally when you attach the top of one widget to another, you attach to the other widget's *bottom* edge. However, if you attach to the opposite side, you attach to the widget's *top* edge, effectively aligning the two widgets. For example:

1. Create a new `XmPushButton` next to the **OK** button and select it for editing.
2. Change its name to `he lp` and set the **Auto position** and **Auto size** toggles in the resource window.
3. Change the **labelString** resource to `He lp` and press the **Rebuild** button.
4. Select the **topAttachment** resource and press the  button next to the text field.
5. Set the **opposite edge** toggle button.
6. Attach the widget to the **OK** button. Note the widget moves to the same space as the **OK** button.
7. Clear the no offset toggle, set the offset to zero, then press the **Rebuild** button. Since we are attaching *in parallel*, rather than *next to* another widget, we do not want to use the default spacing of two pixels.
8. Clear the **opposite edge** toggle.
9. Set the **no offset** toggle.
10. Attach the right side of the widget to the form.
11. Create another push button below the separator, and select it.
12. Change its name to `add`, set the **Auto position** and **Auto size** buttons, change its **labelString** resource to `Add`, then press the **Rebuild** button.

13. Open up the attachment box and attach the top of the widget to the opposite side of the **OK** button. Do this by setting the opposite edge toggle, the widget toggle, and then click on ok in the list of widgets to make it the selection.
14. Clear the **no offset** toggle, set the offset to zero, then press the **Rebuild** button.

9.3.2 Centering on a Position

You use position to specify relative positions in a form. Typically, positions range from 0 to 100, so 50, for example, would be the center of the form. However, since we always attach a side of a widget, we must use the attachment offset to truly center the widget.

1. Choose the left attachment resource. To do this, click in the central cross, move the cursor to the left arrow and click. Notice that the resource name becomes **left attachment**.
2. Set the position toggle button for the attachment type. Note that the position text field is now editable.
3. Enter 50 in the position text field.
4. Click on **Rebuild**. This attaches the left side of the widget to position 50. Note the dashed attachment line that appears.
5. Press the **Cancel** button to return to the resource list and select the **xfmCreate-Callback**.
6. Press the  button and enter the following statement as the callback, then press the **OK** button.

```
self:leftOffset = -(self:width / 2);
```

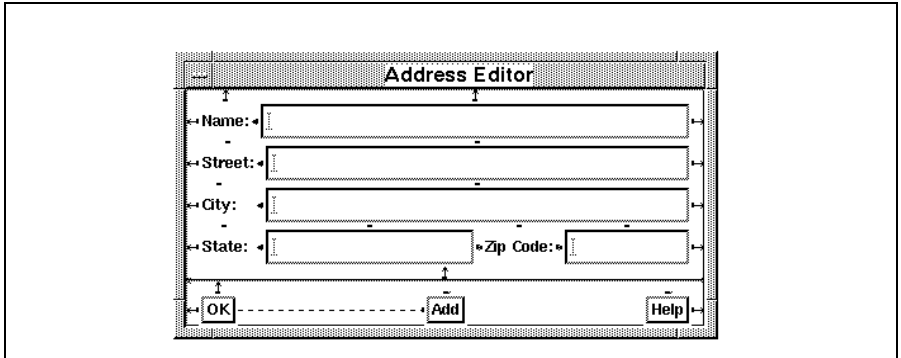
This will center the widget at its position, since its left offset is the negative of half its width.

9.3.3 Separators as Spacing

We will add one last widget in order to make the appearance of our box as automatic as possible. We want the form to decide for itself what its initial size will be.

1. Select the form in the widget tree (or by using the **Custom** mouse button).
2. Set the **Auto position** and **Auto size** buttons in the resource window, then press the **Rebuild** button. The form adjusts its size according to its contents. However, the buttons at the bottom of the form run right up to the edge of the form, as shown in the figure below. One way to solve this problem might be to

attach the **OK** button to the bottom of the form.



3. Select the **OK** button and open up the attachment box.
4. Set the **no offset** toggle.
5. Attach the bottom of the widget to the form. We see that the **OK** button has shrunk in order to accommodate the form's margin.
6. Select the form for editing and press the **Rebuild** button. This causes XFaceMaker to recreate the form, so it will recompute its size. Now the **OK** button is the correct size. However there is still a potential problem.
7. Use the window manager to change the size of the dialog box so it is taller. The **OK** button expands with the height of the dialog box because it is attached on both the top and the bottom. We have to solve this problem in another way.

8. Select the **OK** button for editing, select the **bottomAttachment** resource, and press the  button next to the text field to open up the attachment box.
9. Press the button labeled **Detach this side**. We will find another solution to our problem.
10. Create a new **XmSeparator** widget in the space below the separator, then select it in the widget tree.
11. Set the **Auto position** and **Auto size** toggles.
12. Change the **separatorType** resource to **no_line**, press the **OK** button, then open up the attachment box.
13. Attach the top of the widget to the **OK** button, and reset the offset to zero.
14. Attach the left side of the widget to the form.
15. Attach the bottom of the widget to the form. Make sure the offset is 0.
16. Select the form for editing and press the **Rebuild** button so that it will recompute its size.

Since the separator's type is **no_line**, it will not be visible. It will, however, create a space between the **OK** button and the bottom of the form. We change its top and bottom offsets to zero so that the spacing will not be added to the margin height in the space between the **OK** button and the edge of the form.

CHAPTER 10: Creating the Application

You have now created enough of your interface that you can actually try to build your application. XFaceMaker can help you with this by providing skeletons of the essential files you need to build your application.

10.1 Saving the Interface

In order to use the interface you have built in your application, you must save it to a file, if you have not done so already. XFaceMaker saves your interface in “fm” file format. While you can generate files in other kinds of formats (e.g., C code, RDB data base, UIL), the fm file format provides the basis for all the others. When you need to read your interface back into XFaceMaker to edit it, XFaceMaker always reads the fm file, not the other formats.

Before you save the interface, hide the Address Editor dialog box since you won’t want it to be visible when you first launch your application. To do this, click on the Hide icon in the XFaceMaker main window, bring the cross cursor over the Form contained in the address editor dialogShell, and click. The Address Editor dialog box is hidden.

Save your interface. You can do so as follows:

1. Select the **Save As** item in the **File** menu of the main window. The file selector pops up. The bottom of the file selector contains the current file name (which is `unnamed.fm` by default).
2. Replace the contents of this text field with **addrbook.fm**
3. Press the **OK** button. The message line in the main window should say **addrbook.fm saved.**

If you want, you can examine the file **addrbook.fm** with any standard text editor. The fm file format is normal text, and most of its meaning is obvious, even to those not familiar with the format. However, we do not recommend you change this file with a text editor, since any error may render the file unusable by XFaceMaker, and you will not be able to recover the work saved in it. Since the fm file format is text-based, you can use the same fm file on different machines having different architectures and/or operating systems.

Next, we suggest that you set the **Preserve User Code** toggle in the **Options** menu, then save this status using the **Save Prefs...** option in the **File** menu. The **Save Prefs** option saves your current screen configuration, including the position of the widgetstores, editing boxes, the status of toggles such as **Preserve User Code**, etc. The information is stored in a preference file called `.xfm.startup` in your home directory. XFaceMaker loads this file automatically on startup. No user interface windows are saved.

1. Set the **Preserve User Code** toggle in the **Options** menu of the XFaceMaker main window. This allows us to more easily change the saved files. If you selected **Save Prefs...**, the **Preserve User Code** option will be set automatically each time you re-launch XFaceMaker, because it is included in the `.xfm.startup` file.¹
2. Select **Save Prefs...** in the **File** menu. When the File Selector pops up, the text field at the bottom will contain the file name
`.xfm.startup`
3. Press the **OK** button.

10.2 Application Files

Once you have saved your interface using **Save As**, XFaceMaker uses the name you specified as the current file name. Furthermore, XFaceMaker uses the current file name as a base name for creating application files. You can see this by creating the application files for **addrbook.fm**.

1. Select the **All** item in the **Save Appli Files** sub-menu of the **File** menu in the main window.
2. In another window, list the files in the directory where you saved the interface file **addrbook.fm**. You should see the following files:

```
addrbook.fm
addrbookApp.c
addrbookApp.h
addrbookMain.c
addrbookMake
```

-
1. In this tutorial, we assume that the **Preserve User Code** option is always set. If it is not set automatically, you will have to re-set the **Preserve User Code** option each time you re-launch XFaceMaker, or the instructions given in this tutorial will not yield the expected results.

You can use these files as a basis for your application. Some of these files you will have to edit. Others you can use as they are. Here is a description of each file:

addrbook.fm

This file contains your interface in fm file format.

addrbookApp.c

This file will contain skeletons for any undefined functions in your interface. XFaceMaker expects the application to provide these functions. If you have not yet created these functions, you can fill in the body of the functions in the skeletons provided here. If the functions already exist, you will probably not want to use this file, or at least you will want to delete the definitions for those functions which already exist.

addrbookApp.h

This file declares the functions defined in **addrbookApp.c**.

addrbookMain.c

This is the main program of your application. It initializes XFaceMaker, creates your interface, then calls the “main loop” function which waits for events. Also, if you had any functions declared in **addrbookApp.h**, it makes them available to be called from FACE scripts by “attaching” them, a process we will explain later.

addrbookMake

This is the makefile for your application, the file you will most likely have to edit. It contains variables that specify where to find object files, X libraries, OSF/Motif libraries, XFaceMaker libraries, and anything else you might need to compile and link your application.

You can use these files to build three types of programs:

- m An application that interprets an fm file at run time. You link this type of application with the XFaceMaker library, **-lFm**, which the application uses to load and interpret fm files at run time. This allows you the flexibility of testing your complete application and changing your interface without recompiling it.
- m An application that includes the interface as compiled C code. In this case, XFaceMaker generates C code from the fm file, and you compile this C code and link it into your application. This prevents any change to the interface, but you no longer need to link with XFaceMaker’s interpretive library. Normally, however, you will link with XFaceMaker’s C code support library, **-lFm_c**. If

you take special precautions while building your interface, you can even avoid the need for this library.

Some developers believe that C code runs faster than interpreted code, but the overhead of the OSF/Motif toolkit and the Xt Intrinsics library leaves some room for doubt. A better reason to use C code is to be able to port the C code to a platform other than the one where you developed your interface, perhaps one of the few where XFaceMaker is not available. To aid you in this effort, NSL provides the source to the **-lFm_c** library, so that you do not have to forego the advantages of using this library with your interface.

- m A new version of XFaceMaker that contains application functions. You might want to generate a new version of XFaceMaker because you want to edit your interface as it really operates, with all of your application's functionality. By linking your application with XFaceMaker's editor library, **-lFm_e**, you can use all of XFaceMaker's editing capabilities, such as the FACE language debugger, while testing your application and its interface. Once you have completed your application, you will want to re-link it using one of the other two methods.

10.3 The Application Makefile

Since everyone's installation is different, the application makefile is where you will probably have to do the most customization. If you are completely unfamiliar with the make command and makefiles, you may want to consult your documentation first. We try to point out those parameters in the makefile that are most likely to require your attention.

We explain the file in detail here. First, we show its contents stripped of the template information that is inserted when the Preserve User Code option is set by the user². Then we explain each of its features, line by line.

```
#
# File generated by XFaceMaker Version 3.5.
# Do not remove this comment.
# Timestamp:          908792104
#

#
# XFaceMaker 3.5 Application Makefile.
#

        XFM = xfm
        XFMCFLAGS = -cflags "caC"
        XFMCFLAGS = -cflags "caCg"

MOTIF_INCPATH =
MOTIF_LIBPATH =
MOTIF_LIBNAME = -lXm

        XT_INCPATH =
        XT_LIBPATH =
        XT_LIBNAME = -lXt
```

-
2. In order to generate the application files, XFaceMaker uses pattern (or model) files. If the **Preserve User Code** toggle is not set, new application files are generated from scratch (using the pattern files) each time you select the **Generate Appli Files** menu item. If the **Preserve User Code** toggle is set, the files are generated from scratch the first time (using pattern files) but the pattern information is included in the files generated. These new files are subsequently used as models so that any modifications you make in them remain in subsequent versions of your application files.

```
XLIB_INCPATH =
XLIB_LIBPATH =
XLIB_LIBNAME = -lX11

XEXT_INCPATH =
XEXT_LIBPATH =
XEXT_LIBNAME = -lXext

XFM_I_LIBNAME = -lFm
XFM_C_LIBNAME = -lFm_c
XNSLCTRL_LIBNAME = -lXnsLctrl
XFM_E_LIBNAME = -lFm_e

EXTRA_INCPATH = -I$(NSLHOME)/include $(LOC_INCPATH)
EXTRA_LIBPATH = -L$(NSLHOME)/lib $(LOC_LIBPATH)
EXTRA_I_LIBS =
EXTRA_C_LIBS =
EXTRA_E_LIBS =

XM_INCPATH = $(MRM_INCPATH) $(MOTIF_INCPATH) $(XT_INCPATH)
$(XEXT_INCPATH) $(XLIB_INCPATH)

A_INCPATH = -I. $(EXTRA_INCPATH) $(XM_INCPATH)
E_INCPATH = -I. $(EXTRA_INCPATH) $(XM_INCPATH)
X_LIBS = $(XT_LIBPATH) $(XT_LIBNAME) $(XEXT_LIBPATH)\
$(XEXT_LIBNAME) $(XLIB_LIBPATH) $(XLIB_LIBNAME)
MOTIF_LIBS = $(MRM_LIBPATH) $(MRM_LIBNAME) \
$(MOTIF_LIBPATH) $(MOTIF_LIBNAME) $(X_LIBS)

SYS_LIBS =

SYS_CFLAGS =
SYS_LDFLAGS =
CDEBUGFLAGS =

CCODEOPTS = -DCCODE

STD_LIBS = $(MOTIF_LIBS) -lm $(SYS_LIBS)

A_LIBPATH = -L. $(EXTRA_LIBPATH)
E_LIBPATH = -L. $(EXTRA_LIBPATH)

XFM_I_LIBS = $(XFM_I_LIBNAME)
```

```
XFM_C_LIBS = $(XFM_C_LIBNAME) $(XNSLCTRL_LIBNAME)
XFM_E_LIBS = $(XFM_E_LIBNAME)

A_CFLAGS = $(A_INCPATH) $(SYS_CFLAGS) $(CDEBUGFLAGS)
E_CFLAGS = $(A_INCPATH) $(SYS_CFLAGS) $(CDEBUGFLAGS)
CFLAGS = $(E_CFLAGS)

A_LDFLAGS = $(A_LIBPATH) $(SYS_LDFLAGS) $(CDEBUGFLAGS)
E_LDFLAGS = $(E_LIBPATH) $(SYS_LDFLAGS) $(CDEBUGFLAGS)

RM = rm -rf
CP = cp
LN = ln -s

XFM_TARGET_OS = -sunos
XFM_TARGET_MK = $(NSLHOME)/lib/xfm/GenAppli/\
Mk$(XFM_TARGET_OS)
include $(XFM_TARGET_MK)

all:: interpreted compiled

interpreted:: iaddrbook

compiled:: caddrbook

editor:: eaddrbook

remote:: raddrbook

iaddrbook: iaddrbookMain.o addrbookApp.o
@sh -c "if [ -f $$@ ] ; then mv $$@ $$@.bak ; $(RM) $$@.bak ; fi"
$(CC) $(A_LDFLAGS) -o iaddrbook iaddrbookMain.o \
addrbookApp.o \
$(XFM_I_LIBS) $(EXTRA_I_LIBS) $(STD_LIBS)

raddrbook: raddrbookMain.o addrbookApp.o
@sh -c "if [ -f $$@ ] ; then mv $$@ $$@.bak ; $(RM) $$@.bak ; fi"
$(CC) $(A_LDFLAGS) -o raddrbook raddrbookMain.o \
addrbookApp.o \
$(XFM_I_LIBS) $(EXTRA_I_LIBS) $(STD_LIBS)

caddrbook: caddrbookMain.o addrbookApp.o addrbook.o
@sh -c "if [ -f $$@ ] ; then mv $$@ $$@.bak ; $(RM) $$@.bak ; fi"
```

```
$(CC) $(A_LDFLAGS) -o caddrbook caddrbookMain.o \
addrbookApp.o \
    addrbook.o \
$(XFM_C_LIBS) $(EXTRA_C_LIBS) $(STD_LIBS)

eaddrbook: eaddrbookMain.o addrbookApp.o
@sh -c "if [ -f $@ ] ; then mv $@ $@.bak ; $(RM) $@.bak ; fi"
$(CC) $(E_LDFLAGS) -o eaddrbook eaddrbookMain.o \
addrbookApp.o \
$(XFM_E_LIBS) $(EXTRA_E_LIBS) $(STD_LIBS)

iaddrbookMain.o: addrbookMain.c
$(CC) $(A_CFLAGS) -c addrbookMain.c
mv addrbookMain.o iaddrbookMain.o

raddrbookMain.o: addrbookMain.c
$(CC) $(A_CFLAGS) -DREMOTE -c addrbookMain.c
mv addrbookMain.o raddrbookMain.o

caddrbookMain.o: addrbookMain.c
$(CC) $(A_CFLAGS) $(CCODEOPTS) -c addrbookMain.c
mv addrbookMain.o caddrbookMain.o

eaddrbookMain.o: addrbookMain.c
$(CC) $(E_CFLAGS) -DCALLEDITOR -c addrbookMain.c
mv addrbookMain.o eaddrbookMain.o

addrbook.c: addrbook.fm
$(XFM) -compile addrbook.fm \
    -cinclude 'addrbookApp.h' \
    -cfile addrbook.c -cname addrbook $(XFMCFLAGS)

applifiles:
$(XFM) -gen addrbook.fm \
    -cfile addrbook.c -cname addrbook $(XFMCFLAGS)

clean::
$(RM) *.o iaddrbook eaddrbook caddrbook core emergency.fm

cleanall:: clean
$(RM) addrbook.c *~
```

```
#
# Windows source files generation:
#

WINDOWSDIR = Windows
WINOUTDIR = WinOut

WINAPPPFILES = \
$(WINDOWSDIR)/addrbook.mak \
$(WINDOWSDIR)/addrbook.h \
$(WINDOWSDIR)/addrbook.cpp \
$(WINDOWSDIR)/addrbook.rc \
$(WINDOWSDIR)/addrbookApp.h \
$(WINDOWSDIR)/addrbookApp.cpp \
$(WINDOWSDIR)/addrbookRes.h \
$(WINDOWSDIR)/stdafx.cpp \
$(WINDOWSDIR)/stdafx.h

WINADDRBOOKMODFILES = \
$(WINDOWSDIR)/addrbookXfm.h \
$(WINDOWSDIR)/addrbookXfm.cpp \
$(WINDOWSDIR)/addrbookXfm.rc \
$(WINDOWSDIR)/addrbookFmr.h

$(WINAPPPFILES) $(WINADDRBOOKMODFILES):: addrbook.fm
$(XFM) -target Windows addrbook.fm -targetfile $@
@echo Converting $@...
@xfmwincvt $@

windows:: addrbook.fm
$(XFM) -target Windows addrbook.fm
@echo Converting files...
@xfmwincvt

clean::
$(RM) $(WINDOWSDIR)/.last.* $(WINOUTDIR)/*

always:

cleanall:: clean
$(RM) $(WINOUTDIR)
```

10.4 Makefile Variables

The first half of the makefile contains variable declarations used in the rules that follow for building the application, as follows:

XFM

This is the name used to execute XFaceMaker to compile a fm file into C code. If your PATH environment variable does not include the directory where XFaceMaker is installed, you will have to change this to include the full pathname of XFaceMaker. It can be **xfm** or **xfmwf** if you are using the Widget Factory, or another XFaceMaker executable if you are using your own extended XFaceMaker executable.

XFMCFLAGS

These are the flags specified to XFaceMaker when generating C code. We explain these in a later chapter.

XFMCCLFLAGS

These are the flags specified to XFaceMaker when generating C code for an Xt Intrinsics class.

MOTIF_INCPATH

An argument for the compiler to instruct it where to look for OSF/Motif header files. These files are typically in a subdirectory named **Xm**, and the compiler option is normally **-I**. So, if you have the OSF/Motif header files in the directory **/opt/motif/include/Xm**, you would specify this variable as **-I/opt/motif/include**. If you have installed OSF/Motif header files in their normal location for your platform (**/usr/include/Xm** on many platforms), you may leave this value blank.

MOTIF_LIBPATH

An argument for the compiler to instruct it where to look for the OSF/Motif library. Typically, this argument is **-L**, so if you have the OSF/Motif library in the directory **/opt/motif/lib**, you would specify this variable as **-L/opt/motif/lib**. If the library is in a directory normally searched by your compiler, you may leave this value blank.

MOTIF_LIBNAME

The actual argument that specifies the OSF/Motif library. Normally, you will not have to change this variable. However, if your compiler does not allow you to specify search directories and the library is not installed in a standard directory, you may have to specify the full path name of the library here, such as **/opt/**

motif/lib/libXm.a.**XT_INCPATH**

A compiler argument indicating where to look for the Xt Intrinsics header files.

XT_LIBPATH

A compiler argument indicating where to look for the Xt Intrinsics library.

XT_LIBNAME

The compiler argument that specifies the Xt Intrinsics library. Normally, you will not have to change this variable.

XLIB_INCPATH

A compiler argument indicating where to look for the X11 library files. If this is the same as the one specified for the Xt Intrinsics headers, you can leave it blank.

XLIB_LIBPATH

A compiler argument indicating where to look for the X11 libraries. If this is the same as the one specified for the Xt Intrinsics library, you can leave it blank.

XLIB_LIBNAME

The compiler argument that specifies the X11 library. Normally, you will not have to change this variable.

XFM_I_LIBNAME

The compiler argument specifying the XFaceMaker library used to build applications that interpret fm files at run time. Normally, you will not have to change this variable.

XFM_C_LIBNAME

The compiler argument specifying the XFaceMaker library used to build applications that contain C code generated from fm files. Normally, you will not have to change this variable.

XNSLCTRL_LIBNAME

The compiler argument specifying the Control widget library used to build applications that contain C code generated from fm files. Normally, you will not have to change this variable.

XFM_E_LIBNAME

The compiler argument specifying the XFaceMaker library used to build a new version of XFaceMaker linked with your application. Normally, you will not

have to change this variable.

EXTRA_INCPATH

A compiler argument that indicates where to look for include files in addition to the standard includes.

EXTRA_LIBPATH

A compiler argument that indicates where to look for library files in addition to the standard libraries.

EXTRA_I_LIBS

A compiler argument specifying any additional libraries required to build applications that interpret fm files at run time, when using an extended XFaceMaker. Normally, you will not have to change this variable.

EXTRA_C_LIBS

A compiler argument specifying any additional libraries required to build applications that contain C code generated from fm files, when using an extended XFaceMaker. Normally, you will not have to change this variable.

EXTRA_E_LIBS

A compiler argument specifying any additional libraries required to build a new version of XFaceMaker linked with your application, when using an extended XFaceMaker. You should not need to change this variable.

XM_INCPATH

The list of include paths for the Motif and X header files.

A_INCPATH

The list of include paths for header files in application mode. Note that some include paths are defined only when using an extended XFaceMaker (running xfmwf or another extended XFaceMaker executable). If you need to specify directories for your application, you should do so *after* those specified in the value for this variable. Do not remove any of the variables already there.

E_INCPATH

The list of include paths for header files in editor mode. Note that some include paths are defined only when using an extended XFaceMaker (running xfmwf or another extended XFaceMaker executable). If you need to specify directories for your application, you should do so *after* those specified in the value for this variable. Do not remove any of the variables already there.

X_LIBS

The list of X libraries needed to link an application. Normally, you will not have to change this variable. However, if you are using any extensions to X, this may be a good place to specify them. Alternatively, you could specify them with the `XLIB_LIBNAME` variable.

MOTIF_LIBS

The complete list of all X and OSF/Motif libraries needed to link an OSF/Motif application.

SYS_LIBS

Extra library specifications required by your installation. Normally, you do not need to specify anything here, but on some systems you need to specify a compiler-specific library (such as `-lg++`), or a system-specific library (such as `-lpw` or `-lgen`).

SYS_CFLAGS

Any specific compilation flags required by your C compiler.

SYS_LDFLAGS

Any specific flags required by your linker.

CDEBUGFLAGS

Specify the debug flag here if you want to generate your application in debug mode.

CCODEOPTS

The flag used by XFaceMaker to generate a compiled application. You should not change this.

STD_LIBS

The complete list of all libraries needed to compile your application. If you need to specify special libraries for your application, you should do so *after* those specified in the value for this variable. Do not remove any of the variables already there.

A_LIBPATH

The path to find Widget Factory and XFaceMaker libraries used to build an application in compiled or interpreted mode. Some variables used are defined only if you are using the Widget Factory or third party widgets, such as the Xrt widgets from KLGroup.

E_LIBPATH

The path to find Widget Factory and XFaceMaker libraries used to build a new

XFaceMaker linked with your application functions (the editor mode). Some variables used are defined only if you are using the Widget Factory or third party widgets, such as the Xrt widgets from KLGroup.

XFM_I_LIBS

The list of additional specifications needed to include the XFaceMaker interpretive library. Normally, you will not have to change this variable.

XFM_C_LIBS

The list of additional specifications needed to include the XFaceMaker C code support library. Normally, you will not have to change this variable.

XFM_E_LIBS

The list of additional specifications needed to include the XFaceMaker editor library. Normally, you will not have to change this variable.

A_CFLAGS

The list of compiler flags specified when compiling a .c file when building an application. These are *not* specified when linking. You can add any application specific include directories or macro definitions after the value specified in this variable.

E_CFLAGS

The list of compiler flags specified when compiling a .c file when building a new XFaceMaker. These are *not* specified when linking. You can add any application specific include directories or macro definitions after the value specified in this variable.

A_LDFLAGS

The list of compiler flags specified when linking an application. These are *not* specified when compiling.

E_LDFLAGS

The list of compiler flags specified when building a new XFaceMaker linked with your application. These are *not* specified when compiling.

RM

The command used to remove a file. Normally, you will not have to change this variable.

CP

The command used to copy a file. Normally, you will not have to change this variable.

LN

The command used to create a symbolic link. Normally you will not have to change this variable.

XFM_TARGET_OS

Specifies the target platform. You should not need to change this, unless you are porting an application.

XFM_TARGET_MK

Specifies the path where the characteristics of the target platform can be found. You should not need to change this, unless you are porting an application.

10.5 Makefile Rules

The second half of the Makefile contains the rules used to build the various types of applications. There are rules for the three types of programs: an application with an interpreted interface, an application with a compiled interface, and a new version of XFaceMaker. We look at each of the individual rules in turn.

`all: interpreted compiled`

The default rule makes the two application versions. The editor version is made only upon specific request.

To name the files it makes, XFaceMaker uses the base of the `fm` file name and prepends the letters *i*, *c*, and *e* to each as the name of each of the three programs. As you might expect, the *i* indicates the application with the *interpreted* interface, the *c* marks the application with the *compiled* interface, and the *e* prefixes the new version of the interface *editor*.

```
iaddrbook: iaddrbookMain.o addrbookApp.o
@sh -c "if [ -f $@ ] ; then mv $@ $@.bak ; $(RM) $@.bak ; fi"
$(CC) $(A_LDFLAGS) -o iaddrbook iaddrbookMain.o \
    addrbookApp.o (XFM_I_LIBS) $(EXTRA_I_LIBS) $(STD_LIBS)
```

This rule removes the program if it exists, then it links the program **iaddrbook** from the two object files **iaddrbookMain.o** and **addrbookApp.o**.³

A later rule explains how to make the file **iaddrbookMain.o** by compiling the file **addrbookMain.c**. Each program has its own object file based on compiling this C source file with different macros defined on the compile line. For the interpreted interface, the compilation contains no special arguments. However, for the compiled interface, the compilation line contains the argument `-DCCODE`, and for the new editor, the line contains the argument `-DCALLEDITOR`.

```
caddrbook: caddrbookMain.o addrbookApp.o addrbook.o
@sh -c "if [ -f $@ ] ; then mv $@ $@.bak ; $(RM) $@.bak ; fi"
$(CC) $(A_LDFLAGS) -o caddrbook caddrbookMain.o \
    addrbookApp.o addrbook.o $(XFM_C_LIBS) $(EXTRA_C_LIBS) \
    $(STD_LIBS)
```

3. Because some systems will not let you remove an executable file while it is running, the target is first moved from `$@` to `$@.bak`. This way, we are sure that `$@` does not exist and the make command will succeed.

This rule links the program **caddrbook**, the application with the compiled interface. The main difference from **iaddrbook** is that it uses **caddrbookMain.o** and the libraries specified in **XFM_C_LIBS** and **EXTRA_C_LIBS**.

```
eeaddrbook: eaddrbookMain.o addrbookApp.o
@sh -c "if [ -f $@ ] ; then mv $@ $@.bak ; $(RM) $@.bak ; fi"
$(CC) $(E_LDFLAGS) -o eaddrbook eaddrbookMain.o \
    addrbookApp.o $(XFM_E_LIBS) $(EXTRA_E_LIBS) $(STD_LIBS)
```

This rule links the program **caddrbook**, the new version of XFaceMaker. Like **iaddrbook** and **caddrbook**, it has its own object file of the main program module, called **caddrbookMain.o**, and uses its own set of libraries specified in **XFM_E_LIBS** and **EXTRA_E_LIBS**.

```
iaddrbookMain.o: addrbookMain.c
$(CC) $(A_CFLAGS) -c addrbookMain.c
mv addrbookMain.o iaddrbookMain.o
```

This rule makes the main program module **iaddrbookMain.o**, used in the application that interprets the interface **fm** file at run time. It compiles **addrbookMain.c**, then renames the object module. None of the programs in this makefile use **addrbookMain.o** without renaming it first.

```
caddrbookMain.o: addrbookMain.c
$(CC) $(A_CFLAGS) $(CCODEOPTS) -c addrbookMain.c
mv addrbookMain.o caddrbookMain.o
```

This rule makes the main program module **caddrbookMain.o**, used in the application that contains the interface as compiled code. It specifies **\$(CCODEOPTS)** when compiling **addrbookMain.c** to indicate a compiled interface, then moves the resulting object file to **caddrbookMain.o**.

```
eaddrbookMain.o: addrbookMain.c
$(CC) $(E_CFLAGS) -DCALLEDITOR -c addrbookMain.c
mv addrbookMain.o eaddrbookMain.o
```

This rule makes the main program module **caddrbookMain.o**, used in the new version of XFaceMaker. It specifies the argument **-DCALLEDITOR** to indicate that an editor is being built. It, too, renames the object file, in this case to **caddrbookMain.o**.

```
addrbook.c: addrbook.fm
$(XFM) -compile addrbook.fm \
    -cinclude "\"addrbookApp.h\" \" \
    -cfile addrbook.c -cname addrbook $(XFMCFLAGS)
```

This rule makes the C code file **addrbook.c** from the fm file **addrbook.fm**. It calls XFaceMaker with the appropriate arguments.

```
applifiles:
$(XFM) -gen addrbook.fm \
    -cfile addrbook.c -cname addrbook $(XFMCFLAGS)
```

This rule generates the application files for your program from the fm file. It calls XFaceMaker with the appropriate arguments.

```
clean::
$(RM) *.o iaddrbook eaddrbook caddrbook core emergency.fm
```

This rule removes object files, binaries, and other stray files.

```
cleanall:: clean
$(RM) addrbook.c *~
```

This rule removes c files generated from the .fm interface description files.

The makefile file also includes rules to generate the Windows translation of the interface. We will not explain the XFaceMaker/Win option in this tutorial.

10.6 Making the Program

You are now ready to make your application. You can use this makefile to make any of the three programs. However, at this time, we only look at the first type, an application with an interpreted interface. We will come back to the others in later chapters.

To make your application, enter the following command:

```
make -f addrbookMake interpreted
```

The output of Make will show you the commands executed to make your application. On our system, we see the following:

```
cc -I. -I/news/md/include -c addrbookMain.c
mv addrbookMain.o iaddrbookMain.o
cc -I. -I/news/md/include -target sun4 -c addrbookApp.c
cc -L. -L/news/md/lib -o iaddrbook iaddrbookMain.o \
addrbookApp.o -lFm -lXm -lXt -lXext -lX11 -lm
```

The actual commands you see will depend on the directory in which you installed XFaceMaker, on the operating system you are using and whether you modified your makefile. Assuming that your **make** succeeded, you can test your application by entering the command:

```
./iaddrbook
```

Your interface appears on the screen, and you can test it in the same way you did in XFaceMaker's try mode. This time, however, when you choose the **Exit** item in the **File** menu, the application really does exit!

What if your **make** did not succeed? Most often, this is due to a configuration variable not being set correctly in your makefile. For example, if the compiler cannot find a particular **.h** file, you might check the variables whose names end with **INC-**PATH****. Or, if the compiler cannot find a particular library, you might check the variables whose names end with **LIBPATH**. Finally, you might check to ensure that you have specified all necessary compiler arguments for your particular installation. Ask your system administrator or consult your compiler documentation for help.

CHAPTER 11: Generating C Code

There are two ways to generate C code for your interface. In this chapter, we discuss the interactive way to generate C code using XFaceMaker's C code options. In later chapters, we will show you how you can call XFaceMaker from the command line to translate your `fm` file to C code, in much the same way you would invoke a compiler.

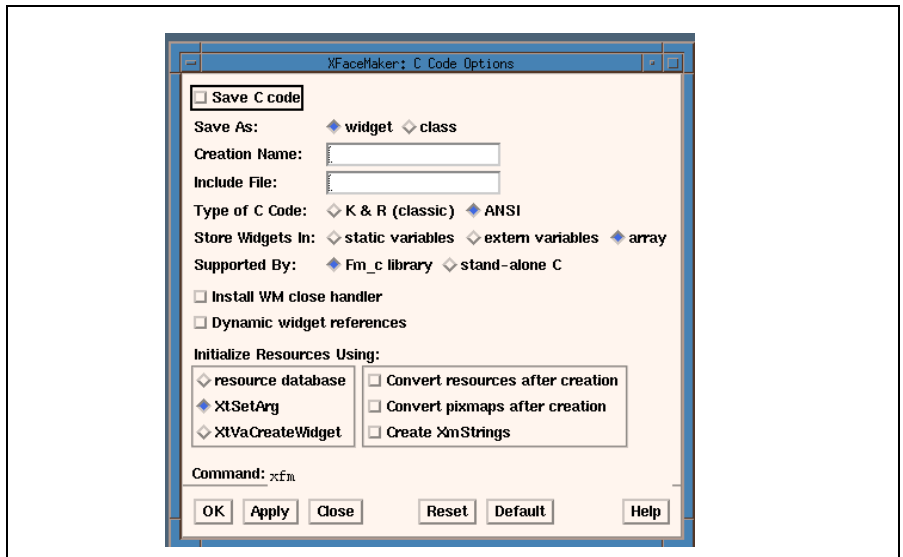
If you are not already doing so, edit your interface with XFaceMaker. You can either specify your `fm` file to XFaceMaker when you launch it, like so:

xfm addrbook.fm

Or, you can edit the `fm` file by selecting **Open** in the **File** menu. This will bring up the file selector box. In this box, you may either double-click the **Select** mouse button on the file `addrbook.fm` in the right hand list of file names, or select the name with a single click, then press the **OK** button.

11.1 Option Box Basics

Select the **C Code** button in the **Options** menu to open the **C Code Options** dialog box.



All option dialog boxes (the ones above the separator in the **Options** menu) have these buttons at the bottom for taking actions:

OK	Apply any changes made in the dialog box and close it.
Apply	Apply any changes made in the dialog box but leave it open.
Close	Close the dialog box without applying any changes.
Reset	Reset the dialog box to the options last applied.
Default	Reset the dialog box to XFaceMaker's default values.
Help	Get some help for this dialog box.

Note that none of the changes you make will take effect until you press either **OK** or **Apply**. This includes the **Default** button, so you can compare your current settings with XFaceMaker's defaults.

All option boxes also have a **Command** field at the bottom. This field shows the command line equivalent to the options currently set in this session of XFaceMaker. This command shows the options in *all* option boxes, not just the one you currently have open. When you press **Apply** (or **OK**), XFaceMaker updates the command line to reflect the values you have chosen in the dialog box.

Many of the option boxes deal with additional files that you can save with your `fm` file. For example, the **C Code Options** box lets you save a C code version of your interface. These boxes will all have a toggle at the top left corner of the box. You must set this toggle in order to save the special file or files associated with that option box. For example, in the **C Code Option** box, the toggle is labeled **Save C code**.

11.2 C Code Options

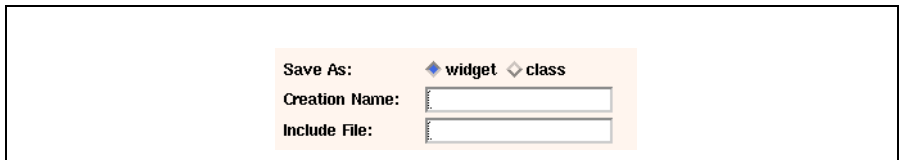
There are many options for C code. For the most part, you will only need to set the **Save C code** toggle, and leave the rest as their default value. If you are not familiar with programming OSF/Motif or Xt Intrinsics applications in C, then you should probably just rely on the defaults, as much of the discussion in this section will not mean very much to you.

XFaceMaker's capabilities have developed over a long time, and this has caused a certain amount of redundancy to creep into its features and functionalities. At one time, many of the C code options dealt with how the application would use that portion of the interface, for example, whether it would load it once or many times, or whether it described a widget hierarchy or a widget class.

For the most part, XFaceMaker now sets the appropriate options automatically when you save a file, according to what you want to save. Only when you save from the **Save** or **Save As** command in the **File** menu does XFaceMaker apply the options in this dialog box without modification.

For these reasons, we will explain the most important options here, and gloss over some of the others. You should see the *XFaceMaker User's Guide* or *XFaceMaker Reference Manual* if you want complete details.

We have already talked about the **Save C code** toggle, so we will proceed to the next portion of the dialog box, where there are the three fields:



At this time, these fields are best left alone. Briefly, the fields are:

Save As

Indicates whether to generate C code for a widget hierarchy or a widget class. Normally, you would use the **Classes** sub-menu under the **Modules** menu to save a widget class, so you can always leave this as its default value, with the widget toggle set.

Creation Name

This name determines the name of the function that the application calls to create the widget hierarchy. If blank (the default), the name of the function is **FmCreatename**, where *name* is the name of the fm file with the suffix removed. In our example, the *name* would be **FmCreateaddrbook**.

Include File

This indicates any extra **#include** directives to insert in the generated source code.

The next field indicates what type of C code to generate:

Type of C Code:

☐ K & R (classic)

☒ ANSI

Select either K & R (classic) or ANSI C, according to your compiler. If you do not know what kind of compiler you have, ask your system administrator.

The **Store Widgets In** field indicates where the generated C code will save the widget ID's of the widgets it creates.

Store Widgets In:

☐ static variables

☐ extern variables

☒ array

The options are as follows:

static variables

Statically allocated space that is also statically linked. The application cannot access the widget ID's directly from other modules.

extern variables

Statically allocated space that is externally linked. XFaceMaker uses the names of the widgets to create the names of the global variables.

array

Dynamically allocated space. The application cannot access the widget ID's directly from other modules.

By default, XFaceMaker selects **array**, the most flexible option. Normally, you will not need to change the default value.

The **Supported By** field indicates whether or not you will link the generated C code with the **Fm_c** library.

Supported By: ☒ **Fm_c library** ☐ **stand-alone C**

If you will link with XFaceMaker's library, choose the **Fm_c** toggle, otherwise the **stand-alone C** toggle. Normally, you will want to link with the **Fm_c** library, since it contains many convenience functions that make FACE programming easier. You would only need to use stand-alone C if you plan to port the generated C code to another system where you have not also ported the **Fm_c** library.

If set, the **Install WM close handler** toggle indicates that the generated C code will contain and install XFaceMaker's default close handler for shell widgets to handle **WM_DELETE_WINDOW** message from the window manager.

☐ **Install WM close handler**

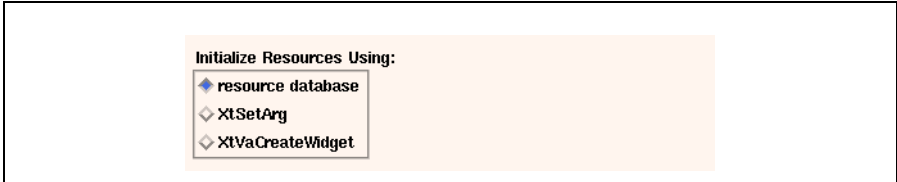
The default close handler exits the application if the shell is an applicationShell, otherwise it hides the shell. If an application interprets the fm file at run-time, the **Fm** library automatically installs the default close handler. You might want to set this toggle so that the compiled interface behavior matches the interpreted interface.

The **Dynamic widget references** toggle instructs XFaceMaker whether or not to use dynamic widget references in order to resolve widget references in FACE scripts.

☐ **Dynamic widget references**

If not set, XFaceMaker will use the widget ID returned at the creation of the interface to refer to a widget within the interface described in the same **fm** file. Dynamic widget references are resolved at run time using the Xt intrinsics function **XtNameToWidget**. Normally, you will want to leave this unset since there is no reason to resolve widget references dynamically for widgets within the same fm file.

The box labeled **Initialize Resources Using** indicates how the generated C code should set resource values.



resource database

Create resource database entries for each resource value and add them to the resource database for the running application. The resource handler takes care of converting the string values into actual resource values.

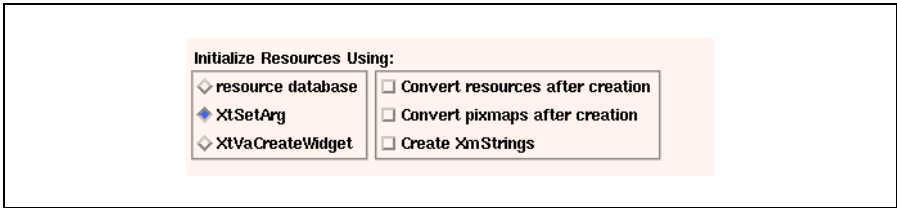
XtSetArg

Use **XtSetArg** to create an array of resource values, and pass that array to **XtCreateWidget**. If XFaceMaker cannot know the correct resource value at the time of code generation (e.g., for pointers to data structures or ID's returned from the X server), it generates a call to the resource converter to determine the resource value. This selection is the default.

XtVaCreateArg

Call **XtVaCreateWidget** to create the widget. XFaceMaker generates the arguments to **XtVaCreateWidget** in the same way it would for **XtSetArg**, except that resource values that require conversion are passed using **XtVaTypedArg** instead of an explicit call to a converter.

If you select **XtSetArg**, another toggle box appears to the right of the radio box.



This toggle box contains options for how the generated code should convert resource values:

Convert resources after creation

For all resources which require a call to a converter, XFaceMaker generates code that calls the converter after creating the widget, passing the created widget

as the argument to the conversion function. When XFaceMaker generates code that calls the converter before the widget's creation, it uses another widget as the argument to the function. You may not want to do this if some of the resources depend on widget-specific properties. However, by calling the converter after the widget's creation, the caller cannot override any values by specifying arguments to the creation function generated by the C code. Also, the widget may “flash” if setting the resource value after creation drastically changes the appearance of the widget, although this is rather unlikely.

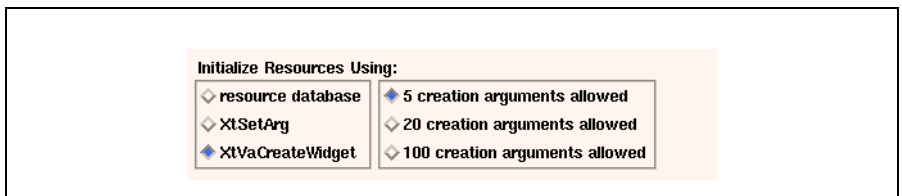
Convert pixmaps after creation

XFaceMaker generates code that converts pixmap resources after the widget's creation. This is the most common type of resource to require conversion after creation, since it uses the background and foreground colors of the widget in the conversion process. Naturally, if you set the previous toggle, XFaceMaker places all resource conversions after the widget creation, so this toggle has no effect.

Create XmStrings

Normally, XFaceMaker uses the resource converter to convert a resource of the type **XmString**. If this toggle is set, it uses the OSF/Motif function **XmStringCreateLtoR** with the character set **XmSTRING_DEFAULT_CHARSET**. Normally, using the converter is better since it caches converted resource values.

On the other hand, if you choose the **XtVaCreateWidget** toggle, a different set of toggle buttons appears.



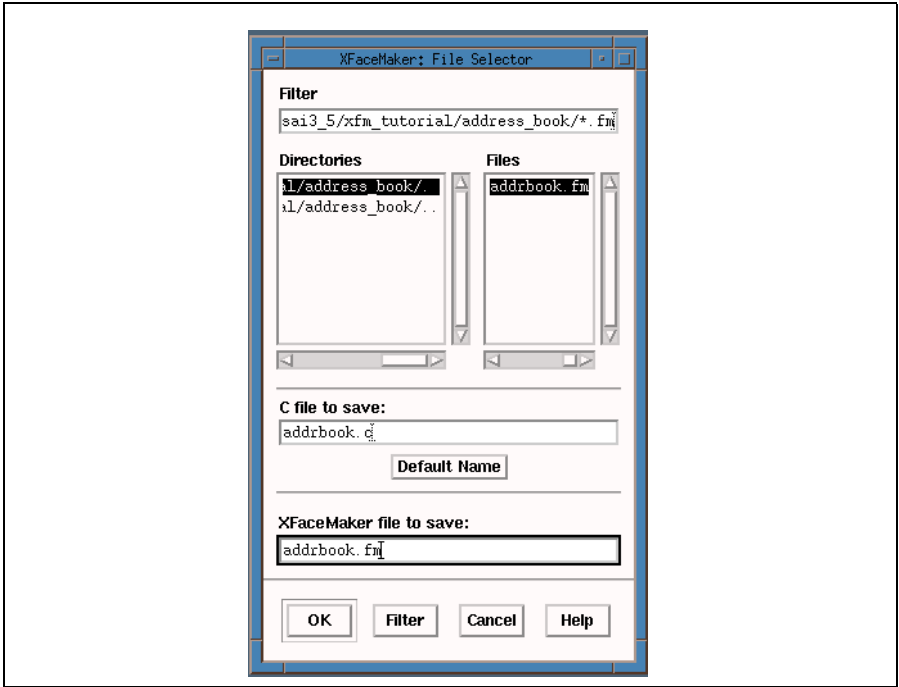
It selects the maximum number of arguments the caller can give to the **FmCreate-*name*** function that creates the interface described in the **fm** file.

11.3 Saving C Code

After all that, we will recommend again, especially for this example, to set only the **Save C code** toggle button at the top of the dialog box and leave the rest of the box with its default settings.

You have already saved your interface once. When you chose the file name then, XFaceMaker made it the current file name. So you could now choose the **Save** item in the **File** menu to save your interface using the current file name. If you did, you would generate two files, `addrbook.fm` and `addrbook.c`.

However, we want to show you how the file selector changes when you set the option to save code, so you should instead chose the **Save As** item in the **File** menu. The file selector appears as in the figure below.



Here are some things to note about the new file selector.

- m There is a new section for the C code file between two indented separators. This section contains any additional files to be saved with the **fm** file.
- m At the bottom of this new section is a **Default Name** button. Use this button to derive the default names for the additional files if you change the XFaceMaker file name.
- m The XFaceMaker file name field contains the current file name.
- m The C file name contains the default name derived from the XFaceMaker file name.

Here are some things you can do to see how this new file selector works.

1. Change the filename portion of the XFaceMaker file from **addrbook.fm** to **addr.fm**.
2. Press the **Default Name** button. The C file name changes to **addr.c**, the default name derived from the XFaceMaker file name.
3. Select the file named **addrbook.fm** in the right-hand list of file names. The XFaceMaker file name and the C file name change to reflect your selection.
4. Press the **OK** button to save the file.
5. Since you used the **Save As** command instead of **Save** to save the file, XFaceMaker will confirm that you will really want to overwrite the fm file. Confirm by pressing the **OK** button. Since you have not yet created a C file, it will not confirm this file.
6. Return to your terminal session and enter the command:

```
make -f addrbookMake compiled
```

Your application should compile successfully and you can test it by entering the command

```
./caddrbook
```

We went through these manipulations of the C Options box to show you how they work. You could have generated the C code for your fm file with the make command directly. The makefile has a rule to generate the C code for the fm file. In our case, it was:

```
addrbook.c: addrbook.fm
$(XFM) -compile addrbook.fm \
    -cinclude 'addrbookApp.h' \
    -preserve \
```

```
-cfile addrbook.c -cname addrbook $(XFMCFLAGS)
```

If you set the options for the C code generation before saving the makefile, the rule in the makefile will reflect the options that you set.

We suggest that, in general, and in this manual, you set your C code options as desired, without setting the Save C Code toggle, and use the makefile to generate the C code for the fm files.

If you agree, open the C code options box, and clear the Save C Code toggle.

CHAPTER 12: FACE Constructs

With the interface basics finished, you can proceed to more advanced FACE programming. To build the address book proper, you will need in particular to use the two FACE constructs for structures and arrays. You will represent the address book as an array of structures.

12.1 Structures

You declare a structure in FACE similarly to the way you would in C, but there are some subtle differences. First, we will look at the declaration:

```
struct tag [ C-tag ] {  
    type name;  
    ...  
};
```

Here are the differences from C:

- m You *must* provide the structure name *tag*. FACE uses this name as the type of a *pointer* to the structure. In FACE, you always reference structures through a pointer. You never access a structure directly.
- m There is a second optional tag. This is the name of a C type which refers to a pointer to the structure, if it differs from *tag*. You may specify this name as a string, and you have to if it includes special characters.

Be careful of the difference in type use between C code and the resource data base. Typically, a structure's name in C code refers to a pointer to that structure when used as a type in the resource data base. For example, the name **FontStruct** refers to the structure in C, but a pointer to the structure in the resource data base. FACE tends to follow the resource data base conventions.

- m Only simple *type/name* pairs are allowed; you may not specify composite types such as array or structure definitions as fields, nor may you specify more than one field name on the same declaration. To declare an array field in a structure, use the type **FaceArray**.
- m You may not declare a variable at the same time you declare the structure. FACE only lets you declare pointers to structures, not the structures themselves. You can *allocate* a structure to point to by calling the function **new_struct** with the pointer type name as its argument.

So, what will our address book structure look like? It will contain the five fields that the user supplied in the address editor dialog box:

```
struct Address {  
    String name;  
    String street;  
    String city;  
    String state;  
    String zip;  
};
```

How do we use this structure? We use the type name **Address** to declare pointers to it and the function **new_struct** to allocate the structure. Furthermore, we always refer to fields using the `->` operator. This reminds us that we always refer to structures through pointers, and it eliminates confusion with the `.` operator that we use for referencing the widget hierarchy. Here is some FACE code using the structure **Address**:

```
Address a = new_struct ("Address");  
a->name = "Jean-Michel Dupont";  
a->street = "99 Bedford Street";  
a->city = "Boston";  
a->state = "Massachusetts";  
a->zip = "02111";
```

12.2 Declarations

In the previous code, when we declared the type of variable **a**, we assigned its value at the same time. In C parlance, we would say we are defining the variable, but in FACE, the distinction means little, since we could repeat the operation later, and the effect would be the same as assigning the variable again. However, by declaring the variable with a type, we have fixed the type of that variable: the variable is *statically* typed. If you want to use a structure pointer to access the fields within the structure, you must declare it this way (with or without assigning it) before referring to one of its fields.

Another reason the declaration/definition distinction is less important in FACE is that you do not need to declare a variable before using it. For example, you can say

```
w = self;
```

without declaring **w** first. In this case, **w** takes the type of the expression to the right of the `=` operator: it is *dynamically* typed.

The other aspect of variable declarations is scope. Up to now, we have only declared variables for use in a particular script. If we want to use the same variable in more than one script, we must declare the variable to be *global*. For example:

```
global TimeOfDay;  
TimeOfDay = parent.time:value;  
global Int NumberOfItems;  
NumberOfItems = parent.list:itemCount;
```

The FACE keyword `global` indicates the global scope of the variable. You may include the type of the variable if you want it to be statically typed. Otherwise it is dynamically typed. Variables which are not declared `global` are local. They cannot be referenced in other scripts. The keyword `local` exists to declare local variables. Most of the time you do not need to use the `local` keyword in FACE declarations.

You cannot make a scope declaration and assignment to a variable at the same time. Thus, while lots of people might want to say:

```
global Widget TopOfTheMorning = root;
```

FACE will not allow this. You must separate this into two statements:

```
global Widget TopOfTheMorning;  
TopOfTheMorning = root;
```

All variables in FACE scripts are permanently allocated, even local ones, so they retain their values between calls to the same scripts. This is one reason why you may want to declare a local variable. Variables are initialized to zero, so you can use this fact in your script to execute some code only once

```
local already;  
if (!already) {  
    already = True;  
    /*  
    * execute this code only once  
    */  
}
```

12.3 Arrays

The FACE language handles arrays of one-dimension only. However, you may use any type you want to index the array, so you may declare associative arrays in FACE. When you declare an array, you declare the type of its index, not its range, in the square brackets;

```
Address Book[Int];
```

In this declaration, we declare an array **Book**, each element is a structure pointer of the type **Address**, and we index the array with integers.

As with structures, this declares **Book** to point to an array, without actually allocating an array. FACE provides three functions to allocate arrays:

new_table()

Return an allocated associative array. If the type of the index is a pointer, the value of the pointer determines the element of the array, not what it points to. The only exception to this is the index type **String**, which FACE recognizes as a zero-byte-terminated string, as in C. In this case, FACE associates elements in the array with the value of the string pointed to by the index.

new_array()

Return an allocated linear array. The type of the index must be a type convertible to integer. In this case, indexing takes place as in C, starting with zero.

new_c_array(address, count)

Map an array in the C programming language into a FACE array. The *address* is the address of the array, and *count* is the number of elements in the array.

For the first two functions, FACE takes care of all memory management for the array. It allocates additional space when needed. In the last function, memory for the array is fixed when the array is allocated.

In the first two arrays, the array has a default value, which FACE returns when you try to access an item which is not yet allocated. In an associative array, FACE returns the default value when you try to read an element in which you have not yet written anything. In a linear array, FACE returns the default value when you try to read an element past the last element in the array, that is, the element written with the largest index. Normally, the default value is 0, but you may specify a different default value when you declare the array:

```
Address Book[Int, NULL];
```

This declares the default value for the array **Book** to be **NULL** instead of zero. (Of course, on many implementations, these values are the same.)

Note: Linear arrays only return the default values when accessing elements past their last element. Reading an unwritten element before the last element in a linear array always returns zero.

12.4 The Address Book

You shall now put your newly acquired knowledge of the FACE programming language to use. First, you will declare the address book array. Since you will need to refer to this array in more than one place in your interface, you should place this declaration in a separate file. Use your favorite text editor to place the following in a file named **addrbook.inc**:

```
struct Address {
    String name;
    String street;
    String city;
    String state;
    String zip;
};
global Address Book[Int, NULL];
```

This declares the structure **Address** and an array of pointers **Book** which contains our address book.

Now you can use this file in your interface:

1. If you are not already doing so, launch an XFaceMaker session, and edit the file **addrbook.fm**.
2. Select the **add** pushbutton in the address editor dialog box.
3. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
4. Press the **Callbacks** button in the resource window.
5. Select the **activate** button in the **Callbacks** menu.

6. Enter the following script in the text widget:

```
include "addrbook.inc";
list = root.xmlMainWindow0.xmlBulletinBoard0.xmlList0;
n = list:itemCount;
if (!Book)
    Book = new_array();
if (!Book[n])
    Book[n] = new_struct ("Address");
Address a = Book[n];
a->name = parent.name:value;
a->street = parent.street:value;
a->city = parent.city:value;
a->state = parent.state:value;
a->zip = parent.zip:value;
xms = FmCreateXmString (a->name);
XmListAddItemUnselected (list,xms,0);
XmStringFree (xms);
```

7. Press the **OK** button.

We will explain the script in detail, then you can try it out.

```
include "addrbook.inc";
```

This line includes the file **addrbook.inc** we created to hold the declaration of the **Address** structure and the address book array.

```
list = root.xmlMainWindow0.xmlBulletinBoard0.xmlList0;
n = list:itemCount;
```

The variable **list** refers to the **XmList** widget we created in the main window. The variable **n** contains the number of items in the list.

```
if (!Book)
    Book = new_array();
```

Allocate the array if we have not already done so.

```
if (!Book[n])
    Book[n] = new_struct ("Address");
```

Allocate a structure for the *n*th item, if we have not already done so.

```
Address a = Book[n];
```

Use the variable **a** to refer to the structure pointed to by the *n*th element in the array. This is for notational convenience.


```
a->name = parent.name:value;  
a->street = parent.street:value;  
a->city = parent.city:value;  
a->state = parent.state:value;  
a->zip = parent.zip:value;
```

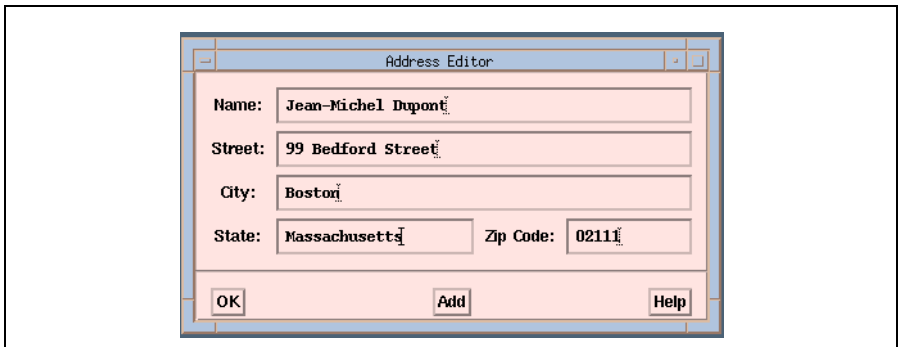
Assign the structure fields by getting the values of the **value** resource in the various text field widgets in the dialog box. When you get the value of the **value** field of an **XmText** or **XmTextField** widget, the widget returns the value in a dynamically allocated string, so we do not have to copy the string to store it in the array.

```
xms = FmCreateXmString (a->name);  
XmListAddItemUnselected (list,xms,0);  
XmStringFree (xms);
```

We create an **XmString** from the name so that we can add the name to the list. After we add it to the list, we free the string, since **XmListAddItemUnselected** copies the **XmString** given to it.

Now, try out the script:

8. Set the **Try** toggle in the main window if it is not already set.
9. If the address box is not visible, open it by selecting the **Edit** item in the **Address** menu of the interface.
10. Enter values in the text fields, for example as in the figure below.
11. Press the **Add** button.



The list in the main window now contains a single item, Jean-Michel Dupont. Unfortunately, on some platforms, the list changed size to accommodate its new contents. We will deal with this problem in the next chapter.

CHAPTER 13: Advanced Editing Techniques

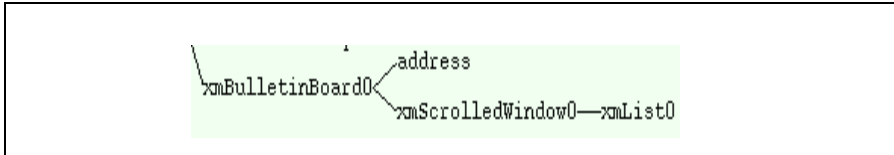
In the last chapter, we discovered that the **XmList** widget automatically adjusts its size according to its contents, on some platforms. One way to deal with this problem is to change its resources so it no longer does that. However, the more common way is to place the list in a scrolled window. That way, the list can still change size, and if it gets too big, the scrolled window will automatically add scroll bars to let the user scroll the contents of the list.

13.1 Reparenting

It seems that we no longer want the list to be a direct child of the bulletin board in the main window, but rather a child of a scrolled window inside the bulletin board. We must move the list widget within the widget hierarchy. People often call this process “reparenting”, since it changes the parent of a widget (or widget sub-tree). You can do this within XFaceMaker using the “cut and paste” editing functions:

1. If you are not already doing so, launch an XFaceMaker session, and edit the file **addrbook.fm**.
2. Clear the **Try** toggle button in the main window if it is set.
3. Make sure the **Composite** button in the **Toolkit** menu is set.
4. Select the list widget.
5. Click on the **XmScrolledWindow** icon.  When the cursor changes to a crosshair, create a scrolled window directly over the list in the main window, and taking up more or less the same space. If the scrolled **XmScrolledWindow** window obscures some or part of the of the list, it is okay. In the widget tree, you will see a child, **xmScrolledWindow0**, created in the bulletin board.
6. With the cursor in the main window of your interface, type **Control-X**. This cuts the list, which is still selected, into XFaceMaker’s cut buffer. The scrolled window does not have any shadowing, so it is not visible in the bulletin board. However, you will still see it in the widget tree.
7. With the cursor still in the main window, type **Control-P** to paste the list.

8. Move the cursor over the scrolled window, and click the **Select** mouse button to place the list in the scrolled window. If you are not sure how far the scrolled window extends in the main window, you can click the **Select** button with the cursor over the name of the scrolled window in the widget tree. In any case, you should check the widget tree to confirm that the list really did get placed as a child of the scrolled window, which should look something like the figure below.



13.2 Changing Class

When building the address editor, we explored the convenience of using forms to control the layout of a dialog box. We can use a form to fix the scrolled window's size in the main window. This will also have the advantage that the user can change the size of the main window, and the window's contents will adjust. To do this, we need to change the bulletin board in the main window to a form:

1. Select the bulletin board in the interface's main window.
2. Make sure the **Composite** button is selected in the **Toolkit** menu.
3. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
4. Click on the **XmForm** icon. When the cursor changes to a crosshair, move it over the **Class** text field in the resource window, and press the **Select** mouse button. The text field contents changes to **XmForm**.
5. Change the **Name** field to **form**, in order to avoid confusion.
6. Press the **Rebuild** button in the resource window.

13.3 Dragging Resources

Often you want to apply resource values in one part of your interface to another part. You can do this by dragging the resources from one widget to another. We will do this to set the margin width and height of the form in the main window.

1. Select the form in the address editor dialog box.
2. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
3. Adjust the resource list so that the **marginHeight** and **marginWidth** resources are visible.
4. Use the OSF/Motif extended selection method to select the two resource names: move the cursor over the **marginHeight** resource name, press the **Select** mouse button and hold it down, drag the cursor until it is over the **marginWidth** name, then release the **Select** button. Both names should change color, indicating their selection.
5. Drag the resources to the form widget in the main window: move the cursor over one of the selected resource names, press the **Drag** mouse button and hold it down, drag the cursor until it is over the name of the form in the widget tree, then release the **Drag** button. While dragging the resources, the cursor will change to the selected resource cursor.
6. Select the form in the main window.
7. Scroll the resource list so that the **marginHeight** and **marginWidth** resources are visible. Their value icon has changed, and their values have changed to **10**, the same as the other form.
8. Scroll the resource list so that both the **marginHeight** and **verticalSpacing** resources are visible. You may have to increase the height of the resource window to do this.
9. Drag the **marginHeight** resource value to the **verticalSpacing** resource: move the cursor over the *value* (10) of the **marginHeight** resource, press the **Drag** mouse button and hold it down, drag the cursor until it is over the **verticalSpacing** resource, then release the **Drag** button. While dragging the resource value, the cursor will change to the resource value cursor. The value of the **verticalSpacing** resource changes to 10, but the change is *not applied*.
10. Press the **Rebuild** button.

13.4 Adjusting the Main Window

We will now attach the label and scrolled window within the form. Since we want the scrolled window to change its height with the main window, it will have to be attached on both the top and bottom. Both widgets will have to be attached on the left and right, since we want both to adjust their width with the main window:

1. Select the label inside the new form.
2. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
3. Set the **Auto position** and **Auto size** toggles.
4. Click **Rebuild**.
5. Attach its top, left, and right sides to the form.
6. Select the **leftAttachment** and **rightAttachment** resources. To select a non-contiguous collection of resources in the resource list, select one resource, then select subsequent resources with the **Control** key pressed. (See OSF/Motif documentation for details on the extended selection process.)
7. Drag the selected resources to the scrolled window widget in the widget tree.
8. Select the scrolled window in the new form.
9. Attach its top to the widget **address** and its bottom to the form.

We need to update the activate callback of the **Add** button in the dialog box to reflect the new configuration of the main window:

10. Select the **add** button in the Address Editor dialog box.
11. Select its **activateCallback** resource for editing.
12. Find the line that looks like this:

```
list = root.xmlMainWindow0.xmlBulletinBoard0.list;
```

13. Delete the reference to the list widget, that is, everything beginning with **root** to just before the semi-colon. Leave the text cursor just before the semi-colon.
14. Press the **Select** push button at the bottom of the callback editor dialog box.

15. When the cursor has changed to , move it over the list widget (either in the interface or the widget tree), then click the **Select** mouse button. The line should now read:

```
list = parent.parent.parent
      .xmMainWindow0.form.xmScrolledWindow0.xmList0;
```

16. Press the **OK** button.

Now you can test your new script.

17. Set the **Try** toggle button in XFaceMaker's main window.
18. Select the **Edit** button in the **Address** menu in your interface to bring up the address editing menu.
19. Fill in the dialog box and press the **Add** button.

Now, the new name is added to the list without the list changing size, even if it did so previously on your platform.

13.5 Displaying the Address

We will now add behavior to the list widget so that it will display the address associated with a name when the user selects one. To do this, we must choose the selection policy of the list. The default is browse selection, which is suitable for selecting a single item at a time in the list. See the OSF/Motif documentation for a complete description of the various selection policies available.

Once we have chosen the selection policy, we must write the appropriate callback. In this case, the **browseSelectionCallback**:

1. Select the list widget in the interface's main window.
2. Open a resource window if one is not already open, and make sure its **Current selection** toggle is set.
3. Scroll the resource list until the **selectionPolicy** resource is visible. Its value should be `browse_select`. If not, change it by pressing the ... button and selecting the `browse_select` item in the list.
4. Scroll the resource list until the **browseSelectionCallback** resource is visible. Edit its value by pressing the ... button and enter the following script:

```
include "addrbook.inc";
if (self:selectedItemCount > 0) {
    XmListCallback cb = $;
    Address a = Book[cb->item_position - 1];
    String buf;
    if (!buf)
        buf = XtMalloc (1000);
    sprintf (buf, "%s\n%s\n%s, %s %s", a->name,
        a->street, a->city, a->state, a->zip);
    xms = FmCreateXmString (buf);
    parent.parent.address:labelString = xms;
    XmStringFree (xms);
}
```

5. Press the **OK** button.

We will take some time here to explain the script in detail.

```
include "addrbook.inc";
```

Include the declarations for the address book structure and array.

```
if (self:selectedItemCount > 0) {
```

This is a sanity check to make sure that something in the list is selected. With the browse selection policy, there is always something selected when the callback gets called. However, this is not always true with other policies. So we have this check here to encourage good programming habits with **XmList** widgets.

```
    XmListCallback cb = $;
```

The special symbol \$ in a callback refers to the call data passed to the callback by the widget. Its type in C is **XmListCallbackStruct**. FACE declares a type for a pointer to this structure as **XmListCallback**, that is, the name with the **Struct** removed. It declares similar types for all OSF/Motif callback structures. The symbol \$ is not a typed variable, so we must copy its value into a statically typed variable, **cb**, to refer to fields within the structure.


```
Address a = Book[cb->item_position - 1];
```

We get the address book entry for the given item. Since we always add items at the end of the list, the position of the item indicates its index in the array. However, **XmList** positions start at one, and our array starts at zero, so we must subtract one. The field **item_position** contains the position of the selected item in the list. See the OSF/Motif documentation of the **XmList** widget for details of the **XmListCallbackStruct** fields.

```
String buf;  
if (!buf)  
    buf = XtMalloc (1000);
```

The variable **buf** holds a buffer where we format the address label from the individual field entries. If it has not been initialized, we allocate space for it. Note that if we had simply said

```
String buf = XtMalloc (1000);
```

It would allocate space *every* time we call this callback, which would cause a memory leak.

```
sprintf (buf, "%s\n%s\n%s, %s %s", a->name,  
        a->street, a->city, a->state, a->zip);
```

We use **sprintf** to format the address from its components.

```
xms = FmCreateXmString (buf);  
parent.parent.address:labelString = xms;  
XmStringFree (xms);  
}
```

We finish the conditional code by setting the label string of the address widget. Since the **labelString** resource is an **XmString**, we must create the **XmString** from the string in our buffer. The widget copies the **XmString** given to it, so we can free the **XmString** after setting the label.

Now that we know what the script should do, we can test it:

6. Set the **Try** toggle in XFaceMaker's main window.
7. Add an entry to the list using the address editor box.
8. Select the entry you added.

What happened? Probably, you will only see the middle line of the address, since the original label only had one line. The form does not let the label increase its height. The easiest way to solve this problem is to reserve space for the address by giving it an initial label string:

9. Select the address widget.
10. Set the **Auto Size** and **Auto Position** toggles
11. Select the **labelString** resource, and press the ... button.
12. Enter the following three lines of text as the value:

Name
Street
City

You should not put a newline at the end of the last line.

13. Press the **OK** button.
14. Set the **Try** toggle if needed.
15. Add an entry to the list using the address editor.
16. Select the entry you added. This time, the complete address appears in the address widget.
17. Save your interface in `addrbook.fm`.

CHAPTER 14: The File Selector

Now that you have provided the user with the basic capabilities to enter and edit addresses, you will want to add the commands to read and write the address list to a file. The user will select the file for this purpose with a file selector.

14.1 Creating a File Selector

Typically, you will want to place the file selector in a `dialogShell`. The OSF/Motif file selector is subclassed from the **XmBulletinBoard** widget, so it interacts with the **XmDialogShell** widget and, through the `dialogShell`, the window manager. Here is how you can create the dialog box for the file selector:

1. If you are not already doing so, launch an `XFaceMaker` session, and edit the file **addrbook.fm**.
2. Clear the **Try** toggle button in the main window if it is set.
3. Make sure the **Shell** button in the **Toolkit** menu is set.
4. Click on the **XmDialogShell** widget icon. 
5. Drag a rectangle for the `dialogShell` that will contain the file selector. The rectangle should be vertically oriented, and about the same size as your interface's main window, or about two-thirds the width and height of `XFaceMaker`'s main window. The top left corner of the rectangle must not be over any window of your interface. The `dialogShell` appears in the widget tree as a child of your `applicationShell`.
6. Click on the **Dialog** button in the **Toolkit** menu.
7. Click on the **XmFileSelectionBox** widget icon. 
8. Move the cursor over the `dialogShell` you just created, and click the **Select** mouse button.
9. If you want, select the file selection box widget for editing and choose its fonts by changing the **buttonFontList**, **labelFontList**, and **textFontList** resources. In this example, we use Helvetica bold 12 pixels for buttons and labels, and Courier medium 12 pixels for text.
10. Hide the file selector. Click on the **Hide** icon in the main window. Move the cursor (picture) over the file selection dialog box, then click the **Select** mouse button.

14.2 More Menu Editing

You will also need to add **Open** and **Save As** buttons to the **File** menu:

1. Open the menu editor by selecting the **Menu Editor** item in the **Windows** menu.
2. Select the exit button for editing. You can use this button as a basis for creating your new buttons.
3. Change the label to **Open . . .** and the mnemonic to **O**. We will change the call-back later.
4. Delete the accelerator text to remove the accelerator specification.
5. Press the **Add** button.

Note that the menu editor places the new widget after the currently selected widget. (If there were none selected, it would have placed it at the beginning of the menu.) Often you want to change the position of a menu item after creating it. You can do this by dragging the widget representation in the menu editor:

6. Move the cursor over the **Open** button you just created, press the **Select** mouse button and hold it down.

7. Drag the  cursor over the **Exit** button, then release the **Select** mouse button.

The **Open** button moves to before the **Exit** button. When you drag a menu item to another item before it in the menu, XFaceMaker places it before that item. Similarly, if you drag a menu item to another item after it, XFaceMaker moves the dragged item after the item where you moved. Try this by moving the **Open** button after the **Exit** button.

If you look at the widget tree, you will see that the widget tree lists the widgets in the same order that they appear in the menu. The widget tree shows the order that the widgets are created, which is also the order that they appear in the menu.

We saw in the **XmForm** widget one way that the creation order of widgets is important. Menus (and row columns in general) are another way. We have seen before that you can drag a widget in the widget tree to change its order within the interface hierarchy. Try dragging the open or exit button in the widget tree to change the order of items in the **filePulldown** widget. Be sure you finish with the **Open** button before the **Exit** button in the menu.

8. Select the **Open** button in the **File** menu in your menu editor.
9. In the menu editor, change the label to **Save AS . . .** and the mnemonic to **A**.

10. Press the **Add** button.

This time, XFaceMaker places the **Save As** button after the **Open** button, so you do not need to move it. You will, however, want to place a separator between the **Save As** and **Exit** buttons, since the OSF/Motif style guide recommends separating “dangerous” operations within a menu.

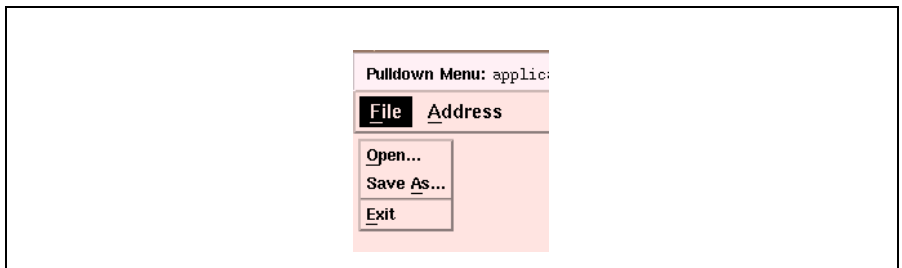
11. Select the **XmSeparator** icon in the menu editor. The bottom part of the dialog becomes a list of separator types.

12. Change the widget name to **line**. Since there is no label, XFaceMaker does not give the widget a default name.

13. If not already selected, select the toggle for **Etched-In Shadow**.

14. Press the **Add** button.

Your file menu in the Menu Editor should look like the figure below.



14.3 Opening the File Selector

Now we will add the callback scripts for opening the file selector. Start by editing the activate callback for the **Open** button in the **File** menu of your interface. You can do this either in the menu editor or in the resource box. Once you are editing the script, remove the line copied from the **Exit** button, and enter this script:

```
XmString label;
if (!label)
    label = FmCreateXmString ("File to open:");
Widget w = parent.parent.parent.parent.xmDialogShell2.
    xmFileSelectionBox0;
w:selectionLabelString = label;
w.parent:title = "Opening Address Book";
FmShowWidget (w);
```

This script sets the label of the file name in the file selector, as well as its title. Since you are using this file selector for both opening an address book and saving one, you should indicate which operation the user is in the process of requesting. Here is a detailed explanation of the script.

```
XmString label;
if (!label)
    label = FmCreateXmString ("File to open:");
```

The variable `label` contains the **label** string for the file selection. We require an **Xm-String** for the label, and in order to avoid creating it and freeing it each time the user presses this button, we create it only once, and save it in a local variable. Remember that local variables are permanently allocated, so they retain their value between callbacks, and they are initialized to zero. So the first time this script gets called, it will set **label**. After that, **label** will retain its (non-zero) value, so the script will no longer set its value.

```
Widget w = parent.parent.parent.parent.xmDialogShell2.
    xmFileSelectionBox0;
```

Store the widget ID of the file selector in `w`. You can use the **Select** button to retrieve the path of this widget.

```
w:selectionLabelString = label;
```

Set the file selection label string in the file selector.

```
w.parent:title = "Opening Address Book";
```

Set dialogShell title, using the title resource in the dialogShell.

```
FmShowWidget (w);
```

Show the dialog box.

You should set the activate callback script for the save button in a similar manner, but with different title strings. Try it, then compare your script to the script below:

```
XmString label;  
if (!label)  
    label = FmCreateXmString ("File to save:");  
Widget w = parent.parent.parent.parent.xmDialogShell2.  
    xmFileSelectionBox0;  
w:selectionLabelString = label;  
w.parent:title = "Saving Address Book";  
FmShowWidget (w);
```

CHAPTER 15: Application Functions

In order to open and save address books, your interface will need to interact with application code. It does this by calling functions in your application. You declare these functions in your FACE scripts, but you write them in separate C source files. To make your application, you combine your C code with your interface in either compiled or interpreted form.

15.1 Function Declarations

In this example, the application will provide one function for reading an address book and three for writing. You would declare them in FACE like so:

```
function Boolean ReadAddressBook(String file) = True;
function Boolean OpenAddressBook(String file) = True;
function None WriteAddress
    (String name, String street,
     String city, String state,
     String zip);
function None CloseAddressBook();
```

Function declarations in FACE are similar to ANSI C, with the following differences:

- The keyword **function** always introduces a function declaration or definition.
- You may specify a *default value* to use in XFaceMaker when testing the script. XFaceMaker uses this value as the value returned by the function if you have not yet provided an actual function to use. This is useful when you first start building your interface and in prototyping because you can usually test most of your script with default values without writing any C code until necessary.
- By convention, the FACE type **None** is used for functions that do not return a value. You may also specify the C type **void**.

15.2 The OK Callback

The file selector has a resource called **okCallback** which it calls when the user presses the **OK** button. Select the file selection box widget for editing, then enter the following script for its **okCallback** resource:

```
xms = self:dirSpec;
name = FmGetString (xms);
XmStringFree (xms);
Boolean done = False;
global Boolean SavingAddressBook;
if (SavingAddressBook) {
function Boolean OpenAddressBook(String file) = True;
if (OpenAddressBook (name)) {
    Int n = parent.parent.XmMainWindow0.form.
        XmScrolledWindow0.XmList0.itemCount;
    for (i = 0; i < n; i = i + 1) {
        include "addrbook.inc";
        Address a = Book[i];
        function None WriteAddress(String
            name, String street,
            String city, String state,
            String zip);
        WriteAddress (a->name,a->street,
            a->city,a->state,a->zip);
    }
    function None CloseAddressBook();
    CloseAddressBook();
    done = True;
} else {
    printf ("Cannot open file for saving.\n");
}
} else {
function Boolean ReadAddressBook(String file) =
    True;
if (ReadAddressBook (name)) {
    done = True;
} else {
    printf ("Cannot open file for reading.\n");
}
}
XtFree (name);
if (done)
    FmHideWidget (self);
```

This script either reads in an address book or writes one out, depending on a global **FACE** variable. We consult this global variable because we use the same file selection box for both opening and saving an address book.

Here is a detailed explanation of the script:

```
xms = self:dirSpec;
name = FmGetString (xms);
XmStringFree (xms);
```

The **dirSpec** resource contains the name of the file selected by the user. It is an **XmString**, so we call the function **FmGetString** to extract the normal zero-byte terminated string. At this point, we no longer need the **XmString**, so we free it.

```
Boolean done = False;
```

The variable **done** will indicate whether we successfully performed the operation requested by the **OK** button. If true, we will close the dialog box, otherwise it will remain open.

```
global Boolean SavingAddressBook;
if (SavingAddressBook) {
```

The global variable **SavingAddressBook** is **True** if we are saving an address book. If it is **False**, we are reading one. We handle the saving case first.

```
function Boolean OpenAddressBook(String file) = True;
if (OpenAddressBook (name)) {
```

We declare the application function **OpenAddressBook**, then call it with the selected file name as the argument. If it returns **True**, the open succeeded and we can write address entries to the file.

```
Int n = parent.parent.xmMainWindow0.form.
xmScrolledWindow0.xmList0:itemCount;
```

We determine the number of entries by getting the size of the list widget from its **itemCount** resource. You can get the path of the widget using the **Select** button in the callback editor.

```
for (i = 0; i < n; i = i + 1) {
```

Then we loop, indexing over the entries in the address book. In **FACE**, we do not have to declare the variable **i**, and we cannot use the **++** operation to increment.

```
include "addrbook.inc";
```

We include the declaration for the address book. FACE does not distinguish between declarations, directives, and statements. All statements are created equal, so you can use any statement at any point in your script.

```
Address a = Book[i];
```

We store the indexed entry in the variable **a**.

```
function None WriteAddress
    (String name, String street,
     String city, String state,
     String zip);
WriteAddress (a->name,a->street,a->city,a->state,
              a->zip);
}
```

We declare the function **WriteAddress**, then call it with the fields of the address book entry and terminate the loop.

```
function None CloseAddressBook();
CloseAddressBook();
done = True;
```

After the loop, we close the address book and set **done** to indicate that we completed successfully.

```
    } else {
        printf ("Cannot open file for saving.\n");
    }
```

If we did not succeed in opening the address book file for writing, we report an error. Later, we will use a message box to report this error, but for now, we just print a message.

```
    } else {
        function Boolean ReadAddressBook(String file) = True;
        if (ReadAddressBook (name)) {
```

If we were not saving a file, but, instead, opening one for reading, we declare and call the application function **ReadAddressBook** with the selected file name.

```
        done = True;
    } else {
        printf ("Cannot open file for reading.\n");
    }
}
```

If we successfully read the address book, we set the flag **done**, otherwise we report an error.

```
XtFree (name);
```

Free the string returned by **FmGetString** since we no longer need it.

```
if (done)  
    FmHideWidget (self);
```

If we were successful, close this window.

15.3 Some Loose Ends

For all of this to work together, we need to clean up a few loose ends.

1. You should provide a cancel callback for the file selector. The widget calls this callback when the user presses the **Cancel** button. In our case, it will simply close the window. Edit the **cancelCallback** resource for the file selector and enter the following script:

```
FmHideWidget (self);
```

2. Set the **dialogStyle** resource of the file selector to **dialog_full_application_modal**. This makes the file selector modal for the entire application, so that the user cannot use any other window in the application until the file selector is closed.
3. You should set the appropriate value for the global FACE variable **SavingAddressBook**. Add these lines to the end of the activate callback of the **Open** button in the **File** menu in your interface:

```
global Boolean SavingAddressBook;  
SavingAddressBook = False;
```

4. Similarly, add these lines to the end of the activate callback of the **Save As** button:

```
global Boolean SavingAddressBook;  
SavingAddressBook = True;
```

You can try out your interface, using the **Open** and **Save As** buttons in the file menu to open the file selector, then pressing either **OK** or **Cancel** to close it. Since we have declared the application functions without actually defining them, XFaceMaker uses the default values for the functions. Thus, when we press the **OK** button in the file selector, XFaceMaker executes our script. If we see no error messages when we press the **OK** button, we can feel a little confident that our script will act properly, assuming that the real application functions live up to their specifications.

CHAPTER 16: Application C Code

With the interaction between your FACE code and application code complete, you can now write the C code for your application functions. We will start with the three functions used to save an address book, **OpenAddressBook**, **WriteAddress**, and **CloseAddressBook**. We leave the function **ReadAddressBook** for a later chapter.

16.1 Saving Files

Before you write your C code, you should save the files that reflect your recent changes. Naturally, you want to save your interface file **addrbook.fm**, and you will want to save some of your application files, but not all of them. Since you may have modified the **Makefile**, and none of the changes you made affect it, you do not need to change it. You do need to change the others, though. You can do so as follows:

1. Select the item **Save** in the **File** menu in XFaceMaker's main window. This saves your interface file, **addrbook.fm**.
2. Select the item **Main** in the **Save Appli Files** sub-menu of the **File** menu. This saves your main program file, **addrbookMain.c**.
3. Select the item **Functions** in the **Save Appli Files** sub-menu. This saves your function definitions file, **addrbookApp.c**.
4. Select the item **Include** in the **Save Appli Files** sub-menu. This saves your function declarations file, **addrbookApp.h**.

Instead of steps 2 to 4, you could have saved all your application files, then restored the backed-up version of the **Makefile**, which XFaceMaker saves in **addrbook-Make~**.

How have these files changed? The main program file now has calls to let the XFaceMaker interpreter know about the application functions. The definitions file contains stubs that you can fill in to define the application functions. The declarations file declares them, so you can include it in other source modules if you want.

The application Makefile generated by XFaceMaker includes this header file in the C code generated from the .fm interface file.

16.2 Attaching Functions

The main change in the main program file is this new portion of code:

```
#ifndef CCODE
    FmAttachFunction("CloseAddressBook",
        (FaceFunctionPtr)CloseAddressBook, "None", 0);
    FmAttachFunction("ReadAddressBook",
        (FaceFunctionPtr)ReadAddressBook, "Boolean", 1,
        "String");
    FmAttachFunction("WriteAddress",
        (FaceFunctionPtr)WriteAddress,
        "None", 5, "String", "String",
        "String", "String", "String");
    FmAttachFunction("OpenAddressBook",
        (FaceFunctionPtr)OpenAddressBook,
        "Boolean", 1, "String");
#endif
```

These lines inform the FACE interpreter of your application functions by calling the XFaceMaker function **FmAttachFunction**. Its arguments are as follows:

```
FmAttachFunction(String name, FaceFunctionPtr function,
    String rtype, int nargs [, arg1type ...] )
```

name The name of the function when called from FACE. Normally, to reduce confusion, this should be the same as its C name.

function A pointer to the actual function.

rtype The FACE type of the returned value.

nargs The number of arguments. -1 indicates a variable number of arguments, and FACE will do no type checking of arguments.

arg1type ... The types of the arguments, one for each.

The code attaches all the functions declared in your interface, using the FACE function declarations to generate the arguments for the calls. It places the calls within `#ifndef CCODE`, since the C code version of the interface does not have a FACE interpreter, and the generated C code calls the application C code directly.

16.3 Filling in the Blanks

The function definition file, **addrbookApp.c**, only contains stubs for the function definitions. You have to write the code for the functions yourself. We provide that code for you here.¹

We use the UNIX standard I/O library to open the file and write to it. Because we will open, write, and close the file with separate function calls, we hold the file pointer in a static variable. We could have also returned the file pointer to the interface code, the interface code does not have any real need for the pointer.

The beginning of the file **addrbookApp.c** contains these lines:

```
#ifndef STANDALONE
#include <Fm.h>
#endif
```

The **Fm.h** file is included only if you have not specified that you want to generate stand-alone C code, i.e., you will link with the **Fm_c** library of convenience functions.

We add some code here so that it now looks like this:

```
#ifndef STANDALONE
#include <Fm.h>
#endif
#include <stdio.h>

static FILE *bookfile = 0;
```

This includes the declarations for the standard I/O library and declares our static file pointer. Although this is not strictly necessary, we explicitly initialize it to zero, to remind us of the fact that we will depend on the initial value of this variable in the upcoming code.

We will describe the functions in the order they appear in the file. **XFaceMaker** writes the definitions in the order they occur in its internal tables, which probably do not correspond to any logical ordering that you can determine from the interface file itself.

1. In these explanations we have stripped the text of any pattern information for better readability. If you have set the **Preserve User Code** toggle, your file will include pattern information (lines starting with `/*##`). Enter your code between the `/*## begin user ##*/` and `/*## end user ##*/` lines.

The function **CloseAddressBook** closes an open address book file. The stub looks like this:

```
#if NeedFunctionPrototypes
void CloseAddressBook()
#else
void CloseAddressBook()

#endif
{
}
```

Although not particularly useful in this case, XFaceMaker uses `#if NeedFunctionPrototypes` to determine whether or not it should specify ANSI-C style function definitions. Between the two curly braces { }, we add the following code:

```
if (bookfile != 0) {
    fclose(bookfile);
    bookfile = 0;
}
```

We check to see if the file is open before closing it, and we clear the pointer after closing it. Thus, the pointer will contain an address only when the file is open.

The function **ReadAddressBook** opens an address book file for reading, reads it, then closes it. We will get into the details of this later. For now, we will just provide a skeleton for the I/O portion of the code. Its stub looks like this:

```
#if NeedFunctionPrototypes
Boolean ReadAddressBook(String file)
#else
Boolean ReadAddressBook(file)
String file;
#endif
{
}
```

In this case, we can see the difference between classic C and ANSI-C function definitions. Note the definition uses the same parameter names that we used in the FACE declaration.

We add this code between the curly braces:

```
CloseAddressBook();
```

```
bookfile = fopen(file, "r");
if (bookfile == 0)
    return False;

/* read file here */

fclose (bookfile);
bookfile = 0;
return True;
```

We leave a comment reminding us where we will do the actual work. Also, we return **False** if we could not successfully open the file. Otherwise, we return **True**.

The function `WriteAddress` writes a single address to the file. We do this with the following C code:

```
if (bookfile != 0) {
    fprintf(bookfile, "Name = %s\n", name);
    fprintf(bookfile, "Street = %s\n", street);
    fprintf(bookfile, "City = %s\n", city);
    fprintf(bookfile, "State = %s\n", state);
    fprintf(bookfile, "Zip = %s\n\n", zip);
}
```

It gets the address from the parameters and writes them as simple statements of the type:

name = *value*

The *name* is the name of the field, and the *value* is its value.

The function **OpenAddressBook** opens an address book file for writing. Its code looks like this.

```
CloseAddressBook();
bookfile = fopen(file, "w");
return bookfile != 0;
```

Your file **addrbookApp.c** should now look something like the following (the timestamp will be different):

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          908964832
 */

/*
 * XFaceMaker 3.5 Application Functions Definitions.
 */

#ifdef STANDALONE
#include <Fm.h>
#endif
#include <stdio.h>

static FILE *bookfile = 0;

#ifdef NeedFunctionPrototypes
void CloseAddressBook()
#else
void CloseAddressBook()

#endif
{
    if (bookfile !=0) {
        fclose(bookfile);
        bookfile = 0;
    }
}

#ifdef NeedFunctionPrototypes
Boolean ReadAddressBook(String file)
#else
Boolean ReadAddressBook(file)
String file;
#endif
{
    CloseAddressBook();
    bookfile = fopen(file, "r");
    if (bookfile == 0)
        return False;
}
```

```
/* read file here */
fclose (bookfile);
bookfile = 0;
return True;
}
#ifdef NeedFunctionPrototypes
void WriteAddress(String name, String street, String city,
                  String state, String zip)

#else
void WriteAddress(name, street, city, state, zip)
String name;
String street;
String city;
String state;
String zip;
#endif
{
    if (bookfile != 0) {
        fprintf(bookfile, "Name = %s\n", name);
        fprintf(bookfile, "Street = %s\n", street);
        fprintf(bookfile, "City = %s\n", city);
        fprintf(bookfile, "State = %s\n", state);
        fprintf(bookfile, "Zip = %s\n", zip);
    }
}
#ifdef NeedFunctionPrototypes
Boolean OpenAddressBook(String file)
#else
Boolean OpenAddressBook(file)
String file;
#endif
{
    CloseAddressBook();
    bookfile = fopen(file, "w");
    return bookfile != 0;
}
```

16.4 Making the Application

You can make your application from a terminal session in the directory where you are building your interface. As explained earlier, to do this, you enter the command

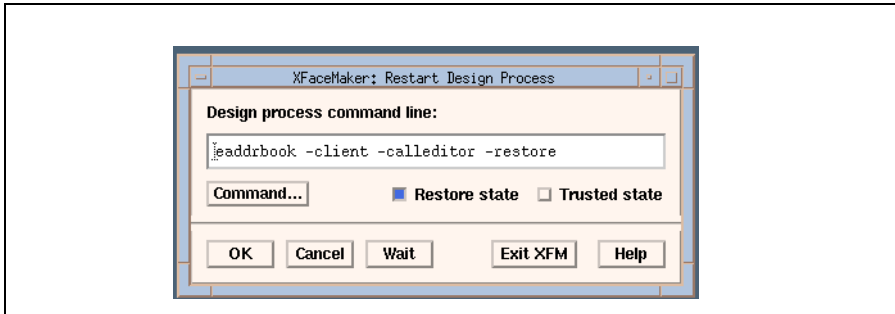
```
make -f addrbookMake
```

This makes two versions of your application. One with an interpreted interface and one with a compiled interface. You can make one linked with a complete XFaceMaker interface editor that is called **eaddrbook** and is of particular interest at this time, since we can use this program to test our application while we work on its interface. To make eaddrbook, enter the command

```
make -f addrbookMake eaddrbook
```

You can also ask XFaceMaker to launch the command to make this program by selecting the item **Build Appli** in the **Design Process** sub-menu of the **File** menu in XFaceMaker's main window. As a short cut, you can also pop up the design process menu by pressing the **Menu** mouse button over the icon portion to the left of the message box at the bottom of the main window.

After making the program **eaddrbook**, you can launch it by selecting the item **Launch Appli** in the **Design Process** menu. This opens the **Restart Design Process** dialog box. We explain the details of this box later.



At this point, you may simply press the **OK** button to confirm you want to restart the design process. XFaceMaker launches **eaddrbook** as the design process, so it will have all of you application functions available for use from FACE scripts while you build your interface.

You can verify the presence of your functions by entering Try mode:

1. Set the **Try** toggle button in XFaceMaker's main window.
2. Select the **Edit** item in the **Address** menu of your interface's main window. This brings up the address editor dialog box.
3. Add some addresses to the address list.
4. Select the **Save** item in the **File** menu of your interface.
5. After the file selector pops up, modify the file name text field to contain the name of an address book file, or select a file from the list of files. If you select an existing file, be sure to select a file whose contents you will not need in the future, since saving the address book in that file will overwrite its contents.

You can use a text editor such as **vi** or **Wx** to verify that the addresses really were written to the file you entered. This means that your application functions were called.

CHAPTER 17: The Dual Process Architecture

What exactly did we just do? We compiled a new version of XFaceMaker and launched it. Yet we did not exit XFaceMaker. Even the windows remained in place. This illustrates the dual process architecture of XFaceMaker's editing session. When you launch XFaceMaker, you in fact launch two processes:

- the XFaceMaker process that contains all of XFaceMaker's windows that you interact with to build your interface
- the design process that contains the interface that you are building.

In order to call your application functions from FACE scripts while you edit your interface, you need to include these functions in the design process, but not the XFaceMaker process. So you can relaunch a modified version of your design process while the rest of XFaceMaker remains in place, waiting to connect to it.

Another advantage of the design process is that it lets you recover from fatal errors that occur while designing your interface. Since XFaceMaker uses genuine Xt and Motif calls to set widget resources, you have the full capability of these libraries, including the capability to cause fatal errors. However, a fatal error will only affect the design process, so the XFaceMaker process remains intact. XFaceMaker detects that the design process has terminated and asks you if you want to restart it. You can also explicitly request the XFaceMaker process to restart the design process by selecting the **Restart** item in the **Design Process** menu.

17.1 Design Process Status

When you launch XFaceMaker, or when you relaunch the design process, you may see messages flash in XFaceMaker's status line. After the connection is complete, and both processes are ready, you see the message

XFaceMaker/UIMS is ready.

Beneath the XFaceMaker logo next to the message box in the main window are three status indicator icons related to XFaceMaker's process communication. The first icon indicates whether or not the design process has saved the current session's latest "state". The indicator shows one of the two icons:



design state saved



design state not saved

The design process periodically saves the state of your session, so that you can recover your session in case of a fatal error in a resource setting or FACE script. You can also request the design process to save its current state by selecting the **Save State** item in the **Design Process** menu. When you request XFaceMaker to restart its design process, it normally restores it to its last saved state.

The next icon indicates the state of XFaceMaker’s dual process system. There are three possible icons:



connected



disconnected



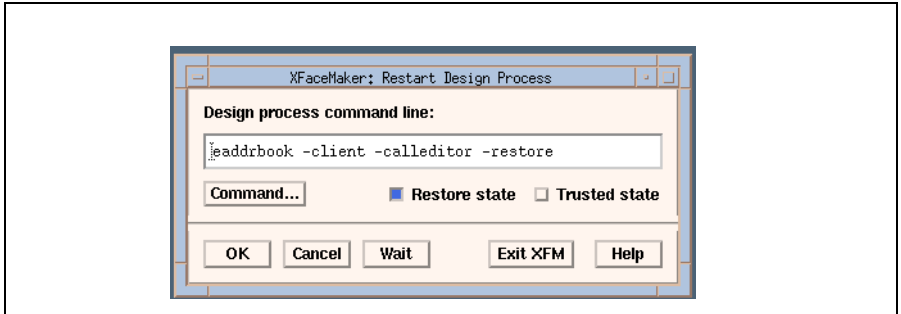
single process

Normally, you will see the “connected” icon, indicating that the XFaceMaker process is connected with the design process. If the design process is not running or not yet connected, you will see the “disconnected” icon.

You can also run XFaceMaker in single process mode, where XFaceMaker performs both sides of the editing session in the same process. However, if a fatal error occurs which would have caused your design process to fail, you will lose your entire editing session, and you will have to relaunch XFaceMaker. You should never have to run XFaceMaker in single process mode.

17.2 Restarting the Design Process

When you ask XFaceMaker to restart the design process, it displays a dialog box to confirm the request.



The dialog box has a text field that contains the command that XFaceMaker runs to start the design process. Normally, the command name is **xfm**, but it may also be your application process name, such as **eaddrbook**. The command also specifies the arguments for the command, typically including one or more of the following:

- client** Launch XFaceMaker as a design process only.
- calleditor** The design process will attach to an editing session.
- restore** Restore the design process to the last restored state.
- trusted** If restoring, use the last trusted state instead of the last restored state.

The toggle buttons **Restore state** and **Trusted state** ask the design process to restore its state and whether to use the last stored state or the last trusted state. The last trusted state is the last state saved by the design process in normal operation. When the design process receives a fatal error, it tries to save its state before exiting. If this state includes whatever caused the fatal error, XFaceMaker will not be able to restore using this state, and you must use the trusted state, even though it may not contain some of your latest changes.

The buttons at the bottom of the box act as follows:

- | | |
|--|--|
|  | Confirm the command and start the design process. |
|  | Do not start a design process. Run XFaceMaker in a single process instead. |
|  | Wait for a connection to the design process. You only need to use this button in the extremely rare event that you launch the design process from a process other than the current XFaceMaker session. |
|  | Leave XFaceMaker. |
|  | Get some help for this window. |

Note that as soon as you request XFaceMaker to restart the design process, it terminates any currently running design process.

CHAPTER 18: Active Values

Application code and interface need to exchange data. You have seen one way to do this, through parameters passed to application functions. Active values are the other way, and provide the basic structure used by XFaceMaker to perform other more complex operations. Although in this chapter, we only describe the most basic use of active values, XFaceMaker also uses them for implementing the OSF/Motif drag-and-drop system, creating resources for new Xt Intrinsic classes, and specifying member functions and data for C++ classes.

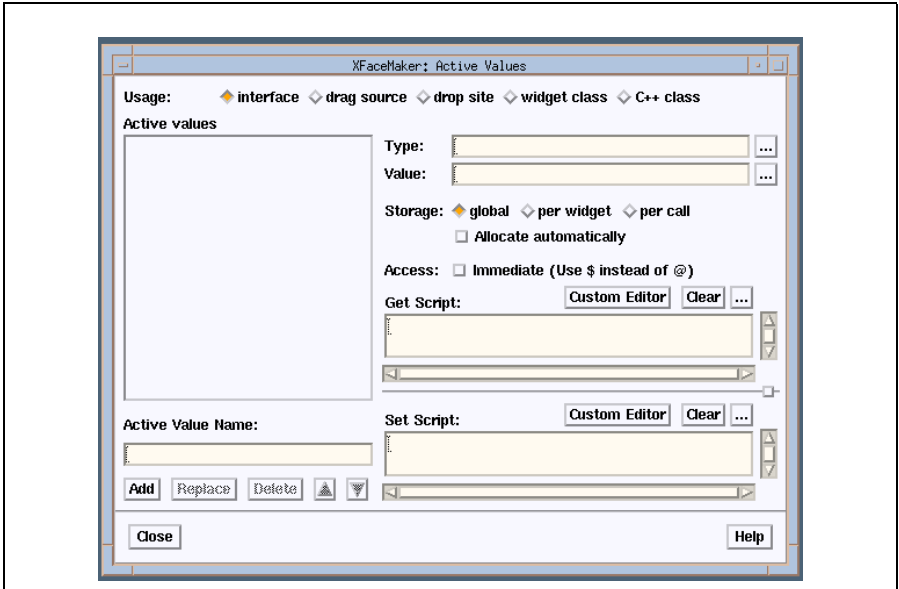
An active value is an address of some memory, usually one word. You choose a name for the active value for the application and interface to use.

The application communicates with active values using two specially designated FACE scripts, a *get* script and a *set* script. By convention, these scripts have specific functions. The application calls the *get* script to tell the interface to update the active value so the application can read the active value. It calls the *set* script to tell the interface when it has changed the active value so that the interface can update its presentation to the user. However, these are only conventions, and not strictly adhered to. For example, it is quite common to use an active value merely as a way for an application to call FACE code in the interface, without actually involving any shared data. In this case, the convention is to use the *set* script.

XFaceMaker associates the scripts for an active value with a widget in the interface. A widget may have more than one active value associated with it, and you may use the same active value with more than one widget.

18.1 The Active Value Window

The active value box, like the menu editor, is a dialog box that works on the current selection. To look at it, select the **Active Values** item in the Windows menu in XFaceMaker's main window. The **Active Values** dialog box appears.



At the top of the dialog is a set of radio buttons that selects the use of the active value. Normally, when you first open the box, the toggle button **interface** is set, indicating a normal active value used for communication between the application and the interface. If you choose another toggle, the dialog box adjusts its contents to reflect the appropriate parameters for that choice.

The left side of the box shows the list of active values for the currently selected widget, and the right side shows the particulars for a specific active value. By selecting an active value in the list, you will see the description of the selected active value in the rest of the box. You also use the rest of the box to provide information to create new active values.

The active value list has operational elements common to many of the lists in XFaceMaker:

- A text field below the list shows the current selection.
- There are three buttons below the current selection to add a new item, replace the

current item, or delete the current item.

In the normal case, each active value also has the following information:

Type The FACE type of the active value.

Value The initial value of the active value.

Storage How storage is allocated for the active value. There are three choices:

global The same memory is used for all widgets.

per widget Each widget has its own storage for the active value.

per call Memory for the active value is dynamically allocated each time the application calls it.

Also, for the first two choices, you can decide whether XFaceMaker will allocate memory for the active value or not. If you set the **allocate automatically** toggle, XFaceMaker will allocate one word for the active value (either globally or per widget). If not, then you must provide the memory for the active value.

Having XFaceMaker allocate the memory lets you read and write the contents of the active value while testing the interface, which lets you perform more testing in XFaceMaker before writing the C code parts of your application. However, you cannot then have the interface directly access application variables.

Access Specifies whether the type refers to the address of the memory location or its contents. For some types, like structure pointers or strings, it is sometimes useful for the type to refer to the address of the memory. Most of the time, however, it is more useful if the type refers to the (one-word) contents of the memory of the active value. Immediate access means that the address has the given type, rather than the contents.

Finally, there are two text widgets for the two FACE scripts. A separator with a sash allows you to adjust the relative size of the two text boxes, where you enter FACE code. There are two special characters in these scripts. \$ refers to the address of the active value, and @ refers to its contents. @ always refers to one word of memory, regardless of the type of the active value or its access type. Both text boxes have the following buttons:

Custom Editor Open a separate editing session to edit the FACE script. This works in the same manner as the **Custom Editor** button in the resource box.

Clear Erase the contents of the text box.

... Expand the text box to take up the whole dialog box. When you press this button, another button, **Back To List**, appears which returns to the active value list.

18.2 Creating Active Values

In our example, we use active values to add addresses read from a file saved earlier. We start by using the active values in the most basic way, so we will not change any of the default active value settings. We will simply provide a name, type, and scripts. Here is how we can fill the address editor dialog box with the values read:

1. If you are not already doing so, start an XFaceMaker session and edit our example, **addrbook.fm**.
2. Open the Active Values window by selecting the **Active Values** button in the **Windows** menu in XFaceMaker's main window.
3. Select the **name** widget in the address editor.
4. Enter **Name** in the **Name** field in the active value box.
5. Enter **String** in the **Type** field.
6. Enter the following set script:
`self:value = @;`
7. Press the **Add** button.

This simple script sets the value of the text field to the string pointed to by the active value. Note that the address of the string is the contents of the active value, so we use the symbol @. We will call this script from our application to specify the name in the address.

8. Select the **street** widget in the address editor.
9. Add an active value named **Street** for this widget. Specify **String** as the type, and enter the same script as in step 6.
10. Select the **city** widget in the address editor and add an active value named **City** in the same manner.
11. In a similar manner, add a **State** active value to the **state** widget and a **Zip** active value to the **zip** widget.
12. Select the **add** widget in the address editor.
13. For this widget, enter **Add** in the name text field and enter the following set script:

```
XtCallCallbacks (self,"activateCallback",0);
parent.name:value = "";
parent.street:value = "";
parent.city:value = "";
```



```
parent.state:value = "";
parent.zip:value = "";
```

This script calls the activate callback for the button, then clears the text fields.

14. Select the list widget in the interface's main window.

15. For this widget, enter **Clear** in the name text field and enter the following set script:

```
include "addrbook.inc";
if (Book) {
    n = self:itemCount;
    for (i = 0; i < n; i = i + 1) {
        Address a = Book[i];
        XtFree (a->name);
        XtFree (a->street);
        XtFree (a->city);
        XtFree (a->state);
        XtFree (a->zip);
        XtFree (String(a));
    }
    destroy (Book);
    Book = NULL;
}
XmListDeleteAllItems (self);
```

Your application will call this script to empty the address book before reading in a new one. It loops through the current address book, freeing all structure fields and the structures themselves for each entry. Then it frees the array and clears the array variable. Finally, it deletes all the items in the list widget.

16. Save your interface by selecting the **Save** item in XFaceMaker's **File** menu.

17. Save your application files by selection the **All** item in the **Save Appli Files** sub-menu of the **File** menu. Since you have modified the **addrbookApp.c** file and you may have modified the **Makefile**, XFaceMaker will ask you to confirm that you want to overwrite these files. Press **OK** to confirm overwriting the **addrbookApp.c** file, but press **Cancel** to cancel overwriting the **Makefile**. Since we have not changed the files involved in our application, we do not need to update the **Makefile**.

18.3 Calling Active Values

You are now ready to call these active value scripts from your application code. Since you set the **Preserve User Code** toggle at the beginning of the tutorial, the code you had entered previously for the application functions is still there, and the declarations for the active values have been added to the **addrbookApp.c** file by XFaceMaker¹.

You call the set script for an active value using the function **FmSetActiveValue**:

```
FmSetActiveValue (Widget w, String name)
```

This calls the set script for the widget *w* for the active value named *name*. If the widget is **NULL**, it calls the named active value script for all widgets. If the name is **NULL**, it calls the set script for all active values for the given widget. They cannot both be **NULL**. The function **FmGetActiveValue** calls the get script for an active value in the same manner.

Often, you will only use an active value in one place, so you will not need to specify the widget. Also, you may want to update the same active value in all places in the interface that might use it. In both of these cases, the widget will be **NULL**. On the other hand, you may want to read all active value data associated with a particular widget. In this case, the name would be **NULL**.

-
1. If you did not set the **Preserve User Code** toggle, you must edit the **addr-bookApp.c** file. Remember that we overwrote it when we saved our application files. However, XFaceMaker saves the previous version of the file in **addr-bookApp.c~**. Compare these two files: the previous file (**addrbookApp.c~**) has all of the code we added for the application functions; the current file (**addr-bookApp.c**) defines the active values using the following lines:

```
XtArgVal Clear;  
String Name;  
String Street;  
String City;  
String Zip;  
String State;  
XtArgVal Add;
```

You need to merge these two versions of the file together, either by adding the variable definitions to the previous version of the file, or by adding the application code to the current version. Whichever way you use, you should put the result in the current version, **addrbookApp.c**.

Now, modify the function **ReadAddressBook** to call the active values. This function will read each line in the file, and if it sees a statement for an address field, it will set the active value for that field, then call the active value's set script. To be able to read the file, you must declare a buffer to hold the line read. Add this line at the beginning of the function **ReadAddressBook**:

```
char line[BUFSIZ];
```

Note: **BUFSIZ** is defined in **stdio.h**.

Then replace the comment that was serving as a placeholder with the following code:

```
FmSetActiveValue(0, "Clear");
while (fgets(line, sizeof(line), bookfile) != 0) {
    char buf[BUFSIZ];

    if (sscanf(line, "Name = %[^\\n]", buf) == 1) {
        if (Name != 0)
            FmSetActiveValue(0, "Add");
        Name = buf;
        FmSetActiveValue(0, "Name");
    }
    else if (sscanf (line, "Street = %[^\\n]", buf) == 1) {
        Street = buf;
        FmSetActiveValue(0, "Street");
    }
    else if (sscanf (line, "City = %[^\\n]", buf) == 1) {
        City = buf;
        FmSetActiveValue(0, "City");
    }
    else if (sscanf (line, "State = %[^\\n]", buf) == 1) {
        State = buf;
        FmSetActiveValue(0, "State");
    }
    else if (sscanf (line, "Zip = %[^\\n]", buf) == 1) {
        Zip = buf;
        FmSetActiveValue(0, "Zip");
    }
}

if (Name != 0) {
    FmSetActiveValue(0, "Add");
    Name = 0;
}
```

First, the code clears the address book list by calling the set script for the active value **Clear**. Then, it uses the function **fgets** to read each line in the file, then **sscanf** to see if the line is an address field description. For every type of field, we set the active value, then call the set script. We do not specify the widget since the active value only exists for one widget, and we do not know the widget ID anyway.

Furthermore, each time we set the active value name, we assume we are starting a new address. If there was already a non-zero value in **Name**, we assume there are fields waiting to be added, and we call the **Add** set script to add the address. We do the same at the end of the loop and clear the **Name** variable so it will be initialized for a subsequent call to **ReadAddressBook**.

Your **addrbookApp.c** file should now look something like the one below. Any pattern information has been removed for better readability.

```
/*
 * C file generated by XFaceMaker Version 3.5.
 * Do not remove this comment.
 * Timestamp:          908985025
 */

/*
 * XFaceMaker 3.5 Application Functions Definitions.
 */

#ifndef STANDALONE
#include <Fm.h>
#endif
#include <stdio.h>

static FILE *bookfile = 0;

XtArgVal Clear;
String Name;
String Street;
String City;
String Zip;
String State;
XtArgVal Add;

#if NeedFunctionPrototypes
void CloseAddressBook()
#else
void CloseAddressBook()

#endif
{
    if (bookfile !=0) {
        fclose(bookfile);
        bookfile = 0;
    }
}

#if NeedFunctionPrototypes
Boolean ReadAddressBook(String file)
#else
Boolean ReadAddressBook(file)
String file;
#endif
```

```
{
char line[BUFSIZ];

CloseAddressBook();
bookfile = fopen(file, "r");
if (bookfile == 0)
    return False;
/* read file here */
FmSetActiveValue(0, "Clear");
while (fgets(line, sizeof(line), bookfile) != 0) {
    char buf[BUFSIZ];
    if (sscanf (line, "Name = %[^\\n]", buf) == 1) {
        if (Name != 0)
            FmSetActiveValue(0, "Add");
        Name = buf;
        FmSetActiveValue(0, "Name");
    }
    if (sscanf (line, "Street = %[^\\n]", buf) == 1) {
        Street = buf;
        FmSetActiveValue(0, "Street");
    }
    if (sscanf (line, "City = %[^\\n]", buf) == 1) {
        City = buf;
        FmSetActiveValue(0, "City");
    }
    if (sscanf (line, "State = %[^\\n]", buf) == 1) {
        State = buf;
        FmSetActiveValue(0, "State");
    }
    if (sscanf (line, "Zip = %[^\\n]", buf) == 1) {
        Zip = buf;
        FmSetActiveValue(0, "Zip");
    }
}

if (Name != 0) {
    FmSetActiveValue(0, "Add");
    Name = 0;
}
fclose (bookfile);
bookfile = 0;
return True;
}
```

```
#if NeedFunctionPrototypes
void WriteAddress(String name, String street, String city,
                  String state, String zip)
#else
void WriteAddress(name, street, city, state, zip)
String name;
String street;
String city;
String state;
String zip;
#endif
{
if (bookfile != 0) {
    fprintf(bookfile, "Name = %s\n", name);
    fprintf(bookfile, "Street = %s\n", street);
    fprintf(bookfile, "City = %s\n", city);
    fprintf(bookfile, "State = %s\n", state);
    fprintf(bookfile, "Zip = %s\n", zip);
}
}
#if NeedFunctionPrototypes
Boolean OpenAddressBook(String file)
#else
Boolean OpenAddressBook(file)
String file;
#endif
{
CloseAddressBook();
bookfile = fopen(file, "w");
return bookfile != 0;
}
```

To see the fruits of your labor:

1. Compile the XFacemaker editor version of your application by selecting the **Build Appli** item in the **Design Process** menu.
1. After the compilation completes successfully, launch your application using the **Launch Appli** item in the **Design Process** menu.
2. Set the **Try** toggle in XFaceMaker's main window.
3. Select the **Open** button in your interface's **File** menu.

4. In the file selector, select a file that you created earlier when you were testing the **Save** button, then press the **OK** button. The addresses you saved are added to the list.
5. If you saved more than one file, you can examine them separately with successive selections of the **Open** button.

CHAPTER 19: Groups

A group is a widget or widget hierarchy that you load into your interface as a separate unit. You can use a group more than once in the same interface, or in several interfaces.

If you create a group using XFaceMaker's interface editor, you can change the elements in the group, and XFaceMaker will keep the changes. However, if you later change the group definition, you will have to reload the group in your interface, and therefore lose all your changes. In this way, an instance of a group, once loaded, is completely independent of the group definition.

19.1 Creating a Group

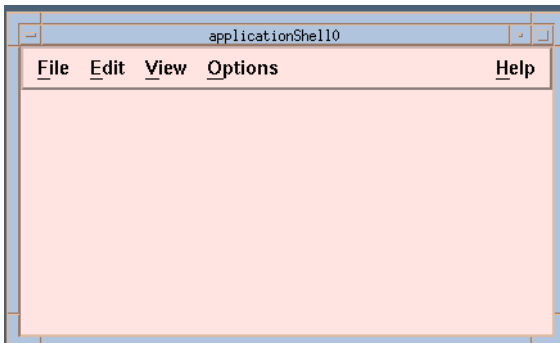
You create a group definition in the same way you would create any other widget. If your group contains more than one widget, they must all be within the same widget hierarchy and therefore have a common ancestor. If the widget you use to define the group (or the top of the hierarchy) is not a shell widget, then you will have to create a shell widget in which to place the widget or widgets that define the group.

In our example, we will create a prototypical main window that you can use as a group. It will contain the standard menus and menu items as recommended by the OSF/Motif Style Guide. This is a good example of a group, because you will usually want to tailor the main window's menu bar to your specific interface, so you will need to modify the instance of the group after you load it into your editing session.

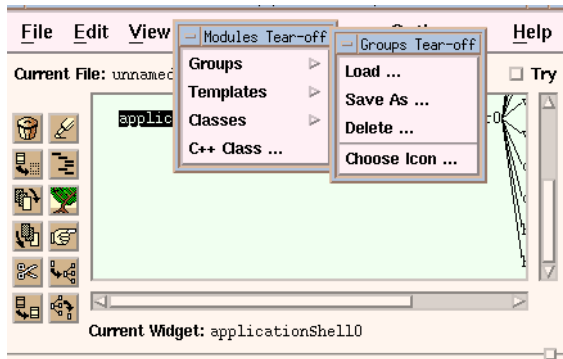
Now, we will create the group definition. As you see, it is exactly like creating an interface:

1. Start an XFaceMaker session, or if you are already running one, save your work and clear the current interface by selecting the **New** item in the **File** menu.
2. Make sure the **Composite** button in the **Toolkit** menu is selected.
3. Create a main window by selecting the **XmMainWindow** widget in the **Toolkit** window, then creating a main window on your desktop. XFaceMaker automatically creates an applicationShell to contain the main window. 
4. Make sure the **Menu** button in the **Toolkit** menu is selected.
5. Create a menu bar as a child of the main window. 
6. Select the menu bar by clicking the **Select** mouse button on its name in the widget tree.

7. Open the menu editor if it is not already open by selecting the **Menu Editor** button in the **Windows** menu.
8. Select the **Pulldown Menu** icon in the menu editor.
9. Enter the name **File** for the label.
10. Enter the letter **F** for the mnemonic.
11. If you wish, select a font for the menu bar by pressing the **Select** button next to the **Font** field and choosing a font with the font selector. We use 14-pixel Helvetica bold.
12. Press the **Add** button to create the **File** menu.
13. Create an **Edit** menu with the mnemonic **E** by changing the **Label** and **mnemonic** fields, then pressing the **Add** button.
14. Similarly, create a **View** menu with the mnemonic **V**, and an **Options** menu with the mnemonic **O**.
15. Select the **Help Menu** icon in the menu editor.
16. Change the label name to **Help** and the mnemonic to **H**, then create the help menu by pressing the **Add** button.



You have now created the most common of the standard OSF/Motif menus with their standard mnemonics. Before filling these menus with menu items, we will save our group. You may notice in the **Groups** sub-menu of the **Modules** menu, there is a **Save As** item, but no **Save** item.



XFaceMaker requires you to save a new file each time, even if you overwrite an existing file, to remind you that your changes will not extend to any previous uses of the group.

17. Select the `applicationShell` in the widget tree. The current selection indicates the widget that contains the definition of the group. If the widget has children, the entire tree from the widget down defines the group.
18. Select the **Save As** item in the **Groups** sub-menu of the **Modules** of XFaceMaker's main window.
19. When the file selector appears, enter the name **AppMain.fm** in the text field labeled **Group file to save**.
20. Press the **Default name** button to create the default group name (**AppMain**).
21. Press the **OK** button to save the group. Since this is a new file, XFaceMaker should save the group without comment. If the file already exists, XFaceMaker will confirm your request.

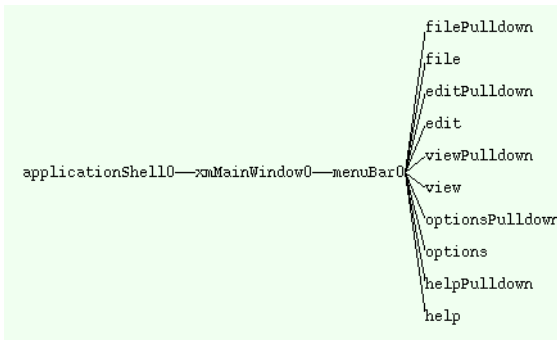
You have now saved your group. By saving the group, you have also defined the group. If you select the **Groups** button in the **Toolkit** menu in XFaceMaker's main window, you will see a new icon for the group you just saved. (If the **Groups** button did not exist before, XFaceMaker will add it to the **Toolkit** menu.)

19.2 Using a Group

Once you have defined a group, you may use it to build an interface.

1. Clear your current work by selecting the **New** button in the **File** menu in XFace-Maker's main window. If the dialog box appears asking you to confirm, press the **OK** button.
2. Make sure the **Group** toggle button is selected in the **Toolkit** menu.
3. Select the **AppMain** icon for the group you just created. Its icon should look something like this. 
4. Create a widget on your desktop in the same manner you would create an applicationShell.

You will see on the desktop and in the widget tree an applicationShell, main window, menu bar, and menus, just as if you had created them yourself individually. You may also freely modify these widgets as if you had created them.



If you have a group file saved from another XFaceMaker session, you can load it into your session:

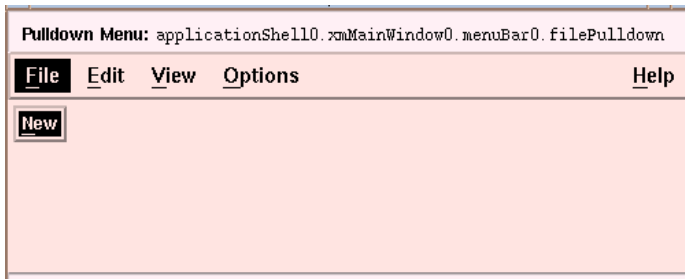
5. Select the **Load** button in the **Group** sub-menu of the **Modules** menu in XFace-Maker's main window.
6. When the file selector appears, select the file **AppMain.fm** in the file name list, then press the **OK** button.
7. When the group name selection box appears, select the name **AppMain**, or enter it in the text field, if it is not already there. Since you have already defined this group once, you will see this message in XFaceMaker's message window: **Re-definition of group AppMain.**

19.3 Modifying a Group

If you want to change a group definition, you simply create an instance of the group, make your modifications, then save your new definition. However, the new definition will not affect any existing instance of the group, only new uses of it.

Let's try modifying our group definition by filling the menus with some of their standard items.

1. Clear your current work by selecting the **New** button in the **File** menu in XFace-Maker's main window.
2. Make sure the **Groups** button is selected in the **Toolkit** menu.
3. Create an instance of your group **AppMain**.
4. Open the Menu Editor window.
5. Select the widget **filePulldown** in the widget tree.
6. Select the **Push Button** icon in the menu editor.
7. Enter the label **New** and the mnemonic **N** in the **Label** field.
8. If you wish, choose a font for the menu buttons. We use 12 point Helvetica bold.
9. Press the **Add** push button. You should now see the **New** button appear in the menu editor.



10. Change the label name to **Open . . .** and the mnemonic to **O**, then press the **Add** button.
11. Similarly, add **Save**, **Save As...**, **Print**, and **Exit** buttons to the **File** menu. Their mnemonics should be **S**, **A**, **P**, and **x** respectively. Your **File** menu is complete.

12. Select the **Edit** menu by double clicking on its button in the menu editor's menu bar.

13. Select the **Push Button** icon in the menu editor.



14. Change the label name to **Undo**, the mnemonic to **U**, select a font, and enter **Alt+Backspace** in the accelerator. To enter the accelerator, move the cursor over the text field, click the **Select** mouse button if you have explicit keyboard focus, then while holding down one of the **Alt** keys, type a **Backspace**. You should see **Mod1<KeyPress>BackSpace** appear in the **Accelerator** field and **Alt+Backspace** appear in the corresponding **text** field. If you want, you can change the accelerator text to read **Alt+Backspace** (with a lower case *s*) to match the OSF/Motif Style Guide.

15. Press the **Add** button to create the **Undo** button in the **Edit** menu.

16. Add a separator by selecting the **Separator** icon, setting the **Etched-In Shadow** toggle if needed, changing the **Widget Name** field to **sep1**, then pressing the **Add** button.



17. Select the **Push Button** icon again in the menu editor.



18. Change the **Label** name to **Cut**, its mnemonic to **t**, and the accelerator to **Shift+Delete**. If you wish, you may change the accelerator text to **Shift+Del** to match the OSF/Motif Style Guide. Then, press the **Add** button to add this item.

19. Similarly, add a **Copy** button with the mnemonic **C** and accelerator **Ctrl+Insert**. Press **Add**. You may change the accelerator text to **Ctrl+Ins** to match the style guide.

20. Then, add a **Paste** button with mnemonic **P** and accelerator **Shift+Insert**. Press **Add**.

Note: If the **Insert** key is on the numeric keypad and entering **Shift-Insert** does not result in the accelerator text **Shift - Insert** you should use another accelerator, such as **Control-P**, which would display the text **^P**.

21. Add another separator as you did in step 16, only specify **sep2** as its name. Press **Add**.

22. Select the **Push Button** icon again in the menu editor and add a **Clear** button with mnemonic **e**.



23. To remove the accelerator, empty the text field for the accelerator text. Press **Add**.

24. Finally, add a **Delete** button with mnemonic **D** and no accelerator. Press **Add**.

You have now finished your standard **Edit** menu. Although the OSF/Motif Style guide mentions other standard buttons for the **Edit** menu, these are the ones most commonly used.

You can add standard **Help** menu items to your group as well. Using the method you just used, add the following buttons to your help menu. Normally there are no separators and no accelerators.

On Context

On Window

On Keys

Index

On Help

Tutorial

On Version

You now have a fairly extensive interface starting point for any application, and the most commonly used functions have their standard position and representation as recommended by the OSF/Motif Style Guide. You could save this group for use by other developers so they have a good starting point, and so that they use the correct mnemonics and accelerators for standard OSF/Motif commands. A developer can add and remove menus and menu items for the interface as needed.

You save your group definition in the same way as before, by selecting the applicationShell, then invoking the **Save As...** item in the **Groups** sub-menu of the **Modules** menu. As explained earlier, XFaceMaker will confirm that you want to overwrite the existing file, and will print a message when it redefines the group **AppMain**.

CHAPTER 20: Templates

Templates are similar to groups: they describe a widget or widget hierarchy that you can use over and over again in one or more interfaces. However, unlike groups, when you change the definition of a template, you also change all instances of that template that are present in the current interface. Thus, while groups serve as useful prototypes for elements that the interface designer uses as starting points, templates form more fundamental interface elements, much like widgets do.

You can specify resource settings for a template like you would for a widget. In this case, they would apply to the widget created by the template, or the top widget of a widget hierarchy. The definition of a widget may include some resource settings. When you define a template, you indicate whether or not its user may override these resource settings or not.

This aspect of templates' resources make them useful for defining standard resource settings; for example, standard colors, margins, borders, or fonts.

For this reason, you can apply more than one template to a given widget on your interface. Thus, one template may specify one set of standard resource settings (say, colors), and another template may specify a different set (say, fonts).

20.1 Creating a Template

First we will create a template for reuse in one or more interfaces. Let us suppose that you had a product logo and name that your marketing people wanted to place in all of your windows. You could put this in two label widgets, one for a pixmap containing the logo, and another for the name. Like a group a template must be either a widget or widget hierarchy, so we will place these two widgets inside a bulletin board.

1. Start an XFaceMaker session, or if you are already running one, save your work and clear the current interface by selecting the **New** item in the **File** menu.
2. Create a bulletin board to work in by selecting the **XmBulletinBoard** icon in the **Toolkit** pane in XFaceMaker's main widow, then dragging out a medium-sized rectangle on the root window. 
3. Create another bulletin board inside the one you created.
4. Select the bulletin board you just created, then open a resource window.
5. Change the name to **xlogo** the margin width and height to 2, select the **Auto position** toggle, and press the **Rebuild** button.

The labels already have built-in margins, so we leave only enough margin in the bulletin board to accomodate the shadow thickness that will be added later. Also, we do not want to specify the position in the template, so that users of the template may place it where they want to.



6. Create a label inside the bulletin board, and select it for editing.
7. Change the **Name** field to **pixmap** the **labelPixmap** resource to **xlogo64**, and the **labelType** resource to **pixmap**.
8. Set the **x** and **y** fields to zero.
9. Select the **Auto size** toggle, press the **Rebuild** button, clear the toggle, then press the **Rebuild** button again.

We use the X Window System logo for our “product”. If you cannot find the pixmap **xlogo64**, you may use any other pixmap available, such as XFaceMaker’s icon **XfmIcon**. We do the trick with the **Auto size** toggles to find the widget’s “natural” size, then fix that size in place. Normally, for the **xlogo64** pixmap this results in a width and height of 68 pixels.



10. Create another label inside the bulletin board, and select it for editing.
11. Change the **Name** field to **name**, the **x** text field zero, and the **y** value to **68**, or whatever the value of the **height** field is in the pixmap label widget, then press the **Rebuild** button.
12. Select the **labelString** resource then press the . . . button to open up an editing window.
13. Enter the following text into the editor:

**Window
System**

Do not put a newline after the second line.

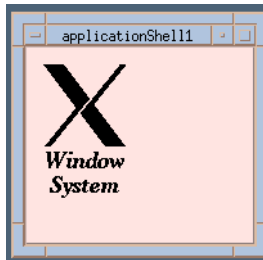
14. Press **OK** to set the resource and return to the resource list.
15. Choose a font for the text. We use times bold italic, 18 pixels.

16. Select the **Auto size** toggle, press the **Rebuild** button, clear the toggle, then press the **Rebuild** button again.

We add the text “Window System” beneath the stylized X logo, then find its natural size. If you used some other pixmap, you may want to use more appropriate text. Normally, the text and logo do not have the same width, so we adjust the width of the smaller to match the larger. In our case, the logo has a width of 68 pixels, and the text has a width of 66 pixels.

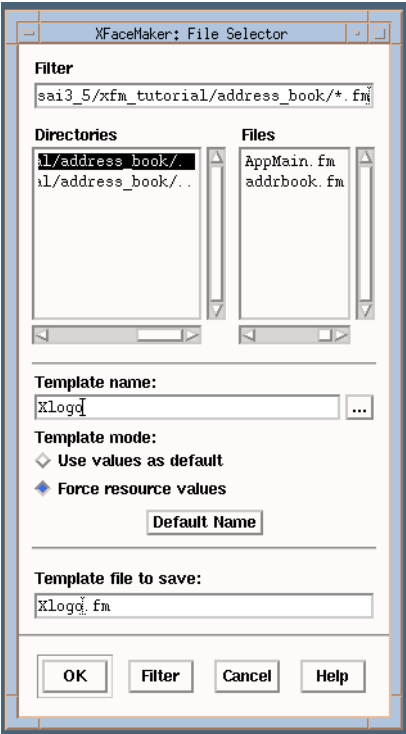
17. Select the label widget (or the pixmap if it is smaller) for editing.
18. Change its width to **68** (or the width of the larger widget).
19. Press the **Rebuild** button.
20. Select the **xlogo** bulletin board that contains the two labels.
21. Select the **Auto size** toggle, press the **Rebuild** button, clear the toggle, then press the **Rebuild** button again.

After we have matched the widths of the two labels, we find the bulletin board’s natural size and fix that into place. Your logo should look something like this:



Now we can save the template:

22. Select the **xlogo** widget in the interface. This is the top of the hierarchy you will save.
23. Select the **Save As** item in the **Templates** sub-menu in the **Modules** menu in XFaceMaker’s main window.
24. In the file selector, enter the name **Xlogo . fm** in the text field labeled **Template file to save**.



25. Press the **Default Name** button to select the default template name, and set the **Force resource values** toggle.

26. Press the **OK** button to save the template.

You now have a template named **Xlogo** ready for use. We choose to force the resource values to prevent the user from accidentally changing them.

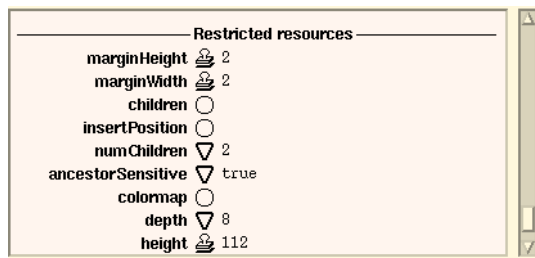
20.2 Using a Template

XFaceMaker places templates under a special heading of the Toolkit menu. You use this menu to create instances of your new template.

1. Select the **New** item in XFaceMaker's **File** menu. If XFaceMaker asks you to confirm your request, do so by pressing the **OK** button.
2. Create a dialogShell, and place a form in the dialogShell.
3. Select the **Templates** toggle button in the **Toolkit** menu.
4. Create an instance of the **Xlogo** template by selecting its icon then dragging out the instance inside the form. Note that beneath the name of the template is the comment (**override**) indicating that all resources set in the template overrides any user value. In our case, this includes the size of the template, so it always takes the same size regardless of whether or not you try to specify a size by dragging a rectangle when you create it.
5. Select the **xlogo** widget you just created and open a resource window if there is not one already open.

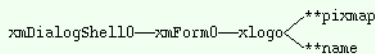
The **Class** field contains the actual class of the widget, **XmBulletinBoard**, while the **Template** field shows the template name, **Xlogo**. Also, any resources set by the template appear under the **Restricted resources** heading in the resource list, since we cannot change these resources. Furthermore, XFaceMaker indicates those resources set by the template by placing the  icon by their value.

Note: if the template icon does not appear, click on the **Sort** button at the bottom of the Resources window to display the Sort Order Options, and make sure that the **Show Template Resources** option is selected.



20.3 Modifying a Template Instance

If you look at the widget tree, you will see that the children of the **xlogo** widget have two asterisks (**) in front of their names. These indicate that they were created as part of a template definition. Any changes you make to these widgets will disappear the next time you load this interface for editing, or the next time the template definition changes.



You can change resource settings within the children of a template, as well as in the widget at the top of the hierarchy, and XFaceMaker will remember the changes, even if the definition of the template itself changes. However, if the template is an *override* template, that is, the toggle button **Force resource values** was set when the template was defined, then you cannot change any resource which has been set as part of the template definition.

1. Select the **xlogo** widget if not already selected and open a resource window if needed.
2. Select the **shadowThickness** resource in the resource list and change its value to **2**.
3. Press the **Rebuild** button.

You now see a shadow border around the logo and text. Only those resources specified in the template definition get placed under the **Restricted resources** heading where you cannot change them.

Sometimes, you may want to specify additional resource values in your template simply to prevent the user from changing their value. You can change a resource's *default* value to its *specified* value by changing its value icon from ▽ to ◆ without changing the value itself, which will then retain its default value. To do so, press mouse Button 3 while the pointer is over the resource you want to change. Then select the item in the “Value” popup menu. The item will be either *XFM File* or *default*, depending on the resource and object. Then click **Rebuild**.

Let's try changing another resource:

4. Select the **background** resource in the resource list, and change its value to **yellow**.

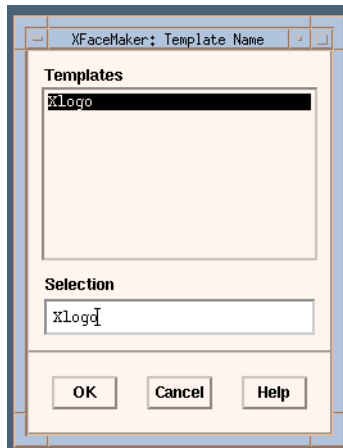
5. Press the **Rebuild** button.

Note that the appearance of the logo does not change. Changing the template's **background** resource changes the bulletin board but not the labels, which have their own **background** resource. In this case, you can get around this problem by using label *gadgets* instead of ordinary widgets. Then, the gadgets will take on the same coloring as the bulletin board. To do this, we need to modify the template definition.

20.4 Modifying a Template Definition

You often need to modify the definition of a template during the course of designing your interface. To do so, you need to load its definition, change the definition, then save it again.

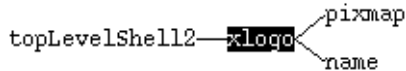
1. Select the **Edit** item in the **Templates** sub-menu of the **Modules** menu in XFaceMaker's main window.
2. When the **Template Name** dialog box appears, select the **Xlogo** item in the list of templates.



3. When the file selector appears, verify that the information is correct, then press the **OK** button.

XFaceMaker automatically creates a top-level shell in which it places the widgets that make up the template definition. You can edit this widget hierarchy to change the definition:

4. Select the **xlogo** widget in the hierarchy you just loaded. It is the child of the top-level shell created by XFaceMaker.



5. Select the **Gadget** item in the **Edit** menu of XFaceMaker's main window. XFaceMaker displays the following message:

2 Widgets changed.

The **Gadget** editing command acts on the current selection. It changes all possible widgets in the hierarchy contained by the widget (including itself) to gadgets. XFaceMaker changes only those widgets that have gadget counterparts, and only those widgets that do not have non-gadget resources specified.

6. Select the **pixmap** child of the **xlogo** widget. Open a resource window if one is not already open. You can see in the resource window that the **Class** field now contain **XmLabelGadget** and the **Gadget** toggle is set.
7. Select the **name** child of the **xlogo** widget. You can see that it, too, is a gadget.
8. Select the **xlogo** widget.
9. Select the **Save** item in the **Templates** sub-menu of the **Modules** of XFaceMaker's main window. You see the following in XFaceMaker's message window:

Template Xlogo redefined

XFaceMaker remembers that the current selection is the definition of a module and automatically saves the definition of the template without bringing up the file selector. It also updates the module definition, which you can see because the logo in your interface has changed to yellow.

20.5 Template Resources

Now that you have seen how to use a template to define a pre-defined collection of widgets, we will show you another method of using templates, as collections of resources. You could use this method to present a standard look for your interfaces across several interfaces, or perhaps for use by more than one developer. Let's look first at a standard button font:

1. Start an XFaceMaker session, or if you are already running one, save your work and clear the current interface by selecting the **New** item in the **File** menu.
2. Create a bulletin board to work in using the **XmBulletinBoard** icon in the **Toolkit** pane in XFaceMaker's main window. 
3. Create an **XmPushButton** widget inside the bulletin board. 
4. Select the push button widget for editing, and open a resource window if one is not already open.
5. Select the font Helvetica bold, 17 pixels, for the **fontList** resource.
6. Set the **marginHeight** resource to **3** and the **marginWidth** resource to **12**.
7. Set the **Auto position** and **Auto Size** resources, change the **Name** field to **buttonFont**, then press the **Rebuild** button.
8. Select the **Save As** item in the **Templates** sub-menu of the **Modules** menu.
9. Enter **ButtonFont . fm** as the file name, press the **Default Name** button, then set the **Use values as default** toggle if needed.
10. Press the **OK** button to save the template.

This template sets the font list and appropriate margins for a push button. You could then create another template for a particular color scheme:

11. Create another push button in the bulletin board. 
12. Select the new push button.
13. Change its **background** resource to **yellow**, its **foreground** resource to **blue**, its **topShadowColor** resource to **lemonchiffon**, and its **bottomShadowColor** to **goldenrod**.
14. Set the **Name** field to **calColors**, set the **Auto position** and **Auto size** toggles, then press the **Rebuild** button.
15. Select the **Save As** item in the **Templates** sub-menu of the **Modules** menu.
16. Enter **CalColors . fm** as the file name, then press the **Default Name** button,

then set the **Use values as default** toggle if needed.

17. Press the **OK** button to save the template.

You have just saved resource settings for a color scheme in a template named **CalColors**. Templates that contain resource settings are good examples for combining templates. To combine templates, you simply enter the template names in the **Templates** field of the resource box, separated by commas.

18. Start a new interface by selecting the **New** button in the **File** menu.

19. Create a new main window using the **XmMainWindow** icon.



20. Place a menu bar in the main window.



21. Place a bulletin board in the main window.



22. Create a push button in the bulletin board.



23. Select the push button for editing, and open a resource window if there one is not already open.

24. Enter **ButtonFont, CalColors** in the **Templates** field. (Do not put a space after the comma.)

25. Press the **Rebuild** button.

The button now takes the font and color scheme of our templates. Because these resources are *default* values and do not *override* the resource values, they do not appear under the heading of **Restricted resources**, but rather in their normal place in the resource list. However, they still have the  icon next to their value, indicating that the value came from a template.

A widget that uses a template does not have to have the same class as the template. XFaceMaker will still apply the resources in the template to the widget. If the widget does not have one or more of the resources in the template definition, it ignores those resources.

26. Select the menu bar for editing, then use the menu editor to create an empty **File** menu in the menu bar. (See Chapter 4 if you need a reminder on how to do this.)
27. Select the cascade button named **file** (for the **File** menu) in the widget tree if it is not already selected.
28. In the resource window, enter **CalColors, ButtonFont** in the **Templates** field, then press the **Rebuild** button.

Note that the order does not matter, and the widget class may differ from that of the template. Of course, if you use a template icon in the toolkit window to create a new widget, it will always have the same class as the template.

If two or more templates specified in the template list have children, the widget applying the templates will have all the children of all the templates.

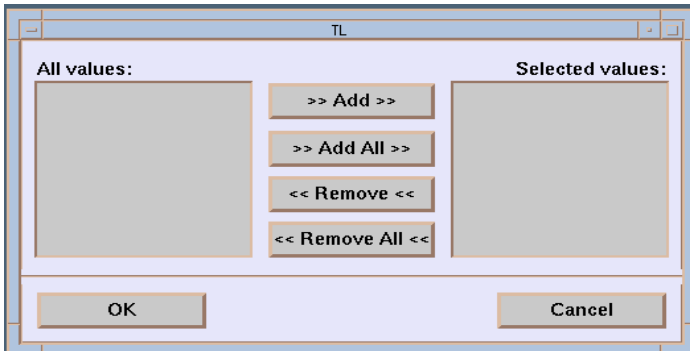
Normally, you will only want to use one template with children. Also, you must use a class that allows children or a template that already has some, so that XFaceMaker will be able to create the children for the widget specified in the template(s).

CHAPTER 21: Creating a Widget Class

A class is similar to a template, in that you define a class as a widget or a widget hierarchy that you load into your interface. Like a template, when you change a class definition, you change all instances of the class in your interface.

However, a class is a full-fledged intrinsics library construct. You can add resources to the class definition, including callback resources, and you can create new classes by subclassing from it. If you generate C code for the class, you can use this class in the same way as any Xt intrinsics class. XFaceMaker generates the three standard files (class.c, class.h, classP.h) so that the class can be used in any application.

The class that we are going to create looks like this:



It consists of two lists placed within scrolled windows with buttons that let you move selected items from one list to the other. The list on the left will contain the **source** items, i.e. items from which the user will make his selection of items to move to the list on the right, the list of selected items, or **destination** list. The two buttons at the bottom, labelled **OK** and **Cancel** will activate the corresponding callbacks for the class.

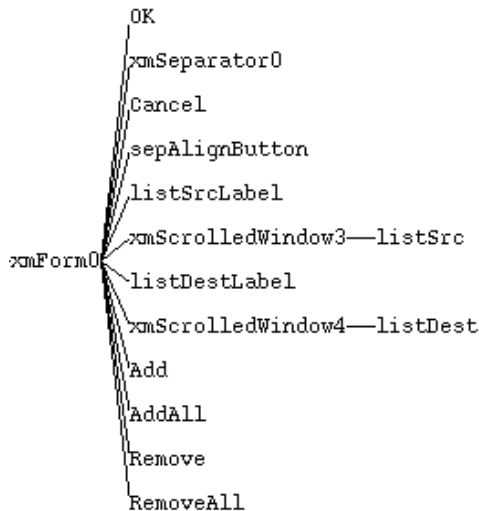
To build this class, we will need to:

1. build the class sub-tree.
2. add internal behavior to transfer items between the lists.
3. define the class resources and their scripts.
4. save the class.

21.1 Build the class sub-tree

The new class will be subclassed from `XmForm`. We will not go into the detail of the widget attachments that should be done in order that the widgets be placed correctly when the class is instantiated: attachments have already been covered in this tutorial.

The widget sub-tree that you should create is shown below:



21.2 Add Internal Behavior

Items are transferred from the source list to the destination list by:

1. selecting one or several items in the list and activating the **Add** button; this will transfer all the selected items.
2. double clicking on one of the items in the list; this will transfer the selected item.
3. activating the **AddAll** button; this will transfer all the items in the list.

Similarly, items are transferred from the destination list back to the source list by:

1. selecting one or several items in the list and activating the **Remove** button; this will transfer all the selected items.
2. double clicking on one of the items in the list; this will transfer the selected item.
3. activating the **RemoveAll** button; this will transfer all the items in the list.

Moreover, the buttons should be insensitive whenever the list that they transfer from is empty.

Let's first look at the creation scripts for the buttons. Since they are created after the lists, we can decide, at the time of creation, if they should be sensitive or not. Thus, in the `xfmCreateCallback` for the **Add** button, we can write the following script:

```
1:  if (parent.xmlScrolledWindow3.listSrc.itemCount > 0)
2:      self:sensitive = true;
3:  else
4:      self:sensitive = false;
```

The first line verifies if the number of items in the **source** list is larger than 0. If there are items in the list, the button is made sensitive, if not, the button is made insensitive.

The same script can be written in the **AddAll** button `xfmCreateCallback`.

Similarly, the **Remove** and **RemoveAll** buttons should be made sensitive or insensitive conditioned by the number of items in the destination list where the selected items are transferred. At the time they are created, we can assume that the list is empty since our class will not let this list be initialized by the application: we can set the two buttons insensitive when they are created. Alternatively, we can write a creation script similar to the one for the **Add** and **AddAll** buttons so that, if someone modifies the class and lets the list be initialized, the buttons will still be in the correct state. The script is:

```
1:  if (parent.xmlScrolledWindow4.listDest.itemCount > 0)
2:      self:sensitive = true;
3:  else
4:      self:sensitive = false;
```

Now, let's write the `activateCallback` for the **Add** button. We want to transfer the items selected in the source list into the destination list, and set the buttons sensitive or insensitive as appropriate. The script is:

```
1:  type XmlStringPtr;
2:  wsrc = parent.xmlScrolledWindow3.listSrc;
```

```
3:  wdest = parent.xmScrolledWindow4.listDest;
4:  XmString sel_it[Int];
5:  sz = wsrc:selectedItemCount;
6:  sel_it=new_c_array(Pointer(wsrc:selectedItems),sz);
7:  XmlListAddItems(wdest,XmStringPtr(data(sel_it)),
    sz,0);
8:  XmlListDeleteItems(wsrc,XmStringPtr(data(sel_it)),
    sz);
9:  destroy(sel_it);
10:  if (wsrc:itemCount < 1){
11:      self:sensitive = false;
12:      parent.AddAll:sensitive = false;
13:  }else {
14:      self:sensitive = true;
15:      parent.AddAll:sensitive = true;
16:  }
17:  if (wdest:itemCount <1){
18:      parent.Remove:sensitive = false;
19:      parent.RemoveAll:sensitive = false;
20:  }else {
21:      parent.Remove:sensitive = true;
22:      parent.RemoveAll:sensitive = true;
23:  }
```

In line 1, we declare a new FACE type, `XmStringPtr` which we will use to cast the Pointer passed to the `XmlList` functions we use.

Next (lines 2 and 3) we initialize variables **wsrc** and **wdest** to the widget ID's of the two lists.

Line 4, we declare a Face array whose elements are of type `XmString`, which is indexed by `Int`. We will use this array to transfer the items.

Line 5 initializes variable **sz** to the number of items selected in the source list. This will determine the number of elements in the Face array that we will create.

Line 6 creates a linear Face array that contains the selected items. The size of the array is **sz**, the address of its elements is the **selectedItems** resource of the list.

Line 7 adds the items in the destination list. We have to cast the Pointer returned by **data** to **XmStringPtr** to have the right type of argument for the function. The number of items added is **sz**. The last argument (0) specifies the items are to be added after the last item in the list.

Line 8 deletes the items from the source list; the same Face array is used to pass the items that are to be deleted.

Line 9 destroys the array now that it will no longer be needed.

The rest of the script sets the **sensitive** resource in the transfer buttons, according to the number of items in each of the lists.

The activateCallback for the **Remove** button is similar; the source and destination list widgets are switched. The script becomes:

```
type XmStringPtr;
wsrc = parent.xmlScrolledWindow4.listDest;
wdest = parent.xmlScrolledWindow3.listSrc;
XmString sel_it[Int];
sz = wsrc.selectedItemCount;
sel_it=new_c_array(Pointer(wsrc.selectedItems),sz);
XmListAddItems(wdest,XmStringPtr(data(sel_it)),sz,0);
XmListDeleteItems(wsrc,XmStringPtr(data(sel_it)),sz);
destroy(sel_it);
if (wsrc.itemCount < 1){
self:sensitive = false;
parent.RemoveAll:sensitive = false;
}else {
self:sensitive = true;
parent.RemoveAll:sensitive = true;
}
if (wdest.itemCount <1){
parent.Add:sensitive = false;
parent.AddAll:sensitive = false;
}else {
parent.Add:sensitive = true;
parent.AddAll:sensitive = true;
}
```

The activateCallback for the **AddAll** button is not very different. It builds the array from the **items** resource instead of the **selectedItems** resource, and it deletes all the items from the source list, instead of removing just the selected items. Here is the script:

```
type XmStringPtr;
wsrc = parent.xmlScrolledWindow3.listSrc;
wdest = parent.xmlScrolledWindow4.listDest;
XmString items[Int];
sz = wsrc.itemCount;
if (sz != 0) {
```

```
items=new_c_array(Pointer(wsrc:items),sz);
XmListAddItems(wdest,XmStringPtr(data(items)),sz,0);
XmListDeleteAllItems(wsrc);
destroy(items);
parent.Remove:sensitive = true;
parent.RemoveAll:sensitive = true;
}
self:sensitive = false;
parent.Add:sensitive = false;
```

The activateCallback for the RemoveAll button is almost identical:

```
type XmStringPtr;
wsrc = parent.xmScrolledWindow4.listDest;
wdest = parent.xmScrolledWindow3.listSrc;
XmString items[Int];
sz = wsrc:itemCount;
if (sz !=0) {
items=new_c_array(Pointer(wsrc:items),sz);
XmListAddItems(wdest,XmStringPtr(data(items)),sz,0);
XmListDeleteAllItems(wsrc);
destroy(items);
parent.Add:sensitive = true;
parent.AddAll:sensitive = true;
}
self:sensitive = false;
parent.Remove:sensitive = false;
```

Both lists will work in extended select mode. You should set the **selectionPolicy** resource of both lists to **extended_select**. In order to have the scroll bars displayed only when they are useful, you should set the **scrollBarDisplayPolicy** resource to **as_needed**.

As we saw in the specifications, we want to be able to transfer an item from one list to the other by just double clicking on the item. Double-clicking on an item in an **XmList** widget activates the **defaultActionCallback**. When this callback is activated in the source list, we want to take the same action as that obtained when clicking on the **Add** button. Thus, the **defaultActionCallback** will just have to call the **Add** button **activateCallback**. We can enter the following script for the source list defaultActionCallback:

```
w = parent.parent.Add;
XtCallCallbacks(w, "activateCallback", NULL);
```

The destination list **defaultActionCallback** has a similar script:

```
w = parent.parent.Remove;  
XtCallCallbacks(w, "activateCallback", NULL);
```

This time, the action we want is the same as that of the **Remove** button **activateCallback**.

We have finished specifying the internal behavior of the widgets for our class. You should save the interface fragment you have just created (e.g. as **dlist_base.fm**), then test its behavior to verify that everything works as expected. For this, you should place items in the source list:

1. In the resources window, select the **items** resource of widget **listSrc**
2. Enter a comma separated list of items, e.g. aa,bb,cc,dd,ee,ff,gg,hh,ii,jj,kk,ll,mm,nn
3. Be sure to enter enough items to cause the vertical scroll bar to be displayed so that you can test scrolling along with the behavior.
4. Now switch XFaceMaker to Try mode and verify the interface behaves as expected.

Once you are satisfied that the interface fragment works as expected, you are ready to create a class from it. You should keep a copy of the file **dlist_base.fm** in a safe place, so that you can use it as your base class for the C++ class example in the next chapter.

21.3 Define Class Resources

Our Xt class needs the following resources and scripts:

1. A resource to initialize the source list. We will call this resource **sourceList**. For it, we will specify *set* script and *get* script: the set script will copy the string received (when the set script is being executed, the string is in the special Face variable *@*) into the source list's **items** resource; it will also control the sensitivity of the **Add** and **AddAll** buttons. The get script will copy the source list's **items** resource, as a String, into the *@* special variable.
2. A resource to read the destination list. We will call this resource **destList**. This resource will have only a *get* script: we don't want the application using the class to be able to write into this list.
3. A callback that is called when the user clicks on the **OK** button. We will call this resource **okCallback**.
4. A callback that is called when the user clicks on the **Cancel** button. We will call this resource **cancelCallback**.

We are going to add these resources on the topmost widget of the sub-tree, i.e. on the Form widget. When we have finished creating our class, the resources that will be accessible are the XmForm widget resources, plus the resources we are about to add.

To add resources to the class:

1. Select the Form widget
2. Open the Active Values Box from the Windows Menu
3. Set the **widget class** toggle in the Active Values Box: we are about to specify widget class resources.

To add the **sourceList** resource:

1. In the text field under New Resource Name: enter **sourceList**.
2. Specify the type of the resource by typing **String** in the text input field next to the Type: label. Alternatively, you can click on the ellipsis to open the Type Selector Box and select the **String** item. The items in the list will be specified as a string containing comma-separated items (the same format as in the FACE convenience function **FmListSetItems**).
3. We don't specify a default value for this resource.
4. Specify the *get* script. The *get* script is executed whenever the resource is read with **XtGetValues**. If the list contains elements, the string of comma-separated elements is returned, by the FACE function **FmListGetItems**. If the string is emp-

ty, an empty string is returned. The script you should enter is:

```
1:   if (self.xmlScrolledWindow3.listSrc:itemCount > 0)
2:     @=FmListGetItems(self.xmlScrolledWindow3.listSrc);
3:   else
4:     @ = "";
```

5. Specify the *set* script. The *set* script is executed whenever the resource is set with `XtSetValues` and at the time of widget creation, when the class `Initialize` method is called. If the list of items provided is not empty (i.e. `@ != 0`), the script will initialize the source list with the string passed in `@`, using the `FACE` function **FmListSetItems**; then it will set the **Add** and **AddAll** buttons to sensitive or insensitive according to the presence of items in the source list. The script you should enter is:

```
1:   if (@ != 0)
2:     FmListSetItems(self.xmlScrolledWindow3.listSrc,@);
3:   if ( self.xmlScrolledWindow3.listSrc:itemCount < 1) {
4:     self.Add:sensitive = false;
5:     self.AddAll:sensitive = false;
6:   } else {
7:     self.Add:sensitive = true;
8:     self.AddAll:sensitive = true;
9:   }
```

6. Click on the **Add** button in the **Active Values Box** to validate the resource **sourceList**

To add the **destList** resource:

1. In the **New Resource Name** field, enter **destList**.
2. Set the type of the resource to **String**, as above.
3. Click on the **Clear** buttons just above the **get** and **set** script input fields to reset the **get** and **set** scripts.
4. Specify the *get* script. The script should do the same as for the **sourceList** resource; the only difference is that we now read the destination list widget. Enter the script:

```
1:   if (self.xmlScrolledWindow4.listDest:itemCount > 0)
2:     @=FmListGetItems(self.xmlScrolledWindow4.listDest);
3:   else
4:     @ = "";
```

5. Click the **Add** button to validate this resource. As discussed before, we are not

specifying a *set* script for this resource, since we don't want to allow the resource to be set externally.

Now, we want to add the **okCallback** resource. To do so:

1. In the New Resource Name field, enter **okCallback**.
2. Set the type of the resource to **Callback**.
3. Click on the Clear buttons just above the get and set script input fields to reset the get and set scripts.
4. Do not enter any *set* or *get* script, just click on the **Add** button to validate this resource.

The **okCallback** callback has to be called when the user clicks on the OK button in the class sub-tree. To implement this:

1. Select the **OK** button in the sub-tree and select its **activateCallback** resource for editing.
2. We want the OK button activateCallback to call the class okCallback, i.e. the okCallback defined in the Form widget. The following script will do just that.

```
XtCallCallbacks(parent, "okCallback", $);
```

In this context, the \$ special variable is a pointer to the callback structure passed to the OK button activateCallback.

3. Click the OK button at the bottom of the script editing window to register this script.

To add the cancelCallback, proceed exactly in the same way:

1. Select the Form widget.
2. In the New Resource Name field, enter **cancelCallback**.
3. Set the type of the resource to **Callback**.
4. Do not enter any *set* or *get* script, just click on the **Add** button to validate this resource.

The **cancelCallback** callback has to be called when the user clicks on the Cancel button in the class sub-tree. To implement this:

1. Select the **Cancel** button in the sub-tree and select its **activateCallback** resource for editing.
2. Enter the following script:

```
XtCallCallbacks(parent, "cancelCallback", $);
```

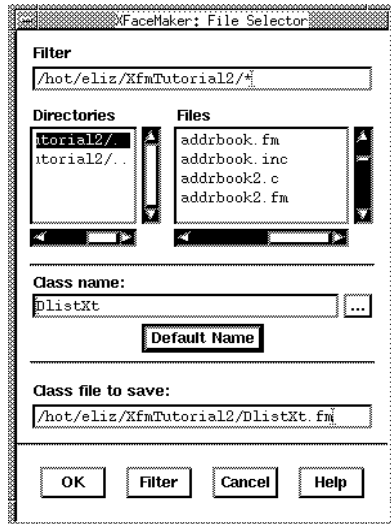
3. Click the OK button at the bottom of the script editing window to register this script.
4. Select the Form widget.

Before saving the class, we need to set the auto position toggle for the Form: we do not want our class to have default x and y resources. We do not set the auto size toggle; we want the class to have a default size, just as standard Motif widget classes do.

21.4 Save the Class

When saving a class, you have to make sure that the top widget of the sub-tree is the currently selected widget: the class created will comprise the entire sub-tree and is sub-classed from the top widget of the hierarchy. To save this class, we will use the Modules menu.

1. Open the **Modules** Menu in the XFaceMaker main window.
2. Select the **Classes** sub-menu
3. Click the **Save As** option. This will open the XFaceMaker File Selector for widget classes. Specify the name of the .fm file where the class will be saved, and the name of the class. The leading character of the file name should be upper-case: Xt class names built with XFaceMaker should always start with an upper case character. The default class name is the name of the .fm file, without the extension
4. Enter DlistXt.fm for the file name and accept the default name for the class name. Click on the **OK** button.



5. Now, you can verify that the class is indeed saved, and known to XFaceMaker. In the XFaceMaker main window, open the User Defined toolkit. Your new class should be there, with the new class default icon.

If you like, you can specify an icon to represent you class. This is done with the Choose Icon option in the Modules menu.

Now, we can build a small interface to use our new class, and verify that we can access its resources.

1. Clear the XFaceMaker design process by clicking New in the File menu.
2. In a BulletinBoard within an applicationShell, draw out an instance of our new class, DlistXt.
3. Open a Resource Window, select the dlitstXt0 widget and verify that it has the new resources we defined: **cancelCallback**, **destList**, **okCallback**, **sourceList** in addition to the standard XmForm resources.
4. Try setting the sourceList resource. The source list in the widget instance should reflect your change.

5. Try specifying an `okCallback` script, and a `cancelCallback` script, switch to Try mode, try the widget out. Your callback script could be, for example:

```
printf("selection: %s\n", self:destList);
```

Notice that, to access the new resource, you proceed exactly in the same way as for the standard Motif widget classes.

As you will notice, you do not have access to the resources of the widgets that are enclosed in the class. When defining a class like this, you might want to let a user modify the labels or the colors of the widgets. To do so, you need only specify new resources, whose *set* and *get* scripts will transfer the information between the `@` special variable and the widgets. Let's see how we can specify a resource for the label of the OK button as an example.

1. To specify new resources, we will have to *edit* our new class: in the Modules menu, select Classes, then Edit. A box is popped to let you choose the class you want to edit, among the classes currently loaded. Here, we have only one. Select it and click OK.
2. The file selector is popped to let you specify the name of the .fm file you want to load.
3. Your class model is loaded, the top level widget is selected, ready to be modified. Open the Active Values box, make sure the Usage is set to widget class. Check that the class name is correct, observe that the resources that we have already defined are listed under New Resources.
4. Enter a new resource name, e.g. `okLabel`
5. Specify the resource type to be `XmString`.
6. The *get* script has to copy the `labelString` resource of the OK button into the `@` variable. Enter the following script in the get script editing field:

```
@ = self.OK:labelString;
```

7. The *set* script has to copy the `XmString` passed in `@` into the button's `labelString` resource. Enter the following script in the set script:

```
self.ok:labelString = @;
```

8. Save this new version of the class; this time, use the Save button in the Active Values box; this will have the same effect as the Save button in the Modules->Class menu. XFaceMaker outputs a message in the status window, warning you that you have redefined the class DlistXt.

Now, you can check that the new resource is indeed defined and can be edited, by selecting the instance of our class in your test interface and editing the okLabel resource in the Resource Window. You may want to specify resources for the other labels of our class.

If you want to specify a resource for the background color, you will have to write a set script which sets the background color in each of the widgets of our subtree: setting the background color of the Form will not be sufficient.

21.5 Generate the C Code

To generate the C code for the class, we can set the Save C Code toggle in the C Options Box (Option->C Code) and save the class again. This time, we will save the files:

- m DlistXt.c which contains the C code for the definition of the widget class, including the function FmCreateDlistXt which can be used to create an instance of the widget.
- m DlistXt.h which contains the public declarations used by programs that create instances of the class, i.e. widget class pointer declaration and symbolic resource names.
- m DlistXtP.h which contains the private declarations used to derive the widget class, i.e. widget and class record definitions.

in addition to the .fm file.

Now that the class has been saved, we would like to save the C code for the interface. Before doing so, we should remove the class model from the design process (note that XFaceMaker will remind you to do so if you attempt to save your interface file while a class model is loaded).

Because we have set the toggle to save C code also, as we save the interface file, we save the .c file along with it. Let's call the interface **test_xtclass.fm**.

Now, if you save the application files (**File** menu, **Save Appli, All**), you can compile the interface; type

```
make -f test_xtclassMake compiled
```

After the command has completed, you should launch the resulting executable, **ctest_xtclass** and check that it works as expected.

You may want to look at the makefile that XFaceMaker generated (test_xtclassMake), as well as the main program (test_xtclassMain.c). In particular, you should observe how the makefile handles generating the C code from the .fm files, using an xfm executable that does not know the new class. This is done using the **classes** command line option.

Because our interface was very simple, we did not use any C functions to run it, so did not need to edit the test_xtclassApp.c file.

If you want to instruct XFaceMaker to load your DlistXt class next time you launch it, do a **Save Prefs** from the **File** menu. This will modify the XFaceMaker startup file so that it will load your class automatically. Alternately, you can load your class into XFaceMaker manually, using the Modules -> Class -> Load command.

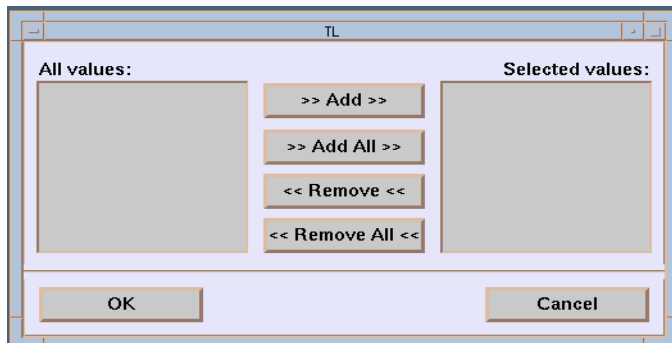
CHAPTER 22: Creating a C++ Class

When you work with an object model, you want to define your whole application using this concept, on both the application and graphic side. You want to define new graphic objects, having an internal behavior and functions to interact with these objects.

This chapter will show you how to define a C++ class in XFaceMaker, and how to use it in an interface.

22.1 The “DoubleList” class

The class that we are going to create looks like this:



It consists of two lists placed within scrolled windows with buttons that let you move selected items from one list to the other. The list on the left will contain the **source** items, i.e. items from which the user will make his selection of items to move to the list on the right, the list of **selected** items.

The source items will be stored in a public data member and we will provide public access functions to set and to read the data.

The selected items will be stored in a public data member and we will provide only a public function to read the data. So, this is read-only data.

We will define a public function to display the instance until the user has clicked on the **Ok** or **Cancel** button. On the **Ok** button, the function returns a string containing the selected items. On the **Cancel** button, the function returns an empty string.

We also define a public function allowing you to change the label of the components of the box.

To implement this class, we will need to define a private data member which contains the status of the box. We will define three states: the user is selecting the items, the user has clicked on the **Ok** button or the user has clicked on the *Cancel* button.

In this example, we will show you how to create a C++ class containing public member functions and private data and how we can define functions to access this data.

To use this class, we will create a small interface and the corresponding application, in order to see how to instantiate a C++ class and to call member functions from FACE.

To build this class, we will need to:

1. build the class sub-tree.
2. add internal behavior to transfer items between the lists.
3. define the class members.
4. define the OK and Cancel scripts.
5. save the class.

The class subtree of this C++ class is the same as the one used for the Xt class in the preceding chapter, so for steps 1 and 2, refer to that chapter. We only redefine the activate callback for the **Ok** and **Cancel** buttons.

22.2 Defining the class members

The `DoubleList` class will have the following members:

- data members
 - m *String* **listsrc**: stores the items of the source list. These items are in a string and they are separated by a comma.
 - m *String* **listdest**: stores the items of the destination list. These items are in a string and they are separated by a comma.
 - m **Int status**: a private data member where values are:
 - -1: the double list is in use
 - 0: the user has clicked on the **Ok** button
 - 1: the user has clicked on the **Cancel** button
- public member functions
 - m *String* **showdialog()**: displays the `DoubleList` object until the user has clicked on **Ok** or **Cancel**. This function returns the destination list items after **Ok**, otherwise an empty string after **Cancel**.
 - m *void* **SetLabels (String LListSrc, String LListDest, String LAdd, String LAddAll, String LRemove, String LRemoveAll, String LOk, String LCancel)**: sets the label of the components of the `DoubleList` class.

22.2.1 Data members

To define the class members (data and functions), select the `XmForm0` widget, open the Active Values window and set the *C++ class usage* toggle.

- *String* **listsrc**

The member name is **listsrc**, the type is **String** and the access is **public**. So, enter **listsrc** in the **Member Name** field, **String** in the **Type** field, select **public** in the **Access** toggle.

The member kind is *data*: select **data** in the **Member Kind** field.

We will want to define functions to set and to get the source list items. Therefore, set the toggle **Generate access functions**. These functions are not those defined implicitly by `XFaceMaker`. We want to define them ourselves, so that their definition will be contained in the set and get scripts.

These functions are called **set<Name>** and **get<Name>** where Name is the member name beginning with an uppercase. In our case, we have defined the *setListsrc* and *getListsrc* functions to set and to get the *listsrc* private data. The get script contains:

```
@ = FmListGetItems(self.xmlScrolledWindow3.listSrc);
```

When we read the *listsrc* data, we fetch the items of the source list.

The set script contains:

```
1: ListSetItems(self.xmlScrolledWindow3.listSrc, @);
2:
3: if (strcmp(@, "")) {
4:   self.Add:sensitive = true;
5:   self.AddAll:sensitive = true;
6: } else {
7:   self.Add:sensitive = false;
8:   self.AddAll:sensitive = false;
9: }
10:
```

When we set the *listsrc* data, we set the items of the source list. If the list is empty, the buttons **Add** and **AddAll** should be made insensitive, otherwise sensitive.

- *String listdest*

The *listdest* data member is similar to the *listsrc* data member except for one thing: we don't allow this data to be set, so we just define a get script and the set script is empty.

The get script of *listdest* is:

```
@ = FmListGetItems(self.xmlScrolledWindow4.listDest);
```

- *Int status*

Enter **status** in the **Member Name:** field

This private data member has type Int, so select **private** in the **Access** toggle and enter **Int** in the **Type** field.

We need to generate access functions because the value of **status** will be set in the **Ok** and **Cancel** callbacks, outside of the member functions. Select the **Generate access functions** toggle.

The set and get scripts are empty because we don't need to do anything special when we set or we read this private value.

22.2.2 Member functions

- **String = showdialog();**

showdialog() is a public function, to specify this, select **function** in **Member Kind** and **public** in **Access**.

Enter *showdialog* in the **Member Name** field.

This function has no argument and returns a string. So, the argument structure in the set script is defined like this:

```
struct showdialog_args {String ret;;};
```

The first field of the structure represents the return value. (Its type is String).

In the next member function, we will see how to define a function with arguments.

In the field **Type**: enter the name of the argument structure: *showdialog_args*.

Now, here is the entire contents of the set script:

```
1: struct showdialog_args {String ret;;};
2: showdialog_args args = @;
3:
4: /* show the dialog */
5: type Dlist "class Dlist *";
6: function None "Dlist::show()";
7: Dlist dlist;
8: dlist = self:$this;
9: dlist->show();
10:
11: self:@status = -1;
12: while (self:@status == -1)
13:   FmDoEvent();
14:
15: if (self:@status == 0) { /* OK */
16:   args->ret = self:@listdest;
17: } else { /* Cancel */
18:   args->ret = "";
19: }
20:
21: /* hide the dialog */
22: function None "Dlist::hide()";
23: dlist->hide();
```

```
24:
25:  /* reinitialize the destination list */
26:  ListSetItems(self.xmlScrolledWindow4.listDest, "");
```

line 2: defines a variable called *args*, its type is the argument structure, and its contents is the value of the current active value.

lines4-9: displays the instance of the *DoubleList* class.

line 5: declares a type *Dlist* which is a pointer to the *DoubleList* class.

line 6: declares the public member function *show* inherited by the class.

In effect, we will use the NSL predefined “style”, so that the class generated is derived from a predefined base class, called *NslUIC*, which provides constructors and useful member functions such as *show*, *hide*, *getWidget*, etc.

Note that the Rogue Wave style is also supported.

line 7: defines a *dlist* variable of type *Dlist*.

line 8: initializes the *dlist* variable to the current class object using the implicit **this** pointer. Note the syntax **widget:\$this**.

line 9 calls the public function *show*, note the syntax: **dlist->show()**.

lines 11-13: initializes the private data *status* to -1 and process the events until the **Ok** or the **Cancel** button has been activated.

lines 15-19: sets the return value:

- if *OK*: returns the contents of the destination list. It is the value of the public data *listdest*, which we can access using *self:@listdest* because we are in a method of the class.
- if *Cancel*: returns an empty string.

lines 21-23: defines and calls the public member function *hide* inherited by the class. it is the same remark as the *show* function.

- **void SetLabels (String LListSrc, String LListDest, String LAdd, String LAddAll, String LRemove, String LRemoveAll,String LOk, String LCancel):**

SetLabels() is a public function, which has as arguments the label of each of the push buttons of the *DoubleList* box. This function returns nothing.

Here is the set script:

```
1:  struct setlabels_args {None ret; String LListSrc;
2:  String LListDest; String LAdd; String LAddAll;
```

```
3:   String LRemove; String LRemoveAll; String LOk;
4:   String LCancel;};

5:   setlabels_args args = @;
6:
7:   self.listSrcLabel:labelString = args->LListSrc;
8:   self.listDestLabel:labelString = args->LListDest;
9:   self.Add:labelString = args->LAdd;
10:  self.AddAll:labelString = args->LAddAll;
11:  self.Remove:labelString = args->LRemove;
12:  self.RemoveAll:labelString = args->LRemoveAll;
13:  self.OK:labelString = args->LOk;
14:  self.Cancel:labelString = args->LCancel;
```

line 1-4: declares the argument structure, each *String* argument is a field of the structure, except for the first which is the return value (here we have *None ret* because this function returns nothing).

If you want to call a member function with arguments, you must define a structure where each field represents an argument and the first field the return value. Then to access an argument, we use the syntax to access a field of a structure.

line 5 declares a *args* variable of type *setlabels_args*.

line 7-14: sets the labels with the contents of the arguments.

Now, don't forget to enter *setlabels_args* in the *Type* field.

Now you have seen how to define public and private data members and public member functions.

22.3 Defining the OK and Cancel scripts

22.3.1 The OK button

In the activate callback, enter:

```
1:  function None "Dlist::setStatus"(Int);
2:  Dlist dlist;
3:  dlist = parent:$this;
4:  dlist->setStatus(0);
```

We just change the value of the private data *status*, it is now 0. For this, we call the public data member function *setStatus*.

line 1: declares the public data member function *setStatus*.

line 2: declares a *dlist* variable of type *Dlist*.

line 3: initializes the *dlist* variable with the pointer to the class object.

line 4: calls the *setStatus* function with 0 in the argument.

22.3.2 the Cancel button

The activate callback of the **Cancel** button is similar to the activate callback of the **Ok** button. The only change is the value of the private data *status*, it is 1 instead of 0.

Here is the activate callback script:

```
1:  function None "Dlist::setStatus"(Int);
2:  Dlist dlist;
3:  dlist = parent:$this;
4:  dlist->setStatus(1);
```

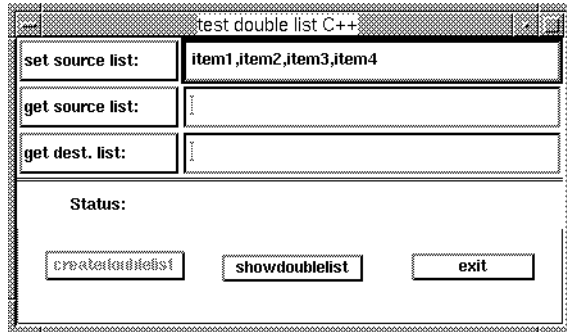
22.4 Saving the class

To save the class, select the *XmForm0* widget and choose the C++ class item from the Modules menu. Specify the name of the Fm file (*Dlist.fm*).

Click on the **Default Names** button, so that the C file containing the class will be called *Dlist.C*, the include file *Dlist.h* and the class *Dlist*.

22.5 Using the class

We are going to build a small interface allowing you to create an instance of the DList class, display this instance and access the class members.



The *createdoublelist* button calls two C functions which create an XmDialogShell widget and an instance of the DList class.

The *showdoublelist* button initializes the public data *listsrc*, changes the labels of the class instance, and displays this instance.

If the user has clicked on the **Ok** button of the DList instance, the selected items are displayed in the *get dest. list* field and the string “**ok**” is written in the **Status** field.

If the user has clicked on the *Cancel* button of the DList instance, the *get dest. list* field is reset and the string “*Cancel*” is written in the *Status* field.

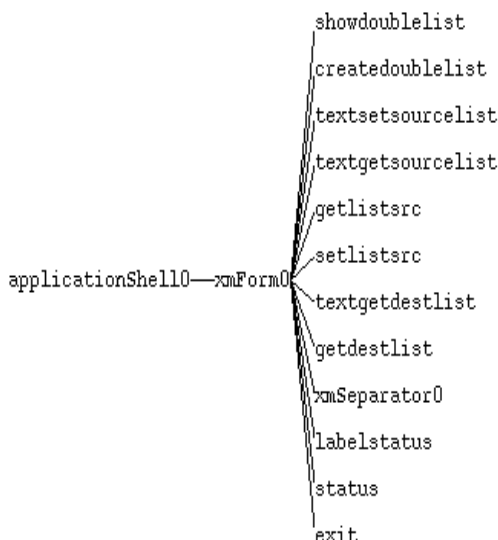
The *set source list:* button sets the public data *listsrc* with the contents of the text field next to it.

The *get source list:* button reads the public data *listsrc* and displays its value in the text field next to it.

The *get dest. list:* button reads the public data *listdest* and displays its value in the text field next to it.

22.5.1 In detail

- The widget tree



- The *createdoublelist* button

The contents of the activate callback are:

```

1:  /*create dialog shell */
2:  function Widget create_dialogshell(Widget parent);
3:  dialogshell = create_dialogshell(root);
4:
5:  /* create double list instance*/
6:  type Dlist "class Dlist *";
7:  function Dlist create_dlist(Widget parent);
8:  global Dlist dlist;
9:  dlist = create_dlist(dialogshell);
10:
11:  if (dlist != 0) {
12:    self:sensitive = false;
13:    parent.setlistsrc:sensitive=true;
14:    parent.getlistsrc:sensitive=true;
  
```

```
15:   parent.getdestlist:sensitive=true;
16:   parent.showdoublelist:sensitive=true;
17: }
```

The *create_dialogshell* function is a C function:

```
return( XtCreatePopupShell("xmDialogShell0",
                           xmDialogShellWidgetClass, parent, 0, 0));
```

The function above will be defined in the *<name>App.c* file in the next step (Saving the interface and the application files). The skeleton of the function is created by XFaceMaker when you save the application files; you only need to enter its contents.

The *create_dlist* function is a C function:

```
1:   /#if NeedFunctionPrototypes
2:   class Dlist * create_dlist(Widget parent)
3:   #else
4:   class Dlist * create_dlist(parent)
5:   Widget parent;
6:   #endif
7:   {
8:   /*## begin user  ##*/
9:
10:   Widget w; Arg arg;
11:   class Dlist *dlist;
12:
13:   dlist = new Dlist("dlist", parent);
14:
15:   w = dlist->getWidget();
16:   XtSetArg(arg, "defaultPosition", False);
17:   XtSetValues(w, &arg, 1);
18:
19:
20:   return (dlist);
21:
22:   /*## end user  ##*/
23: }
```

The function above will be defined in the *<name>App.c* file in the next step (Saving the interface and the application files).

line 13 creates an instance of the DList class.

line 15: calls the public function member *getWidget* which returns the toplevel widget of the class instance. This function is inherited by the class if we use the NSL style.

line 16-17: sets the *defaultPosition* resource of the toplevel widget of the class instance.

- **The *showdoublelist* button**

We are now going to see how to initialize a data member, to call member functions without argument but with a return value (like *showdialog()*), and with arguments but no return value (like *SetLabels()*).

The contents of the activate callback are:

```
1:  global Dlist dlist;
2:
3:  function None "Dlist::setListsrc" (String);
4:  dlist->setListsrc(parent.textsetsourcelist:value);
5:
6:  function None "Dlist::SetLabels"(String, String,
String,
7:  String, String, String, String, String);
8:  dlist->SetLabels("Source", "Destination", ">> Add
>>",
9:  ">> Add All >>", "<< Remove <<", "<< Remove All <<",
"OK", "Cancel");
10:  function String "Dlist::showdialog"();
11:
12:  ret = dlist->showdialog();
13:  if (ret !=0) {
14:  if (strcmp(ret, "")){
15:    parent.labelstatus:labelString = "ok";
16:    parent.textgetdestlist:value = ret;
17:  } else {
18:    parent.labelstatus:labelString = "cancel";
19:    parent.textgetdestlist:value = "";
20:  }
21:  }
```

lines 3-4: sets the public data *listsrc* with the contents of the text field *textset-sourcelist*. We call the data member function *setListsrc*.

lines 6-9: Calls the public function member *SetLabels* to change some labels of the instance.

lines 10-12: Calls the public function member *showdialog*.

lines 13-21: The string “**ok**” or “**cancel**” is displayed in the label status field in function of the *textgetdestlist* return value. If **ok**, the destination list items are displayed in the *textgetdestlist* text field.

- **The *setlistsrc* button**

The contents of the activate callback is:

```
1:  global Dlist dlist;
2:  function None "Dlist::setListsrc" (String);
3:  dlist->setListsrc(parent.textsetsourcelist:value);
```

This button updates the *listsrc* public data with the contents of the *textsetsourcelist* widget.

- **The *getlistsrc* button**

The contents of the activate callback is:

```
1:  global Dlist dlist;
2:  function String "Dlist::getListsrc" ();
3:
4:  listsource = dlist->getListsrc();
5:  parent.textgetsourcelist:value = listsource;
```

This button updates the *textgetsourcelist* text field with the value of the public data *listsrc*.

- **The *getlistdest* button**

The contents of the activate callback are:

```
1:  global Dlist dlist;
2:  function String "Dlist::getListdest"();
3:
4:  listdest = dlist->getListdest();
5:  parent.textgetdestlist:value = listdest;
```

This button updates the *textgetdestlist* text field with the value of the *listdest* public data.

- **Saving the interface and the application files**

Set the *Preserve User Code* toggle in the Options menu, so your modifications in the application files will be preserved when you save.

Save the interface in testdlist.fm.

Save the application files.

Enter the *create_dialogshell* and *create_dlist* functions in the testdlistApp.c file.

Now, we will modify the makefile to include the compilation of the Dlist class.

First of all: define the macro CC, for example, CC = g++.

We are only generate a compiled version of the application, not the interpreted version.

```
1:  all:: compiled
2:
3:  compiled:: ctestdlist
4:
5:  ctestdlist: Dlist.o ctestdlistMain.o  testdlistApp.o
6:                                     testdlist.o
7:  @sh -c "if [ -f $@ ] ; then mv $@ $@.bak ; $(RM)
8:                                     $@.bak ; fi"
9:  $(CC) $(LDFLAGS) $(CDEBUGFLAGS) -o ctestdlist
10:                                     ctestdlistMain.o testdlistApp.o\
11:                                     testdlist.o Dlist.o\
12:                                     $(XFM_C_LIBS) $(STD_LIBS)
13:
14:  ctestdlistMain.o: testdlistMain.c
15:  $(CC) $(CFLAGS) $(CDEBUGFLAGS) -DCCODE -c testdlistMain.c
16:  mv testdlistMain.o ctestdlistMain.o
17:
18:  testdlist.c: testdlist.fm
19:  $(XFM) -compile testdlist.fm \
20:  -cinclue "\"testdlist.h\" \"
21:  -preserve \
22:  -cfile testdlist.c -cname testdlist $(XFMCFLAGS)
23:
24:  Dlist.o: Dlist.C
25:  $(CC) $(CFLAGS) $(CDEBUGFLAGS) -c Dlist.C
```

```
25:
26:  Dlist.C: Dlist.fm
27:  $(XFM) -c++ Dlist.fm \
28:    -cinclude \"testdlist.h\" \
29:    -preserve \
30:    -c++name Dlist
```

lines 5-9: Adds Dlist.o (class object file) to build ctestdlist

lines 17-21: generates the testdlist.c file corresponding to the interface, note that the include file is *testdlist.h* instead of *testdlistApp.h*.

The *testdlist.h* include file contains:

```
1:  #include "Dlist.h"
2:  #include "testdlistApp.h"
```

We need to do this because the C source file needs to know the class definition (Dlist.h) and the definition of functions and variables used by the interface (testdlistApp.h). The testdlist.h file is created using a text editor.

lines 23-30: generates and compile the Dlist class.

Note that the argument is *-c++* instead of *-compile* and that we also include *testdlist.h*.

Now, do the following command:

```
make -f testdlistMake compiled
```

This will generate an executable called *ctestdlist*.

Launch it, create a Dlist instance with the *createdoublelist* button (this button becomes insensitive), initialize the source list in the first text field, then click on the *showdoublelist* button.

Select the items and press on the **Ok** button, check the *status* field, click again in the *showdoublelist* button, and then select the **Cancel** button.

This is one use of this class. Now, you can imagine an interface creating and using several instances of this class.

CHAPTER 23: Motif - General Concepts

23.1 Introduction

The purpose of this part of the document is to provide an overview of Motif widgets and gadgets for users of XFaceMaker. The information contained here is based on the Motif documentation but it does not claim to replace any of the standard Motif books. It presents the widgets and their resources in an often simplified manner, gives illustrations of their use, and shows how XFaceMaker can be used to implement interface fragments, where resource settings and instantiation techniques are not at first obvious.

23.2 Widgets and Gadgets

Motif provides a set of widgets (window objects) that are used as building blocks for Graphic User Interfaces. Each widget has an X window associated to it. For some widgets, among the ‘Primitive’ widgets, Motif also provides gadget equivalents. Motif gadgets behave in the same way and have the same specific resources as their corresponding widget, but they have no associated X window. Events generated in a gadget are received by their parent widget which dispatches the event to the child gadget. This is totally transparent to the end user who sees no difference in the behavior of a widget or a gadget.

In XFaceMaker, you always instantiate widgets. There is no gadget toolkit. However, you can change a widget into a gadget by setting the gadget toggle in the resources window. The toggle is sensitive only if you can ‘gadgetize’ a widget: it has a gadget equivalent, and you have not set resources that require an X window for the object.

23.2.1 Resources

The Motif objects (widgets and gadgets) have properties known as *resources*. For example, the width and height of a widget on the screen are resources. Resources can be specified when a widget is instantiated. They can be modified during the widget’s life within the interface.

In XFaceMaker, use the resource window to specify resource settings for a widget or gadget that you are instantiating. All the resources available for the current widget are listed in the resource window; the order in which they are listed is configurable.

In this document, resources belonging to different levels of the class hierarchy are listed separately. *You might want to use the same classification in the XFaceMaker resource window. That is the default setting.*

When editing a resource in the resource window, you can type the value directly, or open the specific resource editing window for that type of resource. You do not need to know the type of a resource, nor its acceptable values if you use the dedicated editing windows. You need only choose among the options in the window.

*There is one exception: when specifying resources of type Widget, you have to make sure that the widget that you are specifying as the resource value exists when you instantiate the widget you are editing. This is never the case when the resource value is a child of the current widget. To set such a resource with XFaceMaker, write a FACE script in the **xfmCreateCallback** of the widget. The script is executed once, when the widget is instantiated, after all its children have been created.*

To set resources dynamically, after a widget has been created, use FACE scripts associated to callbacks. In many cases, you don't need to know more than the resource's name and acceptable values. In some instances, you may have to worry about type conversion, e.g. between String and XmString, or Int and String. XFaceMaker generates error messages when resource conversions are needed and not done.

23.2.2 Classes

Widgets are organized in a class hierarchy. A widget class, say Class *b*, is derived from another class, say Class *a*, if it has all the resources defined by Class *a*, plus any additional resources defined by Class *b*. Class *b* is a *subclass* of Class *a*, and Class *a* is the *superclass* of Class *b*. A widget class can be subclassed from several levels of the hierarchy. It has the resources of all the classes in its hierarchy. Widget classes inherit not only resources, but also behavior from the classes above them. For example, PushButton is a subclass of Label; it inherits the ability to display labels.

All widgets of a class have the same resources. Several widget instances of a given class can/will have different resource *settings*.

Several classes cannot be instantiated: they serve as a *base* class from which other classes are derived. These do not appear in the XFaceMaker toolkits. Consider them as place holders for resources (and behavior) that will be available in their subclasses when these are instantiated. These will be referred to as *metaclasses* in this document.

23.3 Motif Class Hierarchy


```
| | | XmToggleButtonGadget *  
| | | XmCascadeButtonGadget *  
| | | XmArrowButtonGadget *  
| | | XmSeparatorGadget *
```

The widget classes that can be instantiated are marked with an asterisk, *along with the toolkit from which they can be fetched in XFaceMaker. There is no special toolkit for gadgets: they should be fetched as widgets from the Primitive Toolkit, and the gadget toggle should be set in the Resource Window.*

XFaceMaker provides additional Toolkits:

- m *Menu* - Menus are a variety of RowColumns. They are not a new class, so are not listed in the Motif class hierarchy tree; when creating menus with XFaceMaker, fetch the appropriate type of RowColumn from the Menu toolkit, rather than instantiating a RowColumn and setting its resources as needed. You can use the Menu Editor to fill in the menu items.
- m *Global* - The XmDisplay and XmScreen classes contained in this toolkit can be used to set the resources for the Display object and the Screen object associated to the application. These objects are not part of your interface. They are created at the same time as the first shell of the application (by XtInitialize or XtAppCreateShell) and they are used to access variables that would not otherwise be accessible. Most user interfaces do not deal with these classes.

And toolkits that are configuration dependent:

- m *UserDefined* - This includes any third party widgets that have been integrated into the tool, by NSL, or on site, by the user of the tool as well as new widget classes built with XFaceMaker and loaded into the tool.
- m *NSL* - This toolkit includes any widgets developed by NSL which are integrated into the tool, e.g in the Widget Factory option.
- m *NSL Draw Gadgets* - The NSL Draw Gadgets are part of the Widget Factory option.
- m *Group* - This toolkit includes any groups (widget subtrees) that are loaded into the tool. By default, no groups are loaded. The distribution includes groups such as ScrollText, ScrollList, MenuBar; the tool can be configured to load them. These groups can be found in the directory \$NSLHOME/xfm/Fm. Users can define their own groups. These, too, are accessible through the Group toolkit.
- m *Template* - This toolkit includes any templates that are loaded into the tool. Templates are similar to groups, but, unlike groups, when the model of the template is modified, all instances of the template in the interface are modified. Use tem-

plates to inforce a company style in interface building. No default templates are provided in the distribution; users can start from groups to build templates, or build templates from scratch.

CHAPTER 24: Motif - The Metaclasses

24.1 The Core Class

Core resources are common to all classes. We present the Core class editable resources in three categories: geometric resources, color and texture resources, and basic behavior resources.

Core also defines resources that are not editable in XFaceMaker: some are read-only resources, some are determined by the environment. In XFaceMaker, they are listed under the restricted resources.

Geometric resources deal with a widget's geometry - its size, its position. By default, geometric resources are specified in pixels. All of these resources are available for gadgets.

- m **borderWidth** - Specifies the width, in pixels, of the border surrounding the widget on all four sides.
- m **height** - Specifies the inside height (excluding the border) of the widget.
- m **width** - Specifies the inside width (excluding the border) of the widget.
- m **x** - Specifies the x-coordinate of the widget's upper left outside corner relative to its parent's upper left inside corner.
- m **y** - Specifies the y-coordinate of the widget's upper left outside corner relative to its parent's upper left inside corner.

*With XFaceMaker, a widget's size and position are specified as it is drawn in the work area. When you select a widget, the handles that are drawn at each of its corners can be used to change its size; its position can be changed by dragging the widget to a new position. Another method is to edit the x,y and w,h fields of the resources box directly. Finally, by setting a widget's auto position and auto size toggles in the resources box, you can remove **x,y** and/or **width, height** resource settings from the interface description file, letting the widget take its default size and position. You should set these toggles whenever a widget's size and/or position are fully constrained by its parent, in order to avoid overloading you interface file with unnecessary resource specifications. For example, a RowColumn with **packing** set to **pack_tight** or **pack_column** fully constrains its children's size and position resources.*

The color and texture resources of Core are available only for widgets. None of them are available in gadgets. They imply that a window is associated to the widget.

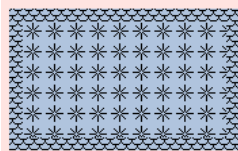
- m **background** - Specifies the color of the background.

m **backgroundPixmap** - Specifies the pattern used to tile the background.

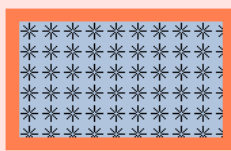
m **borderColor** - Specifies the color of the border.

m **borderPixmap** - Specifies the pattern used to tile the border.

backgroundColor=lightsteelblue



height=90
width = 150
x=10
y=15
borderWidth=10



height=90
width = 150
x=210
y=15
borderWidth=10



height=90
width = 150
x=400
y=15
borderWidth=10

borderPixmap=scales

borderColor=coral

backgroundPixmap=star backgroundPixmap=star

With *XFaceMaker*, you can specify color resources directly with the color name, as 'lightsteelblue' or 'coral'. You can select the color of your choice in a color palette provided in the editing box for color resources, or you can define a new color, using *XFaceMaker*'s color editing dialog box. Similarly, you can specify pixmaps directly with their name, or open *XFaceMaker*'s pixmap editing box to create new pixmaps, or load existing ones. Note that *Motif* uses a pixmaps cache: once a pixmap has been read, it is stored in the cache so that, the next time it is specified, it is fetched from the cache and not from the original file. *XFaceMaker*, being *Motif* based, uses the pixmaps cache as well. You may find this somewhat disturbing when editing pixmaps: any changes you make to a pixmap that has been loaded in the cache will not be reflected in the entities using that pixmap as long as you remain in the same session.

Of the basic behavior resources, only the **destroyCallback** and the **sensitive** resources are available in gadgets.

Callbacks let the application associate additional behavior to a widget. In *XFaceMaker*, you can program a callback directly by writing a *FACE* script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.

m **accelerators** - Specifies keyboard shortcuts for commands; the format is the same as that of the translation table. Accelerators can be installed on destination widgets with the function `XtInstallAccelerators`, so that an event received

in the destination widget triggers the action on the source widget (the widget on which the **accelerators** resource is specified). This is the only way to specify keyboard shortcuts for PushButtons that are not in a menu. For Menus, Motif introduces another resource, **accelerator**, which is defined in the XmLabel class (see section) and is much easier to use; with the **accelerator** resource, the keyboard shortcut activates the appropriate menu item regardless of where the keyboard focus is in the window.

- m **destroyCallback** - Invoked when the widget is destroyed.
- m **mappedWhenManaged** - If *True* (the default), the widget is visible as soon as it is both realized and managed. If *False*, the widget has to be explicitly mapped and/or unmapped.
- m **sensitive** - If *True*, the widget can receive X events; these include mouse and keyboard events, window enter/leave events, and focus events. When *False*, the widget can receive none of these events. When a widget is set insensitive, its label is usually dimmed. When a component becomes unavailable for user interaction, you can set it insensitive; another way is to hide the component altogether. In general, isolated components that become unavailable are set insensitive, whereas entire windows which become unavailable are unmapped.
- m **translations** - Specifies the table which associates X events to widget actions. Widgets have a default translation table which is normally sufficient: they are designed for a specific purpose, they have callbacks that correspond to their intended use. There are instances, however, where the interface designer would like to modify or add to a widget's default behavior. This can be done by redefining the translation table: you can entirely redefine the translations for a widget (use *replace* mode) or add new event to action definitions (use *override* or *augment* modes). *The XFaceMaker translation editing box has the standard Xevent and widget action tables, plus one special action: Eval. With the Eval action, you can associate a full FACE script to an event. As in all FACE scripts, you can call application functions from the Eval action. This is equivalent to adding an event handler to a widget.*

The restricted resources are not editable in XFaceMaker; they are either read-only, or environment dependent. They cannot be modified interactively. The restricted resources for Core class are:

- m **ancestorSensitive** - Specifies whether the immediate parent of the widget is sensitive.
- m **colormap** - Specifies the colormap used for the widget
- m **depth** - Specifies the number of planes, i.e. the number of bits that can be used

for each pixel.

m **screen** - Specifies the screen on which the widget instance resides.

24.2 The Composite Class

Composite widgets are designed to be containers for other widgets; they can have an arbitrary number of children. They are responsible for:

- the management of their children, from creation to destruction
- the destruction of their descendants when they themselves are destroyed.
- managing their children's geometry.
- mapping and unmapping their children.

This class inherits resources from Core class.

All the resources defined by the Composite class are listed in the restricted resources in XFaceMaker: their values are read-only, except the **insertPosition** resource; with XFaceMaker, you specify this resource by controlling the order in which child widgets are created.

m **children** - The list of children of the widget.

m **insertPosition** - The relative position of the widget in the list of children. This resource is important for some Composite widgets, such as RowColumn where the layout of children depends on their **insertPosition**. In XFaceMaker, the order in which sibling widgets are created is the order in which they appear in the widget tree or list. Moving a widget relative to its siblings in the tree or list changes the setting of this resource.

m **numChildren** - The length of the list of children.

24.3 The Constraint Class

Constraint widgets maintain resources that are transferred to their direct children. All children of a composite widget that defines constraint resources inherit these resources. Inherited resources are handled just like the child widget's regular resources. However, they are used by the parent, usually to control each child's geometry. Constraint resources are never used directly by the widget which has inherited them, they are meant for use by the parent.

This class inherits resources from Core and Composite classes.

The Constraint class does not have any resources of its own.

24.4 The XmManager Class

Manager class specifies resources to support the graphics and traversal mechanisms of its subclasses. It also specifies a callback resource.

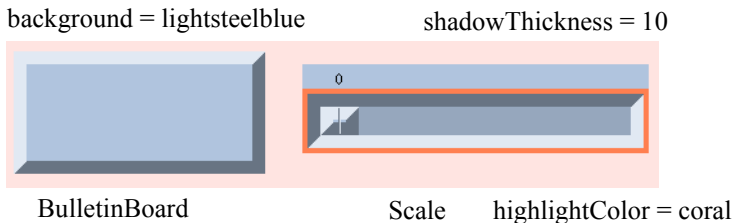
This class inherits resources from Core, Composite and Constraint classes.

Callbacks let the application associate additional behavior to a widget. *In XFace-Maker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The callback resource for Manager is:

- m **helpCallback** - Invoked when the help key sequence is pressed (usually the key labeled *Help* on the keyboard).

Some of the graphics resources defined by Manager are illustrated below.



topShadowColor and bottomShadowColor defaults.

Motif selects the shadow colors with the assumption that the light source is placed in the upper left of the screen, forming a 45 degree angle with the widgets. Notice how the BulletinBoard seems to come out of the screen, because its topShadowColor is lighter than its bottomShadowColor. The Scale slider seems to stick out from the trough which, itself, seems to sink into the screen.

The graphics resources for Manager are:

- m **bottomShadowColor** - Specifies the color used to draw the bottom and right side of the border shadow if no **bottomShadowPixmap** has been specified. If no color is specified, Motif will choose a color based on the background color, to provide the typical three dimensional effect.
- m **bottomShadowPixmap** - Specifies the pixmap used to tile the bottom and right side of the border shadow.
- m **foreground** - Specifies the foreground drawing color.

- m **highlightColor** - Specifies the color of the highlight rectangle if no **highlightPixmap** is specified.
- m **highlightPixmap** - Specifies the pixmap used to tile the highlight rectangle.
- m **shadowThickness** - Specifies the thickness of the border shadow.
- m **topShadowColor** - Specifies the color used to draw the top and left side of the border shadow if no **topShadowPixmap** has been specified. If no color is specified, Motif will choose a color based on the background color, to provide the typical three dimensional effect.
- m **topShadowPixmap** - Specifies the pixmap used to tile the top and left side of the border shadow.
- m **unitType** - Specifies the type of units used with sizing and positioning resources. The default is taken from the parent's `unitType` resource or defaults to `pixels`. Acceptable values are *pixels*, *100th_millimeters*, *1000th_inches*, *100th_points*, *100th_font_units*.

The resources that control traversal are:

- m **initialFocus** - Meaningful only if the nearest shell ancestor's **keyboardFocusPolicy** is set to *explicit*, this resource specifies which of its children receives focus. *In XFaceMaker, use the manager's xfmCreateCallback resource. This callback is executed when the manager is being created, after all its children have been created, i.e. once the widget which should receive initial focus exists. In this callback, to specify that the widget named scale0 should receive initial focus, you could write something like:*

```
self.initialFocus = self.scale0;
```

- m **navigationType** - Determines whether a widget is a tab group, i.e how it should receive keyboard focus using keyboard traversal. The only values you should use are *none* and *tab_group*. The other values have been kept in Motif for backward compatibility; new interfaces should avoid using them. Acceptable values for this resource are:
 - *none* - the widget is not a tab group; it is a member of a tab group; the direction arrows are used to move the focus among widgets within a tab group.
 - *tab_group* - the widget is a tab group header (unless another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The *tab* or *Shift + tab* keys are used to bring focus to it.
 - *sticky_tab_group* - the widget is a tab group header (even if another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The

tab or *Shift + tab* keys are used to bring focus to it.

- *exclusive_tab_group* - the widget is a tab group and widgets in the hierarchy whose **navigationType** is *tab_group* are not tab groups.
- m **traversalOn** - If *True*, traversal is active for the widget, i.e. it can receive keyboard focus using keyboard navigation. If *False*, the widget is removed from the traversal list.

Manager defines an additional resource which is not used internally by the widget, but is a place holder for application specific information:

- m **userData** - Allows the application to attach any necessary specific data to the widget. This resource is not used internally by any of the widgets. *To set this resource in a FACE script, you may need to cast it to Pointer:*

```
w:userData = Pointer(3);
```

24.5 The XmPrimitive Class

Primitive class specifies resources to support border drawing and highlighting, traversal activation and deactivation and a callback needed by its subclasses.

This class inherits resources from Core class.

Callbacks let the application associate additional behavior to a widget. *In XFace-Maker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The callback resource for Primitive is:

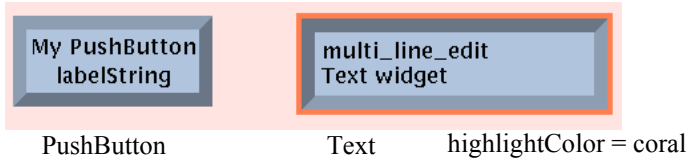
- m **helpCallback** - Invoked when the help key sequence is pressed (usually the key labeled *Help* on the keyboard).

Motif selects the shadow colors with the assumption that the light source is placed in the upper left of the screen, forming a 45 degree angle with the widgets.

Some of the graphic resources defined by Primitive are shown below.

background = lightsteelblue

shadowThickness = 10



topShadowColor and bottomShadowColor defaults

Motif selects the shadow colors with the assumption that the light source is placed in the upper left of the screen, forming a 45 degree angle with the widgets. Notice how the PushButton seems to come out of the screen, because its topShadowColor is lighter than its bottomShadowColor. The Text input field seems to sink into the screen.

The graphics resources for Primitive are:

- m **bottomShadowColor** - Specifies the color used to draw the bottom and right side of the border shadow if no **bottomShadowPixmap** has been specified. If no color is specified, Motif will choose a color based on the background color, to provide the typical three dimensional effect.
- m **bottomShadowPixmap** - Specifies the pixmap used to tile the bottom and right side of the border shadow.
- m **foreground** - Specifies the foreground drawing color.
- m **highlightColor** - Specifies the color of the highlight rectangle if no **highlight-Pixmap** is specified.
- m **highlightOnEnter** - If *True*, and the **keyboardFocusPolicy** is *pointer*, the highlighting rectangle is drawn if the cursor moves into the widget. If *False* (the default), and **keyboardFocusPolicy** is *pointer*, the highlighting rectangle is not drawn if the cursor moves into the widget. If **keyboardFocusPolicy** is *pointer*, this resource is ignored and the highlight rectangle is drawn when the widget has focus.
- m **highlightPixmap** - Specifies the pixmap used to tile the highlight rectangle.
- m **highlightThickness** - Specifies the thickness of the highlight rectangle. Default is 2.
- m **shadowThickness** - Specifies the thickness of the border shadow. Default is 2.
- m **topShadowColor** - Specifies the color used to draw the top and left side of the

border shadow if no **topShadowPixmap** has been specified. If no color is specified, Motif will choose a color based on the background color, to provide the typical three dimensional effect.

- m **topShadowPixmap** - Specifies the pixmap used to tile the top and left side of the border shadow.
- m **unitType** - Specifies the type of units used with sizing and positioning resources. The default is taken from the parent's unitType resource or defaults to pixels. Acceptable values are *pixels*, *100th_millimeters*, *1000th_inches*, *100th_points*, *100th_font_units*.

The resources that control traversal are:

- m **navigationType** - Determines whether a widget is a tab group, i.e how it should receive keyboard focus using keyboard traversal. The only values you should use are *none* and *tab_group*. The other values have been kept in Motif for backward compatibility; new interfaces should avoid using them. Acceptable values for this resource are:
 - *none* - the widget is not a tab group; it is a member of a tab group; the direction arrows are used to move the focus among widgets within a tab group.
 - *tab_group* - the widget is a tab group header (unless another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The *tab* or *Shift + tab* keys are used to bring focus to it.
 - *sticky_tab_group* - the widget is a tab group header (even if another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The *tab* or *Shift + tab* keys are used to bring focus to it.
 - *exclusive_tab_group* - the widget is a tab group and widgets in the hierarchy whose **navigationType** is *tab_group* are not tab groups.
- m **traversalOn** - If *True*, traversal is active for the widget, i.e. it can receive keyboard focus using keyboard navigation. If *False*, the widget is removed from the traversal list.

Primitive defines an additional resource which is not used internally by the widget, but is a place holder for application specific information:

- m **userData** - Allows the application to attach any necessary specific data to the widget. This resource is not used internally by any of the widgets. *To set this resource in a FACE script, you may need to cast it to Pointer:*

```
w:userData = Pointer(3);
```

24.6 The XmGadget Class

Gadget class specifies resources to support border drawing and highlighting, traversal activation and deactivation and a callback needed by its subclasses. It plays, for gadgets, the same role as Primitive for the Primitive widgets.

The color and pixmap resources defined by XmManager are directly used by gadgets, since gadgets do not have an associated window.

This class inherits resources from Object and RectObj classes. These are components of the Core class. In the description of resources for Core class, we have singled out those resources that are inherited by Gadget, i.e. the resources defined by Object and RectObj classes.

Callbacks let the application associate additional behavior to a widget. *In XFace-Maker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The callback resource for Gadget is:

- m **helpCallback** - Invoked when the help key sequence is pressed (usually the key labeled *Help* on the keyboard).

The graphics resources for Gadget are:

- m **bottomShadowColor** - Specifies the color used to draw the bottom and right side of the border shadow. If no color is specified, Motif will choose a color based on the background color, to provide the typical three dimensional effect.
- m **highlightColor** - Specifies the color of the highlight rectangle.
- m **highlightOnEnter** - If *True*, and the **keyboardFocusPolicy** is *pointer*, the highlighting rectangle is drawn when the cursor moves into the gadget. If *False* (the default), and **keyboardFocusPolicy** is *pointer*, the highlighting rectangle is not drawn if the cursor moves into the gadget. If **keyboardFocusPolicy** is *pointer*, this resource is ignored and the highlight rectangle is drawn when the gadget has focus.
- m **highlightThickness** - Specifies the thickness of the highlight rectangle. Default is 2.
- m **shadowThickness** - Specifies the thickness of the border shadow. Default is 2.
- m **topShadowColor** - Specifies the color used to draw the top and left side of the border shadow. If no color is specified, Motif will choose a color based on the

background color, to provide the typical three dimensional effect.

- m **unitType** - Specifies the type of units used with sizing and positioning resources. The default is taken from the parent's **unitType** resource or defaults to pixels. Acceptable values are *pixels*, *100th_millimeters*, *1000th_inches*, *100th_points*, *100th_font_units*.

The resources that control traversal are:

- m **navigationType** - Determines whether a widget is a tab group, i.e how it should receive keyboard focus using keyboard traversal. The only values you should use are *none* and *tab_group*. The other values have been kept in Motif for backward compatibility; new interfaces should avoid using them. Acceptable values for this resource are:
 - *none* - the widget is not a tab group; it is a member of a tab group; the direction arrows are used to move the focus among widgets within a tab group.
 - *tab_group* - the widget is a tab group header (unless another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The *tab* or *Shift + tab* keys are used to bring focus to it.
 - *sticky_tab_group* - the widget is a tab group header (even if another widget in the hierarchy has its **navigationType** set to *exclusive_tab_group*) . The *tab* or *Shift + tab* keys are used to bring focus to it.
 - *exclusive_tab_group* - the widget is a tab group and widgets in the hierarchy whose **navigationType** is *tab_group* are not tab groups.
- m **traversalOn** - If *True*, traversal is active for the widget, i.e. it can receive keyboard focus using keyboard navigation. If *False*, the widget is removed from the traversal list.

Gadget defines an additional resource which is not used internally by the gadget, but is a place holder for application specific information:

- m **userData** - Allows the application to attach any necessary specific data to the widget. This resource is not used internally by any of the widgets. *To set this resource in a FACE script, you may need to cast it to Pointer:*

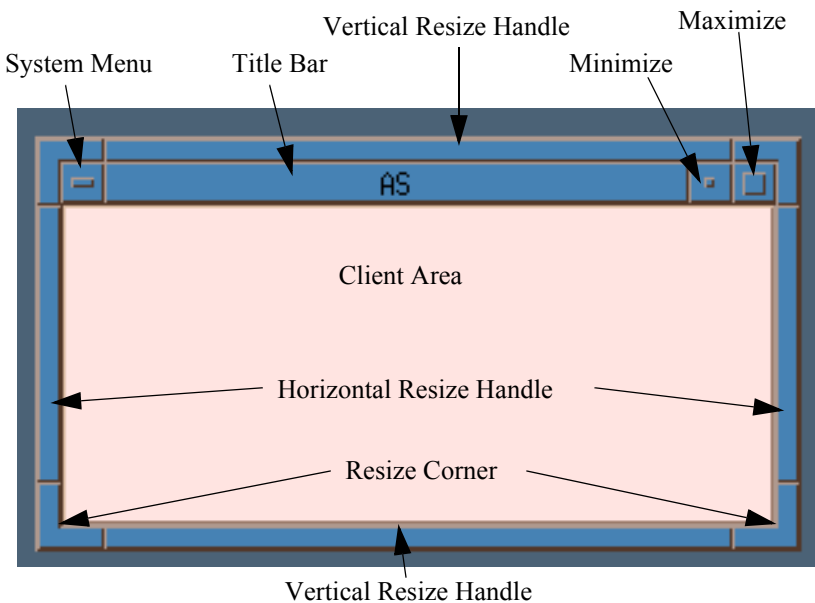
```
w:userData = Pointer(3);:
```


CHAPTER 25: Motif - The Shell Widgets

25.1 Introduction

Shell widgets are composite widgets that can manage only one child. They provide the interface between the window manager and the widgets they manage. Usually, the only child of a shell widget is a composite widget, for example a BulletinBoard. However, there's nothing that prevents that child from being a Primitive widget, say a PushButton. In that case, no other widget can be managed by the shell.

Every window that can be placed independently on the screen, including top-level windows and dialog boxes has as its parent a shell widget. The shell widget and its immediate child have the same size; you can think of the shell as shrink-wrapped around its child widget. Shells have *window decorations* that can be used to interact with the window manager.



The illustration above shows the standard Motif window decorations which allow a user to resize, iconify, move windows, and access the System Menu; this has items to raise or lower the window, move it, and close it.

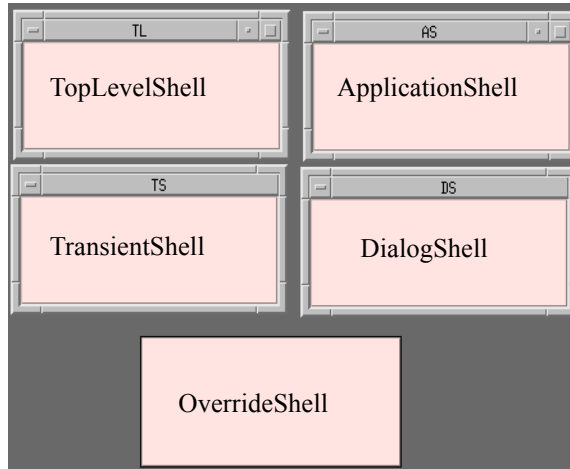
Applications usually have more than one independent window. In addition to the main window, which is in an `ApplicationShell`, it may have one or several dialog boxes, which will use `XmDialogShells`. If the interface has some secondary top level windows (windows that remain more or less permanently on the screen) these will use `TopLevelShells`.

Like other widget classes, shell widgets proceed from a class hierarchy, each level contributing its set of resources. The shell widgets that can be instantiated are leaves of this hierarchy, the intermediate shell classes being *metaclasses*.

The shells that can be instantiated are:

- **ApplicationShell** - An interface should have one and only one `ApplicationShell`. It is the main top-level window of the interface and serves as the top widget of the widget hierarchy that constitutes the interface. All other widgets are in sub-trees, derived from the `ApplicationShell`.
- **TopLevelShell** - If the interface includes secondary main windows, these should be built with a `TopLevelShell`. A `TopLevelShell` looks like an `ApplicationShell`, it interacts with the window manager with the same decorations. However, in most applications, when you close a `TopLevelShell`, you close just that window, while when you close an `ApplicationShell`, you exit the application. When you iconify a `TopLevelShell`, you iconify its sub-tree.
- **TransientShell** - As its name implies, `TransientShell` is intended for windows that are not permanent, e.g. to display dialogs where the user is asked a question and the window is closed after the user has replied. In Motif applications you should use the `XmDialogShell` (which is subclassed from `TransientShell`) rather than `TransientShell` to implement dialogs. `TransientShells` cannot be iconified separately. They are iconified along with the `ApplicationShell` or `TopLevelShell` that is above them in the widget hierarchy. A `TransientShell` window is always displayed above its parent, i.e. it cannot be hidden by its parent.
- **XmDialogShell** - This class is a Motif-defined widget class that is subclassed from `TransientShell`. `DialogShells` can be modal: no user interaction is allowed with other windows in the application as long as the modal window is up. Much of the functionality of `DialogShell` assumes that its child is a `BulletinBoard` or `BulletinBoard` subclass; similarly, several of `BulletinBoard`'s resources are meaningful only if its parent is an `XmDialogShell`. This is why it is better to build dialogs with `XmDialogShell` than with `TransientShell` in Motif interfaces.
- **OverrideShell** - This shell does not interact with the Window Manager. It has no decorations. Few interfaces make use of `OverrideShell`, except to display menus. Motif defines `XmMenuShell` as a subclass of `OverrideShell`. All menus in Motif

interfaces have a MenuShell hidden behind them. MenuShells are not accessible to the interface developer, they are handled directly by Motif. *With XFaceMaker, MenuShells do not appear in the widget hierarchy and they are not in the Shell Toolkit since they cannot be instantiated. The OverrideShell can be instantiated, it is in the Shell Toolkit but it is very seldom used in interfaces.*



Notice, in the illustration above, that TransientShell and DialogShell do not have the minimize and maximize buttons. The OverrideShell has no Window Manager decorations at all.

25.2 The Shell Class

Shell is a top-level widget with only one managed child that encapsulates interaction with the window manager. The **width** and **height** of the Shell and its child are the same once the Shell is realized and its child managed. If **width** and **height** are not specified explicitly in the Shell, the values are taken from the child. If, on the contrary, Shell's **width** and **height** have been specified, those values are used for both widgets. *With XFaceMaker use the autosize toggle to remove width and height specifications on the shell and have its size determined by the child widget it contains.*

This class inherits resources from Core and Composite classes.

Callbacks let the application associate additional behavior to a widget. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The callback resources defined for Shell are:

- m **popdownCallback** - Invoked when the instance is popped down by XtPopdown.
- m **popupCallback** - Invoked when the instance is popped up by XtPopup.

The editable resources defined for Shell are:

- m **allowShellResize** - If *False*, the Shell refuses any geometry requests from its children once it has been realized. If *True*, and the window manager allows it, the Shell will grow when its child wants to grow, even once it has been realized.
- m **geometry** - Specifies the values of **x**, **y**, **width**, **height** when the Shell is unrealized and a manager has changed the number of its managed children. The format of this resource is:
`widthxheight±xoff±yoff`
- m **overrideRedirect** - If *True*, specifies that the widget instance is a temporary window that should be ignored by the window manager. You should not set this resource directly. It is set when you instantiate an `OverrideShell`.
- m **saveUnder** - If *True*, specifies that the contents of the screen beneath the widget instance should be saved, in order to avoid expose events.

Shell has two restricted resources, that you cannot edit with `XFaceMaker`:

- m **createPopupChildProc** - A pointer to a procedure that creates a child widget when the Shell is popped up instead of creating it when the interface fragment is instantiated. *With XFaceMaker, the C code generated always creates all the children along with the Shell.*
- m **visual** - Specifies the visual used when creating the widget.

25.3 The OverrideShell Class

`OverrideShell` is used for shell windows that do not interact with the window manager. When an `OverrideShell` is instantiated, the **overrideRedirect** and **saveUnder** resources are forced to *True*.

This class inherits resources from `Core`, `Composite` and `Shell` classes.

It defines no new resources.

25.4 The XmMenuShell Class

The XmMenuShell is designed specifically to contain Popup and Pulldown Menu-Panels. Whenever you create a menu, an XmMenuShell is created in order that the menu not be clipped by its underlying window. *With XFaceMaker, XmMenuShells are not shown in the widget tree since their resources do not need editing, and they are automatically created. In normal situations, you will not encounter XmMenuShells directly.*

This class inherits resources from Core, Composite and Shell classes.

25.5 The WmShell Class

The WmShell defines resources that control interaction with the window manager. It is a meta class; it is not instantiated directly but defines new resources that shell classes inherit.

This class inherits resources from Core, Composite and Shell classes.

It defines new resources that control

- m size constraints
- m the aspect of icons when the window is iconified
- m behavior parameters

It defines no callback resources.

The resources that constrain the size of the window are:

- m **baseHeight** - Defines the base of progression for preferred heights. Preferred heights are **baseHeight** plus a multiple of **heightInc**.
- m **baseWidth** - Defines the base of progression for preferred widths. Preferred widths are **baseWidth** plus a multiple of **widthInc**.
- m **heightInc** - Defines the values allowed for the widget's height. Height values are **baseHeight** plus a multiple of **heightInc**.
- m **maxAspectX** - Specifies the numerator of the max aspect ratio (X/Y) that the shell instance should have.
- m **maxAspectY** - Specifies the denominator of the max aspect ratio (X/Y) that the shell instance should have.
- m **maxHeight** - Specifies the maximum height the shell instance can have.
- m **maxWidth** - Specifies the maximum width the shell instance can have.

- m **minAspectX** - Specifies the numerator of the min aspect ratio (X/Y) that the shell instance should have.
- m **minAspectY** - Specifies the denominator of the min aspect ratio (X/Y) that the shell instance should have.
- m **minHeight** - Specifies the minimum height the shell instance can have.
- m **minWidth** - Specifies the minimum width the shell instance can have.
- m **widthInc** - Specifies the values allowed for the widget's width. Width values are **baseWidth** plus a multiple of **widthInc**.

The resources that control icons for the window are:

- m **iconMask** - Specifies the bitmap to be used as a mask that will clip the **iconPixmap**. Use this to define nonrectangular icons.
- m **iconPixmap** - Specifies the bitmap used by the window manager to represent the application when it is iconified.
- m **iconWindow** - Specifies the ID of a window that could be used as the application's icon - seldom used.
- m **iconX** - Specifies a hint to the window manager for the x coordinate of the application's icon, in root window coordinates. This may be ignored by the window manager.
- m **iconY** - Specifies a hint to the window manager for the y coordinate of the application's icon, in root window coordinates. This may be ignored by the window manager.

The resources that control behavior of the shell instance are:

- m **initialState** - Specifies the state in which the window should start: *normalstate* or *iconicstate*. The default is *normalstate*.
- m **input** - Specifies the application's input model for this widget and its descendants.
- m **title** - Specifies the application name displayed by the window manager - in mwm, the text in the title bar area. The default is the icon name, if it is specified, and, otherwise, the name of the application.
- m **titleEncoding** - Specifies the property that represents the encoding of the title string. *This resource cannot be edited with XFaceMaker, it appears in the restricted resources of the resource window.*
- m **transient** - A boolean value that is true for transient windows (TransientShell and XmDialogShell instances). The Motif window manager (mwm) does not

iconify transient shells independently. This resource is not normally set by the user, it is set to the correct value according to the type of shell being instantiated. However, if this resource is set to *True* for a *ToplevelShell*, the shell will be iconified along with its parent *ApplicationShell*.

- m **waitForWm** - When true, specifies that the system should wait for the time specified by **wmTimeout** before assuming that there is no window manager.
- m **windowGroup** - Specifies the window identifier of a window to which this widget is associated. The window manager may treat all windows of a group in the same way, e.g. iconify them all. The default is the window of the first realized ancestor widget in the parent hierarchy when the widget is realized.
- m **winGravity** - Specifies the window gravity used by the window manager when positioning the widget.
- m **wmTimeout** - Specifies how long the window manager should be waited for before assuming there is none present. The default is *5000 msec*.

25.6 The VendorShell Class

The *VendorShell* defines resources that interact directly with the Motif window manager. It is a meta class; it is not instantiated directly but defines new resources that shell classes inherit.

This class inherits resources from *Core*, *Composite*, *Shell* and *WMSHELL* classes.

The most often used resources defined by *VendorShell* are:

- m **buttonFontList** - Specifies the default fontList that should be used for *Button* descendants of the widget instance. The resource is initialized to the value of **defaultFontList** if it is non-NULL. If **defaultFontList** is NULL, the parent hierarchy is searched for an ancestor subclass of *BulletinBoard*, *VendorShell*, or *MenuShell*. If such an ancestor is found, the resource is initialized to the **buttonFontList** value of the ancestor. If no such ancestor is found, the default is implementation dependent.
- m **defaultFontList** - Specifies a font list for the widget's descendants. This resource is kept for compatibility with earlier releases; you should use **buttonFontList**, **labelFontList**, and **textFontList** instead.
- m **keyboardFocusPolicy** - Specifies how widgets in the hierarchy rooted at this window receive keyboard focus. Acceptable values are *explicit* (for click-to-type policy) and *pointer* (for pointer-driven policy). The default is *explicit*.
- m **labelFontList** - Specifies the default fontList that should be used for *Label* de-

scendants of the widget instance. The resource is initialized to the value of **defaultFontList** if it is non-NULL. If **defaultFontList** is NULL, the parent hierarchy is searched for an ancestor subclass of BulletinBoard, VendorShell, or MenuShell. If such an ancestor is found, the resource is initialized to the **labelFontList** value of the ancestor. If no such ancestor is found, the default is implementation dependent.

- m **textFontList** - Specifies the default fontList that should be used for text and List descendants of the widget instance. The resource is initialized to the value of **defaultFontList** if it is non-NULL. If **defaultFontList** is NULL, the parent hierarchy is searched for an ancestor subclass of BulletinBoard, VendorShell, or MenuShell. If such an ancestor is found, the resource is initialized to the **textFontList** value of the ancestor. If no such ancestor is found, the default is implementation dependent.

Other specific resources for this widget are:

- m **audibleWarning** - Specifies whether an action generates its associated audible cue. Acceptable values are *bell* (the default) or *none*.
- m **deleteResponse** - Determines which action the shell takes in response to a WM_DELETE_WINDOW message. Possible values are *destroy* (the default for all shells except XmDialogShell), *unmap* (the default for XmDialogShell) and *do_nothing*.
- m **inputMethod** - Specifies the string that sets the locale modifier for the input method .
- m **mwmDecorations** - Used to specify the components of the mwm decorations that should be displayed on the shell. This is the bitwise OR of one or more flag bits. *To specify this resource with XFaceMaker, you have to provide the arithmetic value resulting from the OR of the flag bits you want to set.* The flag bit constants are defined in Xm/MwmUtil.h. The flag bit names are:
 - MWM_DECOR_ALL
 - MWM_DECOR_BORDER - window border
 - MWM_DECOR_RESIZEH - resize handles
 - MWM_DECOR_TITLE - title bar
 - MWM_DECOR_MENU - window menu button
 - MWM_DECOR_MINIMIZE - minimize window button
 - MWM_DECOR_MAXIMIZE - maximize button
- m **mwmFunctions** - Used to specify the function flags to apply to the window for

the `_MOTIF_WM_HINTS` property. To specify this resource with *XFaceMaker*, you have to provide the arithmetic value resulting from the OR of the function bits you want to set. The function bit constants are defined in `Xm/MwmUtil.h`. The function bit names are:

- `MWM_FUNC_ALL`
- `MWM_FUNC_RESIZE`
- `MWM_FUNC_MOVE`
- `MWM_FUNC_MINIMIZE`
- `MWM_FUNC_MAXIMIZE`
- `MWM_FUNC_CLOSE`

m **mwmInputMode** - Specifies the input mode flags (application modal or system modal input constraints). This resource is normally not set directly. It is controlled by the **dialogStyle** resource in the `BulletinBoard`.

m **mwmMenu** - Specifies the menu items that the Motif window manager should add to the end of the window menu. The syntax for each item is: **Label [mnemonic] [accelerator] function** each item being entered on a separate line.

m **preeditType** - Specifies the input method styles available to the input manager. The syntax, values accepted, and default value are implementation dependent.

m **shellUnitType** - Determines the interpretation of geometric resources. Acceptable values are:

- `pixels`
- `100th_millimeters`
- `1000th_inches`
- `100th_points`
- `100th_font_units`

m **useAsyncGeometry** - When this is set to *True*, the geometry manager sets **waitForWm** to *False* and **wmTimeout** to *0*, and relies on asynchronous notification. The default value is *False*.

25.7 The TransientShell Class

TransientShells are used for windows that have a short life on the screen, like dialogs; they cannot be iconified separately and are iconified along with their TopLevelShell or ApplicationShell ancestor. Motif defines an XmDialogShell class which is subclassed from TransientShell and should be used for dialog windows rather than TransientShell.

This class inherits resources from Core, Composite, Shell, WMShell and VendorShell classes.

This class overrides the **saveUnder** resource in Shell, setting it to *True*, and the **transient** resource in WMShell, setting it to *True*.

It defines only one specific resource:

- m **transientFor** - Specifies a widget for which the shell is a popup. *With XFaceMaker, you don't have to specify this resource.*

25.8 The XmDialogShell Class

DialogShells can be modal or modeless. They are used for windows that are transient in nature. Like TransientShells, DialogShells are not iconified separately from their TopLevelShell or ApplicationShell ancestor. Some of the resources of BulletinBoard are meaningful only if its parent is a DialogShell. See “XmBulletinBoard” on page 260.

This class inherits resources from Core, Composite, Shell, WMShell, VendorShell, and TransientShell classes.

It defines no new resources.

25.9 The TopLevelShell Class

TopLevelShell is used for windows that are meant to remain on the screen, i.e. any top level windows in addition to the ApplicationShell. When a TopLevelShell is iconified, it takes with it any transient descendants. A TopLevelShell can be iconified separately from the ApplicationShell to which it is associated. An application can have only one ApplicationShell but as many TopLevelShells as required.

*TopLevelShells are a good place to subdivide interface description files when building a project with XFaceMaker. To build a project, load the full interface file, select a TopLevel, save it in an .fm file with the **SaveSelectedAs** button in the File menu, select the next TopLevel, save it, etc.. until all toplevels have been saved individually.*

*Then save the ApplicationShell with the **SaveAs** button, then save the project with the **SaveProjectAs**. You can now work on the individual files as required and generate the full interface by generating the project.*

This class inherits resources from Core, Composite, Shell, WMSHELL and VendorShell classes.

It defines the following new resources:

- m **iconic** - If *True*, indicates to the window manager that the window would like to start in the iconified state, regardless of the **initialState** resource setting.
- m **iconName** - Specifies the short name that is displayed under the icon when the window is in the iconified state.
- m **iconNameEncoding** - Specifies the property that represents the encoding of the icon name. This resource cannot be edited with XFaceMaker, it appears in the restricted resources of the resource window.

25.10 The ApplicationShell Class

This is the main window of the interface. An application has one and only one ApplicationShell. Any other main windows in the application will use TopLevelShells. When the ApplicationShell is iconified, it takes its sub-tree with it, including any transient children. TopLevelShells are iconified with their ancestor ApplicationShell if their **transient** resource is set to *True*.

This class inherits resources from Core, Composite, Shell, WMSHELL, VendorShell, and TopLevelShell classes.

It defines the following new resources:

- m **argc** - This resource is set by the XtInitialize function. It specifies the number of arguments given in the **argv** resource.
- m **argv** - This resource is set by the XtInitialize function. It is the argument list passed to XtInitialize, normally the command line options passed when launching the application. This resource can be used by the session manager to re-launch the application if it is killed.

CHAPTER 26: Motif - The Composite Widgets

26.1 Introduction

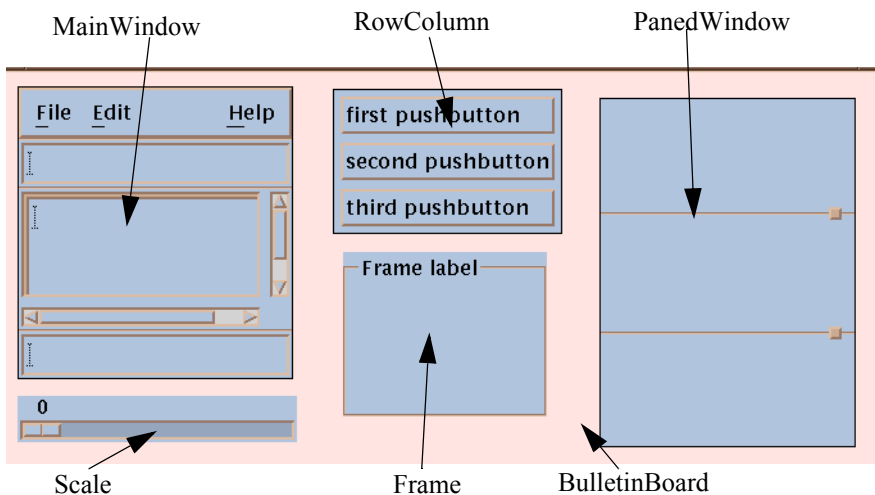
The composite or manager widgets are containers in which you can place widgets and gadgets. One of their functions is to control the position and size, i.e. the *geometry* of their children. Most composites accept primitive widgets (and gadgets) and other composite widgets as children. You can consider composite widgets as boxes in which you put things. In these boxes, you can put other boxes, which, in turn, will contain things. Composite widgets are a way to arrange the components of an interface in an orderly fashion. In addition, some of the composite widgets, such as Form, PanedWindow, RowColumn, handle changes in the size and shape of their window so as to best allocate the space available to their children.

Composite widgets also have all the required resources to handle gadgets, in particular to dispatch events to any gadget children.

In summary, composite widgets have two purposes:

- m manage the size and position of the widgets they contain.
- m provide special support for gadgets.

The main composite widgets are shown below.



The composite widgets inherit resources from Core, Composite, Constraint, and XmManager classes. They include XmBulletinBoard, XmForm, XmRowColumn, XmScrolledWindow, XmMainWindow, XmDrawingArea, XmPanedWindow, XmFrame, XmScale.

26.2 XmBulletinBoard

BulletinBoard is a composite widget that provides simple geometry management for its children. Basically, it lets you place children widgets in any position within the BulletinBoard. It can be configured to reject geometry requests that result in overlapping children. It is the base widget for most dialog widgets and it can be used as a simple container widget.

Modal or modeless dialogs are implemented as collections of widget that include a DialogShell, a BulletinBoard (or subclass) and various components. If its parent is a DialogShell, BulletinBoard passes title and input mode (based on dialog style) information to the DialogShell.

This class inherits resources from Core, Composite, Constraint, XmManager classes.

Callbacks let the application associate additional behavior to the BulletinBoard. They are useful when the BulletinBoard is used in a dialog. In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.

The following callbacks are defined for this widget:

- m **focusCallback** - Invoked when the widget or one of its descendants accepts the input focus.
- m **mapCallback** - Invoked when the parent is a DialogShell and the widget is mapped.
- m **unmapCallback** - Invoked when the parent is a DialogShell and the widget is unmapped.

The main specific resources for this widget are:

- m **allowOverlap** - If *True* (the default), the BulletinBoard will allow its children to overlap.
- m **autoUnmanage** - Controls whether a BulletinBoard whose parent is a DialogShell is automatically unmanaged after a button is activated. If this resource is *True* when the widget is created, and its parent is a DialogShell, child PushButtons, PushButtonGadgets, and DrawnButtons (except Apply and Help but-

tons of BulletinBoard subclasses) will unmanage the BulletinBoard when they are activated. If this resource is *False* when the widget is created, or if the BulletinBoard's parent is not a XmDialogShell, the BulletinBoard is not automatically unmanaged.

- m **buttonFontList** - Specifies the font used for buttons that are descendants of the BulletinBoard. It is more efficient to use this resource than to set the fontList resource of each Button widget. The default is taken from the parent hierarchy and, if no **buttonFontList** specification is found the font used is implementation dependent.
- m **defaultButton** - Specifies which of the BulletinBoard's button children will be activated when the Return key is pressed anywhere in a BulletinBoard that has focus. This resource is set by Motif Message Box dialogs; it is useful when the BulletinBoard's parent is a DialogShell. *When building custom dialogs, you may want to set this resource. To do so in XFaceMaker, use the BulletinBoard's **xfmCreateCallback** resource. This callback is executed when the BulletinBoard is being created, after all its children have been created, i.e. once the PushButton you will specify exists. For example, to specify that the button called okbutton is the **defaultButton**, you could write, in the **xfmCreateCallback** of the BulletinBoard:*

```
self.defaultButton = self.okbutton;
```

- m **defaultPosition** - If *True*, and the parent of BulletinBoard is a DialogShell, the BulletinBoard is centered on the DialogShell's parent when the BulletinBoard is mapped and managed. If *False*, it is not automatically positioned.
- m **dialogStyle** - Forced to *dialog_work_area* when the BulletinBoard's parent is not a DialogShell. When the parent is a DialogShell, this controls if the dialog is modal (i.e., must be responded to before any other input) or modeless. Acceptable values are:
 - *dialog_system_modal* - must be responded to before any interaction in any application running on the same display - avoid using this!
 - *dialog_full_application_modal* - must be responded to before any other interactions in the application.
 - *dialog_application_modal* - similar to full application modal but when dialogs are cascaded, any member of the cascade can be responded to.
 - *dialog_modeless* - the dialog does not interrupt any interactions - use this for help dialogs.
 - *dialog_work_area* - Used for BulletinBoard widgets whose parent is not a DialogShell.

- m **dialogTitle** - Specifies the title of the BulletinBoard's parent shell, i.e. the title that will be displayed by the Window Manager in the shell's title field.
- m **labelFontList** - Specifies the font used for labels that are descendants of the BulletinBoard. It is more efficient to use this resource than to set the fontList resource of each Label widget. The default is taken from the parent hierarchy and, if no **labelFontList** specification is found the font used is implementation dependent.
- m **noResize** - If *True*, and the parent is a subclass of VendorShell (this includes all shells except OverrideShell) mwm window manager will not include resize controls in the Shell's frame. If *False* (the default), the resize controls are included.
- m **textFontList** - Specifies the font used for texts that are descendants of the BulletinBoard. It is more efficient to use this resource than to set the fontList resource of each Text or TextField widget. The default is taken from the parent hierarchy and, if no **textFontList** specification is found, the font used is implementation dependent.
- m **textTranslations** - Adds translations to any text widget or text widget subclass that is added as a child of the BulletinBoard.

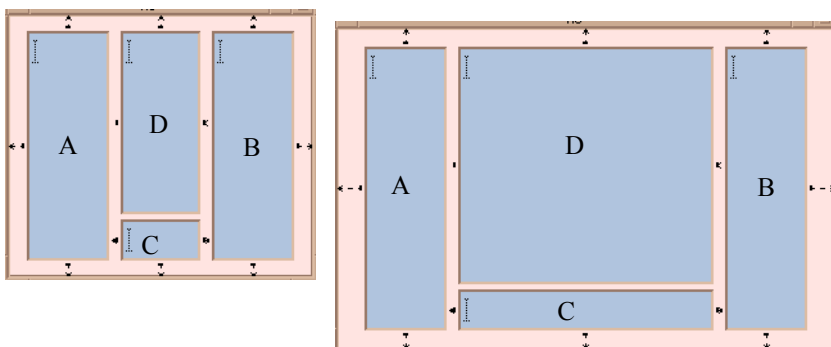
Other specific resources for this class are:

- m **cancelButton** - Specifies which widget is the Cancel button. This resource is used by Motif dialogs which have a Cancel button. It has no effect in the BulletinBoard.
- m **marginHeight** - Specifies the minimum spacing between the top or the bottom of the widget and any child. Default is *10*.
- m **marginWidth** - Specifies the minimum spacing between the left or the right edge of the widget and any child. Default is *10*.
- m **resizePolicy** - Specifies if the BulletinBoard should attempt to resize itself according to its children. Acceptable values are: *resize_none*, *resize_any* (the default), *resize_grow*.
- m **shadowType** - Specifies the shadow drawing style for the BulletinBoard. Acceptable values are: *shadow_in*, *shadow_etched_in*, *shadow_out* (the default), *shadow_etched_out*.

26.3 XmForm

Form is a composite widget that applies constraints on its children: widgets whose parent is a Form have additional resources, called constraint resources. These define attachments for the child's four sides. Attachments can be to the Form, to a relative position in the Form, to a sibling widget. The attachments determine the layout behavior of the Form when it is resized.

Use Forms when you want a particular child widget to change its size when the Form is resized. For example, if you want a widget to change its width when the Form's width changes, attach both its left and its right side.



This is an illustration of the way attachments can be used to change the relative size of widgets in a Form when the Form is resized. Widgets labeled A and B have their top, bottom and one side attached to the Form; they change size vertically. Widget C has its bottom attached to the Form and its left and right sides are attached to A and B; it changes size horizontally. Widget D is attached on all sides: its bottom, left and right sides to C, its top to the Form; it changes both its width and its height when the Form is resized. Note: *the attachments are materialized by arrows in this figure; that is an option in XFaceMaker; they are not visible in the interface.*

Since Form is subclassed from BulletinBoard, Forms used in dialogs have the same behavior as BulletinBoards in dialogs.

Form inherits resources from Core, Composite, Constraint, XmManager, XmBulletinBoard classes.

Form has no specific callbacks.

The specific resources for this widget are:

- m **fractionBase** - Specifies the denominator used to calculate the relative position

of a child widget. The default is *100*. The value must not be *0*.

- m **horizontalSpacing** - Specifies the offset for right and left attachments when no offset resource is specified (when attaching to a widget) and no margin resource is specified (when attaching to the Form).
- m **rubberPositioning** - Specifies the default top and left attachments for a child. If *False*, the widget top and left attachments are *attach_form* and top and left Offset are the x and y values for the widget. If *True*, the widget top and left attachments are *attach_position* and the top and left Position are a value proportional (determined by **fractionBase** value) to the x and y values divided by the width and height of the Form.
- m **verticalSpacing** - Specifies the offset for top and bottom attachments when no offset resource is specified (when attaching to a widget) and no margin resource is specified (when attaching to the Form).

The constraint resources applied to the children of Form are described below. Form defines the same attachment resources for **top**, **bottom**, **left** and **right** sides. Only the **bottom** attachment resources are described in detail here. Extrapolate from these for the other sides.

- m **bottomAttachment** - Specifies the type of attachment for a widget whose parent is a Form. Acceptable values are:
 - *attach_none* - do not attach bottom.
 - *attach_form* - attach bottom to bottom of the Form.
 - *attach_widget* - attach bottom to top of the sibling widget specified in the **bottomWidget** resource.
 - *attach_opposite_widget* - attach bottom to bottom of the sibling widget specified in the **bottomWidget** resource.
 - *attach_position* - attach bottom to a position that is relative to the top of the Form. The position is determined by the **bottomPosition** and **fractionBase** resources.
 - *attach_self* - same as *attach_position*, but the **bottomPosition** is set to a value proportional to the y position of the bottom divided by the Form's height.
- m **bottomOffset** - Specifies the constant offset between the bottom of a widget child of Form and the object to which it is attached. Default is the value of **verticalSpacing**.
- m **bottomPosition** - Specifies the position of the bottom of a widget child of Form relative to the Form, when **bottomAttachment** is set to *attach_position*.

The position of the bottom relative to the top of the Form is:

$$y = (\text{FormHeight} * \text{bottomPosition}/\text{fractionBase}) + \text{bottomOffset}$$

m **bottomWidget** - Specifies the sibling widget to which the bottom of a widget child of Form is attached when **bottomAttachment** is set to *attach_widget* or *attach_opposite_widget*.

When attaching to widgets, make sure that:

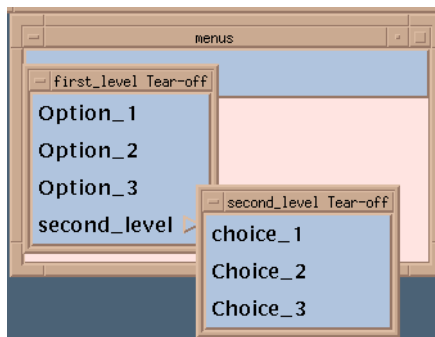
- the widget to which you are attaching is created before the widget whose attachments you are specifying.
- you do not do circular attachments, i.e. attach widget A to widget B for one side, and widget B to widget A for another side.

XFaceMaker has an attachments editing box. Use that, rather than attempting to specify individual resources; as described above, attachment resources are interdependent. While using the attachment editing box, if you want to modify the attachments you have already specified, use the 'Detach this Side' or 'Detach All Sides' toggle before specifying new attachments, to make sure that interdependent resources will all be changed as appropriate.

26.4 XmRowColumn

RowColumn is a general-purpose manager capable of containing any widget as child. It has several layout options and can, in particular, lay its children out in rows and columns.

RowColumn can be configured as a menu, in which case, it accepts only certain children and it lays them out in a fixed way. The menus supported are MenuBar, Pull-Down or Popup MenuPanes and OptionMenu. Pulldown and Popup MenuPanes support tear-off menus, which enable the user to retain a MenuPane on the display.



Above, an illustration of two torn-off cascaded menus.

With XFaceMaker, create menus from the Menu Toolkit. Do not build a menu directly from a RowColumn picked up from the Composite Toolkit.

A RowColumn containing ToggleButtons can be configured to behave like a RadioBox (set **radioBehavior** to *True*), or like a CheckBox (set **radioBehavior** to *False*).

This class inherits resources from Core, Composite, Constraint, and XmManager classes.

Callbacks let the application associate additional behavior to the RowColumn. In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.

The following callbacks are defined for this widget:

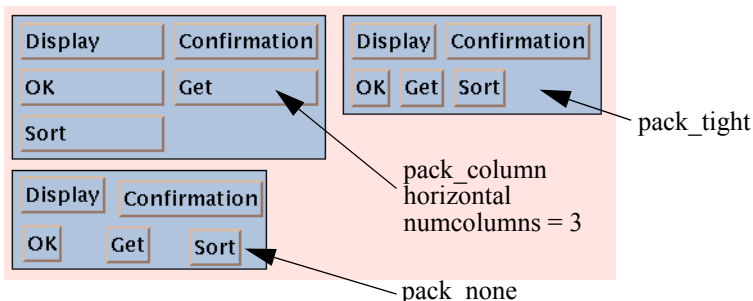
- m **entryCallback** - If a callback script is specified for this resource, the **activateCallback** and **valueChangedCallback** callbacks for all CascadeButton, DrawnButton, PushButton and ToggleButton children are revectorized to the **entryCallback** callback. This allows an application to supply a single callback routine for handling all items contained in a RowColumn.
- m **mapCallback** - Invoked when the window associated with the RowColumn is about to be mapped.
- m **tearOffMenuActivateCallback** - Invoked when the tear-off MenuPane is about to be activated. This precedes the tear-off's map callback.
- m **tearOffMenuDeactivateCallback** - Invoked when the tear-off MenuPane is about to be deactivated. This follows the tear-off's unmap callback.
- m **unmapCallback** - Invoked after the window associated with the RowColumn has been unmapped.

The most important specific resources of RowColumn are:

- m **entryAlignment** - Specifies text alignment for children that are subclasses of Label, when **isAligned** is *True*. This overrides any **alignment** resource setting specified for RowColumn's children. Acceptable values are: *alignment_beginning* (the default), *alignment_center*, *alignment_end*.
- m **isAligned** - When *True*, text alignment for children that are subclassed from Label is done according to **entryAlignment**. This applies to all children of RowColumn except Label children of Popup or Pulldown MenuPanes, where

the Label is treated as the title within the MenuPane and the text is aligned as specified in the Label's **alignment** resource.

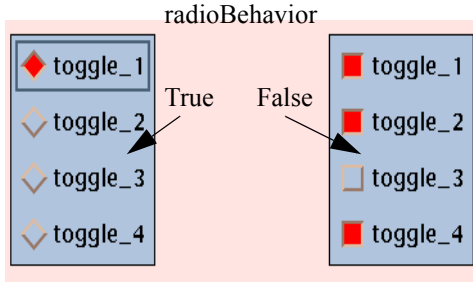
- m **numColumns** - If **packing** is *pack_column*, specifies the number of minor dimension extensions that are made to accomodate children: columns for vertical orientation, rows for horizontal orientation. As children are placed in RowColumn, the number of entries per column (row) is adjusted to maintain this number of columns, if possible. The default value is *1*; it is forced to *1* in OptionMenus. This resource is meaningless if **packing** is not set to *pack_column*.
- m **orientation** - Specifies whether layout is row-major (*horizontal*) or column-major (*vertical*). The default is *vertical*, except for MenuBar and OptionMenu, where the default is *horizontal*.
- m **packing** - Specifies how to pack items within RowColumn. The default is *pack_tight* except for RadioBoxes where the default is *pack_column*. Acceptable values are:
 - *pack_tight* - Entries are placed one after the other in the major dimension (e.g. vertical if orientation is vertical) until the RowColumn must wrap, i.e. when there is no room left for a complete child in that dimension.
 - *pack_column* - All entries are placed in identically sized boxes. The size of the boxes is based on the largest height and width values of all the children widgets. The value of **numColumns** determines how many boxes are placed in the major dimension before extending in the minor dimension.
 - *pack_none* - No packing is performed. The RowColumn attempts to become large enough to accomodate all entries.



- m **radioAlwaysOne** - If *True*, forces a toggle that has just been unselected to be selected again if no other toggle was activated. If *False*, an active toggle may

be deselected.

- m **radioBehavior** - If *True*, specifies that the RowColumn should enforce RadioBox-type behavior on all its ToggleButton children. ToggleButton indicators are set to *one_of_many*, and when a toggle is selected by the user, a previously selected toggle is unselected automatically (this behavior is not enforced when the application, rather than the user, changes the state of a toggle).

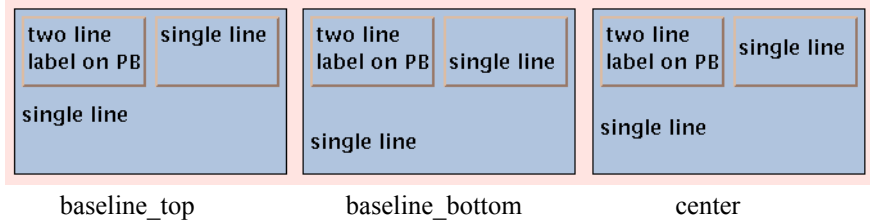


- m **rowColumnType** - Specifies the type of RowColumn. *With XFaceMaker, this resource is automatically set at the time the widget is created. If a RowColumn is created from the Composite Toolkit, it is a work_area RowColumn. If it is created from the Menu Toolkit, it is a menu_bar, menu_pulldown, menu_popup, or menu_option according to the type of menu selected in the toolkit.*

Other specific resources of RowColumn are:

- m **adjustLast** - If *True* (the default), the widget extends the last row of children to the bottom of the RowColumn if **orientation** is *horizontal*, or the last column to the right edge if **orientation** is *vertical*.
- m **adjustMargin** - If *True* (the default), forces the margin of children subclassed from Label to the value of the largest margin. If **orientation** is *horizontal*, the top and bottom margins within a row are set to the largest margin so as to align texts in the row. If **orientation** is *vertical*, the left and right margins in a column are set to the largest margin, so as to align texts in a column.
- m **entryBorder** - Specifies a uniform border width upon all children. The default is 0.
- m **entryClass** - When the **isHomogeneous** resource is *True*, specifies the only class of widgets that can be added. When **rowColumnType** is *menu_bar*, **entryClass** is forced to *xmCascadeButtonWidgetClass*.
- m **entryVerticalAlignment** - Specifies the type of vertical alignment for children subclassed from Label or Text. This resource is ineffective if **orientation** is *vertical* and **packing** is *pack_tight* because this layout preserves varying wid-

get heights. Acceptable values are: *alignment_baseline_bottom*, *alignment_baseline_top*, *alignment_contents_bottom*, *alignment_contents_top*, *alignment_center*.



- m **isHomogeneous** - If *True*, only the widgets that are of the class indicated by **entryClass** are allowed as children of the `RowColumn`. This is used most when creating a `MenuBar`, where only `CascadeButtons` are accepted.
- m **labelString** - When `rowColumnType` is *menu_option*, specifies the **labelString** displayed to the left of the selection area. The default value is no label on most platforms. To change the label after widget creation, get the `LabelGadget` with the function `XmOptionLabelGadget`. *With XFaceMaker, you can add the following in the OptionMenu xfmCreateCallback (this callback is executed when the optionMenu is being created, after all its children have been created):*

```
labelGadgetID = XmOptionLabelGadget(self);
labelGadgetID:labelString = mystring;
```
- m **marginHeight** - Specifies the margin between the top edge of `RowColumn` and the first item, and the bottom edge and the last item in each column. The default is 3, except for `Pulldown` and `Popup MenuPanels` where it is 0.
- m **marginWidth** - Specifies the margin between the left edge of `RowColumn` and the first item, and the right edge and the last item in each row. The default is 3, except for `Pulldown` and `Popup MenuPanels` where it is 0.
- m **menuAccelerator** - For a `MenuBar`, if **popupEnabled** is *True*, when the menuAccelerator is typed by the user (default is `KMenuBar`, e.g. <KeyUp>F10) the first item in the `MenuBar` is highlighted and traversal is enabled in the `MenuBar`.
- m **menuHelpWidget** - If `rowColumnType` is *menu_bar*, specifies which of the `CascadeButtons` is treated as the Help widget. *In XFaceMaker, use the MenuBar's xfmCreateCallback to specify the Help widget's ID; you could write, for example:*

```
self:menuHelpWidget = self.help;
```

- m **menuHistory** - If **rowColumnType** is *menu_option*, specifies the current selection. More generally, specifies the widget ID of the last menu entry activated. In a `RadioBox`, contains the widget ID of the last toggle whose state has changed from *off* to *on*.
- m **menuPost** - Specifies an X event description indicating a button event that posts a menu system (e.g. `<Btn3Down>`) for a `Popup MenuPane`.
- m **mnemonic** - If **rowColumnType** is *menu_option*, specifies which letter of the `labelString`, when typed along with the `MAIt` modifier, will post the associated `Pulldown MenuPane`.
- m **mnemonicCharSet** - Specifies the character set of the mnemonic for an `OptionMenu`. Default is *font_list_default_tag*.
- m **popupEnabled** - If *True*, **menuAccelerator** is enabled.
- m **resizeHeight** - If *True*, requests a new height if necessary.
- m **resizeWidth** - If *True*, requests a new width if necessary.
- m **spacing** - Specifies horizontal and vertical spacing between children. Default is 3 for *menu_option* and *work_area*, 0 for other **rowColumnType** settings.
- m **subMenuId** - For `OptionMenus`, specifies the associated `Pulldown MenuPane`.
- m **tearOffModel** - if **rowColumnType** is *menu_pulldown* or *menu_popup*, specifies whether tear-off functionality is enabled (*tear_off_enabled*) or disabled (*tear_off_disabled* - the default).
- m **whichButton** - This resource is equivalent to and replaced by **menuPost**.

`RowColumn` applies one constraint resource to its children:

- m **positionIndex** - Specifies the position of the widget in its parent's list of children. Values range from 0 (first position) to *n-1*, where *n* is the number of children in the `RowColumn`. In `XFaceMaker`, this is determined by the order of the sibling widgets in the widget tree or list.

26.5 XmScrolledWindow

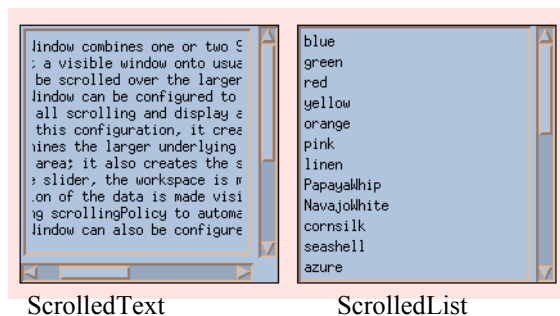
`ScrolledWindow` combines one or two `ScrollBars` and a viewing area to implement a visible window onto a usually larger data display. The viewport can be scrolled over the larger display using the `ScrollBars`.

`ScrolledWindow` can be configured to operate automatically, so that it performs all scrolling and display actions, without application intervention. In this configuration, it creates a clipping window through which the user examines the larger underlying

widget containing the data, called the work area; it also creates the scroll bars automatically. When the user moves the slider, the workspace is moved under the viewing area and a new portion of the data is made visible. This mode of operation is obtained by setting **scrollingPolicy** to *automatic*.

ScrolledWindow can also be configured to provide minimal support for application implemented scrolling. In this case, the underlying data need only be loaded as needed and scrolling consists in bringing in the visible portion of data. To implement scrolling, the application must respond to user input on the ScrollBars, i.e. associate behavior to the callbacks of each of the ScrollBars.

The Text and the List widget implement their own scrolling and display when the ScrolledWindow **scrollingPolicy** is set to *application_defined*, i.e. they can replace the application in handling their own scrolling. To create a ScrolledText or a ScrolledList, just create a ScrolledWindow and a Text or List child. *With XFaceMak-*



er, you can control the size of the ScrolledWindow through resource settings in the Text or the List rather than through width and height resource settings.

Specifying size through resource settings for ScrolledText:

1. *in the Text widget, specify the width with the columns resource, the height with the rows resource.*
2. *Set editMode to multi_line_edit .*
3. *Set the autoposition and autosize toggles for the Text and the autosize toggle for the parent ScrolledWindow.*
4. *Rebuild the parent ScrolledWindow. It resizes itself so as to exactly accomodate the Text widget of the size specified (provided its parent does not constrain ScrolledWindow's size).*

Specifying size through resource settings for ScrolledList:

1. *in the List widget, specify the height with the visibleItemCount resource.*

2. *The width of the list is controlled by the width of the longest item and the default is 1; thus, in order to specify a width larger than one, you should enter at least one item in the items resource of the widget.*
3. *Set the autoposition and autosize toggles for the List and the autosize toggle for the parent ScrolledWindow.*
4. *Rebuild the parent ScrolledWindow. It resizes itself so as to exactly accomodate the List widget with the size specified (provided its parent does not constrain ScrolledWindow's size).*

This class inherits resources from Core, Composite, Constraint, and XmManager classes.

ScrolledWindow has only one specific callback. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callback is defined for this widget:

- m **traverseObscuredCallback** - When **scrollingPolicy** is *automatic*, is invoked when traversing to a widget that is obscured because of its position in the work area relative to the viewport. If no callback is specified (this resource is NULL), an obscured widget cannot be traversed to.

The most important specific resources of ScrolledWindow are:

- m **scrollBarDisplayPolicy** - Must be *static* if **scrollingPolicy** is *application_defined*. If **scrollingPolicy** is *automatic*, it defaults to *as_needed*, where the ScrollBars are displayed only if the work area exceeds the clip area in one or both directions. It can be set to *static*, where the ScrollBars are displayed regardless of the relative sizes of clip window and work area. The latter is sometimes preferable to avoid jumpiness in the display due to scroll bars being displayed, then removed as the work area changes size.
- m **scrollBarPlacement** - Specifies the positioning of the ScrollBars relative to the work window. Acceptable values are: *top_left*, *bottom_left*, *top_right*, *bottom_right* (the default).
- m **scrollingPolicy** - If *automatic*, ScrolledWindow automatically creates the ScrollBars, attaches callbacks to the ScrollBars, sets the **visualPolicy** to *constant*, automatically moves the work area through the clip window as the user interacts with the ScrollBars. The application can add its own callbacks to the ScrollBars, thus being informed of scrolling actions without having to perform layout. If *application_defined*, the application is responsible for all scrolling,

including the creation of the ScrollBars. The Text and List widgets implement their own scrolling and create the ScrollBars, when they are child of ScrolledWindow with *application_defined* **scrollingPolicy**.

- m **visualPolicy** - Must be *variable* if **scrollingPolicy** is *application_defined*. In *automatic* scrolling, it is usually set to *constant*, indicating that the ScrolledWindow should not resize itself when the work area shrinks or grows and should work with a clip window. If it is set to *variable*, the ScrolledWindow expands itself to fit the entire work window, thus nullifying the need for an automatic ScrolledWindow.

Other resources defined for ScrolledWindow are:

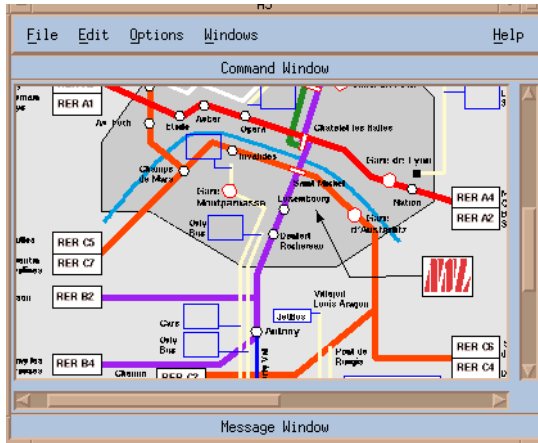
- m **clipWindow** - Specifies the widget ID of the clipping area. This is automatically created by ScrolledWindow when **visualPolicy** is *constant* and can only be read. This resource is NULL when **visualPolicy** is set to *variable*.
- m **horizontalScrollBar** - Specifies the widget ID of the horizontal ScrollBar. This is automatically created when **scrollingPolicy** is *automatic*.
- m **scrolledWindowMarginHeight** - Specifies the margin height on the top and the bottom of the ScrolledWindow.
- m **scrolledWindowMarginWidth** - Specifies the margin width on the left and the right sides of the ScrolledWindow.
- m **spacing** - Specifies the distance between the ScrollBars and the work window. The default is 4.
- m **verticalScrollBar** - Specifies the widget ID of the vertical ScrollBar. This is automatically created when **scrollingPolicy** is *automatic*.
- m **workWindow** - Specifies the widget ID of the work area.

26.6 XmMainWindow

MainWindow provides standard layout for a window with the following optional items: MenuBar, Commandwindow, Message window, separators, and a work area. In MainWindow, the MenuBar can be on several lines, if the window is not wide enough to accomodate all items on a single line. The work area behaves like a ScrolledWindow.

However, you cannot create a `ScrolledText` or a `ScrolledList` by placing a `Text` or `List` widget directly in the work area, because `Text` and `List` check that their parent is a `ScrolledWindow`. To create a `ScrolledText` or a `ScrolledList` in a `MainWindow`, you will have to place a `ScrolledWindow` in the `MainWindow` work area, and add the `Text` or `List` child.

A fully loaded `MainWindow` is shown below, with a map of the Paris underground transportation system in the work area.



This class inherits resources from `Core`, `Composite`, `Constraint`, `XmManager` and `XmScrolledWindow` classes.

This widget has no specific callbacks.

The specific resources of `MainWindow` are:

- m **commandWindow** - Widget ID of the widget to be laid out as the command window. *With XFaceMaker, specify this in the `MainWindow`'s `xfmCreateCallback`. This callback is executed when the `MainWindow` is being created, after all its children have been created*
- m **commandWindowLocation** - Controls the position of the command window. Acceptable values are: `command_above_workspace` (the default) and `command_below_workspace`
- m **mainWindowMarginHeight** - Specifies the margin height on the top and the bottom of the `MainWindow`; overrides the margin height resource inherited from `ScrolledWindow`.
- m **mainWindowMarginWidth** - Specifies the margin width on the left and the

right side of the MainWindow; overrides the margin width resource inherited from ScrolledWindow.

- m **menuBar** - Widget ID of the widget to be laid out as the MenuBar. *With XFaceMaker, you don't need to specify this explicitly. You just need to instantiate a MenuBar as child of the MainWindow; the MenuBar will be handled properly by the MainWindow.*
- m **messageWindow** - Widget ID of the widget to be laid out as the message window. *With XFaceMaker, specify this in the MainWindow's xfmCreateCallback. This callback is executed when the MainWindow is being created, after all its children have been created*
- m **showSeparator** - If *True*, displays separators between the widget's components.

26.7 XmDrawingArea

DrawingArea supports callbacks to notify the application that graphics should be redrawn. Is a manager widget, and supports minimal geometry management for multiple child widgets. It is an otherwise empty widget; the application is responsible for controlling its appearance in response to the exposure and resize callbacks.

This widget inherits resources from Core, Composite, Constraint, and XmManager classes.

Callbacks let the application respond to user interaction with DrawingArea. In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.

The following callbacks are defined for this widget:

- m **exposeCallback** - Invoked when the widget receives an exposure event.
- m **inputCallback** - Invoked when the widget receives a mouse or keyboard event (key or button up or down).
- m **resizeCallback** - Invoked when the widget is resized.

The specific resources for DrawingArea are:

- m **marginHeight** - Specifies the minimum spacing between the top or bottom edge of the widget and any child.
- m **marginWidth** - Specifies the minimum spacing between the left or right edge of the widget and any child.

- m **resizePolicy** - Controls the widget's resize policy. Acceptable values are: *resize_none*, *resize_grow*, *resize_any* (the default).

26.8 XmPanedWindow

This widget lays its children out vertically, from top to bottom. `PanedWindow` grows to match the width of the widest of its children, and all the children's widths are forced to this width. The widget provides control sashes between its children, to allow the user to control the size of a pane. The `XFaceMaker` main window is a `PanedWindow` with three panes.

`PanedWindow` applies constraints on its children: widgets whose parent is a `PanedWindow` have additional resources, called constraint resources. These control the size and position of the children.

This class inherits resources from `Core`, `Composite`, `Constraint`, and `XmManager` classes.

This widget has no specific callbacks.

The specific resources of `PanedWindow` are:

- m **marginHeight** - Specifies the minimum spacing between the top or bottom edge of the widget and any child.
- m **marginWidth** - Specifies the minimum spacing between the left or right edge of the widget and any child.
- m **refigureMode** - If *True* (the default), the panes' positions are recomputed when programmatic changes are made to the `PanedWindow`.
- m **sashHeight** - Specifies the height of the sash. Default is *10*.
- m **sashIndent** - Specifies the horizontal placement of the sashes. Positive values are an offset from the left of the window, negative values an offset from the right of the window. The default is *-10*.
- m **sashShadowThickness** - Specifies the thickness of shadows on the sashes.
- m **sashWidth** - Specifies the width of the sashes. Default is *10*.
- m **separatorOn** - If *True* (the default), a separator is drawn between each of the panes.
- m **spacing** - Specifies the distance between each child pane. Default is *8*.

The constraint resources applied to the children of `PanedWindow` are:

- m **allowResize** - If *True*, the `PanedWindow` tries to honor requests from a child to

alter the height of its pane. The default is *False*, all pane requests to resize are denied.

m **paneMaximum** - Specifies the maximum size of the pane. Default is *1000*.

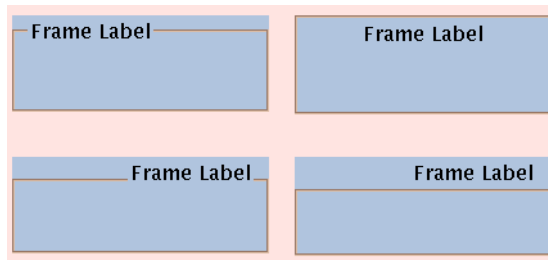
m **paneMinimum** - Specifies the minimum size of the pane. Default is *1*.

m **positionIndex** - Specifies the position of the widget in its parent's list of children (not including sashes). Values range from 0 (first position) to *n-1*, where *n* is the number of children.

m **skipAdjust** - If *True*, specifies that the parent *PanedWindow* should not automatically resize this pane. The default is *False*.

26.9 XmFrame

Frame provides a decorative border and optional title. It encloses a single work area child (which can itself be a manager). It has constraint resources that are used to designate a child as the Frame title. The title is displayed only at the top of the Frame. It can be centered vertically and horizontally in various ways.



With XFaceMaker, here is how you can go about creating a Frame with a Label as `frame_title_child`, and a BulletinBoard as `frame_workarea_child`.

1. *Instantiate a Frame widget.*
2. *In the widget tree, place a Label as child of the Frame.*
3. *Select the Label and, in the resources box, set the auto position and auto size toggles, set the **childType** resource to `frame_title_child`, set the alignment constraints as desired, specify the **labelString**, click the Set Values or the Rebuild button .*
4. *Now, place a BulletinBoard as child of the Frame, by dropping it over the Frame in the widget tree (don't worry about widget sizes at this stage).*
5. *Select the BulletinBoard, set the auto position and auto size toggles, click the Set Values or the Rebuild button.*
6. *Finally, select the Frame, use the selection handles to specify its size.*

This class inherits resources from Core, Composite, Constraint, and XmManager classes.

This widget has no specific callbacks.

The specific resources of Frame are:

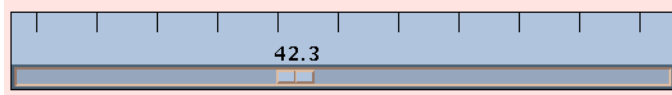
- m **marginHeight** - Specifies the space between the top or bottom edge of the Frame's shadow and the workarea child. At the top, the distance the title extends below the shadow is added to the empty space.
- m **marginWidth** - Specifies the space on the left and right sides between the Frame's shadow drawing and the workarea child.
- m **shadowType** - Specifies the shadow's drawing style. Acceptable values are: *shadow_in*, *shadow_out*, *shadow_etched_in*, *shadow_etched_out*.

The constraint resources applied to the children of Frame are:

- m **childHorizontalAlignment** - If the **childType** is *frame_title_child*, specifies the horizontal alignment of the title in the Frame shadow. Acceptable values are: *alignment_beginning*, *alignment_center*, *alignment_end*.
- m **childHorizontalSpacing** - Specifies the minimum distance between the edge of the title and the inner edge of the Frame shadow. Default is the **marginWidth** of the Frame.
- m **childType** - Specifies whether a child is title or work area. Frame supports only one title and one workarea child. Acceptable values are: *frame_title_child*, *frame_workarea_child*, *frame_generic_child*. The latter is ignored by the Frame geometry manager.
- m **childVerticalAlignment** - If the **childType** is *frame_title_child*, specifies the vertical alignment of the title in the Frame shadow. Acceptable values are: *alignment_baseline_bottom*, *alignment_baseline_top*, *alignment_widget_top*, *alignment_center*, *alignment_widget_bottom*.

26.10 XmScale

Scale is used to set or display a value in a range. It consists of a slider moving in a trough, and an optional label in which the value is displayed. Scale can have Separator children; they are distributed along the side of the widget to be used as tick marks.



This widget inherits resources from Core, Composite, Constraint, and XmManager classes.

Callbacks let the application respond to user interaction with Scale. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

- m **dragCallback** - Invoked as the slider position changes as a result of being dragged.
- m **valueChangedCallback** - Invoked when the slider position has changed as a result of user interaction with Scale (end of drag, or clicking in the trough).

The most important specific resources for Scale are:

- m **decimalPoints** - Specifies the number of decimal points to shift the value when displaying it. In the example above, **value** was 423, **decimalPoints** was 1, so the value is displayed as 42.3. Default is 0.
- m **maximum** - Specifies the slider's maximum value. Default is 100.
- m **minimum** - Specifies the slider's minimum value. Default is 0.
- m **orientation** - Specifies whether Scale should be displayed vertically or horizontally. Acceptable values are *horizontal* and *vertical* (the default).
- m **processingDirection** - Specifies the position of the maximum value. The values accepted are *max_on_top* (the default if **orientation** is *vertical*), *max_on_bottom*, *max_on_left*, *max_on_right*.
- m **showValue** - If *True*, the current slider value is displayed in a label next to the slider. Default is *False*.
- m **value** - Specifies the slider's position along the scale. It is constrained to lie be-

tween **minimum** and **maximum**.

Other specific resources for Scale are:

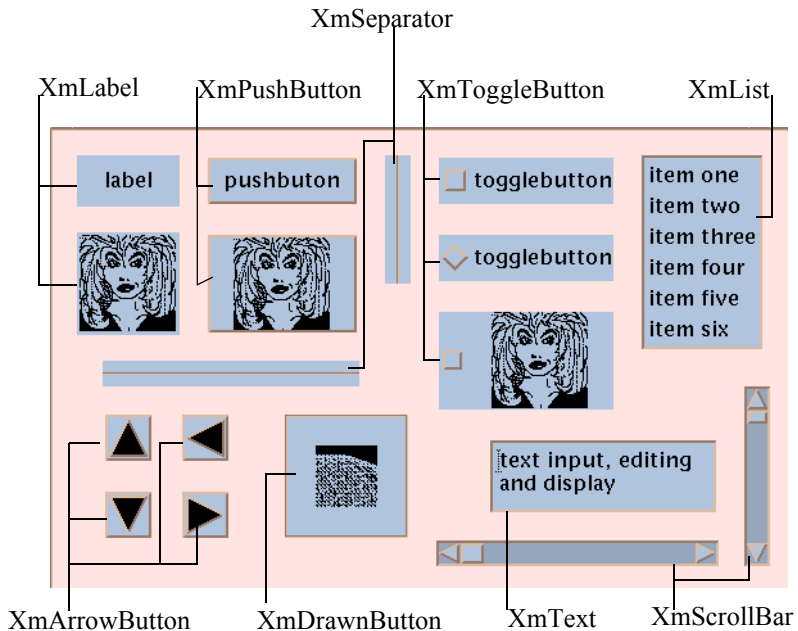
- m **fontList** - Specifies the font used for the title and value labels. Default value is taken from the parent hierarchy **labelFontList** resource.
- m **highlightOnEnter** - If **keyboardFocusPolicy** is *pointer* and this resource is *True*, the highlighting rectangle is drawn around Scale when the pointer moves into the widget. Default is *False*. If **keyboardFocusPolicy** is *explicit*, this resource is ignored, the widget is highlighted when it has the keyboard focus.
- m **highlightThickness** - Specifies the thickness of the highlight rectangle. Default is 2.
- m **scaleHeight** - Specifies the height of the slider.
- m **scaleMultiple** - Specifies the amount to move the slider when the user clicks in the trough. The default is $(\text{max} - \text{min})/10$ with a minimum of 1.
- m **scaleWidth** - Specifies the width of the slider.
- m **titleString** - Specifies the title text string to display in the Scale widget window.

CHAPTER 27: Motif - The Primitive Widgets

27.1 Introduction

Primitive widgets are the widgets with which the user interacts directly in the interface. They are a part of the ‘Command’ widgets, along with the `XmDialog` widgets.

Primitive widgets inherit resources from `Core` and `XmPrimitive` classes. They include `XmLabel`, `XmSeparator`, `XmPushButton`, `XmToggleButton`, `XmDrawnButton`, `XmArrowButton`, `XmCascadeButton`, `XmList`, `XmText`, `XmTextField`, `XmScrollBar`.



One or several examples of the primitive widgets are displayed in the figure. What they are used for and their specific resources are described in the following sections.

27.2 XmLabel

This widget is not meant to interact with the user. It does not accept keyboard or mouse button input and has no specific callbacks. As its name implies, it is used to identify widgets near it in order to help the end-user know what kind of information they represent or contain: you will use Labels to *label* a widget or a group of widgets. Label serves as the superclass for Button widgets.

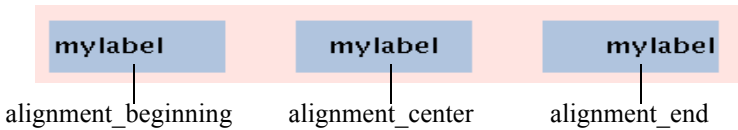
This class inherits resources from Core and XmPrimitive classes.

Labels can display text (single or multi-line) or pixmaps, depending on the setting of the following resources:

- m **labelType** - Specifies the type of information displayed; it can be *pixmap* or *string* (the default value).
- m **labelPixmap** - Specifies the pixmap displayed when the **labelType** is *pixmap*.
- m **labelString** - Specifies the text displayed on the label when the **labelType** is *string*.

Other resources for this class are:

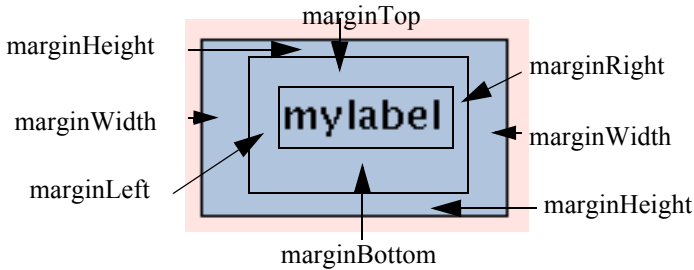
- m **alignment** - Specifies the alignment for the label text. The default value is *alignment_center*, other values are *alignment_beginning* and *alignment_end*. If



the parent is a RowColumn whose **isAligned** resource is true, alignment is forced to the same value as the RowColumn's **entryAlignment** resource.

- m **fontList** - Specifies the font of the text in the widget. Default value is taken from the parent hierarchy: in buttons, the **buttonFontList** resource is used, and in labels, the **labelFontList** resource. If neither is defined, **defaultFontList** is used. If no default font is specified in the parent hierarchy, the font used is implementation dependent.

The figure illustrates the margin resources for the Label widget.



- m **marginHeight** - Specifies the margin between the top of the widget and the top margin set by **marginTop**, and an equal margin between the bottom of the widget and the bottom margin set by **marginBottom**. Consider it as an empty space reserved along the top and the bottom of the widget. Default value is 2.
- m **marginWidth** - Specifies the margin between the left edge of the widget and the left margin set by **marginLeft**, and an equal margin between the right edge of the widget and the right margin set by **marginRight**. Consider it as an empty space reserved along the left and the right edges of the widget. Default value is 2.
- m **marginBottom** - Specifies the spacing between the bottom of the label text and the top of the bottom margin specified by **marginHeight**. Default value is 0, but can be modified in buttons, e.g. in CascadeButtons to make room for the cascade pixmap.
- m **marginTop** - Specifies the spacing between the top of the label text and the bottom of the top margin specified by **marginHeight**. Default value is 0, but can be modified in buttons, e.g. in CascadeButtons to make room for the cascade pixmap.
- m **marginLeft** - Specifies the spacing between the left edge of the label text and the right side of the left margin specified by **marginWidth**.
- m **marginRight** - Specifies the spacing between the right edge of the label text and the left side of the right margin specified by **marginWidth**.
- m **recomputeSize** - If *True* (the default) the widget adapts its size to the size of the string or the pixmap.

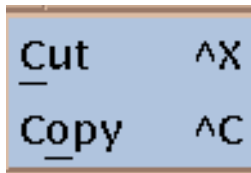
Button widgets inherit from this class; some of the resources defined for XmLabel apply to the button subclasses, rather than to the label itself.

- m **accelerator** - Specifies the accelerator on a button widget placed in a Pull-

downMenu or a PopupMenu. In *XFaceMaker*, use the menu editor to specify an **accelerator** and enter the desired keyboard combination in the accelerator field, as the end user would type it. To clear the **accelerator**, clear the accelerator text.

- m **acceleratorText** - Specifies the text displayed next to the label for the accelerator. In *XFaceMaker*'s menu editor, the **acceleratorText** resource is set automatically from the accelerator entered. To reset the **accelerator**, clear the accelerator text field.
- m **mnemonic** - Specifies a single character shortcut to activate a PushButton placed in a MenuBar, a PulldownMenu or a PopupMenu. The **mnemonic** specified must be one of the characters in the label. The first character matching the **mnemonic** will be underlined. In the *XFaceMaker* menu editor, enter the **mnemonic** directly. It can be handled as an internationalized mnemonic by clicking the GLS button.

Two PushButtons in a PulldownMenu are shown, with mnemonics and accelerator



texts. The *Cut* button has a **mnemonic** set to *c*, and an **accelerator** set to *Control+X*; the **acceleratorText** is *^X*. The *Copy* button has a **mnemonic** set to *o* and an **accelerator** set to *Control+C*; its **acceleratorText** is *^C*.

27.3 XmSeparator

Separator is used to visually separate items in a display. It can have horizontal or vertical orientation, several line drawing styles are provided. Separators are often used in menus to implement visual separation of logically separated items. *The XFaceMaker menu editor lets you specify Separators as menu items, along with the different button types.* Separators are also used in Manager widgets to implement visual separation of logically separated groups of widgets.

This class inherits resources from Core and XmPrimitive classes.

Specific resources of Separator are:

- m **margin** - Specifies the space left between the border of the widget and the line drawn.

- m **orientation** - Specifies the orientation of the separator. If *horizontal* (default), the Separator is displayed horizontally. If *vertical*, the separator is drawn vertically.
- m **separatorType** - Specifies the type of line used to draw the Separator. Acceptable values are: *shadow_etched_in* (default), *single_line*, *double_line*, *single_dashed_line*, *double_dashed_line*, *shadow_etched_in_dash*, *shadow_etched_out_dash*, *no_line*, *shadow_etched_out*.

Separator types:

shadow_etched_in	
single_line	
double_line	
single_dashed_line	
double_dashed_line	
shadow_etched_in_dash	
shadow_etched_out_dash	
shadow_etched_out	

27.4 XmPushButton

PushButton is used to trigger an action when it is pressed with the mouse, or when it receives a KActivate event (usually the Return key on the keyboard). PushButtons have a three dimensional appearance thanks to the color scheme applied to the top and left chamfers versus the bottom and right chamfers. The widget's internal behavior makes it seem to sink into the screen when the mouse button is pressed and to pop back to its initial state when it is released.

This class inherits resources from Core, XmPrimitive, XmLabel classes.

Callbacks let the application associate additional behavior that is triggered when the button is pressed, released or activated. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

- m **activateCallback** - Invoked when the button is pressed and released.
- m **armCallback** - Invoked when the button is pressed.
- m **disarmCallback** - Invoked when the button is released. This callback is trig-

gered after the **activateCallback**.

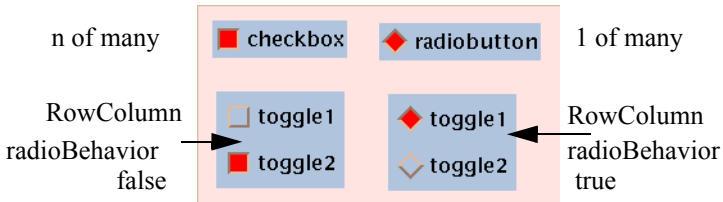
Other specific resources of PushButton are:

- m **armColor** - Specifies the color used to fill the armed button if **fillOnArm** is set to *True*.
- m **armPixmap** - Specifies the pixmap used when the button is armed if **label-Type** is set to *pixmap*.
- m **fillOnArm** - If *True*, the background of the button is filled with the **armColor** when the button is armed. If *False*, only the top and bottom shadow colors are switched when the button is armed, to give the impression the button has sunk into the screen. This resource is ignored and assumed to be *False* when the button is in a menu.
- m **multiClick** - Specifies how multiple clicks are handled. If it is set to *multiclick_keep* (the default when not in a menu) multiple clicks are processed and the click-count field of the callback structure is incremented. If it is set to *multiclick_discard*, any additional click received in the button within the display's multclick time is discarded.
- m **showAsDefault** - If the **defaultButtonShadowThickness** is not 0 and the value of this resource is greater than 0, the button is marked as default with a shadow thickness as specified in **defaultButtonShadowThickness**. If **defaultButtonShadowThickness** is 0, this resource specifies the shadow thickness with which the button is to be marked as default. If this resource is set to 0, the button is not shown as default. This resource is ignored when the button is in a menu.
- m **defaultButtonShadowThickness** - One of the resources that specifies the shadow thickness used to mark a button as default (see **showAsDefault**).

27.5 XmToggleButton

A ToggleButton is a button with two states: *on* and *off*. A graphic indicator next to the label indicates whether the button is in the *on* or in the *off* state. “n of many” toggles, or CheckBoxes, have a square indicator, while “1 of many toggles”, or Radio

Buttons, have a diamond shaped indicator. ToggleButtons are often used in RowColumns; the toggle's CheckBox or RadioBox behavior is controlled with the RowColumn's **radioBehavior** resource.



This class inherits resources from Core, XmPrimitive, XmLabel classes.

Callbacks let the application associate additional behavior that is triggered when the button is pressed, released or its state is changed. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

- m **valueChangedCallback** - Invoked when the button is pressed and released, i.e. the state has changed.
- m **armCallback** - Invoked when the button is pressed.
- m **disarmCallback** - Invoked when the button is released. This callback is triggered after the **valueChangedCallback**.

Other specific resources of ToggleButton are:

- m **fillOnSelect** - If *True*, the indicator is filled with the **selectColor** when the button is *on*. Default is *True*.
- m **indicatorOn** - If *True*, the indicator is displayed. Default is *True*.
- m **indicatorSize** - Specifies the size of the indicator. Default value is based on the size of the label string or pixmap.
- m **indicatorType** - Specifies if the indicator is *n_of_many* (square shaped) or *one_of_many* (diamond shaped). This specifies the visuals only, it does not enforce any behavior. Default value is *n_of_many* if the button is on its own. If the button is in a RowColumn, the default value depends on the **radioBehavior** resource of the RowColumn.
- m **selectColor** - Specifies the color used to fill the indicator, when it is *on*, if **fillOnSelect** is *True*.

- m **selectInsensitivePixmap** - Specifies the pixmap used when the button is *on* and insensitive, and the **labelType** is *pixmap*.
- m **selectPixmap** - Specifies the pixmap used when the button is *on* and the **labelType** is *pixmap*.
- m **set** - Represents the state of the ToggleButton. A value of *False* indicates that the button is *off*, a value of *True* indicates it is *on*. Setting this resource sets the state of the ToggleButton.
- m **spacing** - Specifies the amount of space between the indicator and the label (string or pixmap).
- m **visibleWhenOff** - If *True*, the indicator is visible in the *off* state. The default is *True* in a RadioBox (RowColumn with radioBehavior) and *False* in a menu.

27.6 XmDrawnButton

DrawnButton is an empty widget window surrounded by a border shadow that has PushButton behavior, i.e. it can have PushButton input semantics while providing a graphic area within its shadow border, in which the application can create drawings.

This class inherits resources from Core, XmPrimitive, XmLabel classes.

Callbacks let the application associate additional behavior that is triggered when the button is pressed, released or activated. In addition, expose and resize callbacks are available to inform the application that it should redraw or reposition its graphics. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

- m **activateCallback** - Invoked when the button is pressed and released.
- m **armCallback** - Invoked when the button is pressed.
- m **disarmCallback** - Invoked when the button is released. This callback is triggered after the **activateCallback**.
- m **exposeCallback** - Invoked when the widget receives an expose event.
- m **resizeCallback** - Invoked when the widget receives a resize event.

Other specific resources of DrawnButton are:

- m **multiClick** - Specifies how multiple clicks are handled. If it is set to *multiclick_keep* (the default when not in a menu) multiple clicks are processed and the click-count field of the callback structure is incremented. If it is set to

multiclick_discard, any additional click received in the button within the display's multiclick time is discarded.

m **pushButtonEnabled** - If *True*, enables the three dimensional shadow drawing as in *PushButton*. Default is *False*.

m **shadowType** - Specifies the drawing style. Acceptable values are: *shadow_etched_in* (the default) , *shadow_etched_out*, *shadow_in*, *shadow_out*.

27.7 XmArrowButton

ArrowButton is a directional arrow surrounded by a border shadow that has *PushButton* behavior. The widget's internal behavior makes it seem to sink into the screen when the mouse button is pressed and to pop back to its initial state when it is released.

This class inherits resources from *Core* and *XmPrimitive* classes.

Callbacks let the application associate additional behavior that is triggered when the button is pressed, released or activated. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

m **activateCallback** - Invoked when the button is pressed and released.

m **armCallback** - Invoked when the button is pressed.

m **disarmCallback** - Invoked when the button is released. This callback is triggered after the **activateCallback**.

Other specific resources of *ArrowButton* are:

m **arrowDirection** - Specifies the direction of the arrow. Acceptable values are: *arrow_up* (the default), *arrow_down*, *arrow_left*, *arrow_right*.

m **multiClick** - Specifies how multiple clicks are handled. If it is set to *multiclick_keep* (the default when not in a menu) multiple clicks are processed and the click-count field of the callback structure is incremented. If it is set to *multiclick_discard*, any additional click received in the button within the display's multiclick time is discarded.

27.8 XmCascadeButton

`CascadeButton` is used in menu systems to link two `PulldownMenus` or a `MenuBar` to a `PulldownMenu`. When `CascadeButton` is activated in a `MenuBar`, a `PopupMenu` or a `PulldownMenu`, the submenu associated to the `CascadeButton` is displayed. *Because `CascadeButton` is associated to menus, `XFaceMaker` places this class in the Menu Toolkit, rather than in the Primitive Toolkit. In general, you will not instantiate `CascadeButtons` directly from the toolkit; you will use the menu editor. However, if, instead of using the menu editor, you place a `PulldownMenu` in a `MenuBar`, `XFaceMaker` will automatically create the associated `CascadeButton`.*

This class inherits resources from `Core`, `XmPrimitive`, `XmLabel` classes.

Callbacks let the application associate additional behavior that is triggered when the button is pressed, released or activated. *In `XFaceMaker`, you can program a callback directly by writing a `FACE` script in the resource editing box, or in the menu editor. This script may perform actions of its own and/or call application functions, with/without arguments.*

The following callbacks are defined for this widget:

- m **activateCallback** - Invoked when the user activates the widget and there is no submenu attached to pop up.
- m **cascadingCallback** - Invoked just before the associated submenu is mapped.

Other specific resources of `CascadeButton` are:

- m **cascadePixmap** - Specifies the pixmap displayed on one end of the widget when it is used in a `Popup` or `Pulldown` menu and a submenu is attached. Default is an arrow pointing to the side of the menu where the submenu will appear.
- m **mappingDelay** - Specifies the time in milliseconds between arm and mapping of the submenu when the widget is in a `PopupMenu` or a `PulldownMenu`. Default is 180 msec.
- m **subMenuId** - Specifies the widget ID of the `PulldownMenu` displayed when the `CascadeButton` becomes armed. *In `XFaceMaker`, the setting of this resource is automatic unless the `CascadeButton` is created on its own. In the latter case, if you want to set this resource yourself, make sure the `Pulldown Menu` is created before the `CascadeButton`, or set the resource in the `xfmCreateCallback`.*

27.9 XmList

List allows a user to select one or several items from a group of choices, using the mouse pointer or the keyboard. Use a List when the number of items to choose from is larger than a RadioBox, CheckBox, or OptionMenu can easily handle, or when the items to choose from can change.

Motif defines the XmScrolledList in which the List widget handles its own scrolling when the user interacts with the ScrollBars. *To build an XmScrolledList with XFaceMaker, place a List widget inside a ScrolledWindow with **scrollingPolicy** set to `application_defined`.*

This class inherits resources from Core and XmPrimitive classes.

Callbacks let the application associate additional behavior when items in the list are selected or un-selected. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box, or in the menu editor. This script may perform actions of its own and/or call application functions, with/without arguments. Several functions in the Motif library are dedicated to accessing items in the List widget. Likewise, the FACE language has several convenience functions which access items in the List in various ways.*

The following callbacks are defined for this widget:

- m **browseSelectionCallback** - Invoked when an item is selected in the *browse_select* mode.
- m **defaultActionCallback** - Invoked when an item is double-clicked or activated using KActivate (usually the Return key) and the List has keyboard focus.
- m **extendedSelectionCallback** - Invoked when an item is selected in the *extended_select* mode.
- m **multipleSelectionCallback** - Invoked when an item is selected in the *multiple_select* mode.
- m **singleSelectionCallback** - Invoked when an item is selected in the *single_select* mode.

The most important specific resources of List are:

- m **itemCount** - Specifies the total number of items in the list.
- m **items** - Specifies the array of items, which must contain exactly **itemCount** items. *With XFaceMaker, you can specify the items as a comma-separated list of items in the resource editing field; itemCount will be modified to reflect the list of items entered. You can also use the special resource editing box to add, delete, modify items one by one. This resource is an array of XmStrings; XFaceMaker lets you enter strings which it converts to XmStrings.*

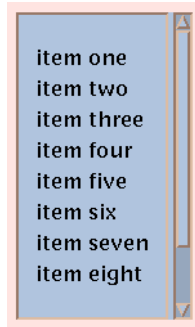
- m **selectedItemCount** - Specifies the total number of selected items in the List.
- m **selectedItems** - Specifies the array of selected items, which must contain exactly **selectedItemCount** items.
- m **selectionPolicy** - Specifies the type of selection. Possible values are: *single_select*, *browse_select* (the default), *multiple_select*, *extended_select*.
- m **visibleItemCount** - Specifies the number of list items that can fit in the List's work area. The widget uses this value to determine its height (*with XFaceMaker, when the autosize toggle is set*).

Other specific resources of List are:

- m **automaticSelection** - If *True*, and **selectionPolicy** is *browse_select* or *extended_select*, the **browseSelectionCallback** or **extendedSelectionCallback** is invoked before the mouse button is released. If *False*, no selection callbacks are invoked until the mouse button is released. Default is *False*.
- m **doubleClickInterval** - Specifies the maximum interval in milliseconds between clicks for double-clicks. Default is the display's multlick time.
- m **fontList** - Specifies the font used to display the items. Default value is taken from the **textFontList** resource in the parent hierarchy. If none is specified, the default is implementation dependent.
- m **listMarginHeight** - Specifies the height of the margin between the List border and the items. Default is 0.
- m **listMarginWidth** - Specifies the width of the margin between the List border and the items. Default is 0.
- m **listSizePolicy** - Controls the behavior of the List when an item grows wider than the List. If *constant*, the viewing area does not grow (and a ScrollBar is added for a ScrolledList); if *variable*, the widget grows to match the size of the longest item (if it is not otherwise constrained). If *resize_if_possible*, the widget attempts to grow or shrink to match the width of the widest item. This resource cannot be changed dynamically after widget creation. Default is *variable*.
- m **listSpacing** - Specifies the spacing between List items. Default is 0.
- m **scrollBarDisplayPolicy** - Controls the display of the vertical ScrollBar in a ScrolledList. If *static*, the vertical ScrollBar is always displayed; if *as_needed* (the default) the vertical ScrollBar is displayed only when the number of items exceeds the number of visible items.
- m **topItemPosition** - Specifies the position of the first visible item in the list. De-

fault is 1.

A `ScrolledList` with **itemCount 10**, **visibleItemCount 8**, **listMarginHeight 20**, **listMarginWidth 10**, **listSizePolicy variable** is shown below.



27.10 XmText

Texts are used for displaying, entering, and modifying text. The text can be single line or multiple line. For single line text, however, it is better to use the `XmTextField` widget which has a smaller overhead than `XmText`.

Motif defines the `XmScrolledText` in which the `Text` widget handles its own scrolling when the user interacts with the `ScrollBars`. *To build an `XmScrolledText` with `XFaceMaker`, place a `Text` widget inside a `ScrolledWindow` with **scrollingPolicy** set to *application_defined*.*

This class inherits resources from `Core` and `XmPrimitive` classes.

Callbacks let the application associate additional behavior when the user interacts with the contents of the text. *In `XFaceMaker`, you can program a callback directly by writing a `FACE` script in the resource editing box, or in the menu editor. This script may perform actions of its own and/or call application functions, with/without arguments. Several functions in the Motif library are dedicated to accessing data in the `Text` widget.*

The following callbacks are defined for this widget:

- m **activateCallback** - Invoked when the user invokes an event that calls the `Activate()` function. In *single_line_edit* mode, the Return key; in *multi_line_edit* mode, Control + Return.
- m **focusCallback** - Invoked when the widget accepts input focus, i.e. when keyboard input will be received by the widget.

- m **gainPrimaryCallback** - Invoked when the widget gains ownership of the primary selection.
- m **losePrimaryCallback** - Invoked when the widget loses ownership of the primary selection.
- m **losingFocusCallback** - Invoked before the widget loses input focus.
- m **modifyVerifyCallback** - Invoked before text is inserted or deleted. The *doit* field in the callback structure lets the application accept or reject the change.
- m **modifyVerifyCallbackWcs** - Invoked before text is inserted or deleted. The *doit* field in the callback structure lets the application accept or reject the change. Wcs stands for *Wide character set*, used to access data represented using the wide character type defined in ANSI-C. If both **modifyVerifyCallback** and **modifyVerifyCallbackWcs** are registered callback lists, **modifyVerifyCallback** is invoked first.
- m **motionVerifyCallback** - Invoked before the insert cursor is moved to a new position. A single action may invoke this callback several times.
- m **valueChangedCallback** - Invoked after the contents of the widget has been modified, i.e. text has effectively been inserted or deleted.

The most important specific resources of Text are:

- m **columns** - Specifies the initial width of the text window as an integer number of characters. Default depends on the value of the **width** resource. *With XFaceMaker, to have the widget's width controled by the **columns** resource, set the auto size toggle in the resouce window; this removes the **width** and **height** resources from the interface description file.*
- m **editable** - If *True*, the user can edit the text; if *False*, the widget beeps if the user attempts to modify the text. Default is *True*.
- m **editMode** - Specifies the keyboard bindings used when editing: *single_line_edit* specifies single line text editing (the default); *multi_line_edit* specifies multiple line text editing.
- m **maxLength** - Specifies the maximum length of the text string that can be entered from the keyboard. This resource is ignored if text is entered using the value resource or the Text convenience functions.
- m **rows** - Specifies the height of the widget, in character heights, if **editMode** is *multi_line_edit*. *With XFaceMaker, to have the widget's height controled by the **rows** resource, set the auto size toggle in the resouce window; this removes the **width** and **height** resources from the interface description file.*

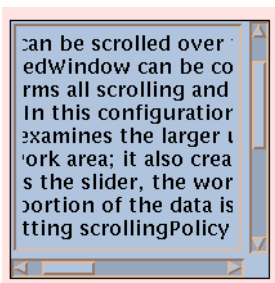
- m **value** - Specifies the content of the Text widget as a **char*** data value. If **value** and **valueWcs** are both defined, the value of **valueWcs** supersedes that of **value**.
- m **valueWcs** - Specifies the content of the Text widget as **wchar_t*** data value. If **value** and **valueWcs** are both defined, the value of **valueWcs** supersedes that of **value**.
- m **wordWrap** - When *True*, and **editMode** is *multi_line_edit*, indicates that lines are to be broken at word breaks.

Other specific resources of Text are:

- m **autoShowCursorPosition** - If *True* (default value), ensures that the visible text contains the insert cursor. The content of the Text may scroll to satisfy this requirement.
- m **blinkRate** - Blink rate of the text cursor in milliseconds. Default is *500* ms. If it is set to *0*, the cursor does not blink.
- m **cursorPosition** - Indicates the position of the insert cursor in the text, as the number of characters from the beginning of the text.
- m **cursorPositionVisible** - If *True*, indicates the the insert cursor position is marked by a blinking text cursor.
- m **fontList** - Specifies the font used to display the text. Default value is taken from the **textFontList** resource in the parent hierarchy. If none is specified, the default is implementation dependent.
- m **marginHeight** - Specifies the margin, in pixels, between the text and the top and bottom of the widget.
- m **marginWidth** - Specifies the margin, in pixels, between the text and the left and right edges of the widget.
- m **pendingDelete** - If *True* (the default), any selected text is deleted when an insertion is made.
- m **resizeHeight** - If *True*, indicates the Text will attempt to resize its height to accommodate all the text contained in the widget, if it is not in a *ScrolledWindow* with the **scrollVertical** resource *True*.
- m **resizeWidth** - If *True*, indicates the Text will attempt to resize the width to accommodate all the text contained in the widget, if **wordWrap** is *False*.
- m **selectThreshold** - Specifies the number of pixels of motion required to select the next character in click-drag mode of selection. Default is *5*.
- m **selectionArray** - Specifies the array of actions for multiple clicks. The actions

are: *select_position*, *select_word*, *select_line*, *select_all*.

- m **selectionArrayCount** - Indicates the number of elements in the **selectionArray** resource. Default is 4.
- m **source** - Specifies the source with which the widget displays text. This can be used to share text sources between Text widgets. If no source is specified, the widget creates a default string source.
- m **topCharacter** - Specifies the text position at the top of the window, i.e. on which character text display should start. Position is determined by the number of characters from the beginning of the text.



Above is an example of a ScrolledText.

27.11 XmTextField

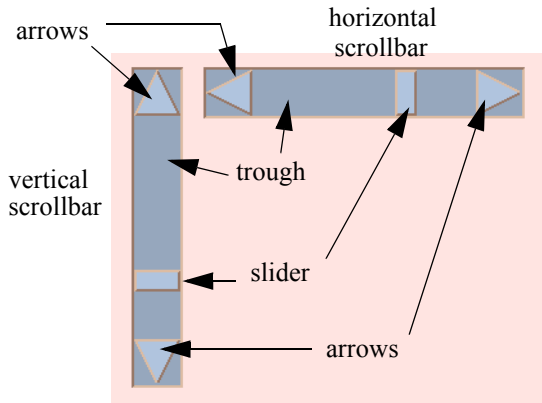
TextFields, like Texts, are used for displaying, entering, and modifying text. The text can only be single line. Because none of the multiple line editing capabilities are implemented in TextField, it has a smaller overhead than Text.

TextField and Text have the same callbacks and resources. Please refer to “Texts are used for displaying, entering, and modifying text. The text can be single line or multiple line. For single line text, however, it is better to use the XmTextField widget which has a smaller overhead than XmText.” on page 293.

27.12 XmScrollBar

The scrollbar allows the user to view data that is too large to be displayed all at once. ScrollBars are usually located inside a ScrolledWindow and adjacent to the data to be displayed. ScrollBars can have vertical or horizontal orientation.

ScrollBars and their components are shown in the illustration. Refer to “XmScrolled-Window” on page 269 to see ScrollBars as they are used in ScrolledWindows.



This class inherits resources from Core and XmPrimitive classes.

Callbacks let the application associate additional behavior when the user interacts with the ScrollBar components. *In XFaceMaker, you can program a callback directly by writing a FACE script in the resource editing box, or in the menu editor. This script may perform actions of its own and/or call application functions, with/without arguments. Several functions in the Motif library are dedicated to interacting with ScrollBars.*

The following callbacks are defined for this widget:

- m **decrementCallback** - Invoked when the user takes an action that decreases the slider position by one, e.g. clicks on the top or left arrow (depends on **processingDirection**).
- m **dragCallback** - Invoked when the user drags the slider.
- m **incrementCallback** - Invoked when the user takes an action that increases the slider position by one, e.g. clicks on the bottom or right arrow (depends on **processingDirection**).
- m **pageDecrementCallback** - Invoked when the user takes an action that decreases the slider position by one page, e.g. clicks in the trough above or to the left of the slider (depends on **processingDirection**).
- m **pageIncrementCallback** - Invoked when the user takes an action that increases the slider position by one page, e.g. clicks in the trough below or to the right

of the slider (depends on **processingDirection**).

- m **toBottomCallback** - Invoked when the user takes an action that brings the slider to its maximum position, e.g. Control + click below or to the right of the slider (depends on **processingDirection**).
- m **toTopCallback** - Invoked when the user takes an action that brings the slider to its minimum position, e.g. Control + click above or to the left of the slider (depends on **processingDirection**).

The most important specific resources of ScrollBar are:

- m **maximum** - Specifies the slider's maximum value.
- m **minimum** - Specifies the slider's minimum value.
- m **orientation** - Specifies the ScrollBar's orientation. The values accepted are *horizontal* and *vertical* (the default value).
- m **processingDirection** - Specifies the position of the maximum value. The values accepted are *max_on_bottom* (the default if **orientation** is *vertical*), *max_on_top*, *max_on_left*, *max_on_right*.
- m **value** - Specifies the slider's position. It is constrained to lie between *minimum* and *maximum*. The initial value is the larger of 0 and *minimum*.

Other specific resources of ScrollBar are:

- m **increment** - Specifies the amount by which the value increases or decreases when the slider is moved by one increment (click on one of the arrows).
- m **initialDelay** - Specifies the time in milliseconds to wait before starting slider motion after a user action. The default is 250 ms.
- m **pageIncrement** - Specifies the amount by which the value increases or decreases when the slider is moved by one page increment (click in the trough).
- m **repeatDelay** - Specifies the time in milliseconds to wait between subsequent slider motions after the **initialDelay** has been processed. This controls, in particular, slider motion speed when the user keeps the mouse button pressed on one of the arrows.
- m **showArrows** - If *True* (the default), the arrows are displayed.
- m **sliderSize** - Specifies the size of the slider. Default value is (*maximum* - *minimum*) divided by 10.
- m **troughColor** - Specifies the color of the slider trough.

Index

Symbols

\$ 136, 167

as pointer 206

* in widget specification 71

* operator 71

** 190

. operator 70, 124

: operator 71

= operator 124

-> operator 124

@ 167, 204, 205

[bottom]Atachment 264

[bottom]Offset 264

[bottom]Position 264

[bottom]Widget 265

A

A_CFLAGS, in makefile 106

A_INCPATH, in makefile 104

A_LDFLAGS, in makefile 106

A_LIBPATH, in makefile 105

accelerator(s) 236, 283

accelerators, in menu 182–183

acceleratorText 284

accessing, C++ class members
221

action(s)

Eval 237

for selectionArray

select_all 296

select_line 296

select_position 296

select_word 296

widget, and X events 237

activateCallback 74, 134, 199, 201,
206, 285, 288, 289, 290, 293

active values 165–176

\$ 167

@ 167

calling 169–176

creating 168–169

window 166–168

addressPulldown 74

adjustLast 268

adjustMargin 268

alignent_beginning 282

alignment 282

alignment_baseline_bottom 269,
278

alignment_baseline_top 269, 278

alignment_beginning 266, 278

alignment_center 266, 269, 278

alignment_center, as default 282

alignment_contents_bottom 269

alignment_contents_top 269

alignment_end 266, 278, 282

alignment_widget_bottom 278

alignment_widget_top 278

allocating, arrays 126

allowOverlap 260

allowResize 276

allowShellResize 250

ancestorSensitive 237

application

C Code 151–159

code 145

creating 93–111

- files 94–96
 - for building 93
 - saving 151, 169
- functions 145–150, 165
- launching 93, 175
- makefile 97–??
- making 158–159
- OSF/Motif 13
- testing 95, 121, 158
- versions 108, 158
- application_definedSee scrollingPolicy
- application_defined, and scrollingPolicy 273
- ApplicationShell 248, 257
- applicationShell 20
- Apply Options 29–30
 - box 29
- architecture, dual process 161–164
- argc 257
- argv 257
- armCallback 74, 285, 287, 288, 289
- armColor 286
- armPixmap 286
- arrays 116, 126–127
 - allocating 126
 - associative 126
 - declaring 126
 - indexed by String 126
 - indexing 126
 - linear 126
 - memory for 126
- arrow_down 289
- arrow_left 289
- arrow_right 289
- arrow_up, as default 289
- ArrowButtonSee XmArrowButton
- arrowDirection 289
- as_needed 272
- attach_form 264
- attach_none 264
- attach_opposite_widget 264
- attach_position 264
- attach_self 264
- attach_widget 264
- attaching
 - functions 151
 - widgets, to adjust main window 134
- attachment(s)
 - circular 265
 - defined by constraint resources 263
- attachments
 - building from right to left 84
 - copying 81
 - dialog box 64
 - making 59–66, 84–92
 - removing 84
 - resources 66
- audibleWarning 254
- augment mode 237
- Auto position 27
 - toggle, and saving a class 207
- auto position toggle, and default position 235

- Auto size 28
- auto size toggle, and default size 235
- automatic, and scrollingPolicy 272
- automaticSelection 292
- autoShowCursorPosition 295
- autosize toggle, and List height 292
- autoUnmanage 260
- B
- background 235
- backgroundPixmap 236
- baseHeight 251
- baseWidth 251
- bell 254
- blinkRate 295
- borderColor 236
- borderPixmap 236
- borderWidth 235
- bottom_left 272
- bottom_right 272
- bottomAttachment 92, 264
- bottomOffset 264
- bottomPosition 264
- bottomShadowColor 239, 242, 244
- bottomShadowPixmap 239, 242
- bottomWidget 265
- box, pixmap editing 236
- browse_select, as default 292
- browseSelectionCallback 135, 291
- BUFSIZ 171
- Build Appli 158, 175
- building
 - basic menu (procedures) 42–44
 - class sub-tree 197
- bulletin board 52, 54–55
 - and list widget 131
 - resources 54
- BulletinBoard 14, 248
 - unmanage, autoUnmanage 261
 - with DialogShell parent 260
 - without DialogShell parent 261
 - See also XmBulletinBoard
- buttonFontList 253, 261
- buttons
 - adding to File menu 140
 - Auto position 27
 - Auto size 27
 - createdoublelist 222–224
 - creating 88
 - creation scripts for, in building a class 199
 - ellipsis, and editing resources 29
 - font, standard 193
 - in option dialog boxes 114
 - mouse 4
 - of Restart Design dialog box 164
 - Rebuild, and applying resource changes 29
 - Restore state 163
 - Set Values, and applying re-

- source changes 29
 - setting to sensitive or insensitive 199
 - showdoublelist 224–225
 - Trusted state 163
- C
- C Code
- and defining variables 124
 - and porting to other platform 96, 117
 - and setting resource values 118
 - and structures 123
 - for application 151–159
 - generating 113–121
 - and compiler type 116
 - for a class 210–211
 - option box 113–114
 - options 115–119
 - save options, and widget ID's 116
 - saving 120–121
 - type 116
- C source files 145, 227
- C++ classes 165
- creating 213–227
 - instantiating 214
 - members, accessing 221
 - saving 220
 - sub-tree 214
 - usage toggle, and defining class members 215
 - using 221–227
- cache, pixmaps 236
- callback
- activateCallback 285, 288, 289, 293
 - armCallback 285, 287, 288, 289
 - browseSelectionCallback 291
 - decrementCallback 297
 - defaultActionCallback 291
 - disarmCallback 285, 287, 288, 289
 - dragCallback 279, 297
 - exposeCallback 288
 - extendedSelectionCallback 291
 - focusCallback 260, 293
 - gainPrimaryCallback 294
 - incrementCallback 297
 - losePrimaryCallback 294
 - losingFocusCallback 294
 - mapCallback 260
 - modifyVerifyCallback 294
 - modifyVerifyCallbackWcs 294
 - motionVerifyCallback 294
 - multipleSelectionCallback 291
 - pageDecrementCallback 297
 - pageIncrementCallback 297
 - popdownCallback 250
 - popupCallback 250
 - resizeCallback 288
 - singleSelectionCallback 291
 - toBottomCallback 298
 - toTopCallback 298
 - unmapCallback 260

- valueChangedCallback 287, 294
- Callback resource
 - adding to a class 206
- callbacks 69–77
 - activate
 - and application functions 150
 - and creating active values 169
 - cancel, for file selector 150
 - editing 142
 - writing, and selection policy 135–138
- calling
 - active values 169–176
 - functions 145
 - member function 219
- cancelButton 262
- cancelCallback 150
 - as class resource 204
- CascadeButtonSee XmCascadeButton
- cascadePixmap 290
- cascadingCallback 290
- CCODEOPTS, in makefile 105
- CDEBUGFLAGS, in makefile 105
- changing
 - class 132
 - template definition 191
- CheckBox 266, 287
- childHorizontalAlignment 278
- childHorizontalSpacing 278
- children
 - and FACE 66
 - creating 13
 - creating in interface 14
 - of templates 195
 - of widget 12
 - of XmMainWindow widget 13
 - overlapping 260
- childType 278
- childVerticalAlignment 278
- circular attachments 265
- class
 - adding internal behavior to 198–203
 - behavior, internal, of 197
 - changing 132
 - creating, C++ 213–227
 - creating, Xt 197–211
 - editing, new 209
 - files, standard 197
 - members 214
 - members, defining 215–219
 - saving 207–210
 - saving, and auto position toggle 207
 - saving, and Modules menu 207
 - sub-tree 197
 - sub-tree, building 197
 - widget, creating 197–211
 - Xt, resources and scripts 204–207
- class hierarchy

- and resources 230
- Motif 230–233
- class(es) 230
 - ApplicationShell 257
 - metaclass
 - Composite 238
 - Constraint 238
 - Core 235
 - XmGadget 244
 - XmManager 239
 - XmPrimitive 241
 - OverrideShell 250
 - shellmetaclass
 - Shell 249
 - VendorShell 253
 - WMSHELL 251
 - TopLevelShell 256
 - TransientShell 256
 - XmDialogShell 256
 - XmMenuShell 251
- clipWindow 273
- close windows 247
- code
 - application 145
 - FACE, initializing and executing 72
- color(s), shadow 239
- colormap 237
- colors 36
- columns 294
- Command window option 273
- commandWindow resource 274
- commandWindowLocation resource 274
- compilation 175
- compiled interface 108
- Composite class 238
 - children
 - management of 238
 - mapping and unmapping of 238
 - descendants, destruction of 238
 - geometry, children's, managing 238
- Composite Toolkit 231
 - and creating a RowColumn 268
- composite widgets 259–280
- Composite, as widget category 9
- Constraint class 238
- constraint(s)
 - on XmForm children 263
 - on XmPanedWindow children 276
- constructs, FACE 123–129
- container widget 20
- container(s), composite widgets as 259
- Core class 235–238
- CP, in makefile 106
- createPopUpChildProc 250
- creating
 - active values 168–169
 - buttons 88
 - C++ class 213–227
 - dialog box, for file selector 139

- file selector 139
- groups 177–179
- templates 185–188
- widget class 197–211
- window, scrolled 131
- Creation Name, and generating C Code 116
- creation order, of sibling widgets 238
- creation order, of widgets 140
- current file name 94, 120
- current selection
 - and active value dialog box 166
 - and changing template definition 192
 - and menu editor 80, 140
 - notion in XFaceMaker 80
- current session, saving state of 161
- CurrentSelection toggle 23, 80
- cursorPosition 295
- Custom Editor 33
- cutting, and pasting 79–83
 - and reparenting 131
- D
- decimalPoints 279
- declarations 124–125
 - function 145
- declaring
 - arrays 126
 - functions 145
 - structures, in FACE 123
 - types, for callback structures 136
 - variables 123
- decorations 248
 - window decorations 247
- decrementCallback 297
- default values
 - of C Code options 115
- defaultActionCallback 291
- defaultActionCallback, activating 202
- defaultButton 261
- defaultButtonShadowThickness 286
- defaultFontList 253
- defaultPosition 261
- defining
 - member, private data 215–216
 - members, class 215–219
 - resources, class 204–207
- definitions, basic 1–3
- deleteResponse 254
- depth 237
- design process
 - relaunchingSee restarting
 - restarting 163–164
- Design Process menu 161
 - and making application 158
- destList 204
 - resource, adding to a class 205
- destroy 254
- destroyCallback 237
- Detach All Sides 265
- Detach this Side 265

- dialog boxes 51–57
 - creating, for file selector 139
- dialog(s)
 - MessageBox 261
 - modal 260
 - modeless 260
- Dialog, as widget category 9
- dialog_application_modal 261
- dialog_full_application_modal 261
- dialog_modeless 261
- dialog_system_modal 261
- dialog_work_area 261
- dialogShell 51–53
 - and creating file selector 139
 - and creating window 52
 - resources 53
- DialogShellSee XmDialogShell
- dialogStyle 150, 261
 - and dialog_work_area 261
- dialogTitle 262
- dimension, major or minor, and value of numColumns 267
- disarmCallback 285, 287, 288, 289
- DM_LICENSE_NAME 5
- do_nothing 254
- double_dashed_line 285
- double_line 285
- doubleClickInterval 292
- dragCallback 279, 297
- DrawingAreaSee XmDrawingArea
- DrawnButtonSee XmDrawnButton
- dual process architecture 161–164
- dual process state 162
- Duplicate (command)
 - copying and pasting widgets, using 82
- duplicating, widgets 82
- E
- E_CFLAGS, in makefile 106
- E_INCPATH, in makefile 104
- E_LDFLAGS, in makefile 106
- E_LIBPATH, in makefile 105
- editable 294
- editing 17–30, 79–92
 - advanced 131–138
 - and current selection 80
 - callbacks 142
 - copying 80
 - cutting 79
 - duplicating widgets 82
 - fm file 113
 - operations, on selected objects
 - Copy 80
 - Cut 80
 - Delete 80
 - Duplicate 80
 - pasting 79, 80
 - resources 31–37
- editing box, pixmap 236
- editing resources 230
- editMode 294
- ellipsis button, and editing resources 29
- entryAlignment 266

- entryBorder 268
- entryCallback 266
- entryClass 268
- entryVerticalAlignment 268
- enumerations 35
- environment variable
 - DM_LICENSE_NAME 5
 - LD_LIBRARY_PATH 5
 - NSL_LICENSE_HOST 5
 - NSLHOME 5
 - PATH 5
 - WX_LICENSE_NAME 5
 - XFM_LICENSE_NAME 5
- errors, fatal
 - and design process 163
 - recovering from 161
- Eval action 237
- event handler, adding to a widget 237
- Exit (command) 7, 77
- exiting, and saving 163
- explicit 253
- exposeCallback 275, 288
- extended_select 292
- extendedSelectionCallback 291
- extern variables 116
- EXTRA_C_LIBS, in makefile 104
- EXTRA_E_LIBS, in makefile 104
- EXTRA_I_LIBS, in makefile 104
- EXTRA_INCPATH, in makefile 104
- EXTRA_LIBPATH, in makefile 104
- F
- FACE
 - and C programming language 70
 - and widget resources 71
- arrays 126–127
- code, initializing and executing 72
- constructs 123–129
- declarations 124–125
- dynamically typed variables 125
- function calls 73
- function declarations
 - differences between C and 145
- global keyword and variable scope 125
- introduction to 70–71
- keywords, and widgets 70
- language debugger 96
- local keyword and variable scope 125
- operators 70–71
- pointer and structures 123
- programming language 66, 69–77
- programming, advanced 123–129
- resource names 71
- scripts and active values 165
- statements 148

- statically typed variables 124
- syntax 70
- variables, typing 124–125
- widget references 71
- fgets 172
- file selector 139–143
 - and cancel callback resource 150
 - and okCallback resource 146–149
 - and saving code 120–121
 - creating 139
 - opening 142–143
- filePulldown 74, 140, 181
- files
 - application 94–96
 - saving 169, 226–227
 - C source 145
 - changing with text editor 93
 - for application building 93
 - name, current 94, 120
 - object 108
 - saving, and applying C Code dialog box options 115
 - saving, with option box toggle 114
 - selecting, with file selector 139
- fillOnArm 286
- fillOnSelect 287
- fm file 93, 95, 121
 - and C Code options 117
 - editing 113
- fm file format
 - and text editors 93
- Fm library 95, 117
- Fm_c
 - library, and linking with generated C Code 117
- FmAttachFunction 152
- FmCreateDlistXt 210
- FmCreateName 119
- FmCreatename 116, 119
- FmCreateXmString 128, 129, 136, 142, 143
- FmDoEvent 217
- FMEDITOR 33
- FMGetActiveValue 170
- FmGetString 146, 147, 149
- FmHideWidget 146, 149
- FmListGetItems 204, 205, 216
- FmListSetItems 204, 205
- FmSetActiveValue 170, 171
- FmSetTallest 73, 86
 - and xfmCreateCallback 72
- FmSetWidest 87
- FmShowWidget 75, 142, 143
- focusCallback 260, 293
- Font Selector 45–46
- fontList 280, 282, 292, 295
- fonts
 - default, for form widgets 61
 - selecting 44–46, 139
- foreground 239, 242
- form layout 60
- form widget 59
 - adding resources to 204

- and constraints 59
- FormSee XmForm
- forms 59–67
 - and changing class 132
 - and default layout parameters 61
 - attaching to 63–65
 - size, and separators 90
- fractionBase 263
- FrameSee XmFrame
- frame_generic_child 278
- frame_title_child 278
- frame_workarea_child 278
- function keyword 145
- functions 165
 - application 145–150, 165
 - application, description of 153–155
 - application, including in design process 161
 - attaching 151
 - calling 145
 - declarations 145
 - declaring 145
 - definitions, code for 153–??
 - member, calling 219
 - member, defining 217–219
 - public member 214
- G
- GadgetSee XmGadget
- gadget(s)
 - toggle 229, 232
- gadget(s) vs widget(s) 229–230
- gadgets 28
 - and changing sub-tree 192
 - label 191
- gainPrimaryCallback 294
- generating
 - C Code 113–121
 - interactive way, and from fm file 113
 - C Code, for a class 210–211
- geometry 250
- geometry, of children 259
- get 216
- get script 165
 - and XtGetValues 204
 - resetting, in adding Callback 206
- GLS button 33
- graphics and DrawingArea 275
- groups 177–183
 - creating 177–179
 - definition, saving 183
 - modifying 181–183
 - saving 179
 - using 180
- H
- height 235
- heightInc 251
- helpCallback
 - for XmGadget 244
 - for XmManager 239
 - for XmPrimitive 241
- helpPulldown 75
- highlightColor 240, 242, 244
- highlightOnEnter 242, 244, 280
- highlightPixmap 240, 242

- highlightThickness 242, 244, 280
- horizontal orientation, in Row-Column 267
- horizontalScrollBar 273
- horizontalSpacing 264
- I
- I/O library, UNIX standard 153
- library
 - Fm_c, and linking with generated C Code 117
- iconic 257
- iconicstate 252
- iconify windows 247
- iconMask 252
- iconName 257
- iconNameEncoding 257
- iconPixmap 252
- iconWindow 252
- iconX 252
- iconY 252
- increment 298
- incrementCallback 297
- indexing, arrays 126
- indicator, of indicatorType
 - l_of_many 287
 - n_of_many 287
- indicatorOn 287
- indicatorSize 287
- indicatorType 287
- initialDelay 298
- initialFocus 240
- initialState 252
- input 252
- inputMethod 254
- insertPosition 238
- installation 5
- instantiating, C++ class 214
- integers 34
- interface 10
 - and widget hierarchy 11
 - as compiled C code 95
 - building 10, 17–21
 - using group 180
 - compiled 108, 158
 - creating children in 14
 - default name 11
 - editor 108, 158
 - file name 11
 - interpreted 108, 158
 - look, standard, and templates 193
 - saving 93–94, 203, 226–227
 - testing 167, 203
 - testing behavior of 76
- interface pane 11–12
 - and widget tree 11
- icons 11
- internal behavior, adding to class 198–203
- Internationalization 33
- interpreted interface 108, 158
- intrinsics 2
 - See also XtIntrinsics library
- isAligned 266
- isHomogeneous 269
- itemCount 291
- items 291

K

keyboard, key names 4
keyboardFocusPolicy 242, 244,
253

keyword

parent 70
root 70
self 70
toplevel 70

keyword, function 145

L

LabelSee XmLabel

labelFontList 253, 262

LabelGadget 269

labelPixmap 282

labelString 186, 269, 282

labelString, setting 22

labelType 282

Launch Appli 158, 175

launching

application 93, 175
XFaceMaker 5
XFaceMaker, and design
process status 161

LD_LIBRARY_PATH 5

leaving XFaceMaker 7

leftAttachment 134, 264

library

C code support, XFaceMa-
ker's 95
editor, XFaceMaker's 96
Fm 95, 117
Fm_c 95
Fm_e 96

interpretive, XFaceMaker's
95

line type, and separatorType 285

ListSee XmList

listMarginHeight 292

listMarginWidth 292

listSizePolicy 292

listSpacing 292

LN, in makefile 107

loading

template definition 191

losePrimaryCallback 294

losingFocusCallback 294

lower windows 247

M

main window, of interface

adjusting 134–135
and Select button 17
changing position of 10
changing size of 10, 132
creating 10

Main Window, XFaceMaker's
7–12

changing pane size of 7

MainWindowSee XmMainWin-
dow

mainWindowMarginHeight 274

mainWindowMarginWidth 274

make command 97

make, unsuccessful 111

makefile 169

and making application with
interpreted interface
111

- application 97–??
 - modifying, to include compilation 226
 - rules 108–110
 - variables 102–107
- making
 - application 158–159
 - program 111
- ManagerSee `XmManager`
- `mapCallback` 260, 266
- `mappedWhenManaged` 237
- `mappingDelay` 290
- `margin` 284
- `marginBottom` 283
- `marginHeight` 17, 262, 269, 275, 276, 278, 283, 295
 - and sizing widgets 21
- `marginLeft` 283
- `marginRight` 283
- `marginTop` 283
- `marginWidth` 17, 262, 269, 275, 276, 278, 283, 295
 - and sizing widgets 21
- `max_on_bottom` 279, 298
- `max_on_left` 279, 298
- `max_on_right` 279, 298
- `max_on_top` 279, 298
- `maxAspectX` 251
- `maxAspectY` 251
- `maxHeight` 251
- maximize
 - window decoration 247
- maximum 279
 - as `XmScrollBar` resource 298
- `maxLength` 294
- `maxWidth` 251
- member, private data 214
 - defining 215–216
- members
 - C++ class, accessing 221
 - C++ class, defining 214
- menu
 - building 39
 - Design Process 161
 - design process, and making application 158
 - OSF/Motif standard 179
- menu bar
 - and groups 177
 - editing 39–50
- menu editor 39–50
 - and adding buttons 140–141
 - and changing menu item order 140
 - and current selection 80, 140
- menu system, and `menuPost` 270
- Menu Toolkit 266
 - and creating a `RowColumn` 268
- Menu, as widget category 9
- `menu_bar` and Menu Toolkit 268
- `menu_option` and Menu Toolkit 268
- `menu_popup` and Menu Toolkit 268
- `menu_pulldown` and Menu Toolkit 268
- `menuAccelerator` 269

- menuBar 273, 275
- menuHelpWidget 269
- menuHistory 270
- menuPost 270
- message area
 - clearing 8
 - errors 8
 - warnings 8
- Message window 273
- MessageBox dialogs 261
- messageWindow 275
- metaclass, definition of 230
- metaclasses, the 235–245
 - Composite 238
 - Constraint 238
 - Core 235–238
 - Shell 249–250
 - VendorShell 253–255
 - WMShell 251–253
 - XmGadget 244–245
 - XmManager 239–241
 - XmPrimitive 241–243
- minAspectX 252
- minAspectY 252
- minHeight 252
- minimize
 - window decoration 247
- minimum 279
 - as XmScrollBar resource 298
- minWidth 252
- mnemonic 270, 284
- mnemonicCharSet 270
- mnemonics, in standard Edit
- menu 181–183
- modal shell 248
- mode in translations
 - augment 237
 - override 237
 - replace 237
- modifying
 - template definition 191–192
 - template instance 190–191
- modifyVerifyCallback 294
- modifyVerifyCallbackWcs 294
- Modules menu
 - and saving a class 207
- Motif
 - window menu 255
- Motif class hierarchy 230–233
- Motif window manager 253
- MOTIF_INCPATH, in makefile 102
- MOTIF_LIBNAME, in makefile 102
- MOTIF_LIBPATH, in makefile 102
- MOTIF_LIBS 161
- MOTIF_LIBS, in makefile 105
- motionVerifyCallback 294
- mouse buttons 4
- move windows 247
- multi_line_edit 294
- multi_line_edit, and wordWrap 295
- multiClick 286, 288, 289
- multiclick_discard 286, 289
- multiclick_keep 286, 288, 289
- multiple_select 292

- multipleSelectionCallback 291
- mwm window manager 253
- mwm window manager, and re-size controls with noResize 262
- mwmDecorations 254
- mwmFunctions 254
- mwmInputMode 255
- mwmMenu 255
- N
- n_of_many, as default 287
- names, FACE resource 71
- navigationType 240, 243, 245
- new_array (function) 126
- new_c_array (function) 126
- new_struct (function) 124
- new_table (function) 126
- no_line 285
- None
 - FACE type 145
- none 254
- noResize 262
- normalstate 252
- NSL_LICENSE_HOST 5
- NSLHOME 5
- numbers 34
- numChildren 238
- numColumns 267
- O
- object file 108
- objects 213
 - graphic 213
 - model, and defining application 213
- offsets 88–92
- OK Callback 146–149
- okCallback
 - as class resource 204
 - script 146
- 100th_font_units 255
- 100th_millimeters 255
- 100th_points 255
- 1000th_inches 255
- one_of_many 268, 287
- one_of_many, and ToggleButton indicators 268
- Open (command) 15
- opening, file 15
- opposite sides 89
- option dialog box buttons 114
- option, Widget Factory 232
- order, creation, of sibling widgets 238
- orientation 267, 285
 - in RowColumn 267
 - in XmScale 279
 - in XmScrollBar 298
- OSF/Motif
 - drag-and-drop system 165
- OSF/Motif Style Guide 177, 182, 183
- OSF/Motif toolkit 96
- override
 - and default values, in templates 194
 - and template resources 189
- override mode 237
- overrideRedirect 250
- OverrideShell 248, 250

P

- pack_column 267
- pack_none 267
- pack_tight 267
- packing 267
- pageDecrementCallback 297
- pageIncrement 298
- pageIncrementCallback 297
- pane size, and control sashes 276
- PanedWindowSee XmPaned-Window
- paneMaximum 277
- paneMinimum 277
- parent
 - and FACE 70
 - in widget referencing 71
 - keyword 70
- pasting, and cutting 79–83
 - and reparenting 131
- PATH 5
- pendingDelete 295
- pixels 255
- pixmap editing box 236
- pixmap cache 236
- pointer 123, 253
- pointer, and keyboardFocusPolicy 244
- popdownCallback 250
- Popup 28
- popupCallback 250
- popupEnabled 270
- positionIndex 270, 277
- positioning resourcesSee values
- positions 88–92

- and centering widgets 90
- preeditType 255
- Preserve User Code 94
 - and active value scripts 170
- PreserveUserCode
 - option, and makefile 97
 - toggle, and saving 226
- Primitive Toolkit 231
- primitive widgets 281–298
- PrimitiveSee XmPrimitive
- Primitive, as widget category 9
- process
 - architecture, dual 161–162
 - design 161, 163
 - XFaceMaker 161
- processingDirection 279, 298
- push buttonSee buttons
- PushButton
 - See also XmPushButton
 - appearance 285
- pushButtonEnabled 289
- R
- radioAlwaysOne 267
- radioBehavior 268
- RadioBox 266, 270, 287
- raise windows 247
- Rebuild button 29
- recomputeSize 283
- refigureMode 276
- reparenting 131–132
- repeatDelay 298
- replace mode 237
- resize handle
 - window decoration 247

- resize windows 247
- resize_any 262
- resize_grow 262
- resize_none 262
- resizeCallback 275, 288
- resizeHeight 270, 295
- resizePolicy 262, 276
- resizeWidth 270, 295
- resource database 118
- resource list, and class structure 27
- resource values
 - editing 26
 - setting with C Code options 118
- Resource Window 22–30
- resource(s) 229–230
 - and class hierarchy levels 230
 - ApplicationShell
 - argc 257
 - argv 257
 - Composite
 - children 238
 - insertPosition 238
 - numChildren 238
 - Constraint 238
 - constraint, of Form
 - [bottom]Attachment 264
 - [bottom]Offset 264
 - [bottom]Position 264
 - [bottom]Widget 265
 - constraint, of Frame
 - childHorizontalAlign-ment 278
 - childHorizontalSpacing 278
 - childType 278
 - constraint, of PanedWindow 276
 - allowResize 276
 - paneMaximum 277
 - paneMinimum 277
 - positionIndex 277
 - skipAdjust 277
 - constraint, of RowColumn
 - positionIndex 270
- Core 235–238
 - accelerators 236
 - color 235
 - destroyCallback 237
 - geometric 235
 - mappedWhenManaged 237
 - sensitive 237
 - texture 235
 - translations 237
- Core, restricted
 - ancestorSensitive 237
 - colormap 237
 - depth 237
 - screen 238
- editing 230
 - and dedicated editing windows 230
- for VendorShell
 - mwmInutMode 255
- Shell
 - allowShellResize 250

- createPopupChildProc
 - 250
- geometry 250
- overrideRedirect 250
- popdownCallback 250
- popupCallback 250
- saveUnder 250
- visual 250
- TopLevelShell
 - iconic 257
 - iconName 257
 - iconNameEncoding 257
- TransientShell
 - transientFor 256
- type Widget 230
- VendorShell
 - audibleWarning 254
 - buttonFontList 253
 - defaultFontList 253
 - deleteResponse 254
 - inputMethod 254
 - keyboardFocusPolicy 253
 - labelFontList 253
 - mwmDecorations 254
 - mwmFunctions 254
 - mwmMenu 255
 - preeditType 255
 - shellUnitType 255
 - textFontList 254
 - useAsyncGeometry 255
- WMShell
 - baseHeight 251
 - baseWidth 251
 - heightInc 251
- iconMask 252
- iconPixmap 252
- iconWindow 252
- iconX 252
- iconY 252
- initialState 252
- input 252
- maxAspectX 251
- maxAspectY 251
- maxHeight 251
- maxWidth 251
- minAspectX 252
- minAspectY 252
- minHeight 252
- minWidth 252
- title 252
- titleEncoding 252
- transient 252
- waitforwm 253
- widthInc 252
- windowGroup 253
- winGravity 253
- wmTimeout 253
- XmArrowButton
 - activateCallback 289
 - armCallback 289
 - arrowDirection 289
 - disarmCallback 289
 - multiClick 289
- XmBulletinBoard
 - allowOverlap 260
 - autoUnmanage 260
 - buttonFontList 261
 - cancelButton 262

- defaultButton 261
- defaultPosition 261
- dialogStyle 261
- dialogTitle 262
- focusCallback 260
- labelFontList 262
- mapCallback 260
- marginHeight 262
- marginWidth 262
- noResize 262
- resizePolicy 262
- shadowType 262
- textFontList 262
- textTranslations 262
- unmapCallback 260
- XmCascadeButton
 - activateCallback 290
 - cascadePixmap 290
 - cascadingCallback 290
 - mappingDelay 290
 - subMenuId 290
- XmDrawingArea
 - exposeCallback 275
 - inputCallback 275
 - marginHeight 275
 - marginWidth 275
 - resizeCallback 275
 - resizePolicy 276
- XmDrawnButton
 - activateCallback 288
 - armCallback 288
 - disarmCallback 288
 - exposeCallback 288
 - multiClick 288
 - pushButtonEnabled 289
 - resizeCallback 288
 - shadowType 289
- XmForm
 - fractionBase 263
 - horizontalSpacing 264
 - rubberPositioning 264
 - verticalSpacing 264
- XmFrame
 - childVerticalAlignment 278
 - marginHeight 278
 - marginWidth 278
 - shadowType 278
- XmGadget
 - bottomShadowColor 244
 - helpCallback 244
 - highlightColor 244
 - highlightOnEnter 244
 - highlightThickness 244
 - navigationType 245
 - shadowThickness 244
 - topShadowColor 244
 - traversalOn 245
 - unitType 245
 - userData 245
- XmLabel
 - accelerator 283
 - acceleratorText 284
 - alignment 282
 - fontList 282
 - labelPixmap 282
 - labelString 282
 - labelType 282

- marginBottom 283
- marginHeight 283
- marginLeft 283
- marginRight 283
- marginTop 283
- marginWidth 283
- mnemonic 284
- recomputeSize 283
- XmList
 - automaticSelection 292
 - browseSelectionCallback 291
 - defaultActionCallback 291
 - doubleClickInterval 292
 - extendedSelectionCallback 291
 - fontList 292
 - itemCount 291
 - items 291
 - listMarginHeight 292
 - listMarginWidth 292
 - listSizePolicy 292
 - listSpacing 292
 - multipleSelectionCallback 291
 - scrollBarDisplayPolicy 292
 - selectedItemCount 292
 - selectedItems 292
 - selectionPolicy 292
 - singleSelectionCallback 291
 - topItemPosition 292
 - visibleItemCount 292
- XmMainWindow
 - commandWindow 274
 - commandWindowLocation 274
 - mainWindowMarginHeight 274
 - mainWindowMarginWidth 274
 - menuBar 275
 - messageWindow 275
 - showSeparator 275
- XmManager
 - bottomShadowColor 239
 - bottomShadowPixmap 239
 - foreground 239
 - helpCallback 239
 - highlightColor 240
 - highlightPixmap 240
 - initialFocus 240
 - navigationType 240
 - shadowThickness 240
 - topShadowColor 240
 - topShadowPixmap 240
 - traversalOn 241
 - unitType 240
 - userData 241
- XmPanedWindow
 - marginHeight 276
 - marginWidth 276
 - refigureMode 276
 - sashHeight 276
 - sashIndent 276

- sashShadowThickness
 - 276
- sashWidth 276
- separatorON 276
- spacing 276
- XmPrimitive
 - bottomShadowColor 242
 - bottomShadowPixmap
 - 242
 - foreground 242
 - helpCallback 241
 - highlightColor 242
 - highlightOnEnter 242
 - highlightPixmap 242
 - highlightThickness 242
 - navigationType 243
 - shadowThickness 242
 - topShadowColor 242
 - topShadowPixmap 243
 - traversalOn 243
 - unitType 243
 - userData 243
- XmPushButton
 - activateCallback 285
 - armCallback 285
 - armColor 286
 - armPixmap 286
 - defaultButtonShadow-
Thickness 286
 - disarmCallback 285
 - fillOnArm 286
 - multiClick 286
 - showAsDefault 286
- XmRowColumn
 - adjustLast 268
 - adjustMargin 268
 - entryAlignment 266
 - entryBorder 268
 - entryCallback 266
 - entryClass 268
 - entryVerticalAlignment
 - 268
 - isAligned 266
 - isHomogeneous 269
 - labelString 269
 - mapCallback 266
 - marginHeight 269
 - marginWidth 269
 - menuAccelerator 269
 - menuHelpWidget 269
 - menuHistory 270
 - menuPost 270
 - mnemonic 270
 - mnemonicCharSet 270
 - numColumns 267
 - orientation 267
 - packing 267
 - popupEnabled 270
 - radioAlwaysOne 267
 - radioBehavior 268
 - resizeHeight 270
 - resizeWidth 270
 - rowColumnType 268
 - spacing 270
 - subMenuId 270
 - tearOffMenuActivateCallback 266
 - tearOffMenuDeactivate

- teCall- back 266
- tearOffModel 270
- unmapCallback 266
- whichButton 270
- XmScale
 - decimalPoints 279
 - dragCallback 279
 - fontList 280
 - highlightOnEnter 280
 - highlightThickness 280
 - maximum 279
 - minimum 279
 - orientation 279
 - processingDirection 279
 - scaleHeight 280
 - scaleMultiple 280
 - scaleWidth 280
 - showValue 279
 - titleString 280
 - value 279
 - valueChangedCallback 279
- XmScrollBar
 - decrementCallback 297
 - dragCallback 297
 - increment 298
 - incrementCallback 297
 - initialDelay 298
 - maximum 298
 - minimum 298
 - orientation 298
 - pageDecrementCall-
back 297
 - pageIncrement 298
 - pageIncrementCallback 297
 - processingDirection 298
 - repeatDelay 298
 - showArrows 298
 - sliderSize 298
 - toBottomCallback 298
 - toTopCallback 298
 - troughColor 298
 - value 298
- XmScrolledWindow
 - clipWindow 273
 - horizontalScrollBar 273
 - scrollBarDisplayPolicy 272
 - scrollBarPlacement 272
 - scrolledWindowMargin-
Height 273
 - scrolledWindowMar-
ginWidth 273
 - scrollingPolicy 272
 - spacing 273
 - traverseObscuredCall-
back 272
 - verticalScrollBar 273
 - visualPolicy 273
 - workWindow 273
- XmSeparator
 - margin 284
 - orientation 285
 - separatorType 285
- XmText
 - activateCallback 293
 - autoShowCursorPosi-

- tion 295
- blinkRate 295
- columns 294
- cursorPosition 295
- cursorPositionVisible 295
- editable 294
- editMode 294
- focusCallback 293
- fontList 295
- gainPrimaryCallback 294
- losePrimaryCallback 294
- losingFocusCallback 294
- marginHeight 295
- marginWidth 295
- maxLength 294
- modifyVerifyCallback 294
- modifyVerifyCallback-kWcs 294
- motionVerifyCallback 294
- pendingDelete 295
- resizeHeight 295
- resizeWidth 295
- rows 294
- selectionArray 295
- selectionArrayCount 296
- selectThreshold 295
- source 296
- topCharacter 296
- value 295
- valueChangedCallback 294
- valueWcs 295
- wordWrap 295
- XmToggleButton
 - armCallback 287
 - disarmCallback 287
 - fillOnSelect 287
 - indicatorOn 287
 - indicatorSize 287
 - indicatorType 287
 - selectColor 287
 - selectInsensitivePixmap 288
 - selectPixmap 288
 - set 288
 - spacing 288
 - valueChangedCallback 287
 - visibleWhenOff 288
- resources
 - accessing, in new class 208
 - activateCallback 74, 134
 - adding to a class 204
 - attachment 84, 85, 90
 - bottomAttachment 92
 - Callbacks 27
 - changing, and override template 190
 - colors 36
 - constraints 59
 - converting 118
 - deleteResponse 53
 - dragging 133
 - editing 31–37, 76
 - editing, and ellipsis button 29
 - enumerations 35

- fontList 193
- for templates 185
- integers and other numbers
 - 34
- labelString 22, 137, 186
- leftAttachment 134
- looking for 27
- marginHeight 17, 133, 193
- marginWidth 17, 133, 193
- name prefixes 71
- okCallback 204
- pixmap, converting 119
- rightAttachment 134
- Search 27
- selectionPolicy 135
- specification, and : operator
 - 71
- specifying new, in Xtclass
 - creation 209–210
- string 31
- template 193–195
- template default value of, to
 - specified value 190
- title 53
- topAttachment 81
- verticalSpacing 133
- widget 25–28
- xfmCreateCallback 72
- XmString 137
- Xt class, and scripts 204–207
- Xt class, defining 204–207
- Restart Design Process 158
- restarting
 - design process 163–164
 - with Restart Design Process
 - 158
- Restore state 163
- restoring
 - state 163
- Restricted resources
 - and templates 189
- rightAttachment 134, 264
- RM, in makefile 106
- root
 - and FACE 70
 - keyword 70
- RowColumnSee XmRowColumn
- rowColumnType 268
- rows 294
- rubberPositioning 264
- rules, makefile 108–110
- run time
 - and fm file 95
- S
- sash(es) and controlling pane
 - size 276
- sashHeight 276
- sashIndent 276
- sashShadowThickness 276
- sashWidth 276
- Save (command) 15
- Save As (command) 15
- Save Prefs option 94
- saveUnder 250, 256
- saving 15
 - and "fm" file format 93
 - and applying C Code dialog

- box options 115
- and exiting 163
- and Preserve User Code 94
- and status indicator icons 161
- and Try Mode 76
- application files 169, 226–227
- C Code 120–121
- C++ class 220
- class 207–210
- class, new version of 210
- definition, group 183
- design state 161–162
- files, application 151
- files, with option box toggle
 - 114
- groups 179
- interface 93–94, 203, 226–227
- interface, and current file name 120
- state
 - restore 163
 - trusted 163
- state of current session 161
- template definition by current selection 192
- templates 187
- widget ID 116
- ScaleSee XmScale
- scaleHeight 280
- scaleMultiple 280
- scaleWidth 280
- scope, and variable declarations 124
- screen 238
- script
 - active values 165
 - and Custom Editor 167
 - application functions in 145
 - editing, and Select button
 - 142, 147
 - for C++ class 215–220
 - for Xt class 204–207
 - get 165
 - global variables in 124
 - inserting widget name in, with Select button 75
 - local variables in 125
 - set 165
 - testing 129, 135, 137
 - with application function
 - 158, 161
 - with application function default value 145
 - variable allocation 125
- ScrollBarSee XmScrollBar
- scrollBarDisplayPolicy 202, 272, 292
- scrollBarPlacement 272
- ScrolledList 271
 - in MainWindow 274
- ScrolledText 271
 - in MainWindow 274
- ScrolledWindowSee XmScrolledWindow
- scrolledWindowMarginHeight 273
- scrolledWindowMarginWidth 273

- scrolling, application implemented 271
- scrollingPolicy 272
- Select button
 - and editing script 142, 147
- select mode
 - and class building 202
- select_all 296
- select_line 296
- select_position 296
- select_word 296
- selectColor 287
- selectedItemCount 292
- selectedItems 200, 292
- selecting
 - files, with file selector 139
 - fonts 44–46, 139
 - resources, in resource list 134
 - shell 20
 - widget, to copy 81
 - widgets 17, 20
 - widgets, to edit 79
- selectInsensitivePixmap 288
- selection 17–20
- selection method, OSF/Motif extended
 - and dragging resources 133
- selection policy
 - of list widget 135
- selection, browse 135, 136
- selectionArray 295
- selectionArrayCount 296
- selectionPolicy 202, 292
- selectPixmap 288
- selectThreshold 295
- self
 - and FACE 70
 - keyword 70
- sensitive 237
- separating menu operations 141
- SeparatorSee XmSeparator
- separatorOn 276
- separators
 - and menu operations 141
 - and spacing 90–92
 - creating 88
- separatorType 92, 285
- session 5
- set 216, 288
- set script 165
 - and XtSetValues 205
 - resetting in adding Callback 206
- SetValues button 29
- shadow_etched_in 262, 278
- shadow_etched_in, as default 285, 289
- shadow_etched_in_dash 285
- shadow_etched_out 262, 278, 285, 289
- shadow_etched_out_dash 285
- shadow_in 262, 278, 289
- shadow_out 262, 278, 289
- shadowThickness 190, 240, 242, 244
- shadowType 262, 278, 289
- sharing text sourcesSee text sources

- Shell 249
- shell
 - ApplicationShell 248
 - child of 20
 - modal dialog 248
 - Override 248
 - selecting 20
 - top level 52
 - TopLevel 248
 - TransientShell 248
 - widget, and creating groups
 - 177
 - XmDialogShell 248
- Shell Toolkit 231
- shell widgets 247–257
- Shell, as widget category 9
- shellUnitType 255
- showArrows 298
- showAsDefault 286
- showSeparator 275
- showValue 279
- single process mode, and fatal error 162
- single_dashed_line 285
- single_line 285
- single_line_edit, as default 294
- single_select 292
- singleSelectionCallback 291
- sizing resourcesSee values
- skipAdjust 277
- slider, and Scale 279
- sliderSize 298
- source 296
- sourceList 204
- sourceList resource
 - adding to a class 204
- spacing 270, 273, 276, 288
 - between widgets 85
 - using separators 90–92
- sprintf 137
- sscanf 172
- state
 - of current session, saving 161
 - restoring 163
- states, of ToggleButton 286
- static 272
- static variables 116
- status
 - design process 161–162
- status indicator icons 161–162
 - and saving 161
- status indicators 8
- STD_LIBS, in makefile 105
- Strings 31–33
- structures, declaring in FACE 123–124
- style
 - NSL predefined 218
 - Rogue Wave 218
- subclass, definition of 230
- submenu, of cascadingCallback 290
- subMenuId 270, 290
- superclass, definition of 230
- superclasses 27
- SYS_CFLAGS, in makefile 105
- SYS_LDFLAGS, in makefile 105

- SYS_LIBS, in makefile 105
- system menu
 - window decoration 247
- system, dual process 162
- T
- tearOffMenuActivateCallback 266
- tearOffMenuDeactivateCallback 266
- tearOffModel 270
- template definition
 - modifying 191–192
- template instance
 - modifying 190–191
- template, override 190
- Templates 23
- templates 185–195
 - children of 195
 - combining 194
 - creating 185–188
 - resources 193–195
 - saving 187
 - setting color scheme resources for 193
 - using 189
- testing 76, 95
 - application 121, 158
 - interface 111, 167, 203
 - script 135, 137
- text editor
 - and changing file 93
 - and fm file format 93
 - and verifying files 159
- text fields
 - aligning 87
- text sources, sharing between
- Text widgets 296
- TextSee XmText
- TextFieldSee XmTextField
- textFontList 254, 262
- textTranslations 262
- tick marks, and Scale 279
- title 252
- title bar
 - window decoration 247
- title, optional 277
- titleEncoding 252
- titleString 280
- toBottomCallback 298
- toggle(s)
 - "l of many" 286
 - "n of many" 286
 - auto position 235
 - auto size 235
 - autosize, and List height 292
 - Checkbox 287
 - Detach All Sides 265
 - Detach this Side 265
 - gadget 229, 232
 - RadioBox 287
- ToggleButtonSee XmToggleButton
- toolkit 2, 9–10
 - button 9
 - menu 9
 - popup menu 9
 - Single toggle button 9
 - widget categories in 9

- top_left 272
- top_right 272
- topAttachment 81, 264
- topCharacter 296
- topItemPosition 292
- toplevel
 - and FACE 70
 - keyword 70
- TopLevelShell 248, 256
- topShadowColor 240, 242, 244
- topShadowPixmap 243
- toTopCallback 298
- transient 252, 256
- transientFor 256
- TransientShell 248, 256
- translations 237
- traversal 240, 243, 245
- traversalOn 243, 245
- traverseObscuredCallback 272
- trough, and Scale 279
- troughColor 298
- Trusted State 163
- try mode 76–77, 111
 - and edit mode 76
 - and verifying functions 159
- type, and declaring variables 124
- types
 - specifying 123
- typing, variables, statically or dynamically 125
- U
- unitType 240, 243, 245
- unmanage, BulletinBoard, with
- autoUnmanage 260

- unmap 254
- unmapCallback 260, 266
- useAsyncGeometry 255
- userData 241, 243, 245
- using
 - C++ class 221–227
 - templates 189
- V
- value 279
 - as XmScrollBar resource 298
 - as XmText resource 295
- value(s)
 - for [bottom]Attachment
 - attach_form 264
 - attach_none 264
 - attach_opposite_widget 264
 - attach_position 264
 - attach_self 264
 - attach_widget 264
 - for alignment
 - alignment_beginning 282
 - alignment_end 282
 - de-
 - fault:alignment_center 282
 - for arrowDirection
 - arrow_down 289
 - arrow_left 289
 - arrow_right 289
 - arrow_up 289
 - for audibleWarning
 - bell 254
 - none 254

- for childHorizontalAlignment
 - alignment_beginning 278
 - alignment_center 278
 - alignment_end 278
- for childType
 - frame_generic_child 278
 - frame_title_child 278
 - frame_workarea_child 278
- for childVerticalAlignment
 - alignment_baseline_bottom 278
 - alignment_baseline_top 278
 - alignment_center 278
 - alignment_widget_bottom 278
 - alignment_widget_top 278
- for commandWindowLocation
 - command_above_workspace 274
 - command_below_workspace 274
- for deleteResponse
 - destroy 254
 - do_nothing 254
 - unmap 254
- for dialogStyle
 - dialog_application_modal 261
 - dialog_full_application_modal 261
 - dialog_modeless 261
 - dialog_system_modal 261
 - dialog_work_area 261
- for editMode
 - multi_line_edit 294
 - single_line_edit 294
- for entryAlignment
 - alignment_beginning 266
 - alignment_center 266
 - alignment_end 266
- for entryVerticalAlignment
 - alignment_baseline_bottom 269
 - alignment_baseline_top 269
 - alignment_center 269
 - alignment_contents_bottom 269
 - alignment_contents_top 269
- for initialState
 - iconicstate 252
 - normalstate 252
- for keyboardFocusPolicy
 - explicit 253
 - pointer 253
- for listSizePolicy
 - constant 292
 - resize_if_possible 292
 - variable 292
- for mwmDecorations
 - See Xm/MwmUtil.h

- for mwmFunctions
 - See Xm/MwmUtil.h
- for navigationType resource
 - exclusive_tab_group 241, 243, 245
 - none 240, 243, 245
 - sticky_tab_group 240, 243, 245
 - tab_group 240, 243, 245
- for packing
 - pack_column 267
 - pack_none 267
 - pack_tight 267
- for processingDirection
 - max_on_bottom 279, 298
 - max_on_left 279, 298
 - max_on_right 279, 298
 - max_on_top 279, 298
- for resizePolicy
 - resize_any 262
 - resize_grow 262
 - resize_none 262
- for scrollBarDisplayPolicy
 - as_needed 272, 292
 - static 272, 292
- for scrollBarPlacement
 - bottom_left 272
 - bottom_right 272
 - top_left 272
 - top_right 272
- for scrollingPolicy
 - application_defined 273
 - automatic 272
- for selectionPolicy
 - default:browse_select 292
 - extended_select 292
 - multiple_select 292
 - single_select 292
- for separatorType
 - double_dashed_line 285
 - double_line 285
 - no_line 285
 - shadow_etched_in 285
 - shadow_etched_in_dash 285
 - shadow_etched_out 285
 - shadow_etched_out_dash 285
 - single_dashed_line 285
 - single_line 285
- for shadowType
 - shadow_etched_in 262, 278, 289
 - shadow_etched_out 262, 278, 289
 - shadow_in 262, 278, 289
 - shadow_out 262, 278, 289
- for shellUnitType
 - 1000th_inches 255
 - 100th_font_units 255
 - 100th_millimeters 255
 - 100th_points 255
 - pixels 255
- for unitType
 - 1000th_inches 240, 243, 245
 - 100th_font_units 240, 243,

- 245
- 100th_millimeters 240,
243, 245
- 100th_points 240, 243, 245
- pixels 240, 243, 245
- for visualPolicy
constant 273
- valueChangedCallback 279, 287,
294
- values, activeSee active values
- valueWcs 295
- variables
 - cb 136
 - declaring 123, 124
 - FACE, typing 125
 - global or local 124
 - local 142
 - makefile 102–107
 - scope 124
- VendorShell 253
- verticalScrollBar 273
- verticalSpacing 264
- viewing area, of ScrolledWin-
dow 270
- viewport, of ScrolledWindow
270
- visibeItemCount 292
- visibleWhenOff 288
- visual 250
- visualPolicy 273
- W
- waitforwm 253
- Wcs 294
- whichButton 270
- WideCharacterSet 294
- widget
 - as group 177
 - associating active values
with 165
 - attaching opposite edges 89–
90
 - attaching to 65–66
 - attaching, in Form 134
 - attaching, to adjust main
window 134
 - categories 9
 - selecting 9
 - centering on a position 90
 - changing class of 23
 - changing name 23
 - child of, and FACE 66
 - children 12, 13
 - creating 13
 - class
 - creating 197–211
 - generating C Code for
115
 - vs applied template class
194
 - container 20
 - creating in interface 14
 - creation order of 140
 - duplicating on Duplicate 82
 - form 59
 - adding resources to 204
 - hierarchy 18
 - and . operator 70
 - and FACE 66

- and groups 177
- and interface 11
- and templates 185
- creating, and Creation Name 116
- generating C Code for 115
- moving widget within
 - See reparenting
- ID 142
 - saving, and C Code options 116
- identifying 23–24
- inserting name in script 75
- moving 21
 - within hierarchy 131
- moving, within widget hierarchySee reparenting
- naming 12, 63
- parent 23
 - and FACE 66
 - in widget referencing 71
- push button 74
- resources 25–28
 - and : operator 71
 - specifying 71
- selecting 17, 18
 - to copy 81
 - to edit 79
- shell
 - and creating groups 177
- sizing 21
- spacing between two 85
 - specification 71
 - variable 71
- Widget Factory option 232
- widget specification 75
- widget tree 11, 18
 - and attaching 66
 - and menu order 140
 - and order of creation 72
 - and reparenting 131
 - and selecting 80
 - creating children in 13
- widget(s)
 - actions, and X events 237
 - adding event handler to 237
 - Command 281
 - composite 259–280
 - size, position of widgets managed by 259
 - primitive 281–298
 - resources of type 230
 - shell 247–257
 - Text, and sharing text sources between 296
 - XmDialog 281
- widget(s) vs gadget(s) 229–230
- widget, Shell 12
- width 235
- widthInc 252
- window
 - active value 166–168
 - mainSee main window, of application
 - resource 29
 - scrolled, creating 131

- window manager
 - mwm 253
- window(s)
 - clipping 270
 - close 247
 - Command 273
 - decorations 247, 248
 - dedicated editing 230
 - dialog window 248
 - iconify 247
 - lower 247
 - main application window 248
 - manager 247
 - manager, mwm, and resize controls with noResi-ze 262
 - Message 273
 - raise 247
 - resize 247
 - secondary toplevel windows 248
- windowGroup 253
- winGravity 253
- WMShell 251
- wmTimeout 253
- wordWrap 295
- work_area RowColumn, and Composite Toolkit 268
- WX_LICENSE_NAME 5
- X
- X events, and widget actions 237
- X_LIBS, in makefile 104
- x-coordinate 235
- XFaceMaker Reference Manual 115
- XFaceMaker Toolkit(s)
 - Composite 231
 - Global 232
 - Group 232
 - Menu 232
 - NSL 232
 - NSL Draw Gadgets 232
 - Primitive 231
 - and fetching gadgets 232
 - Shell 231
 - Template 232
 - UserDefined 232
- XFaceMaker User's Guide 15, 115
- XFaceMaker's Main Window 7–12
- xfm(command) 5, 163
- XFM, in makefile 102
- xfm.startup file 94
- XFM_C_LIBNAME, in makefile 103
- XFM_C_LIBS, in makefile 106
- XFM_E_LIBNAME, in makefile 103
- XFM_E_LIBS, in makefile 106
- XFM_I_LIBNAME, in makefile 103
- XFM_I_LIBS, in makefile 106
- XFM_LICENSE_NAME 5
- XFM_TARGET_MK, in makefile 107
- XFM_TARGET_OS, in makefile 107

- XFMCFLAGS, in makefile 102
- XFMCLFLAGS, in makefile 102
- xfmCreateCallback 71, 72, 199, 230, 240, 261, 269, 275
 - and copying widgets 81
- XLIB_INCPATH, in makefile 103
- XLIB_LIBNAME, in makefile 103
- XLIB_LIBPATH, in makefile 103
- Xm/MwmUtil.h 254
- XM_INCPATH, makefile 104
- XmArrowButton 289
- XmBulletinBoard 52, 59, 60, 139, 189, 260–262
- XmCascadeButton 290
- XmDialog 281
- XmDialogShell 52, 60, 139, 221, 248, 256, 261
- XmDrawingArea 275–276
- XmDrawnButton 288–289
- XmForm 59, 60, 140, 263–265
- XmFrame 277–278
- XmGadget 244–245
- XmLabel 17, 52, 282–284
- XmLabelGadget 192
- XmList 17, 131, 136, 137, 200, 291–293
 - and array variables 128
- XmListAddItemUnselected 129
- XmListCallback 136
- XmListCallbackStruct 136, 137
- XmMainWindow 13, 273–275
 - children 13
- XmMainWindow, as widget class 13
- XmManager class 239–241
- XmMenuShell 251
- XmOptionLabelGadget 269
- XmPanedWindow 276–277
- XmPrimitive class 241–243
- XmPushButton 88, 193, 285–286
- XmRowColumn 265–270
- XmScale 279–280
- XmScrollBar 296–298
- XmScrolledList 291
- XmScrolledText 293
- XmScrolledWindow 131, 270–273
- XmSeparator 92, 284–285
- XmString 119, 129, 142, 147, 200
- XmStringCreateLtoR 119
- XmStringFree 129
- XmStringPtr 200
- XmStrings 31–33
- XmText 129, 293–296
- XmTextField 129, 296
- XmToggleButton 286–288
- XNSLCTRL_LIBNAME, in makefile 103
- Xt Intrinsics, as widget category 9
- XT_INCPATH, in makefile 103
- XT_LIBNAME 96, 161
- XT_LIBNAME, in makefile 103
- XT_LIBPATH, in makefile 103
- XtManageChild 28
- XtNameToWidget 117

XtSetArg 118

XtVaCreateArg 118

XtVaCreateWidget 119

Y

y-coordinate 235